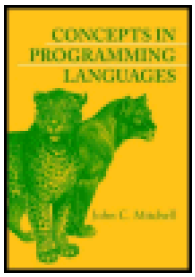
Team-Fly



.**Concepts in Programming Languages**

by John C. Mitchell ISBN:0521780985

Cambridge University Press © 2003 (529 pages)

This book provides a better understanding of the issues and trade-offs that arise in programming language design and a better appreciation of the advantages and pitfalls of the programming languages used.

Table of Contents

Concepts in Programming Languages

Preface

Part 1 - Function and Foundations

Chapter 1 - Introduction

Chapter 2 - Computability

Chapter 3 - Lisp—Functions, Recursion, and Lists

Chapter 4 - Fundamentals

Part 2 - Procedures, Types, Memory Mangement, and Control

Chapter 5 - The Algol Family and ML

Chapter 6 - Type Systems and Type Inference

Chapter 7 - Scope, Functions, and Storage Management

Chapter 8 - Control in Sequential Languages

Part 3 - Modularity, Abstraction, and Object-Oriented Programming

Chapter 9 - Data Abstraction and Modularity

Chapter 10 - Concepts in Object-Oriented Languages

Chapter 11 - History of Objects—Simula and Smalltalk

Chapter 12 - Objects and Run-Time Efficiency— C++

Chapter 13 - Portability and Safety—Java

Part 4 - Concurrency and Logic Programming

Chapter 14 - Concurrent and Distributed Programming

Chapter 15 - The Logic Programming Paradigm and Prolog

Appendix A - Additional Program Examples

Glossary

Index

List of Figures

List of Tables

Team-Fly

Team-Fly

Chapter 1: Introduction

"The Medium Is the Message"
--Marshall McLuhan

1.1 PROGRAMMING LANGUAGES

Programming languages are the medium of expression in the art of computer programming. An ideal programming language will make it easy for programmers to write programs succinctly and clearly. Because programs are meant to be understood, modified, and maintained over their lifetime, a good programming language will help others read programs and understand how they work. Software design and construction are complex tasks. Many software systems consist of interacting parts. These parts, or software components, may interact in complicated ways. To manage complexity, the interfaces and communication between components must be designed carefully. A good language for large-scale programming will help programmers manage the interaction among software components effectively. In evaluating programming languages, we must consider the tasks of designing, implementing, testing, and maintaining software, asking how well each language supports each part of the software life cycle.

There are many difficult trade-offs in programming language design. Some language features make it easy for us to write programs quickly, but may make it harder for us to design testing tools or methods. Some language constructs make it easier for a compiler to optimize programs, but may make programming cumbersome. Because different computing environments and applications require different program characteristics, different programming language designers have chosen different trade-offs. In fact, virtually all successful programming languages were originally designed for one specific use. This is not to say that each language is good for only one purpose. However, focusing on a single application helps language designers make consistent, purposeful decisions. A single application also helps with one of the most difficult parts of language design: leaving good ideas out.



I hope you enjoy using this book. At the beginning of each chapter, I have included pictures of people involved in the development or analysis of programming languages. Some of these people are famous, with major awards and published biographies. Others are less widely recognized. When possible, I have tried to include some personal information based on my encounters with these people. This is to emphasize that programming languages are developed by real human beings. Like most human artifacts, a programming language inevitably reflects some of the personality of its designers.

As a disclaimer, let me point out that I have not made an attempt to be comprehensive in my brief biographical comments. I have tried to liven up the text with a bit of humor when possible, leaving serious biography to more serious biographers. There simply is not space to mention all of the people who have played important roles in the history of programming languages.

Historical and biographical texts on computer science and computer scientists have become increasingly available in recent years. If you like reading about computer pioneers, you might enjoy paging through *Out of Their Minds: The Lives and Discoveries of 15 Great Computer Scientists* by Dennis Shasha and Cathy Lazere or other books on the history of computer science.

John Mitchell

Even if you do not use many of the programming languages in this book, you may still be able to put the conceptual framework presented in these languages to good use. When I was a student in the mid-1970s, all "serious" programmers (at my university, anyway) used Fortran. Fortran did not allow recursion, and recursion was generally regarded as too inefficient to be practical for "real programming." However, the instructor of one course I took argued that recursion was still an important idea and explained how recursive techniques could be used in Fortran by managing data in an array. I am glad I took that course and not one that dismissed recursion as an impractical idea. In the 1980s, many people considered object-oriented programming too inefficient and clumsy for real programming. However, students who learned about object-oriented programming in the 1980s were certainly happy to know about these "futuristic" languages in the 1990s, as object-oriented programming became more widely accepted and used.

Although this is not a book about the history of programming languages, there is some attention to history throughout the book. One reason for discussing historical languages is that this gives us a realistic way to understand programming language trade-offs. For example, programs were different when machines were slow and memory was scarce. The concerns of programming language designers were therefore different in the 1960s from the current concerns. By imaging the state of the art in some bygone era, we can give more serious thought to why language designers made certain decisions. This way of thinking about languages and computing may help us in the future, when computing conditions may change to resemble some past situation. For example, the recent rise in popularity of handheld computing devices and embedded processors has led to renewed interest in programming for devices with limited memory and limited computing power.

When we discuss specific languages in this book, we generally refer to the original or historically important form of a language. For example, "Fortran" means the Fortran of the 1960s and early 1970s. These early languages were called Fortran I, Fortran II, Fortran III, and so on. In recent years, Fortran has evolved to include more modern features, and the distinction between Fortran and other languages has blurred to some extent. Similarly, Lisp generally refers to the Lisps of the 1960s, Smalltalk to the language of the late 1970s and 1980s, and so on.

Team-Fly



1.2 GOALS

In this book we are concerned with the basic concepts that appear in modern programming languages, their interaction, and the relationship between programming languages and methods for program development. A recurring theme is the trade-off between language expressiveness and simplicity of implementation. For each programming language feature we consider, we examine the ways that it can be used in programming and the kinds of implementation techniques that may be used to compile and execute it efficiently.

1.2.1 General Goals

In this book we have the following general goals:

- To understand the *design space* of programming languages. This includes concepts and constructs from past programming languages as well as those that may be used more widely in the future. We also try to understand some of the major conflicts and trade-offs between language features, including implementation costs.
- To develop a better understanding of the languages we currently use by comparing them with other languages.
- To understand the programming techniques associated with various language features. The study of programming languages is, in part, the study of conceptual frameworks for problem solving, software construction, and development.

Many of the ideas in this book are common knowledge among professional programmers. The material and ways of thinking presented in this book should be useful to you in future programming and in talking to experienced programmers if you work for a software company or have an interview for a job. By the end of the course, you will be able to evaluate language features, their costs, and how they fit together.

1.2.2 Specific Themes

Here are some specific themes that are addressed repeatedly in the text:

- *Computability:* Some problems cannot be solved by computer. The undecidability of the halting problem implies that programming language compilers and interpreters cannot do everything that we might wish they could do.
- *Static analysis:* There is a difference between compile time and run time. At compile time, the program is known but the input is not. At run time, the program and the input are both available to the run-time system. Although a program designer or implementer would like to find errors at compile time, many will not surface until run time. Methods that detect program errors at compile time are usually conservative, which means that when they say a program does not have a certain kind of error this statement is correct. However, compile-time error-detection methods will usually say that some programs contain errors even if errors may not actually occur when the program is run.
- *Expressiveness versus efficiency:* There are many situations in which it would be convenient to have a programming language implementation do something automatically. An example discussed in Chapter 3 is memory management: The Lisp run-time system uses garbage collection to detect memory locations no longer needed by the program. When something is done automatically, there is a cost. Although an automatic method may save the programmer from thinking about something, the implementation of the language may run more slowly. In some cases, the automatic method may make it easier to write programs and make programming less prone to error. In other cases, the

resulting slowdown in program execution may make the automatic method infeasible.

Team-Fly



1.3 PROGRAMMING LANGUAGE HISTORY

Hundreds of programming languages have been designed and implemented over the last 50 years. As many as 50 of these programming languages contained new concepts, useful refinements, or innovations worthy of mention. Because there are far too many programming languages to survey, however, we concentrate on six programming languages: Lisp, ML, C, C++, Smalltalk, and Java. Together, these languages contain most of the important language features that have been invented since higher-level programming languages emerged from the primordial swamp of assembly language programming around 1960.

The history of modern programming languages begins around 1958-1960 with the development of Algol, Cobol, Fortran, and Lisp. The main body of this book covers Lisp, with a shorter discussion of Algol and subsequent related languages. A brief account of some earlier languages is given here for those who may be curious about programming language prehistory.

In the 1950s, a number of languages were developed to simplify the process of writing sequences of computer instructions. In this decade, computers were very primitive by modern standards. Most programming was done with the native machine language of the underlying hardware. This was acceptable because programs were small and efficiency was extremely important. The two most important programming language developments of the 1950s were Fortan and Cobol.

Fortran was developed at IBM around 1954-1956 by a team led by John Backus. The main innovation of Fortran (a contraction of formula translator) was that it became possible to use ordinary mathematical notation in expressions. For example, the Fortran expression for adding the value of i to twice the value of j is i + 2*j. Before the development of Fortran, it might have been necessary to place i in a register, place j in a register, multiply j times 2 and then add the result to i. Fortran allowed programmers to think more naturally about numerical calculation by using symbolic names for variables and leaving some details of evaluation order to the compiler. Fortran also had subroutines (a form of procedure or function), arrays, formatted input and output, and declarations that gave programmers explicit control over the placement of variables and arrays in memory. However, that was about it. To give you some idea of the limitations of Fortran, many early Fortran compilers stored numbers 1, 2, 3 … in memory locations, and programmers could change the values of numbers if they were not careful! In addition, it was not possible for a Fortran subroutine to call itself, as this required memory management techniques that had not been invented yet (see Chapter 7).

Cobol is a programming language designed for business applications. Like Fortran programs, many Cobol programs are still in use today, although current versions of Fortran and Cobol differ substantially from forms of these languages of the 1950s. The primary designer of Cobol was Grace Murray Hopper, an important computer pioneer. The syntax of Cobol was intended to resemble that of common English. It has been suggested in jest that if object-oriented Cobol were a standard today, we would use "add 1 to Cobol giving Cobol" instead of "C++".

The earliest languages covered in any detail in this book are Lisp and Algol, which both came out around 1960. These languages have stack memory management and recursive functions or procedures. Lisp provides higher-order functions (still not available in many current languages) and garbage collection, whereas the Algol family of languages provides better type systems and data structuring. The main innovations of the 1970s were methods for organizing data, such as records (or structs), abstract data types, and early forms of objects. Objects became mainstream in the 1980s, and the 1990s brought increasing interest in network-centric computing, interoperability, and security and correctness issues associated with active content on the Internet. The 21st century promises greater diversity of computing devices, cheaper and more powerful hardware, and increasing interest in correctness, security, and interoperability.

Team-Fly

1.4 ORGANIZATION: CONCEPTS AND LANGUAGES

There are many important language concepts and many programming languages. The most natural way to summarize the field is to use a two-dimensional matrix, with languages along one axis and concepts along the other. Here is a partial sketch of such a matrix:

Language	Expressions	Functions	Heap storage	Exceptions	Modules	Objects	Threads
Lisp	x	x	x				
C	x	x	x				
Algol 60	x	x					
Algol 68	x	x	x				x
Pascal	x	x	x				
Modula-2	x	x	x		x		
Modula-3	x	x	x	x	x	x	
ML	x	x	x	x	x		
Simula	x	x	x			x	x
Smalltalk	x	x	x	x		x	x
C++	x	x	x	x	x	x	
Objective C	x	x	x			x	
Java	x	x	x	x	x	x	x

Although this matrix lists only a fraction of the languages and concepts that might be covered in a basic text or course on the programming languages, one general characteristic should be clear. There are some basic language concepts, such as expressions, functions, local variables, and stack storage allocation that are present in many languages. For these concepts, it makes more sense to discuss the concept in general than to go through a long list of similar languages. On the other hand, for concepts such as objects and threads, there are relatively few languages that exhibit these concepts in interesting ways. Therefore, we can study most of the interesting aspects of objects by comparing a few languages. Another factor that is not clear from the matrix is that, for some concepts, there is considerable variation from language to language. For example, it is more interesting to compare the way objects have been integrated into languages than it is to compare integer expressions. This is another reason why competing object-oriented languages are compared, but basic concepts related to expressions, statements, functions, and so on, are covered only once, in a concept-oriented way.

Most courses and texts on programming languages use some combination of language-based and concept-based presentation. In this book a concept-oriented organization is followed for most concepts, with a language-based organization used to compare object-oriented features.

The text is divided into four parts:

Part 1: Functions and Foundations	(Chapters 1-4)
Part 2: Procedures, Types, Memory Management, and Control	(5-8)

Part 3: Modularity, Abstraction and Object-Oriented Programming (9-13)

Part 4: Concurrency and Logic Programming (14 and 15)

In Part 1 a short study of Lisp is presented, followed by a discussion of compiler structure, parsing, lambda calculus, and denotational semantics. A short chapter provides a brief discussion of computability and the limits of compile-time program analysis and optimization. For C programmers, the discussion of Lisp should provide a good chance to think differently about programming and programming languages.

In Part 2, we progress through the main concepts associated with the conventional languages that are descended in some way from the Algol family. These concepts include type systems and type checking, functions and stack storage allocation, and control mechanisms such as exceptions and continuations. After some of the history of the Algol family of languages is summarized, the ML programming language is used as the main example, with some discussion and comparisons using C syntax.

Part 3 is an investigation of program-structuring mechanisms. The important language advances of the 1970s were abstract data types and program modules. In the late 1980s, object-oriented concepts attained widespread acceptance. Because object-oriented programming is currently the most prominent programming paradigm, in most of Part 3 we focus on object-oriented concepts and languages, comparing Smalltalk, C++, and Java.

Part 4 contains chapters on language mechanisms for concurrent and distributed programs and on logic programming.

Because of space limitations, a number of interesting topics are not covered. Although scripting languages and other "special-purpose" languages are not covered explicitly in detail, an attempt has been made to integrate some relevant language concepts into the exercises.

Team-Fly



Previous Next

Chapter 2: Computability

Some mathematical functions are computable and some are not. In all general-purpose programming languages, it is possible to write a program for each function that is computable in principle. However, the limits of computability also limit the kinds of things that programming language implementations can do. This chapter contains a brief overview of computability so that we can discuss limitations that involve computability in other chapters of the book.

2.1 PARTIAL FUNCTIONS AND COMPUTABILITY

From a mathematical point of view, a program defines a function. The output of a program is computed as a function of the program inputs and the state of the machine before the program starts. In practice, there is a lot more to a program than the function it computes. However, as a starting point in the study of programming languages, it is useful to understand some basic facts about computable functions.

The fact that not all functions are computable has important ramifications for programming language tools and implementations. Some kinds of programming constructs, however useful they might be, cannot be added to real programming languages because they cannot be implemented on real computers.

2.1.1 Expressions, Errors, and Nontermination

In mathematics, an expression may have a defined value or it may not. For example, the expression 3 + 2 has a defined value, but the expression 3/0 does not. The reason that 3/0 does not have a value is that division by zero is not defined: division is defined to be the inverse of multiplication, but multiplication by zero cannot be inverted. There is nothing to try to do when we see the expression 3/0; a mathematician would just say that this operation is undefined, and that would be the end of the discussion.

In computation, there are two different reasons why an expression might not have a value:





Alan Turing was a British mathematician. He is known for his early work on computability and his work for British Intelligence on code breaking during the Second World War. Among computer scientists, he is best known for the invention of the Turing machine. This is not a piece of hardware, but an idealized computing device. A Turing machine consists of an infinite tape, a tape read-write head, and a finite-state controller. In each computation step, the machine reads a tape symbol and the finite-state controller decides whether to write a different symbol on the current tape square and then whether to move the read-write head one square left or right. The importance of this idealized computer is that it is both very simple and very powerful.

Turing was a broad-minded individual with interests ranging from relativity theory and mathematical logic to number theory and the engineering design of mechanical computers. There are numerous published biographies of Alan Turing, some emphasizing his wartime work and others calling attention to his sexuality and its impact on his professional career.

The ACM Turing Award is the highest scientific honor in computer science, equivalent to a Nobel Prize in other fields.

- *Error termination:* Evaluation of the expression cannot proceed because of a conflict between operator and operand.
- *Nontermination:* Evaluation of the expression proceeds indefinitely.

An example of the first kind is division by zero. There is nothing to compute in this case, except possibly to stop the computation in a way that indicates that it could not proceed any further. This may halt execution of the entire program, abort one thread of a concurrent program, or raise an exception if the programming language provides exceptions.

The second case is different: There is a specific computation to perform, but the computation may not terminate and therefore may not yield a value. For example, consider the recursive function defined by

```
f(x:int) = if x = 0 then 0 else x + f(x-2)
```

This is a perfectly meaningful definition of a *partial* function, a function that has a value on some arguments but not on all arguments. The expression `f(4)` calling the function f above has value 4 + 2 + 0 = 6, but the expression `f(5)` does not have a value because the computation specified by this expression does not terminate.

2.1.2 Partial Functions

A partial function is a function that is defined on some arguments and undefined on others. This is ordinarily what is meant by function in programming, as a function declared in a program may return a result or may not if some loop or sequence of recursive calls does not terminate. However, this is not what a mathematician ordinarily means by the word function.

The distinction can be made clearer by a look at the mathematical definitions. A reasonable definition of the word function is this: A function $f : A \rightarrow B$ from set A to set B is a rule associating a unique value $y = f(x)$ in B with every x in A. This is almost a mathematical definition, except that the word rule does not have a precise mathematical meaning. The notation $f : A \rightarrow B$ means that, given arguments in the set A, the function f produces values from set B. The set A is called the *domain* of f, and the set B is called the *range* or the *codomain* of f.

The usual mathematical definition of function replaces the idea of rule with a set of argument-result pairs called the graph of a function. This is the mathematical definition:

A *function* $f : A \rightarrow B$ is a set of ordered pairs $f \subseteq A \times B$ that satisfies the following conditions:

1. If ?x, y? ? f and ?x, z?? f, then $y = z$.
2. For every x ? A, there exists y ? B with ?x, y?? f.

When we associate a set of ordered pairs with a function, the ordered pair ?x, y? is used to indicate that y is the value of the function on argument x. In words, the preceding two conditions can be stated as (1) a function has at most one value for every argument in its domain, and (2) a function has at least one value for every argument in its domain.

A partial function is similar, except that a partial function may not have a value for every argument in its domain. This is the mathematical definition:

A *partial function* $f : A \rightarrow B$ is a set of ordered pairs $f \subseteq A \times B$ satisfying the preceding condition

1. If ?x, y?? f and ?x, z?? f, then $y = z$.

In words, a partial function is single valued, but need not be defined on all elements of its domain.

Programs Define Partial Functions

In most programming languages, it is possible to define functions recursively. For example, here is a function f defined in terms of itself:

```
f(x:int) = ifx=0 then 0 else x + f(x-2);
```

If this were written as a program in some programming language, the declaration would associate the function name f with an algorithm that terminates on every even$x \geq 0$, but diverges (does not halt and return a value) if x is odd or negative. The algorithm for f defines the following mathematical function f, expressed here as a set of ordered pairs:

f ={?x, y?| x is positive and even, y = 0 + 2 + 4 +...+ x}.

This is a partial function on the integers. For every integer x, there is at most one y with $f(x) = y$. However, if x is an odd number, then there is no y with $f(x) = y$. Where the algorithm does not terminate, the value of the function is undefined. Because a function call may not terminate, this program defines a partial function.

2.1.3 Computability

Computability theory gives us a precise characterization of the functions that are computable in principle. The class of functions on the natural numbers that are computable in principle is often called the class of *partial recursive functions*,

as recursion is an essential part of computation and computable functions are, in general, partial rather than total. The reason why we say "computable in principle" instead of "computable in practice" is that some computable functions might take an extremely long time to compute. If a function call will not return for an amount of time equal to the length of the entire history of the universe, then in practice we will not be able to wait for the computation to finish. Nonetheless, computability in principle is an important benchmark for programming languages.

Computable Functions

Intuitively, a function is *computable* if there is some program that computes it. More specifically, a function $f : A \rightarrow B$ is computable if there is an algorithm that, given any $x ? A$ as input, halts with $y = f(x)$ as output.

One problem with this intuitive definition of computable is that a program has to be written out in some programming language, and we need to have some implementation to execute the program. It might very well be that, in one programming language, there is a program to compute some mathematical function and in another language there is not.

In the 1930s, Alonzo Church of Princeton University proposed an important principle, called Church's thesis. Church's thesis, which is a widely held belief about the relation between mathematical definitions and the real world of computing, states that the same class of functions on the integers can be computed by any general computing device. This is the class of partial recursive functions, sometimes called the class of *computable functions*. There is a mathematical definition of this class of functions that does not refer to programming languages, a second definition that uses a kind of idealized computing device called a *Turing machine*, and a third (equivalent) definition that uses lambda calculus (see Section 4.2). As mentioned in the biographical sketch on Alan Turing, a Turing machine consists of an infinite tape, a tape read-write head, and a finite-state controller. The tape is divided into contiguous cells, each containing a single symbol. In each computation step, the machine reads a tape symbol and the finite-state controller decides whether to write a different symbol on the current tape square and then whether to move the read-write head one square left or right. Part of the evidence that Church cited in formulating this thesis was the proof that Turing machines and lambda calculus are equivalent. The fact that all standard programming languages express precisely the class of partial recursive functions is often summarized by the statement that *all programming languages are Turing complete*. Although it is comforting to know that all programming languages are universal in a mathematical sense, the fact that all programming languages are Turing complete also means that computability theory does not help us distinguish among the expressive powers of different programming languages.

Noncomputable Functions

It is useful to know that some specific functions are not computable. An important example is commonly referred to as the *halting problem*. To simplify the discussion and focus on the central ideas, the halting problem is stated for programs that require one string input. If *P* is such a program and *x* is a string input, then we write *P*(*x*)for the output of program *P* on input *x*.

> *Halting Problem:* Given a program *P* that requires exactly one string input and a string *x*, determine whether *P* halts on input *x*.

We can associate the halting problem with a function f_{halt} by letting f_{halt} (*P*, *x*) = "halts" if *P* halts on input and f_{halt}(*P*, *x*) = "does not halt" otherwise. This function f_{halt} can be considered a function on strings if we write each program out as a sequence of symbols.

The *undecidability of the halting problem* is the fact that the function f_{halt} is not computable. The undecidability of the halting problem is an important fact to keep in mind in designing programming language implementations and optimizations. It implies that many useful operations on programs cannot be implemented, even in principle.

Proof of the Undecidability of the Halting Problem. Although you will not need to know this proof to understand any other topic in the book, some of you may be interested in proof that the halting function is not computable. The proof is surprisingly short, but can be difficult to understand. If you are going to be a serious computer scientist, then you will want to look at this proof several times, over the course of several days, until you understand the idea behind it.

> Step 1: Assume that there is a program *Q* that solves the halting problem. Specifically, assume that program *Q* reads two inputs, both strings, and has the following output:

$$Q(P, x) = \begin{cases} \text{halts} & \text{if } P(x) \text{ halts} \\ \end{cases}$$

$$Q(P,x) = \begin{cases} \text{halts} & \text{if } P(x) \text{ does not halt} \\ \text{does not halt} & \text{if } P(x) \text{ does not} \end{cases}.$$

An important part of this specification for *Q* is that *Q*(*P*, *x*) always halts for every *P* and *x*.

Step 2: Using program *Q*, we can build a program *D* that reads one string input and sometimes does not halt. Specifically, let *D* be a program that works as follows:

D(*P*) = if *Q*(*P*, *P*) = halts then *run forever* else *halt*.

Note that *D* has only one input, which it gives twice to *Q*. The program *D* can be written in any reasonable language, as any reasonable language should have some way of programming if-then-else and some way of writing a loop or recursive function call that runs forever. If you think about it a little bit, you can see that *D* has the following behavior:

$$D(P) = \begin{cases} \text{halt} & \text{if } P(P) \text{ runs forever} \\ \text{run forever} & \text{if } P(P) \text{ halts} \end{cases}.$$

In this description, the word halt means that *D*(*P*) comes to a halt, and runs forever means that *D*(*P*) continues to execute steps indefinitely. The program *D*(*P*) halts or does not halt, but does not produce a string output in any case.

Step 3: Derive a contradiction by considering the behavior *D*(*D*) of program *D* on input *D*. (If you are starting to get confused about what it means to run a program with the program itself as input, assume that we have written the program *D* and stored it in a file. Then we can compile *D* and run *D* with the file containing a copy of *D* as input.) Without thinking about how *D* works or what *D* is supposed to do, it is clear that either *D*(*D*) halts or *D*(*D*) does not halt. If *D*(*D*) halts, though, then by the property of *D* given in step 2, this must be because *D*(*D*) runs forever. This does not make any sense, so it must be that *D*(*D*) runs forever. However, by similar reasoning, if *D*(*D*) runs forever, then this must be because *D*(*D*) halts. This is also contradictory. Therefore, we have reached a contradiction.

Step 4: Because the assumption in step 1 that there is a program *Q* solving the halting problem leads to a contradiction in step 3, it must be that the assumption is false. Therefore, there is no program that solves the halting problem.

Applications

Programming language compilers can often detect errors in programs. However, the undecidability of the halting problem implies that some properties of programs cannot be determined in advance. The simplest example is halting itself. Suppose someone writes a program like this:

```
i=0;
while (i != f(i)) i = g(i);
printf( ... i ...);
```

It seems very likely that the programmer wants the while loop to halt. Otherwise, why would the programmer have written a statement to print the value of i after the loop halts? Therefore, it would be helpful for the compiler to print a warning message if the loop will not halt. However useful this might be, though, it is not possible for a compiler to determine whether the loop will halt, as this would involve solving the halting problem.

2.2 CHAPTER SUMMARY

Computability theory establishes some important ground rules for programming language design and implementation. The following main concepts from this short overview should be remembered:

- *Partiality:* Recursively defined functions may be partial functions. They are not always total functions. A function may be partial because a basic operation is not defined on some argument or because a computation does not terminate.
- *Computability:* Some functions are computable and others are not. Programming languages can be used to define computable functions; we cannot write programs for functions that are not computable in principle.
- *Turing completeness:* All standard general-purpose programming languages give us the same class of computable functions.
- *Undecidability:* Many important properties of programs cannot be determined by any computable function. In particular, the halting problem is undecidable.

When the value of a function or the value of an expression is undefined because a basic operation such as division by zero does not make sense, a compiler or interpreter can cause the program to halt and report the error. However, the undecidability of the halting problem implies that there is no way to detect and report an error whenever a program is not going to halt.

There is a lot more to computability and complexity theory than is summarized in the few pages here. For more information, see one of the many books on computability and complexity theory such as *Introduction to Automata Theory, Languages, and Computation* by Hopcroft, Motwani, and Ullman (Addison Wesley, 2001) or *Introduction to the Theory of Computation* by Sipser (PWS, 1997).

```
(compose (lambda (x) (+ x x)) (lambda (x) (* x x)))
```

Example 3.5

A maplist is a function that takes a function and list and applies the function to every element in the list. The result is a list that contains all the results of function application. Using define to define a recursive function, we can write the maplist as follows:

```
(define maplist (lambda (f x)
 (cond ((eq x nil) nil) (true (cons (f (car x)) (maplist f (cdr x)))))))
```

We cannot say whether the maplist is a second-order or third-order function, as the elements of the list might be atoms, functions, or higher-order functions. As an example of the use of this function, we have

```
(maplist square '(1 2 3 4 5)) => (1 4 9 16 25),
```

where the symbol → means "evaluates to."

Higher-order functions require more run-time support than first-order functions, as discussed in some detail in Chapter 7.

3.4.8 Garbage Collection

In computing, *garbage* refers to memory locations that are not accessible to a program. More specifically, we define garbage as follows:

> At a given point in the execution of a program *P*, a memory location *m* is *garbage* if no completed execution of *P* from this point can access location *m*. In other words, replacing the contents of *m* or making this location in accessibleto *P* cannot affect any further execution of the program.

Note that this definition does not give an algorithm for finding garbage. However, if we could find all locations that are garbage (by this definition), at some point in the suspended execution of a program, it would be safe to deallocate these locations or use them for some other purpose.

In Lisp, the memory locations that are accessible to a program are cons cells. Therefore the garbage associated with a running Lisp program will be a set of cons cells that are not needed to complete the execution of the program. Garbage collection is the process of detecting garbage during the execution of a program and making it available for other uses. In garbage-collected languages, the run-time system receives requests for memory (as when Lisp cons cells are created) and allocates memory from some list of available space. The list of available memory locations is

called the free list. When the run-time system detects that the available space is below some threshold, the program may be suspended and the garbage collector invoked. In Lisp and other garbage-collected languages, it is generally not necessary for the program to invoke the garbage collector explicitly. (In some modern implementations, the garbage collector may run in parallel with the program. However, because concurrent garbage collection raises some additional considerations, we will assume that the program is suspended when the garbage collector is running.)

The idea and implementation of automatic garbage collection appear to have originated with Lisp.

Here is an example of garbage. After the expression

```
(car (cons e1 e2 ))
```

is evaluated, any cons cells created by evaluation of e_2 will typically be garbage. However, it is not always correct to deallocate the locations used in a list after applying car to the list. For example, consider the expression

```
((lambda (x) (car (cons x x))) '(A B))
```

When this expression is evaluated, the function car will be applied to a cons cell whose "a" and "d" parts both point to the same list.

Many algorithms for garbage collection have been developed over the years. Here is a simple example called mark-and-sweep. The name comes from the fact that the algorithm first marks all of the locations reachable from the program, then "sweeps" up all the unmarked locations as garbage. This algorithm assumes that we can tell which bit sequences in memory are pointers and which are atoms, and it also assumes that there is a tag bit in each location that can be switched to 0 or 1 without destroying the data in that location.

Mark-and-Sweep Garbage Collection

1. Set all tag bits to 0.
2. Start from each location used directly in the program. Follow all links, changing the tag bit of each cell visited to 1.
3. Place all cells with tags still equal to 0 on the free list.

Garbage collection is a *very* useful feature, at least as far as programmer convenience goes. There is some debate about the efficiency of garbage-collected languages, however. Some researchers have experimental evidence showing that garbage collection adds of the order of 5% overhead to program execution time. However, this sort of measurement depends heavily on program design. Some simple programs could be written in C without the use of any user-allocated memory, but when translated into Lisp could create many cons cells during expression evaluation and therefore involve a lot of garbage-collection overhead. On the other hand, explicit memory management in C and C++ (in place of garbage collection) can be cumbersome and error prone, so that for certain programs it is highly advantageous to have automatic garbage collection. One phenomenon that indicates the importance and difficulty of memory management in C programs is the success of program analysis tools that are aimed specifically at detecting memory management errors.

Example. In Lisp, we can write a function that takes a list lst and an entry x, returning the part of the list that follows x, if any. This function, which we call select, can be written as follows:

```
(define select (lambda (x lst)
  (cond ((equal lst nil) nil)
    ((equal x (car lst)) (cdr lst))
    (true (select x (cdr lst)))
)))
```

Here are two analogous C programs that have different effects on the list they are passed. The first one leaves the list alone, returning a pointer to the cdr of the first cell that has its car equal to x:

```
typedef struct cell cell;
struct cell {
  cell * car, * cdr;
} ;
cell * select (cell *x, cell *lst) {
  cell *ptr;
  for (ptr=lst; ptr != 0; ) {
    if (ptr->car = = x) return(ptr->cdr);
    else ptr = ptr->cdr;
  };
};
```

A second C program might be more appropriate if only the part of the list that follows x will be used in the rest of the program. In this case, it makes sense to free the cells that will no longer be used. Here is a C function that does just this:

```
cell * select1 (cell *x; cell *lst) {
  cell *ptr, *previous;
  for (ptr=lst; ptr != 0; ) {
    if (ptr->car = = x) return(ptr->cdr);
    else previous = ptr;
    ptr = ptr->cdr;
    free(previous);
  }
}
```

An advantage of Lisp garbage collection is that the programmer does not have to decide which of these two functions to call. In Lisp, it is possible to just return a pointer to the part of the list that you want and let the garbage collector figure out whether you may need the rest of the list ever again. In C, on the other hand, the programmer must decide, while traversing the list, whether this is the last time that these cells will be referenced by the program.

Question to Ponder. It is interesting to observe that programming languages such as Lisp, in which most computation is expressed by recursive functions and linked data structures, provide automatic garbage collection. In contrast, simple imperative languages such as C require the programmer to free locations that are no longer needed. Is it just a coincidence that function-oriented languages have garbage collection and assignment-oriented languages do not? Or is there something intrinsic to function-oriented programming that makes garbage collection more appropriate for these languages? Part of the answer lies in the preceding example. Another part of the answer seems to lie in the problems associated with storage management for higher-order functions, studied in Section 7.4.

3.4.9 Pure Lisp and Side Effects

Pure Lisp expressions do not have side effects, which are visible changes in the state of the machine as the result of evaluating an expression. However, for efficiency, even early Lisp had expressions with side effects. Two historical functions with side effects are rplaca and rplacd:

(rplaca x y) - replace the address field of cons cell x with y,

(rplacd x y) - replace the decrement field of cons cell x with y.

In both cases, the value of the expression is the cell that has been modified. For example, the value of

---

```
(rplaca (cons 'A 'B) 'C)
```

---

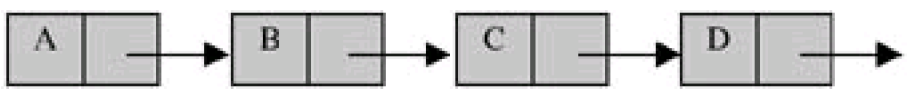
is the cons cell with car ‘C and cdr ‘B produced when a new cons cell is allocated in the evaluation of (cons ‘A ‘B) and then the car ‘A is replaced with ‘C.

With these constructs, two occurrences of the same expression may have different values. (This is really what side effect means.) For example, consider the following code:

```
(lambda (x) (cons (car x) (cons (rplaca x c) (car x)))) (cons a b)
```

The expression (car x) occurs twice within the function expression, but there will be two different values in the two places this expression is evaluated. When rplaca and rplacd are used, it is possible to create circular list structures, something that is not possible in pure Lisp.

One situation in which rplaca and rplacd may increase efficiency is when a program modifies a cell in the middle of a list. For example, consider the following list of four elements:



Team-Fly

Chapter 4: Fundamentals

OVERVIEW

In this chapter some background is provided on programming language implementation through brief discussions of syntax, parsing, and the steps used in conventional compilers. We also look at two foundational frameworks that are useful in programming language analysis and design: lambda calculus and denotational semantics. Lambda calculus is a good framework for defining syntactic concepts common to many programming languages and for studying symbolic evaluation. Denotational semantics shows that, in principle, programs can be reduced to functions.

A number of other theoretical frameworks are useful in the design and analysis of programming languages. These range from computability theory, which provides some insight into the power and limitations of programs, to type theory, which includes aspects of both syntax and semantics of programming languages. In spite of many years of theoretical research, the current programming language theory still does not provide answers to some important foundational questions. For example, we do not have a good mathematical theory that includes higher-order functions, state transformations, and concurrency. Nonetheless, theoretical frameworks have had an impact on the design of programming languages and can be used to identify problem are as in programming languages. To compare one aspect of theory and practice, we compare functional and imperative languages in Section 4.4.

Team-Fly

Previous Next

Team-Fly

4.1 COMPILERS AND SYNTAX

A program is a description of a dynamic process. The text of a program itself is called its syntax; the things a program does comprise its semantics. The function of a programming language implementation is to transform program syntax into machine instructions that can be executed to cause the correct sequence of actions to occur.

4.1.1 Structure of a Simple Compiler

Programming languages that are convenient for people to use are built around concepts and abstractions that may not correspond directly to features of the underlying machine. For this reason, a program must be translated into the basic instruction set of the machine before it can be executed. This can be done by a *compiler*, which translates the entire program into machine code before the program is run, or an *interpreter*, which combines translation and program execution. We discuss programming language implementation by using compilers, as this makes it easier to separate the main issues and to discuss them in order.



An early pioneer, John Backus became a computer programmer at IBM in 1950. In the 1950s, Backus developed Fortran, the first high-level computer language, which became commercially available in 1957. The language is still widely used for numerical and scientific programming. In 1959, John Backus invented Backus naur form (BNF), the standard notation for defining the syntax of a programming language. In later years, he became an advocate of pure functional programming, devoting his 1977 ACM Turing Award lecture to this topic.

I met John Backus through IFIP WG 2.8, a working group of the International Federation of Information Processing on functional programming. Backus continued to work on functional programming at IBM Almaden through the 1980s, although his group was disbanded after his retirement. A mild-mannered and unpretentious individual, here is a quote that gives some sense of his independent, pioneering spirit:

"I really didn't know what the hell I wanted to do with my life. I decided that what I wanted was a good hi fi set because I liked music. In those days, they didn't really exist so I went to a radio technicians' school. I had a very nice teacher - the first good teacher I ever had - and he asked me to cooperate with him and compute the characteristics of some circuits for a magazine."

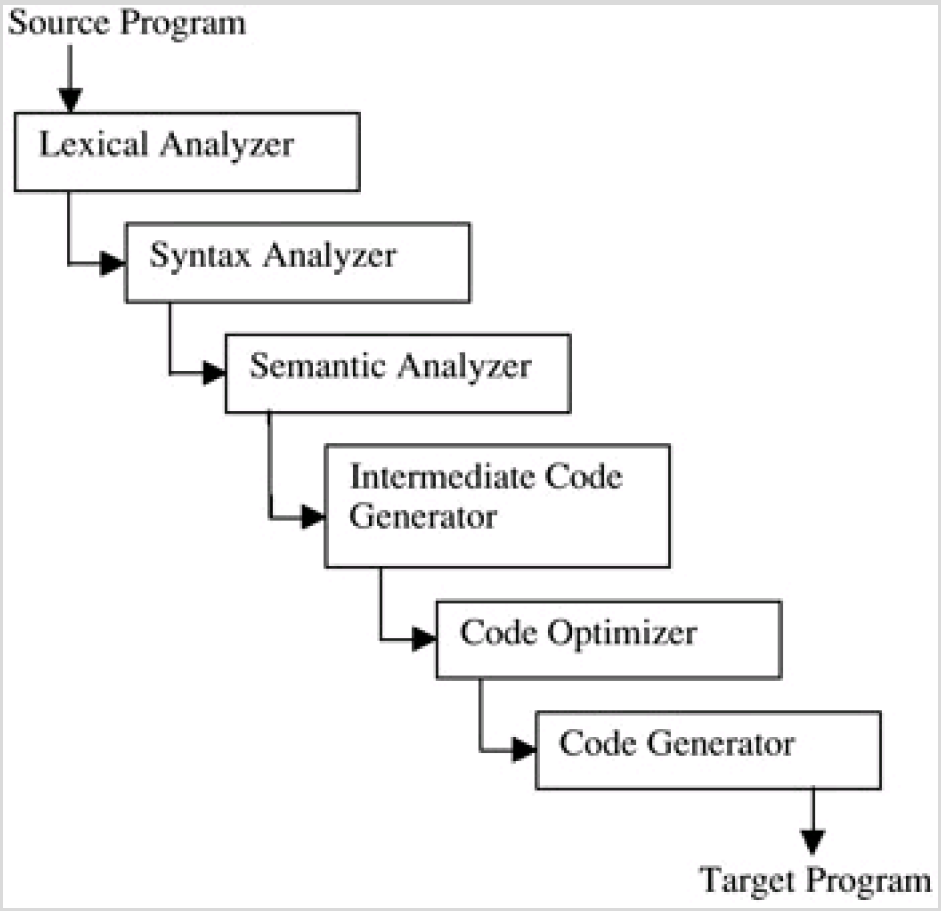
"I remember doing relatively simple calculations to get a few points on a curve for an amplifier. It was laborious and tedious and horrible, but it got me interested in math. The fact that it had an application -that interested me."

The main function of a compiler is illustrated in this simple diagram:



Given a program in some *source language*, the compiler produces a program in a *target language*, which is usually the instruction set, or machine language, of some machine.

Most compilers are structured as a series of phases, with each phase performing one step in the translation of source program to target program. A typical compiler might consist of the phases shown in the following diagram:



Each of these phases is discussed briefly. Our goal with this book is only to understand the parts of a compiler so that we can discuss how different programming language features might be implemented. We do not discuss how to build a compiler. That is the subject of many books on compiler construction, such as *Compilers: Principles, Techniques and*

Tools by Aho, Sethi, and Ullman (Addison-Wesley, 1986),and *Modern Compiler Implementation in Java/ML/C* by Appel (Cambridge Univ. Press, 1998).

Lexical Analysis

The input symbols are scanned and grouped into meaningful units called *tokens.* For example, lexical analysis of the expression temp := x+1, which uses Algol-style notation := for assignment, would divide this sequence of symbols into five tokens: the identifier temp, the assignment "symbol" :=, the variable x, the addition symbol+, and the number 1. Lexical analysis can distinguish numbers from identifiers. However, because lexical analysis is based on a single left-to-right (and top-to-bottom)scan, lexical analysis does not distinguish between identifiers that are names of variables and identifiers that are names of constants. Because variables and constants are declared differently, variables and constants are distinguished in the semantic analysis phase.

Syntax Analysis

In this phase, tokens are grouped into syntactic units such as expressions, statements, and declarations that must conform to the grammatical rules of the programming language. The action performed during this phase, called *parsing*, is described in Subsection 4.1.2. The purpose of parsing is to produce a data structure called a parse tree, which represents the syntactic structure of the program in a way that is convenient for subsequent phases of the compiler. If a program does not meet the syntactic requirements to be a well formed program, then the parsing phase will produce an error message and terminate the compiler.

Semantic Analysis

In this phase of a compiler, rules and procedures that depend on the context surrounding an expression are applied. For example, returning to our sample expression temp := x+1, we find that it is necessary to make sure that the types match. If this assignment occurs in a language in which integers are automatically converted to floats as needed, then there are several ways that types could be associated with parts of this expression. In standard semantic analysis, the types of temp and x would be determined from the declarations of these identifiers. If these are both integers, then the number 1 could be marked as an integer and + marked as integer addition, and the expression would be considered correct. If one of the identifiers, say x, is a float, then the number 1 would be marked as a float and + marked as a floating-point addition. Depending on whether temp is a float or an integer, it might also be necessary to insert a conversion around the subexpression x+1. The output of this phase is an augmented parse tree that represents the syntactic structure of the program and includes additional information such as the types of identifiers and the place in the program where each identifier is declared.

Although the phase following parsing is commonly called *semantic analysis*, this use of the word *semantic* is different from the standard use of the term for *meaning*. Some compiler writers use the word semantic because this phase relies on context information, and the kind of grammar used for syntactic analysis does not capture context information. However, in the rest of this book, the word semantics is used to refer to how a program executes, not the essentially syntactic properties that arise in the third phase of a compiler.

Intermediate Code Generation

Although it might be possible to generate a target program from the results of syntactic and semantic analysis, it is difficult to generate efficient code in one phase. Therefore, many compilers first produce an intermediate form of code and then optimize this code to produce a more efficient target program.

Because the last phase of the compiler can translate one set of instructions to another, the intermediate code does not need to be written with the actual instruction set of the target machine. It is important to use an intermediate representation that is easy to produce and easy to translate into the target language. The intermediate representation can be some form of generic low-level code that has properties common to several computers. When a single generic intermediate representation is used, it is possible to use essentially the same compiler to generate target programs for several different machines.

Code Optimization

There are a variety of techniques that may be used to improve the efficiency of a program. These techniques are

usually applied to the intermediate representation. If several optimization techniques are written as transformations of the intermediate representation, then these techniques can be applied over and over until some termination condition is reached.

The following list describes some standard optimizations:

- *Common Subexpression Elimination:* If a program calculates the same value more than once and the compiler can detect this, then it may be possible to transform the program so that the value is calculated only once and stored for subsequent use.
- *Copy Propagation:* If a program contains an assignment such as x=y, then it may be possible to change subsequent statements to refer to y instead of to x and to eliminate the assignment.
- *Dead-Code Elimination:* If some sequence of instructions can never be reached, then it can be eliminated from the program.
- *Loop Optimizations:* There are several techniques that can be applied to remove instructions from loops. For example, if some expression appears inside a loop but has the same value on each pass through the loop, then the expression can be moved outside the loop.
- *In-Lining Function Calls:* If a program calls function f, it is possible to substitute the code for f into the place where f is called. This makes the target program more efficient, as the instructions associated with calling a function can be eliminated, but it also increases the size of the program. The most important consequence of in-lining function calls is usually that they allow other optimizations to be performed by removing jumps from the code.

Code Generation

The final phase of a standard compiler is to convert the intermediate code into a target machine code. This involves choosing a memory location, a register, or both, for each variable that appears in the program. There are a variety of register allocation algorithms that try to reuse registers efficiently. This is important because many machines have a fixed number of registers, and operations on registers are more efficient than transferring data into and out of memory.

4.1.2 Grammars and Parse Trees

We use grammars to describe various languages in this book. Although we usually are not too concerned about the pragmatics of parsing, we take a brief look in this subsection at the problem of producing a parse tree from a sequence of tokens.

Grammars

Grammars provide a convenient method for defining infinite sets of expressions. In addition, the structure imposed by a grammar gives us a systematic way of processing expressions.

A *grammar* consists of a start symbol, a set of nonterminals, a set of terminals, and a set of productions. The nonterminals are symbols that are used to write out the grammar, and the terminals are symbols that appear in the language generated by the grammar. In books on automata theory and related subjects, the productions of a grammar are written in the form $s \rightarrow tu$, with an arrow, meaning that in a string containing the symbol s, we can replace s with the symbols tu. However, here we use a more compact notation, commonly referred to as BNF.

The main ideas are illustrated by example. A simple language of numeric expressions is defined by the following grammar:

```
e ::= n | e+e | e-e
n ::= d | nd
d ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

where e is the *start symbol*, symbols e, n, and d are nonterminals, and 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, and - are the terminals. The language defined by this grammar consists of all the sequences of terminals that we can produce by starting with the start symbol e and by replacing nonterminals according to the preceding productions. For example, the first preceding production means that we can replace an occurrence of e with the symbol n, the three symbols e+e, or the three symbols e-e. The process can be repeated with any of the preceding three lines.

Some expressions in the language given by this grammar are

0, 1+3+5, 2+4 - 6 - 8

Sequences of symbols that contain nonterminals, such as

---

e, e+e, e+6 - e

---

are not expressions in the language given by the grammar. The purpose of non-terminals is to keep track of the form of an expression as it is being formed. All nonterminals must be replaced with terminals to produce a well-formed expression of the language.

Derivations

A sequence of replacement steps resulting in a string of terminals is called a *derivation*.

Here are two derivations in this grammar, the first given in full and the second with a few missing steps that can be filled in by the reader (be sure you understand how!):

$$e \rightarrow n \rightarrow nd \rightarrow dd \rightarrow 2d \rightarrow 25$$

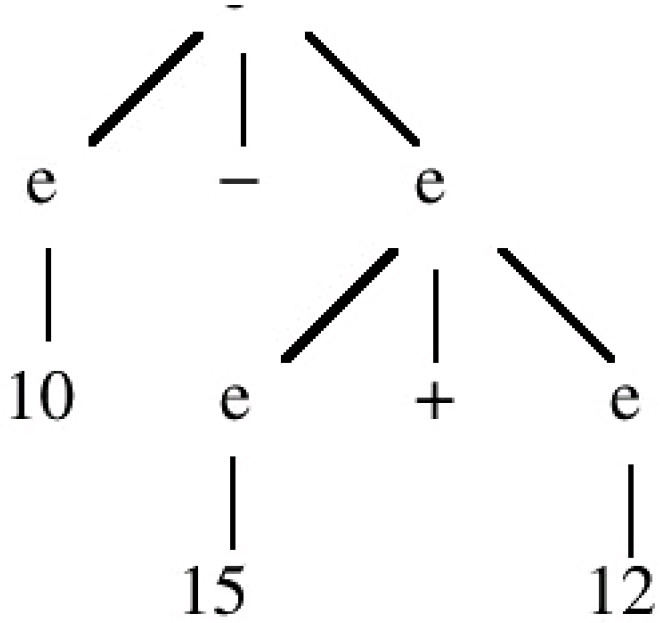
$$e \rightarrow e - e \rightarrow e - e+e \rightarrow \ldots \rightarrow n-n+n \rightarrow \ldots \ldots 10-15+12$$

Parse Trees and Ambiguity

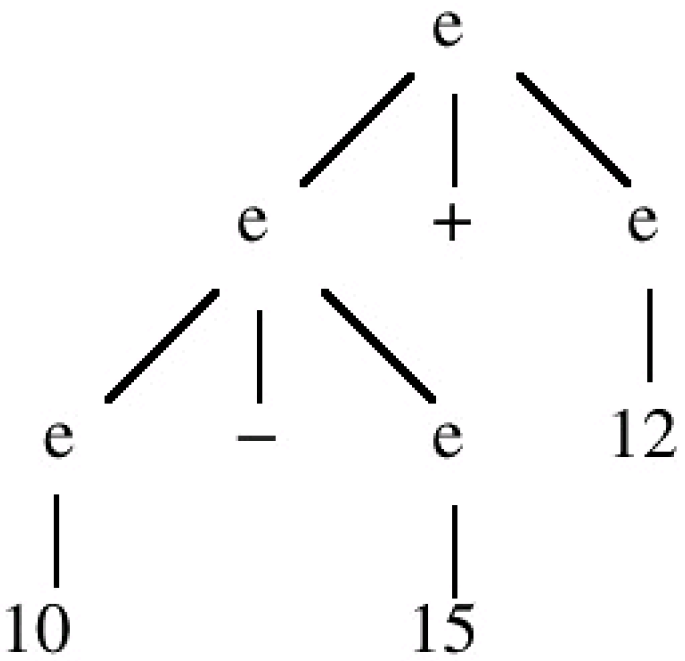
It is often convenient to represent a derivation by a tree. This tree, called the *parse tree* of a derivation, or *derivation tree*, is constructed with the start symbol as the root of the tree. If a step in the derivation is to replace s with $x_1,\ldots, x_n$, then the children of s in the tree will be nodes labeled $x_1,\ldots, x_n$.

The parse tree for the derivation of 10–15+12 in the preceding subsection has some useful structure. Specifically, because the first step yields e-e, the parse tree has the form





where we have contracted the subtrees for each two-digit number to a single node. This tree is different from



which is another parse tree for the same expression. An important fact about parse trees is that each corresponds to a unique parenthesization of the expression. Specifically, the first tree corresponds to 10-(15+12) whereas the second corresponds to (10-15)+12. As this example illustrates, the value of an expression may depend on how it is parsed or parenthesized.

A grammar is *ambiguous* if some expression has more than one parse tree. If every expression has at most one parse tree, the grammar is *unambiguous*.

Example 4.1

There is an interesting ambiguity involving if-then-else. This can be illustrated by the following simple grammar:

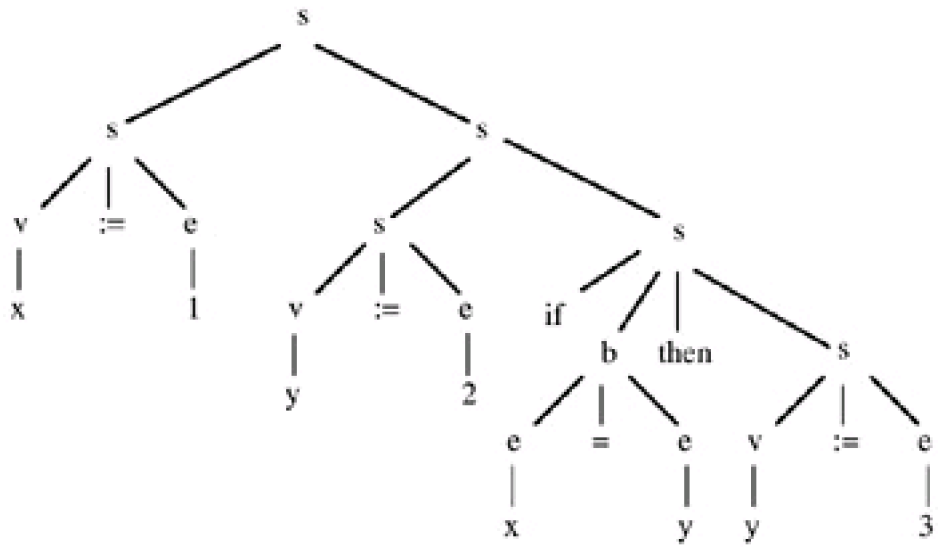
---

s ::= v := e | s;s | if b then s | if b then s else s

```
v ::= x | y | z
e ::= v | 0 | 1 | 2 | 3 | 4
b ::= e=e
```

---

where s is the start symbol, s, v, e, and b are nonterminals, and the other symbols are terminals. The letters s, v, e, and b stand for statement, variable, expression, and Boolean test, respectively. We call the expressions of the language generated by this grammar *statements*. Here is an example of a well-formed statement and one of its parse trees:



```
x:=1;y:=2;if x=y then y:=3
```

This statement also has another parse tree, which we obtain by putting two assignments to the left of the root and the if-then statement to the right. However, the difference between these two parse trees will not affect the behavior of code generated by an ordinary compiler. The reason is that $s_1;s_2$ is normally compiled to the code for s_1 followed by the code for s_2. As a result, the same code would be generated whether we consider $s_1;s_2;s_3$ as $(s_1;s_2);s_3$ or $s_1;(s_2;s_3)$.

A more complicated situation arises when if-then is combined with if-then-else in the following way:

---

if b_1 then if b_2 then s_1 else s_2

What should happen if b_1 is true and b_2 is false? Should s_2 be executed or not? As you can see, this depends on how the statement is parsed. A grammar that allows this combination of conditionals is ambiguous, with two possible

meanings for statements of this form.

4.1.3 Parsing and Precedence

Parsing is the process of constructing parse trees for sequences of symbols. Suppose we define a language *L* by writing out a grammar *G*. Then, given a sequence of symbols *s*, we would like to determine if *s* is in the language *L*. If so, then we would like to compile or interpret *s*, and for this purpose we would like to find a parse tree for *s.* An algorithm that decides whether *s* is in *L*, and constructs a parse tree if it is, is called a *parsing algorithm* for *G*.

There are many methods for building parsing algorithms from grammars. Many of these work for only particular forms of grammars. Because parsing is an important part of compiling programming languages, parsing is usually covered in courses and text books on compilers. For most programming languages you might consider, it is either straight forward to parse the language or there are some changes in syntax that do not change the structure of the language very much but make it possible to parse the language efficiently.

Two issues we consider briefly are the syntactic conventions of precedence and right or left associativity. These are illustrated briefly in the following example.

Example 4.2

A programming language designer might decide that expressions should include addition, subtraction, and multiplication and write the following grammar:

```
e ::= 0 | 1 | e+e | e-e | e*e
```

This grammar is ambiguous, as many expressions have more than one parse tree. For expressions such as 1–1+1, the value of the expression will depend on the way it is parsed. One solution to this problem is to require complete parenthesization. In other words, we could change the grammar to

```
e ::= 0 | 1 | (e+e) | (e-e) | (e*e)
```

so that it is no longer ambiguous. However, as you know, it can be awkward to write a lot of parentheses. In addition, for many expressions, such as 1+2+3+4, the value of the expression does not depend on the way it is parsed. Therefore, it is unnecessarily cumbersome to require parentheses for every operation.

The standard solution to this problem is to adopt parsing conventions that specify a single parse tree for every expression. These are called *precedence* and *associativity*. For this specific grammar, a natural precedence convention is that multiplication(*) has a higher precedence than addition (+) and subtraction (–). We incorporate precedence into parsing by treating an unparenthesized expression $e\ op_1\ e\ op_2\ e$ as if parentheses are inserted around the operator of higher precedence. With this rule in effect, the expression 5*4-3 will be parsed as if it were written as (5*4)-3. This coincides with the way that most of us would ordinarily think about the expression5*4-3. Because there is no standard way that most readers would parse 1+1-1, we might give addition and subtraction equal precedence. In this case, a compiler could issue an error message requiring the programmer to parenthesize 1+1-1. Alternatively, an

expression like this could be disambiguated by use of an additional convention.

Associativity comes into play when two operators of equal precedence appear next to each other. Under left associativity, an expression e op_1 e op_2 e would be parsed as (e op_1 e) op_2 e, if the two operators have equal precedence. If we adopted a right-associativity convention instead, e op_1 e op_2 e would be parsed as e op_1(e op_2 e).

Expression	Precedence	Left Associativity	Right Associativity
5*4-3	(5*4)-3	(no change)	(no change)
1+1-1	(no change)	(1+1)-1	1+(1-1)
2+3-4*5+2	2+3-(4*5)+2	((2+3)-(4*5))+2	2+(3-((4*5)+2))

Team-Fly

Previous Next

Team-Fly

4.2 LAMBDA CALCULUS

Lambda calculus is a mathematical system that illustrates some important programming language concepts in a simple, pure form. Traditional lambda calculus has three main parts: a notation for defining functions, a proof system for proving equations between expressions, and a set of calculation rules called reduction. The first word in the name lambda calculus comes from the use of the Greek letter lambda(λ) in function expressions. (There is no significance to the letter λ.) The second word comes from the way that reduction may be used to *calculate* the result of applying a function to one or more arguments. This calculation is a form of symbolic evaluation of expressions. Lambda calculus provides a convenient notation for describing programming languages and may be regarded as the basis for many language constructs. In particular, lambda calculus provides fundamental forms of parameterization (by means of function expressions) and binding (by means of declarations). These are basic concepts that are common to almost all modern programming languages. It is therefore useful to become familiar enough with lambda calculus to regard expressions in your favorite programming language as essentially a form of lambda expression. For simplicity, the untyped lambda calculus is discussed; there are also typed versions of lambda calculus. In typed lambda calculus, there are additional type-checking constraints that rule out certain forms of expressions. However, the basic concepts and calculation rules remain essentially the same.

4.2.1 Functions and Function Expressions

Intuitively, a function is a rule for determining a value from an argument. This view of functions is used informally in most mathematics. (See Subsection 2.1.2 for a discussion of functions in mathematics.) Some examples of functions studied in mathematics are

$$f(x) = x^2 + 3,$$
$$g(x + y) = \sqrt{x^2 + y^2}.$$

In the simplest, pure form of lambda calculus, there are no domain-specific operators such as addition and exponentiation, only function definition and application. This allows us to write functions such as

h(x) = f (g(x))

because *h* is defined solely in terms of function application and other functions that we assume are already defined. It is possible to add operations such as addition and exponentiation to pure lambda calculus. Although purists stick to pure lambda calculus without addition and multiplication, we will use these operations in examples as this makes the functions we define more familiar.

The main constructs of lambda calculus are *lambda abstraction* and *application*. We use lambda abstraction to write functions: If *M* is some expression, then λ *x.M* is the function we get by treating *M* as a function of the variable *x*. For example,

λ*x .x*

is a lambda abstraction defining the identity function, the function whose value at *x* is *x*. A more familiar way of defining the identity function is by writing

I(x) = x.

However, this way of defining a function forces us to make up a name for every function we want. Lambda calculus lets us write anonymous functions and use them inside larger expressions.

In lambda notation, it is traditional to write function application just by putting a function expression in front of one or more arguments; parentheses are optional. For example, we can apply the identity function to the expression *M* by writing

$$(\lambda x\ .x)M.$$

The value of this application is the identity function, applied to M, that just ends up being M. Thus, we have

$$(\lambda x\ .x)M = M.$$

Part of lambda calculus is a set of rules for deducing equations such as this. Another example of a lambda expression is

$$\lambda f.\lambda g.\lambda x.\ f(gx).$$

Given functions f and g, this function produces the composition $\lambda x.\ f\ (g\ x)$of f and g.

We can extend pure lambda calculus by adding a variety of other constructs. We call an extension of pure lambda calculus with extra operations an *applied lambda calculus*. A basic idea underlying the relation between lambda calculus and computer science is the slogan

Programming language = applied λ-calculus

= pure λ-calculus + additional data types.

This works even for programming languages with side effects, as the way a program depends on the state of the machine can be represented by explicit data structures for the machine state. This is one of the basic ideas behind Scott-Strachey denotational semantics, as discussed in Section 4.3.

4.2.2 Lambda Expressions

Syntax of Expressions

The syntax of untyped lambda expressions may be defined with a BNF grammar. We assume we have some infinite set V of *variables* and use x, y, z ,…, to stand for arbitrary variables. The grammar for lambda expressions is

$$M ::= x \mid MM \mid \lambda x.M$$

where x may be any variable (element of V). An expression of the form $M_1\ M_2$ is called an *application*, and an expression of the form $\lambda\ x.M$ is called a *lambda abstraction.* Intuitively, a variable x refers to some function, the particular function being determined by context; $M_1\ M_2$ is the application of function M_1 to argument M_2; and $\lambda\ x.M$ is the function that, given argument x, returns the value M. In the lambda calculus literature, it is common to refer to the expressions of lambda calculus as *lambda terms*.

Here are some example lambda terms:

$\lambda\ x.x$	a lambda abstraction called the*identity function*
$\lambda\ x.(\ f\ (gx))$	another lambda abstraction
$(\lambda\ x.x)5$	an application

There are a number of syntactic conventions that are generally convenient for lambda calculus experts, but they are confusing to learn. We can generally avoid these by writing enough parentheses. One convention that we will use, however, is that in an expression containing a λ, the scope of λ extends as far to the right as possible. For example, $\lambda\ x\ .\ xy$ should be read as $\lambda\ x.(xy)$, not $(\lambda\ x.x)y$.

Variable Binding

An occurrence of a variable in an expression may be either free or bound. If a variable is *free* in some expression, this means that the variable is not declared in the expression. For example, the variable x is free in the expression $x + 3$. We can not evaluate the expression $x + 3$ as it stands, without putting it inside some larger expression that will associate some value with x. If a variable is not free, then that must be because it is *bound*.

The symbol λ is called a *binding operator*, as it binds a variable within a specific scope (part of an expression). The variable x is bound in $\lambda\ x.M$. This means that x is just a place holder, like x in the integral $\int f\ (x)\ \mathrm{d}x$ or the logical formula $\forall x.P(x)$, and the meaning of $\lambda\ x.M$ does not depend on x. Therefore, just as $\int f\ (x)\mathrm{d}\ x$ and $\int f\ (y)\mathrm{d}y$ describe the same

integral, we can rename a λ-bound *x* to *y* without changing the meaning of the expression. In particular,

λ*x.x* defines the same function as λ *y . y*.

Expressions that differ in only the names of bound variables are called ? equivalent. When we want to emphasize that two expressions are ? equivalent, we write λ *x.x* = ? λ *y.y*, for example.

In λ *x.M*, the expression *M* is called the *scope* of the binding λ *x*. A variable *x* appearing in an expression *M* is bound if it appears in the scope of some λ *x* and is free otherwise. To be more precise about this, we may define the set FV(*M*)of *free variables of M* by induction on the structure of expressions, as follows:

FV(*x*) ={*x*},

FV(*MN*) = FV(*M*) ? FV(*N*),

FV(λ *x.M*) = FV(*M*) - *x*,

where - means set difference, that is, *S* - *x* ={*y* ? *S* | *y* ≠ *x*}. For example, FV(λ *x.x*) = ? because there are no free variables in λ *x.x*, whereas FV**(** λ *f* .λ *x*.(*f* (*g*(*x*))**)** ={*g*}.

It is sometimes necessary to talk about different occurrences of a variable in an expression and to distinguish free and bound occurrences. In the expression

λx.(λ*y.xy*)*y*,

the first occurrence of *x* is called a *binding occurrence*, as this is where *x* becomes bound. The other occurrence of *x* is a bound occurrence. Reading from left to right, we see that the first occurrence of *y* is a binding occurrence, the second is a bound occurrence, and the third is free as it is outside the scope of the λ *y* in parentheses.

It can be confusing to work with expressions that use the same variable in more than one way. A useful convention is to rename bound variables so that all bound variables are different from each other and different from all of the free variables. Following this convention, we will write (λ *y*.(λ *z.z*)*y*)*x* instead of (λ *x*.(λ *x.x*)*x*)*x*. The variable convention will be particularly useful when we come to equational reasoning and reduction.

Lambda Abstraction in Lisp and Algol

Lambda calculus has an obvious syntactic similarity to Lisp: the Lisp lambda expression

---

(lambda (x) *function_body*)

---

looks like the lambda calculus expression

λx.*function_body*,

and both expressions define functions. However, there are some differences between Lisp and lambda calculus. For example, lists are the basic data type of Lisp, whereas functions are the only data type in pure lambda calculus. Another difference is evaluation order, but that topic will not be discussed in detail.

Although the syntax of block-structured languages is farther from lambda calculus than from Lisp, the basic concepts of declarations and function parameterizations in Algol like languages are fundamentally the same as in lambda calculus. For example, the C program fragment

---

```
int f (int x) {return x };
block_body;
```

---

with a function declaration at the beginning of a block is easily translated into lambda calculus. The translation is easier to understand if we first add declarations to lambda calculus.

The simple let declaration,

let $x = M$ in N,

which declares that x has value M in the body N, may be regarded as syntactic sugar for a combination of lambda abstraction and application:

let $x = M$ in N *is sugar for* $(\lambda x.N)M$.

For those not familiar with the concept of *syntactic sugar*, this means that let $x = M$ in N is "sweeter" to write in some cases, but we can just think of the syntax let $x = M$ in N as standing for $(\lambda x.N)M$.

Using let declarations, we can write the C program from above as

---

let $f = (\lambda x.x)$ in *block body*

Note that the C form expression and the lambda expression have the same free and bound variables and a similar structure. One difference between C and lambda calculus is that C has assignment statements and side effects, whereas lambda calculus is purely functional. However, by introducing *stores*, mappings from variable locations to values, we can translate C programs with side effects into lambda terms as well. The translation preserves the overall structure of programs, but makes programs look a little more complicated as the dependencies on the state of the machine are explicit.

Equivalence and Substitution

We have already discussed α equivalence of terms. This is one of the basic axioms of lambda calculus, meaning that it is one of the properties of lambda terms that defines the system and that is used in proving other properties of lambda terms. Stated more carefully than before, the equational proof system of lambda calculus has the equational axiom

$$\lambda x.M = \lambda y.[y/x]M, \qquad (\alpha)$$

where $[y/x]M$ is the result of substituting y for free occurrences of x in M and y cannot already appear in M. There are three other equational axioms and four inference rules for proving equations between terms. However, we look at only one other equational axiom.

The central equational axiom of lambda calculus is used to calculate the value of a function application $(\lambda x.M)N$ by substitution. Because $\lambda x.M$ is the function we get by treating M as a function of x, we can write the value of this function at N by substituting N for x. Again writing $[N/x]M$ for the result of substituting N for free occurrences of x in M, we have

$$(\lambda x.M)N = [N/x]M, \qquad (\beta)$$

which is called the axiom of β-equivalence. Some important warnings about substitution are discussed in the next subsection. The value of $\lambda f.\, f\, x$ applied to $\lambda y.y$ may be calculated if the argument $\lambda y.y$ is substituted in for the bound

variable f:

$$(\lambda f.f\,x)\,(\lambda y.y) = (\lambda y,y)x$$

Of course, $(\lambda y.y)x$ may be simplified by an additional substitution, and so we have

$$(\lambda f.f\,x)(\lambda y.y) = (\lambda y.y)x = x$$

Any readers old enough to be familiar with the original documentation of Algol 60 will recognize β-equivalence as the *copy rule* for evaluating function calls. There are also parallels between (β) and macroexpansion and in-line substitution of functions.

Renaming Bound Variables

Because λ bindings in M can conflict with free variables in N, substitution $[N/x]M$ is a little more complicated than we might think at first. However, we can avoid all of the complications by following the variable convention: renaming bound variables in $(\lambda x.M)N$ so that all of the bound variables are different from each other and different from all of the free variables. For example, let us reduce the term$(\lambda f.\lambda x.\ f\,(fx))(\lambda y.y+x)$. If we first rename bound variables and then perform β-reduction, we get

$$(\lambda f.\lambda z.f(f\,z))\ (\lambda y.y+x) = \lambda z.((\lambda y.y+x)((\lambda y.y+x)z)) = \lambda z.z+x+x$$

If we were to forget to rename bound variables and substitute blindly, we might simplify as follows:

$$(\lambda f.\ \lambda x.\ f(f\,x))(\lambda y.\ y+x) = \lambda x.((\lambda y.\ y+x)((\lambda y.\ y+x)x)) = \lambda x.x+x+x$$

However, we should be suspicious of the second reduction because the variable x is free in $(\lambda y.y+x)$ but becomes bound in $\lambda x.x+x+x$. *Remember:* In working out$[N/x]M$, we must rename any bound variables in M that might be the same as free variables in N. To be precise about renaming bound variables in substitution, we define the result $[N/x]M$ of substituting N for x in M by induction on the structure of M:

- $[N/x]x = N$,
- $[N/x]y = y$, where y is any variable different from x,
- $[N/x](M_1\,M_2) = ([N/x]M^1)([N/x]M_2)$,
- $[N/x](\lambda x.M) = \lambda x.M$,
- $[N/x](\lambda y.M) = \lambda y.([N/x]M)$, where y is not free in N.

Because we are free to rename the bound variable y in $\lambda y.M$, the final clause $\lambda y.([N/x]M)$ always makes sense. With this precise definition of substitution, we now have a precise definition of β-equivalence.

4.2.3 Programming in Lambda Calculus

Lambda calculus may be viewed as a simple functional programming language. We will see that, even though the language is very simple, we can program fairly naturally if we adopt a few abbreviations. Before declarations and recursion are discussed, it is worth mentioning the problem of multiargument functions.

Functions of Several Arguments

Lambda abstraction lets us treat any expression M as a function of any variable x by writing $\lambda x.M$. However, what if we want to treat M as function of two variables, x and y? Do we need a second kind of lambda abstraction $\lambda_2 x, y.M$ to treat M as function of the pair of arguments x, y? Although we could add lambda operators for sequences of formal parameters, we do not need to because ordinary lambda abstraction will suffice for most purposes.

We may represent a function f of two arguments by a function $\lambda x.(\lambda y.M)$of a single argument that, when applied, returns a second function that accepts a second argument and then computes a result in the same way as f. For example, the function

$$f(g,\,x) = g(x)$$

has two arguments, but can be represented in lambda calculus by ordinary lambda abstraction. We define f_{curry} by

$$f_{curry} = \lambda g.\lambda x.gx$$

The difference between f and f_{curry} is that f takes a pair (g, x) as an argument, whereas f_{curry} takes a single argument g. However, f_{curry} can be used in place of f because

$$f_{curry} g\ x = gx = f(g, x)$$

Thus, in the end, f_{curry} does the same thing as f. This simple idea was discovered by Schönfinkel, who investigated functionality in the 1920s. However, this technique for representing multiargument functions in lambda calculus is usually called *Currying*, after the lambda calculus pioneer Haskell Curry.

Declarations

We saw in Subsection 4.2.2 that we can regard simple let declarations,

$$\text{let } x = M \text{ in } N$$

as lambda terms by adopting the abbreviation

$$\text{let } x = M \text{ in } N = (\lambda x.N)M$$

The let construct may be used to define a composition function, as in the expression

$$\text{let } compose = \lambda f.\lambda g.\lambda x.f(gx) \text{ in } compose\ h\ h$$

Using β-equivalence, we can simplify this let expression to $\lambda x.\ h(hx)$, the composition of h with itself. In programming language parlance, the let construct provides local declarations.

Recursion and Fixed Points

An amazing fact about pure lambda calculus is that it is possible to write recursive functions by use of a self-application "trick." This does not have a lot to do with comparisons between modern programming languages, but it may interest readers with a technical bent. (Some readers and some instructors may wish to skip this subsection.)

Many programming languages allow recursive function definitions. The characteristic property of a recursive definition of a function f is that the body of the function contains one or more calls to f. To choose a specific example, let us suppose we define the factorial function in some programming language by writing a declaration like

```
function f(n) {if n=0 then 1 else n*f(n-1)};
```

where the body of the function is the expression inside the braces. This definition has a straightforward computational interpretation: when f is called with argument a, the function parameter n is set to a and the function body is evaluated. If evaluation of the body reaches the recursive call, then this process is repeated. As definitions of a computational procedure, recursive definitions are clearly meaningful and useful.

One way to understand the lambda calculus approach to recursion is to associate an equation with a recursive definition. For example, we can associate the equation

```
f(n) = if n=0 then 1 else n*f(n-1)
```

with the preceding recursive declaration. This equation states a property of factorial.

Specifically, the value of the expression f(n) is equal to the value of the expression if n=0 then 1 else n*f(n–1) when f is the factorial function. The lambda calculus approach may be viewed as a method of finding solutions to equations in which an identifier (the name of the recursive function) appears on both sides of the equation.

We can simplify the preceding equation by using lambda abstraction to eliminate n from the left-hand side. This gives us

---

f= λ n. if n=0 then 1 else n*f(n-1)

Now consider the function G that we obtain by moving f to the right-hand-side of the equation:

G= λf. λn. if n=0 then 1 else n*f(n-1)

Although it might not be clear what sort of "algebra" is involved in this manipulation of equations, we can check, by using lambda calculus reasoning and basic understanding of function equality, that the factorial function f satisfies the equation

---

f = G(f)

This shows that recursive declarations involve finding fixed points.

A *fixed point* of a function *G* is a value *f* such that *f* = *G*(*f*). In lambda calculus, fixed points may be defined with the *fixed-point operator*:

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(x\ x)).$$

The surprising fact about this perplexing lambda expression is that, for any *f*, the application *Yf* is a fixed point of *f*. We can see this by calculation. By β-equivalence, we have

$$Y\ f = (\lambda\ x\ .\ f\ (x\ x))(\lambda\ x\ .\ f\ (x\ x)).$$

Using β-equivalence again on the right-hand term, we get

$$Y\ f = (\lambda\ x\ .\ f\ (x\ x))(\lambda\ x\ .\ f\ (x\ x)) = f\ (\lambda\ x\ .\ f\ (x\ x))(\lambda\ x\ .\ f\ (x\ x)) = f\ (Y\ f).$$

Thus *Y f* is a fixed point of *f* .

Example 4.3

We can define factorial by *fact* = *YG*, where lambda terms *Y* and *G* are as given above, and calculate *fact* 2 = 2! by using the calculation rules of lambda calculus. Here are the calculation steps representing the first "call" to factorial:

$$
\begin{aligned}
&fact\,2 = (YG)\,2\\
&= G\,(YG)\,2\\
&= (\lambda f.\lambda n.\,\text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))(YG)\,2\\
&= (\lambda n.\,\text{if } n = 0 \text{ then } 1 \text{ else } n*((YG)(n-1))\,2\\
&= \text{if } 2 = 0 \text{ then } 1 \text{ else } 2*((YG)(2-1))\\
&= \text{if } 2 = 0 \text{ then } 1 \text{ else } 2*((YG)\,1)\\
&= 2*((YG)\,1).
\end{aligned}
$$

Using similar steps, we can calculate (*YG*)1 = 1! = 1 to complete the calculation.

It is worth mentioning that *Y* is not the only lambda term that finds fixed points of functions. There are other expressions that would work as well. However, this particular fixed-point operator played an important role in the history of lambda calculus.

4.2.4 Reduction, Confluence, and Normal Forms

The computational properties of lambda calculus are usually described with a form of symbolic evaluation called reduction. In simple terms, reduction is equational reasoning, but in a certain direction. Specifically, although β-equivalence was written as an equation, we have generally used it in one direction to "evaluate" function calls. If we write $\rightarrow$ instead of = to indicate the direction in which we intend to use the equation, then we obtain the basic computation step called β-reduction:

$$(\lambda x.M)N \rightarrow [N/x]M. \qquad (\beta)$$

We say that *M* β-reduces to *N*, and write $M \rightarrow N$, if *N* is the result of applying one β-reduction step somewhere inside *M*. Most of the examples of calculation in this section use β-reduction, i.e., β-equivalence is used from left to right rather than from right to left. For example,

$$(\lambda f.\ \lambda z.f\,(f\,z))(\lambda y\,.y + x) \rightarrow \lambda z.\ ((\lambda y\,.y + x)(\lambda y.y + x)z)) \rightarrow \lambda z.z + x + x.$$

Normal Forms

Intuitively, we think of $M \rightarrow N$ as meaning that, in one computation step, the expression *M* can be evaluated to the expression *N*. Generally, this process can be repeated, as illustrated in the preceding subsection. However, for many expressions, the process eventually reaches a stopping point. A stopping point, or expression that cannot be further evaluated, is called a *normal form*. Here is an example of a reduction sequence that leads to a normal form:

$$
\begin{aligned}
&(\lambda f.\lambda x.\,f(\,fx))(\lambda y.y + 1)\,2\\
&\rightarrow (\lambda x.(\lambda y.y + 1)((\lambda y.y + 1)x))\,2\\
&\rightarrow (\lambda x.(\lambda y.y + 1)(x + 1))\,2\\
&\rightarrow (\lambda x.(x + 1 + 1))\,2\\
&\rightarrow (2 + 1 + 1).
\end{aligned}
$$

This last expression is a normal form if our only computation rule is β-reduction, but not a normal form if we have a computation rule for addition. Assuming the usual evaluation rule for expressions with addition, we can continue with

$2 + 1 + 1$

$\rightarrow 3 + 1$

$\rightarrow 4.$

This example should give a good idea of how reduction in lambda calculus corresponds to computation. Since the 1930s, lambda calculus has been a simple mathematical model of expression evaluation.

Confluence

In our example starting with $(\lambda f.\lambda x.\ f\,(fx))\ (\lambda y.y + 1)\ 2$, there were some steps in which we had to chose from several possible subexpressions to evaluate. For example, in the second expression,

$$(\lambda x.(\lambda y.y + 1)((\lambda y.y + 1)x))2,$$

we could have evaluated either of the two function calls beginning with λy. This is not an artifact of lambda calculus itself, as we also have two choices in evaluating the purely arithmetic expression

$$2 + 1 + 1.$$

An important property of lambda calculus is called *confluence*. In lambda calculus, as a result of confluence, evaluation order does not affect the final value of an expression. Put another way, if an expression *M* can be reduced to a normal form, then there is exactly one normal form of *M*, independent of the order in which we choose to evaluate subexpressions. Although the full mathematical statement of confluence is a bit more complicated than this, the important thing to remember is that, in lambda calculus, expressions can be evaluated in any order.

4.2.5 Important Properties of Lambda Calculus

In summary, lambda calculus is a mathematical system with some syntactic and computational properties of a programming language. There is a general notation for functions that includes a way of treating an expression as a function of some variable that it contains. There is an equational proof system that leads to calculation rules, and these calculation rules are a simple form of symbolic evaluation. In programming language terminology, these calculation rules are a form of macro expansion(with renaming of bound variables!) or function in lining. Because of the relation to in lining, some common compiler optimizations may be defined and proved correct by use of lambda calculus.

Lambda calculus has the following important properties:

- Every computable function can be represented in pure lambda calculus. In the terminology of Chapter 2, lambda calculus is Turing complete. (Numbers can be represented by functions and recursion can be expressed by *Y*.)
- Evaluation in lambda calculus is order independent. Because of confluence, we can evaluate an expression by choosing any subexpression. Evaluation in pure functional programming languages (see Section 4.4) is also confluent, but evaluation in languages whose expressions may have side effects is not confluent.

Macro expansion is another setting in which a form of evaluation is confluent. If we start with a program containing macros and expand all macro calls with the macro bodies, then the final fully expanded program we obtain will not depend on the order in which macros are expanded.

4.3 DENOTATIONAL SEMANTICS

In computer science, the phrase denotational semantics refers to a specific style of mathematical semantics for imperative programs. This approach was developed in the late 1960s and early 1970s, following the pioneering work of Christopher Strachey and Dana Scott at Oxford University. The term denotational semantics suggests that a meaning or *denotation* is associated with each program or program phrase (expression, statement, declaration, etc.). The denotation of a program is a mathematical object, typically a function, as opposed to an algorithm or a sequence of instructions to execute.

In denotational semantics, the meaning of a simple program like

```
x := 0; y:=0; while x ≤ z do y:=y+x;x:=x+1
```

is a mathematical function from *states* to *states*, in which a state is a mathematical function representing the values in memory at some point in the execution of a program. Specifically, the meaning of this program will be a function that maps any state in which the value of *z* is some nonnegative integer *n* to the state in which *x*=*n*, *y* is the sum of all numbers up to *n*, and all other locations in memory are left unchanged. The function would not be defined on machine states in which the value of *z* is not a nonnegative integer.

Associating mathematical functions with programs is good for some purposes and not so good for others. In many situations, we consider a program correct if we get the correct output for any possible input. This form of correctness depends on only the denotational semantics of a program, the mathematical function from input to output associated with the program. For example, the preceding program was designed to compute the sum of all the non negative integers up to *n*. If we verify that the actual denotational semantics of this program is this mathematical function, then we have proved that the program is correct. Some disadvantages of denotational semantics are that standard denotational semantics do not tell us anything about the running time or storage requirements of a program. This is sometimes an advantage in disguise because, by ignoring these issues, we can sometimes reason more effectively about the correctness of programs.

Forms of denotational semantics are commonly used for reasoning about program optimization and static analysis methods. If we are interested in analyzing running time, then operational semantics might be more useful, or we could use more detailed denotational semantics that also involves functions representing time bounds. An alternative to denotational semantics is called operational semantics, which involves modeling machine states and (generally) the step-by-step state transitions associated with a program. Lambda calculus reduction is an example of operational semantics.

Compositionality

An important principle of denotational semantics is that the meaning of a program is determined from its text *compositionally*. This means that the meaning of a program must be defined from the meanings of its parts, not something else, such as the text of its parts or the meanings of related programs obtained by syntactic operations. For example, the denotation of a program such as *if B then P else Q* must be explained with only the denotations of *B*, *P*, and *Q*; it should not be defined with programs constructed from *B*, *P*, and *Q* by syntactic operations such as substitution.

The importance of compositionality, which may seem rather subtle at first, is that if two program pieces have the same denotation, then either may safely be substituted for the other in any program. More specifically, if *B*, *P*, and *Q* have the same denotations as *B* ', *P* ', and *Q* ', respectively, then *if B then P else Q* must have the same denotation as *if B' then P' else Q'*. Compositionality means that the denotation of an expression or program statement must be detailed enough to capture everything that is relevant to its behavior in larger programs. This makes denotational semantics useful for understanding and reasoning about such pragmatic issues as program transformation and optimization, as these operations on programs involve replacing parts of programs without changing the overall meaning of the whole program.

4.3.1 Object Language and Metalanguage

One source of confusion in talking (or writing) about the interpretation of syntactic expressions is that everything we write is actually syntactic. When we study a programming language, we need to distinguish the programming language we study from the language we use to describe this language and its meaning. The language we study is traditionally called the object language, as this is the object of our attention, whereas the second language is called the metalanguage, because it transcends the object language in some way.

To pick an example, let us consider the mathematical interpretation of a simple algebraic expression such as 3 + 6 – 4 that might appear in a program written in C, Java, or ML. The ordinary "mathematical" meaning of this expression is the number obtained by doing the addition and subtraction, namely 5. Here, the symbols in the expression 3 + 6 – 4 are in our object language, whereas the number 5 is meant to be in our metalanguage. One way of making this clearer is to use an out lined number, such as 1, to mean "the mathematical entity called the natural number 1." Then we can say that the meaning of the object language expression 3 + 6 – 4 is the natural number 5. In this sentence, the symbol 5 is a symbol of the metalanguage, whereas the expression 3 + 6 – 4 is written with symbols of the object language.

4.3.2 Denotational Semantics of Binary Numbers

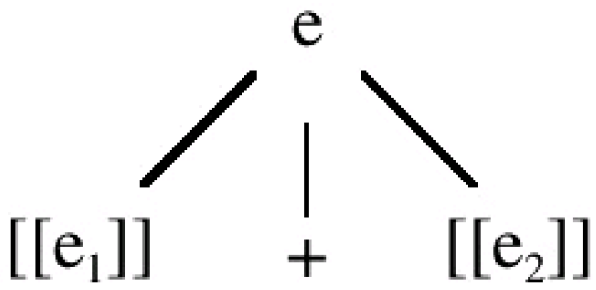
The following grammar for binary expressions is similar to the grammar for decimal expressions discussed in Subsection 4.1.2:

---

e ::= n | e+e | e-e
n ::= b | nb
b ::= 0 | 1

We can give a mathematical interpretation of these expressions in the style of denotational semantics. In denotational semantics and other studies of programming languages, it is common to forget about how expressions are converted into parse trees and just give the meaning of an expression as a function of its parse tree.

We may interpret the expressions previously defined as natural numbers by using induction on the structure of parse trees. More specifically, we define a function from parse trees to natural numbers, defining the function on a compound expression by referring to its value on simpler expressions. A historical convention is to write [[e]]for any parse tree of the expression *e*. When we write $[[e_1+e_2]]$, for example, we mean a parse tree of the form



with $[[e_1]]$ and $[[e_2]]$ as immediate subtrees.

Using this notation, we may define the meaning E[[e]] of an expression e, according to its parse tree [[e]], as follows:

E[[0]] = 0

E[[1]] = 1

E[[nb]] = E[[n]] * 2 + E[[b]]

$E[[e_1+e_2]] = E[[e_1]] + E[[e_2]]$

$E[[e_1-e_2]] = E[[e_1]]-E[[e_2]]$

In words, the value associated with a parse tree of the form $[[e_1+e_2]]$, for example, is the sum of the values given to the subtrees $[[e_1]]$ and $[[e_2]]$. This is not a circular definition because the parse trees $[[e_1]]$ and $[[e_2]]$ are smaller than the

parse tree[[e_1+e_2]].

On the right-hand side of the equal signs, numbers and arithmetic operations *,+, and – are meant to indicate the actual natural numbers and the standard integer operations of multiplication, addition, and subtraction. In contrast, the symbols + and – in expressions surrounded by double square brackets on the left-hand side of the equal signs are symbols of the object language, the language of binary expressions.

4.3.3 Denotational Semantics of While Programs

Without going into detail about the kinds of mathematical functions that are used, let us take a quick look at the form of semantics used for a simplified programming language with assignment and loops.

Expressions with Variables

Program statements contain expressions with variables. Here is a grammar for arithmetic expressions with variables. This is the same as the grammar in Subsection 4.1.2,except that expressions can contain variables in addition to numbers:

---

e ::= v | n | e+e | e-e
n ::= d | nd
d ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
v ::= x | y | z | …

In the simplified programming language we consider, the value of a variable depends on the state of the machine. We model the state of the machine by a function from variables to numbers and write E[[e]](s) for the value of expression e in state s. The value of an expression in a state is defined as follows.

E[[x]](s) = s(x)

E[[0]](s) = 0

E[[1]](s) = 1

… = ?

E[[9]](s) = 9

E[[nd]](s) = E[[n]](s) * 10 + E[[d]](s)

$E[[e_1+e_2]](s) = E[[e_1]](s) + E[[e^2]](s)$

$E[[e_1– e^2]](s) = E[[e^1]](s) – E[[e^2]](s)$

Note that the state matters in the definition in the base case, the value of a variable. Otherwise, this is essentially the same definition as that in the semantics of variable-free expressions in the preceding subsection.

The syntax and semantics of Boolean expressions can be defined similarly.

While Programs

The language of *while* programs may be defined over any class of value expressions and Boolean expressions. Without specifying any particular basic expressions, we may summarize the structure of while programs by the grammar

P ::=x:=e | P; P | if e then P else P | while e do P

where we assume that *x* has the appropriate type to be assigned the value of e in the assignment x :=e and that the test *e* has type *bool* in if-then-else and while statements. Because this language does not have explicit input or output, the effect of a program will be to change the values of variables. Here is a simple example:

x := 0; y:=0; while x ≤ z do (y := y+x; x := x+1)

We may think of this program as having input *z* and output *y*. This program uses an additional variable *x* to set *y* to the sum of all natural numbers up to *z*.

States and Commands

The meaning of a program is a function from states to states. In a more realistic programming language, with procedures or pointers, it is necessary to model the fact that two variable names may refer to the same location. However, in the simple language of while programs, we assume that each variable is given a different location. With this simplification in mind, we let the set *State* of mathematical representations of machine states be

$$State = Variables \rightarrow Values$$

In words, a state is a function from variables to values. This is an idealized view of machine states in two ways: We do not explicitly model the locations associated with variables, and we use infinite states that give a value to every possible variable.

The meaning of a program is an element of the mathematical set *Command* of commands, defined by

$$Command = State \rightarrow State$$

In words, a command is a function from states to states. Unlike states themselves, which are total functions, a command may be a *partial* function. The reason we need partial functions is that a program might not terminate on an initial state.

A basic function on states that is used in the semantics of assignment is *modify*,which is defined as follows:

$$modify(s,x,a) = \lambda\, v \,?\, Variables.\ \text{if } v=x \text{ then } a \text{ else } s(v)$$

In words, *modify*(*s,x,a*) is the state (function from variables to values) that is just like state *s*, except that the value of *x* is *a*.

Denotational Semantics

The denotational semantics of while programs is given by the definition of a function C from parsed programs to commands. As with expressions, we write [[P]] for a parse tree of the program P. The semantics of programs are defined by the following clauses, one for each syntactic form of program:

C[[x :=e]](s) =modify(s,x, E[[e]](s))

C[[P_1;P_2]](s) =C[[P_2]](C[[P_1]](s))

C[[if e then P_1 else P_2]](s) =if E[[e]](s) then C[[P_1]](s) else C[[P_2]](s)

C[[while e do P]](s) =if not E[[e]](s) then s

else C[[while e do P]](C[[P]](s))

Because e is an expression, not a statement, we apply the semantic function E to obtain the value of e in a given state.

In words, we can describe the semantics of programs as follows:

C[[x := e]](*s*) is the state similar to *s*, but with *x* having the value of *e*.

C[[P_1;P_2]](s) is the state we obtain by applying the semantics of P_2 the to the state we obtain by applying the semantics of P_1 to state *s*.

C[[if e then P_1 else P_2]](s) is the state we obtain by applying the semantics of P_1 to *s* if *e* is true in *s* and P_2 to s otherwise.

C[[while e do P]](s) is a recursively defined function *f*, from states to states. In words, *f(s)* is either *s* if *e* is false or the state we obtain by applying *f* to the state resulting from executing P once in *s*.

The recursive function definition in the while clause is relatively subtle. It also raises some interesting mathematical problems, as it is not always mathematically reasonable to define functions by arbitrary recursive conditions. However, in the interest of keeping our discussion simple and straight forward, we just assume that a definition of this form can be made mathematically rigorous.

Example 4.4

We can calculate the semantics of various programs by using this definition. To begin with, let us consider a simple loop-free program,

---

if x>y then x :=y else y :=x

---

which sets both x and y to the minimum of their initial values. For concreteness, let us calculate the semantics of this program in the state s_0, where $s_0(x) = 1$ and $s_0(y)=2$.

Because E[[x>y]](s_0) = false, we have

C[[if x>y then x :=y else y := x]](s_0)

=if E[[x>y]](s_0) then C[[x :=y]](s_0) else C[[y :=x]](s_0)

=C[[y :=x]](s_0)

=modify(s_0, y, E[[x]](s_0))

In words, if the program if x>y then x :=y else y := x is executed in the state s_0, then the result will be the state that is the same as s_0, but with variable y given the value that x has in state s_0.

Example 4.5

Although it takes a few more steps than the previous example, it is not too difficult to work out the semantics of the program

x := 0; y:=0; while x ≤ z do (y := y+x; x := x+1)

in the state s_0, where $s_0(z)=2$. A few preliminary definitions will make the calculation easier. Let s_1 and s_2 be the states

$$s_1 = C[[\ x:= 0\ ]](s_0)$$

$$s_2 = C[[\ y :=0]](s_1)$$

Using the semantics of assignment, as above, we have

$$s_1 = \text{modify}(s_0, x, 0)$$

$$s_2 = \text{modify}(s_1, y, 0)$$

Returning to the preceding program, we have

$$C[[x := 0; y:=0; \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](s_0)$$

$$= C[[\ y:=0; \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](C\ [[x : =0\ ]](s_0))$$

$$= C[[\ y:=0; \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](s_1\ )$$

$$= C[[\ \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](C\ [[y :=0]](s_1))$$

$$= C[[\ \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](s_2)$$

$$= \text{if not } E[[\ x \le z\ ]](s_2) \text{ then } s_2$$
$$\text{else } C[[\ \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](C[[\ y :=y+x;x :=x+1]](s_2))$$

$$= C[[\ \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](s_3)$$

where s_3 has y set to 0 and x set to 1. Continuing in the same manner, we have

$$C[[\ \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](s_3)$$

$$= \text{if not } E[[x \le z\ ]](s_3) \text{ then } s_3$$
$$\text{else } C[[\ \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](C[[y :=y+x;x :=x+1]](s_3))$$

$$= C[[\ \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](s_4)$$

$$= \text{if not } E[[x \le z\ ]](s_4) \text{ then } s_4$$
$$\text{else } C[[\ \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](C[[y :=y+x;x :=x+1]](s_4))$$

$$= C[[\ \text{while } x \le z \text{ do } (y := y+x; x := x+1)\ ]](s_5)$$

$$= s_5$$

where s_4 has y set to 1 and x to 2 and s_5 has y set to 3 and x to 3. The steps are tedious to write out, but you can probably see without doing so that if $s_0(z) = 5$, for example, then this program will yield a state in which the value of x is $0+1+2+3+4+5$.

As these examples illustrate, the denotational semantics of while programs unambiguously associates a partial function from states to states with each program. One important issue we have not discussed in detail is what happens when a loop does not terminate. The meaning C[[while x=x do x := x]] of a loop that does not terminate in any state is a partial function that is not defined on any state. In other words, for any state s, C[[while x=x do x := x]](s) is not defined.

Similarly, C[[while x=y do x := y]](s) is s if $s(x) \neq s(y)$ and undefined otherwise. If you are interested in more information, there are many books that cover denotational semantics in detail.

4.3.4 Perspective and Nonstandard Semantics

There are several ways of viewing the standard methods of denotational semantics. Typically, denotational semantics is given by the association of a function with each program. As many researchers in denotational semantics have observed, a mapping from programs to functions must be written in some metalanguage. Because lambda calculus is a useful notation for functions, it is common to use some form of lambda calculus as a metalanguage. Thus, most denotational semantics actually have two parts: a translation of programs into a lambda calculus (with some extra operations corresponding to basic operations in programs) and a semantic interpretation of the lambda calculus expressions as mathematical objects. For this reason, denotational semantics actually provides a general technique for translating imperative programs into functional programs.

Although the original goal of denotational semantics was to define the meanings of programs in a mathematical way, the techniques of denotational semantics can also be used to define useful "nonstandard" semantics of programs.

One useful kind of nonstandard semantics is called *abstract interpretation*. In abstract interpretation, programs are assigned meaning in some simplified domain. For example, instead of interpreting integer expressions as integers, integer expressions could be interpreted elements of the finite set {0, 1, 2, 3, 4, 5, …, 100, >100}, where>100 is a value used for expressions whose value might be greater than 100. This might be useful if we want to see if two array expressions $A[e_1]$ and $A[e_2]$ refer to the same array location. More specifically, if we assign values to e_1 from the preceding set, and similarly for e_2, we might be able to determine that $e_1=e_2$. If both are assigned the value >100, then we would not know that they are the same, but if they are assigned the same ordinary integer between 0 and 100, then we would know that these expressions have the same value. The importance of using a finite set of values is that an algorithm could iterate over all possible states. This is important for calculating properties of programs that hold in all states and also for calculating the semantics of loops.

Example. Suppose we want to build a program analysis tool that checks programs to make sure that every variable is initialized before it is used. The basis for this kind of program analysis can be described by denotational semantics, in which the meaning of an expression is an element of a finite set.

Because the halting problem is unsolvable, program analysis algorithms are usually designed to be *conservative.* Conservative means that there are no false positivies: An algorithm will output *correct* only if the program is correct, but may sometimes output *error* even if there is no error in the program. We cannot expect a computable analysis to decide correctly whether a program will ever access a variable before it is initialized. For example, we cannot decide whether

---

(complictated error-free program); x := y

---

executes the assignment x:=y without deciding whether complictated error-free program halts. However, it is often possible to develop efficient algorithms for conservative analysis. If you think about it, you will realize that most compiler warnings are conservative: Some warnings could be a problem in general, but are not a problem in a specific program because of program properties that the compiler is not "smart" enough to understand.

We describe initialize-before-use analysis by using an abstract representation of machine states that keep track only of whether a variable has been initialized or not. More precisely, a state will either be a special error state called *error* or a function from variable names to the set { *init*, *uninit* } with two values, one representing any value of initialized variable and the other an uninitialized one. The set *State* of mathematical abstractions of machine states is therefore

$$State = \{error\} \cup (Variables \rightarrow \{init, uninit\})$$

As usual, the meaning of a program will be a function from states to states. Let us assume that E[[e]](s) =*error* if e contains any variable y with s(y) =*uninit* and E[[e]](s) =*Ok* otherwise.

The semantics of programs is given by a set of clauses, one for each program form, as usual. For any program P, we let C[[P]](*error*)=*error*. The semantic clause for assignment in state $s \neq error$ can be written as

C[[x := e]](s) =if E[[e]](s) =*Ok* then modify(s, x, *init*) else *error*

In words, if we execute an assignment x:= e in a state s different from *error*, then the result is either a state that has x initialized or *error* if there is some variable in e that was not initialized in s. For example, let

$s_0 = \lambda v$? *Variables.uninit*

be the state with every variable uninitialized. Then

C[[x :=0]](s_0) = modify(s_0, x, *init*)

is the state with variable x initialized and all other variables uninitialized.

The clauses for sequences P1; P2 are essentially straightforward. For $s \neq error$,

C[[P_1;P_2]](s) = if C[[P_1]](s) = *error* then *error* else C[[P_2]](C[[P_1]](*s*))

The clause for conditional is more complicated because our analysis tool is not going to try to evaluate a Boolean expression. Instead, we treat conditional as if it were possible to execute either branch. (This is the conservative part of the analysis.)Therefore, we change a variable to initialized only if it is initialized in both branches. If s_1 and s_2 are states different from *error*, then let $s_1 + s_2$ be the state

$s_1 + s_2 = \lambda v$? *Variables*. If $s_1(v) = s_2(v)$ = *init* then *init* else *uninit*

We define the meaning of a conditional statement by

C[[if e then P_1 else P_2]](s)

=if E[[e]](s)= error or C[[P_1]](s)= error or C[[P_2]](s) =*error*

then *error*

else C[[P_1]](s) + C[[P_2]](s)

For example, using s_0 as above, we have

C[[if 0=1 then x := 0 else x := 1; y := 2]](s_0) = modify(s_0, x, *init*)

as only x is initialized in both branches of the conditional.

For simplicity, we do not consider the clause for while e do P.

4.4 FUNCTIONAL AND IMPERATIVE LANGUAGES

4.4.1 Imperative and Declarative Sentences

The languages that humans speak and write are called *natural languages* as they developed naturally, without concern for machine readability. In natural languages, there are four main kinds of sentences: imperative, declarative, interrogative, and exclamatory.

In an imperative sentence, the subject of the sentence is implicit. For example, the subject of the sentence

Pick up that fish

is (implicitly) the person to whom the command is addressed. A declarative sentence expresses a fact and may consist of a subject and a verb or subject, verb, and object. For example,

Claude likes bananas

is a declarative sentence.

Interrogatives are questions. An exclamatory sentence may consist of only an interjection, such as *Ugh!* or *Wow!*

In many programming languages, the basic constructs are imperative statements. For example, an assignment statement such as

```
x:=5
```

is a command to the computer (the implied subject of the utterance) to store the value5 in a certain location. Programming languages also contain declarative constructs such as the function declaration

```
function f(int x) {return x+1;}
```

that states a fact. One reading of this as a declarative sentence is that the subject is the name f and the sentence about f is "f is a function whose return value is 1 greater than its argument."

In programming, the distinction between imperative and declarative constructs rests on the distinction between changing an existing value and declaring a new value. The first is imperative, the latter declarative. For example, consider the following program fragment:

```
{ int x=1;              /* declares new x */
    x = x+1;            /* assignment to existing x */
    { int y = x+1;      /* declares new y */
       { int x = y+1;   /* declares new x */
}}}
```

Here, only the second line is an imperative statement. This is an imperative command that changes the state of the machine by storing the value 2 in the location associated with variable x. The other lines contain declarations of new variables.

A subtle point is that the last line in the preceding code declares a new variable with the same name as that of a previously declared variable. The simplest way to understand the distinction between declaring a new variable and changing the value of an old one is by variable renaming. As we saw in lambda calculus, a general principle of binding is that bound variables can be renamed without changing the meaning of an expression or program. In particular, we can rename bound variables in the preceding program fragment to get

```
{ int x=1;              /* declares new x */
    x = x+1;            /* assignment to existing x */
    { int y = x+1;      /* declares new y */
       { int z = y+1;   /* declares new z */
}}}
```

(If there were additional occurrences of x inside the inner block, we would rename them to z also.) After rewriting the program to this equivalent form, we easily see that the declaration of a new variable z does not change the value of any previously existing variable.

4.4.2 Functional versus Imperative Programs

The phrase *functional language* is used to refer to programming languages in which most computation is done by evaluation of expressions that contain functions. Two examples are Lisp and ML. Both of these languages contain declarative and imperative constructs. However, it is possible to write a substantial program in either language without using any imperative constructs.

Some people use the phrase functional language to refer to languages that do not have expressions with side effects or any other form of imperative construct. However, we will use the more emphatic phrase *pure functional language* for declarative languages that are designed around flexible constructs for defining and using functions. We learned in Subsection 3.4.9 that pure Lisp, based on atom, eq, car, cdr, cons, lambda, define, is a pure functional language. If rplaca, which changes the car of a cell, and rplacd, which changes the cdr of a cell, are added, then the resulting Lisp is not a pure functional language.

Pure functional languages pass the following test:

Declarative Language Test: Within the scope of specific declarations of $x_1, \ldots, x_n$, all occurrences

of an expression e containing only variables $x_1, \ldots, x_n$ have the same value.

As a consequence, pure functional languages have a useful optimization property: If expression e occurs several places within a specific scope, this expression needs to be evaluated only once. For example, suppose a program written in pure Lisp contains two occurrences of (cons a b). An optimizing Lisp compiler could compute (cons a b)once and use the same value both places. This not only saves time, but also space, as evaluating cons would ordinarily involve a new cell.

Referential Transparency

In some of the academic literature on programming languages, including some textbooks on programming language semantics, the concept that is used to distinguish declarative from imperative languages is called *referential transparency*. Although it is easy to define this phrase, it is a bit tricky to use it correctly to distinguish one programming language from another.

In linguistics, a name or noun phrase is considered *referentially transparent* if it may be replaced with another noun phrase with the same referent (i.e., referring to the same thing) without changing the meaning of the sentence that contains it. For example, consider the sentence

I saw Walter get into *his car.*

If Walter owns a Maserati Biturbo, say, and no other car, then the sentence

I saw Walter get into *his Maserati Biturbo*

has the same meaning because the noun phrases (in italics) have the same meaning. A traditional counter example to referential transparency, attributed to the language philosopher Willard van Orman Quine, occurs in the sentence

He was called *William Rufus* because of his red beard.

The sentence refers to William IV of England and rufus means reddish or orange in color. If we replace William Rufus with William IV, we get a sentence that makes no sense:

He was called *William IV* because of his red beard.

Obviously, the king was called William IV because he was the fourth William, not because of the color of his beard.

Returning to programming languages, it is traditional to say that a language is referentially transparent if we may replace one expression with another of equal value anywhere in a program without changing the meaning of the program. This is a property of pure functional languages.

The reason referential transparency is subtle is that it depends on the value we associate with expressions. In imperative programming languages, we can say that a variable x refers to its value or to its location. If we say that a variable refers to its location in memory, then imperative languages *are* referentially transparent, as replacing one variable with another that names the same memory location will not change the meaning of the program. On the other hand, if we say that a variable refers to the value stored in that location, then imperative languages are not referentially transparent, as the value of a variable may change as the result of assignment.

Historical Debate

John Backus received the 1977 ACM Turing Award for the development of Fortran. In his lecture associated with this award, Backus argued that pure functional programming languages are better than imperative ones. The lecture and the accompanying paper, published by the Association for Computing Machinery, helped inspire a number of research projects aimed at developing practical pure functional programming languages. The main premise of Backus' argument is that pure functional programs are easier to reason about because we can understand expressions independently of the context in which they occur.

Backus asserts that, in the long run, program correctness, readability, and reliability are more important than other factors such as efficiency. This was a controversial position in 1977, when programs were a lot smaller than they are today and computers were much slower. In the 1990s, computers finally reached the stage at which commercial organizations began to choose software development methods that value programmer development time over

run-time efficiency. Because of his belief in the importance of correctness, readability, and reliability, Backus thought that pure functional languages would be appreciated as superior to languages with side effects.

To advance his argument, Backus proposed a pure functional programming language called FP, an acronym for functional programming. FP contains a number of basic functions and a rich set of combining forms with which to build new functions from old ones. An example from Backus' paper is a simple program to compute the inner product of two vectors. In C, the inner product of vectors stored in arrays a and b could be written as

```
int i, prod;
prod=0;
for (i=0; i < n; i++) prod = prod + a[i] * b[i];
```

In contrast, the inner product function would be defined in FP by combining functions+ and × (multiplication) with vector operations. Specifically, the inner product would be expressed as

```
Inner product = (Insert +) ° (ApplyToAll x) ° Transpose
```

where ° is function composition and Insert, ApplyToAll, and Transpose are vector operations. Specifically, the Transpose of a pair of lists produces a list of pairs and ApplyToAll applies the given operation to every element in a list, like the maplist function in Subsection 3.4.7. Given a binary operation and a list, Insert has the effect of inserting the operation between every pair of adjacent list elements and calculating the result. For example, (Insert +) <1, 2, 3, 4> = 1+2+3+4=10, where <1, 2, 3, 4> is the FP notation for the list with elements 1, 2, 3, 4.

Although the C syntax may seem clearer to most readers, it is worth trying to imagine how these would compare if you had not seen either before. One facet of the FP expression is that all of its parts are functions that we can understand without thinking about how vectors are represented in memory. In contrast, the C program has extra variables i and prod that are not part of the function we are trying to compute. In this sense, the FP program is higher level, or more abstract, than the C code.

A more general point about FP programs is that if one functional expression is equivalent to another, then we can replace one with the other in any program. This leads to a set of algebraic laws for FP programs. In addition to algebraic laws, an advantage of functional programming languages is the possibility of parallelism in implementations. This is subsequently discussed.

In retrospect, Backus' argument seems more plausible than his solution. The importance of program correctness, readability, and reliability has increased compared with that of run-time efficiency. The reason is that these affect the amount of time that people must spend in developing, debugging, and maintaining code. When computers become faster, it is acceptable to run less efficient programs - hardware improvements can compensate for software. However, increases in computer speed do not make humans more efficient. In this regard, Backus was absolutely right about the aspects of programming languages that would be important in the future.

Backus' language FP, on the other hand, was not a success. In the years since his lecture, there was an effort to develop a FP implementation at IBM, but the language was not widely used. One problem is that the language has a difficult syntax. Perhaps more importantly, there are severe limitations in the kind of data structures and control

structures that can be defined. FP languages that allow variable names and binding (as in Lisp and lambda calculus) have been more successful, as have all programming languages that support modularity and reuse of library components. However, Backus raised an important issue that led to useful reflection and debate.

Functional Programming and Concurrency

An appealing aspect of pure functional languages and of programs written in the pure functional subset of larger languages is that programs can be executed concurrently. This is a consequence of the declarative language test mentioned at the beginning of this subsection. We can see how parallelism arises in pure functional languages by using the example of a function call $f(e_1,\ldots,e_n)$, where function arguments $e_1,\ldots,e_n$ are expressions that may need to be evaluated.

> *Functional Programming:* We can evaluate $f(e_1,\ldots,e_n)$ by evaluating $e_1,\ldots,e_n$ in parallel because values of these expressions are independent.
>
> *Imperative Programming:* For an expression such as $f(g(x), h(x))$, the function g might change the value of x. Hence the arguments of functions in imperative
>
> languages must be evaluated in a fixed, sequential order. This order ing restricts the use of concurrency.

Backus used the term *von Neumann bottleneck* for the fact that in executing an imperative program, computation must proceed one step at a time. Because each step in a program may depend on the previous one, we have to pass values one at a time from memory to the CPU and back. This sequential channel between the CPU and memory is what he called the von Neumann bottleneck.

Although functional programs provide the opportunity for parallelism, and parallelism is often an effective way of increasing the speed of computation, effectively taking advantage of inherent parallelism is difficult. One problem that is fairly easy to understand is that functional programs sometimes provide too much parallelism. If all possible computations are performed in parallel, many more computation steps will be executed than necessary. For example, full parallel evaluation of a conditional

---

if e_1 then e_2 else e_3

---

will involve evaluating all three expressions. Eventually, when the value of e_1 is found, one of the other computations will turn out to be irrelevant. In this case, the irrelevant computation can be terminated, but in the meantime, resources will have been devoted to calculation that does not matter in the end.

In a large program, it is easy to generate so many parallel tasks that the time setting up and switching between parallel processes will detract from the efficiency of the computation. In general, parallel programming languages need to provide some way for a programmer to specify where parallelism may be beneficial. Parallel implementations of functional languages often have the drawback that the programmer has little control over the amount of parallelism used in execution.

Practical Functional Programming

Backus' Turing Award lecture raises a fundamental question:

> Do pure functional programming languages have significant practical advantages over imperative languages?

Although we have considered many of the potential advantages of pure FP languages in this section, we do not have a

definitive answer. From one theoretical point of view, FP languages are as good as imperative programming languages. This can be demonstrated when a translation is made of C programs into FP programs, lambda calculus expressions, or pure Lisp. Denotational semantics provides one method for doing this.

To answer the question in practice, however, we would need to carry out large projects in a functional language and see whether it is possible to produce usable software in a reasonable amount of time. Some work toward answering this question was done at IBM on the FP project (which was canceled soon after Backus retired). Additional efforts with other languages such as Haskell and Lazy ML are still being carried out at other research laboratories and universities. Although most programming is done in imperative languages, it is certainly possible that, at some future time, pure or mostly pure FP languages will become more popular. Whether or not that happens, FP projects have generated many interesting language design ideas and implementation techniques that have been influential beyond pure functional programming.

Team-Fly



Previous Next

4.5 CHAPTER SUMMARY

In this chapter, we studied the following topics:

- the outline of a simple compiler and parsing issues,
- lambda calculus,
- denotational semantics,
- the difference between functional and imperative languages.

A standard compiler transforms an input program, written in a source language, into an output program, written in a target language. This process is organized into a series of six phases, each involving more complex properties of programs. The first three phases, lexical analysis, syntax analysis, and semantic analysis, organize the input symbols into meaningful tokens, construct a parse tree, and determine context-dependent properties of the program such as type agreement of operators and operands. (The name semantic analysis is commonly used in compiler books, but is somewhat misleading as it is still analysis of the parse tree for context-sensitive syntactic conditions.) The last three phases, intermediate code generation, optimization, and target code generation, are aimed at producing efficient target code through language transformations and optimizations.

Lambda calculus provides a notation and symbolic evaluation mechanism that is useful for studying some properties of programming languages. In the section on lambda calculus, we discussed binding and α conversion. Binding operators arise in many programming languages in the form of declarations and in parameter lists of functions, modules, and templates. Lambda expressions are symbolically evaluated by use of β-reduction, with the function argument substituted in place of the formal parameter. This process resembles macro expansion and function in lining, two transformations that are commonly done by compilers. Although lambda calculus is a very simple system, it is theoretically possible to write every computable function in the lambda calculus. Untyped lambda calculus, which we discussed, can be extended with type systems to produce various forms of typed lambda calculus.

Denotational semantics is a way of defining the meanings of programs by specifying the mathematical value, function, or function on functions that each construct denotes. Denotational semantics is an abstract way of analyzing programs because it does not consider issues such as running time and memory requirements. However, denotational semantics is useful for reasoning about correctness and has been used to develop and study program analysis methods that are used in compilers and programming environments. Some of the exercises at the end of the chapter present applications for type checking, initialization-before-use analysis, and simplified security analysis. From a theoretical point of view, denotational semantics shows that every imperative program can be transformed into an equivalent functional program.

In pure functional programs, syntactically identical expressions within the same scope have identical values. This property allows certain optimizations and makes it possible to execute independent subprograms in parallel. Because functional languages are theoretically as powerful as imperative languages, we discussed some of the pragmatic differences between functional and imperative languages. Although functional languages may be simpler to reason about in certain ways, imperative languages often make it easier to write efficient programs. Although Backus argues that functional programs can eliminate the von Neumann bottleneck, practical parallel execution of functional programs has not proven as successful as he anticipated in his Turing Award lecture.

Team-Fly

Chapter 6: Type Systems and Type Inference

Programming involves a wide range of computational constructs, such as data structures, functions, objects, communication channels, and threads of control. Because programming languages are designed to help programmers organize computational constructs and use them correctly, many programming languages organize data and computations into collections called types. In this chapter, we look at the reasons for using types in programming languages, methods for type checking, and some typing issues such as polymorphism, overloading, and type equality. A large section of this chapter is devoted to type inference, the process of determining the types of expressions based on the known types of some symbols that appear in them. Type inference is a generalization of type checking, with many characteristics in common, and a representative example of the kind of algorithms that are used in compilers and programming environments to determine properties of programs. Type inference also provides an introduction to polymorphism, which allows a single expression to have many types.

6.1 TYPES IN PROGRAMMING

In general, a *type* is a collection of computational entities that share some common property. Some examples of types are the type int of integers, the type int→int of functions from integers to integers, and the Pascal subrange type [1 .. 100] of integers between 1 and 100. In concurrent ML there is the type int channel of communication channels carrying integer values and, in Java, a hierarchy of types of exceptions.

There are three main uses of types in programming languages:

- naming and organizing concepts,
- making sure that bit sequences in computer memory are interpreted consistently,
- providing information to the compiler about data manipulated by the program.

These ideas are elaborated in the following subsections.

Although some programming language descriptions will say things like, "Lisp is an untyped language," there is really no such thing as an untyped programming language. In Lisp, for example, lists and atoms are two different types: list operations can be applied to lists but not to atoms. Programming languages do vary a great deal, however, in the ways that types are used in the syntax and semantics (implementation) of the language.

6.1.1 Program Organization and Documentation

A well-designed program uses concepts related to the problem being solved. For example, a banking program will be organized around concepts common to banks, such as accounts, customers, deposits, withdrawals, and transfers. In modern programming languages, customers and accounts, for example, can be represented as separate types. Type checking can then check to make sure that accounts and customers are treated separately, with account operations applied to accounts but not used to manipulate customers. Using types to organize a program makes it easier for someone to read, understand, and maintain the program. Types therefore serve an important purpose in documenting the design and intent of the program.

An important advantage of type information, in comparison with comments written by a programmer, is that types may be checked by the programming language compiler. Type checking guarantees that the types written into a program are correct. In contrast, many programs contain incorrect comments, either because the person writing the explanation was careless or because the program was later changed but the comments were not.

6.1.2 Type Errors

A *type error* occurs when a computational entity, such as a function or a data value, is used in a manner that is inconsistent with the concept it represents. For example, if an integer value is used as a function, this is a type error. A common type error is to apply an operation to an operand of the wrong type. For example, it is a type error to use integer addition to add a string to an integer. Although most programmers have a general understanding of type errors, there are some subtleties that are worth exploring.

Hardware Errors. The simplest kind of type error to understand is a machine instruction that results in a hardware error. For example, executing a "function call"

```
x()
```

is a type error if x is not a function. If x is an integer variable with value 256, for example, then executing x() will cause the machine to jump to location 256 and begin executing the instructions stored at that place in memory. If location 256 contains data that do not represent a valid machine instruction, this will cause a hardware interrupt. Another example of a hardware type error occurs in executing an operation

```
float_add(3, 4.5)
```

where the hardware floating-point unit is invoked on an integer argument 3. Because the bit pattern used to represent 3 does not represent a floating-point number in the form expected by the floating-point hardware, this instruction will cause a hardware interrupt.

Unintended Semantics. Some type errors do not cause a hardware fault or interrupt because compiled code does not contain the same information as the program source code does. For example, an operation

```
int_add(3, 4.5)
```

is a type error, as int_add is an integer operation and is applied here to a floating-point number. Most hardware would perform this operation. Because the bits used to represent the floating-point number 4.5 represent an integer that is not mathematically related to 4.5, the operation it is not meaningful. More specifically, int_add is intended to perform addition, but the result of int_add(3, 4.5) is not the arithmetic sum of the two operands.

The error associated with int_add(3, 4.5) may become clearer if we think about how a program might apply integer addition to a floating-point argument. To be concrete, suppose a program defines a function f that adds three to its argument,

```
fun f(x) = 3+x;
```

and someplace within the scope of this definition we also declare a floating-point value z:

```
float z = 4.5;
```

If the programming language compiler or interpreter allows the call f(z) and the language does not automatically convert floating-point numbers to integers in this situation, then the function call f(z) will cause a run-time type error because int_add(3, 4.5) will be executed. This is a type error because integer addition is applied to a noninteger argument.

The reason why many people find the concept of type error confusing is that type errors generally depend on the concepts defined in a program or programming language, not the way that programs are executed on the underlying hardware. To be specific, it is just as much of a type error to apply an integer operation to a floating-point argument as it is to apply a floating-point operation to an integer argument. It does not matter which causes a hardware interrupt on any particular computer.

Inside a computer, all values are stored as sequences of bytes of bits. Because integers and floating-point numbers are stored as four bytes on many machines, some integers and floating-point numbers overlap; a single bit pattern may represent an integer when it is used one way and a floating-point number when it is used another. Nonetheless, a type error occurs when a pattern that is stored in the computer for the purpose of representing one type of value is used as the representation of another type of value.

6.1.3 Types and Optimization

Type information in programs can be used for many kinds of optimizations. One example is finding components of records (as they are called in Pascal and ML) or structs (as they are called in C). The component-finding problem also arises in object-oriented languages. A record consists of a set of entries of different types. For example, a student record may contain a student name of type string and a student number of type integer. In a program that also has records for undergraduate students, these might be represented as related type that also contains a field for the year in school of the student. Both types are written here as ML-style type expressions:

```
Student = {name : string, number : int}
Undergrad = {name : string, number : int, year : int}
```

In a program that manipulates records, there might be an expression of the form r.name, meaning the name field of the record r. A compiler must generate machine code that, given the location of record r in memory at run time, finds the location of the field name of this record at run time. If the compiler can compute the type of the record at compile time, then this type information can be used to generate efficient code. More specifically, the type of r makes it is possible to compute the location of r.name relative to the location r, at compile time. For example, if the type of r is Student, then the compiler can build a little table storing the information that name occurs before number in each Student record. Using this table, the compiler can determine that name is in the first location allocated to the record r. In this case, the expression r.name is compiled to code that reads the value stored in location r+1 (if location r is used for something else besides the first field). However, for records of a different type, the name field might appear second or third. Therefore, if the type of r is not known at compile time, the compiler must generate code to compute the location of name from the location of r at run time. This will make the program run more slowly. To summarize: Some operations can be computed more efficiently if the type of the operand is known at compile time.

In some object-oriented programming languages, the type of an object may be used to find the relative location of parts of the object. In other languages, however, the type system does not give this kind of information and run-time search must be used.

Team-Fly



Previous Next

6.2 TYPE SAFETY AND TYPE CHECKING

6.2.1 Type Safety

A programming language is *type safe* if no program is allowed to violate its type distinctions. Sometimes it is not completely clear what the type distinctions are in a specific programming language. However, there are some type distinctions that are meaningful and important in all languages. For example, a function has a different type from an integer. Therefore, any language that allows integers to be used as functions is not type safe. Another action that we always consider a type error is to access memory that is not allocated to the program.

The following table characterizes the type safety of some common programming languages. We will discuss each form of type error listed in the table in turn.

Safety	Example languages	Explanation
Not safe	C and C++	Type casts, pointer arithmetic
Almost safe	Pascal	Explicit deallocation; dangling pointers
Safe	Lisp, ML, Smalltalk, Java	Complete type checking

Type Casts. Type casts allow a value of one type to be used as another type. In C in particular, an integer can be cast to a function, allowing a jump to a location that does not contain the correct form of instructions to be a C function.

Pointer Arithmetic. C pointer arithmetic is not type safe. The expression *(p+i) has type A if p is defined to have type A*. Because the value stored in location p+i might have any type, an assignment like x = *(p+i) may store a value of one type into a variable of another type and therefore may cause a type error.

Explicit Deallocation and Dangling Pointers. In Pascal, C, and some other languages, the location reached through a pointer may be deallocated (freed) by the programmer. This creates a *dangling pointer*, a pointer that points to a location that is not allocated to the program. If p is a pointer to an integer, for example, then after we deallocate the memory referenced by p, the program can allocate new memory to store another type of value. This new memory may be reachable through the old pointer p, as the storage allocation algorithm may reuse space that has been freed. The old pointer p allows us to treat the new memory as an integer value, as p still has type int. This violates type safety. Pascal is considered "mostly safe" because this is the only violation of type safety (after the variant record and other original type problems are repaired).

6.2.2 Compile-Time and Run-Time Checking

In many languages, type checking is used to prevent some or all type errors. Some languages use type constraints in the definition of legal program. Implementations of these languages check types at compile time, before a program is started. In these languages, a program that violates a type constraint is not compiled and cannot be run. In other languages, checks for type errors are made while the program is running.

Run-Time Checking. In programming languages with run-time type checking, the compiler generates code so that, when an operation is performed, the code checks to make sure that the operands have the correct type. For example, the Lisp language operation car returns the first element of a cons cell. Because it is a type error to apply car to something that is not a cons cell, Lisp programs are implemented so that, before (car x) is evaluated, a check is made to make sure that x is a cons cell. An advantage of run-time type checking is that it catches type errors. A disadvantage is the run-time cost associated with making these checks.

Compile-Time Checking. Many modern programming languages are designed so that it is possible to check expressions for potential type errors. In these languages, it is common to reject programs that do not pass the compile-time type checks. An advantage of compile-time type checking is that it catches errors earlier than run-time checking does: A program developer is warned about the error before the program is given to other users or shipped as a product. Because compile-time checks may eliminate the need to check for certain errors at run time, compile-time checking can make it possible to produce more efficient code. For a specific example, compiled ML code is two to four times faster than Lisp code. The primary reason for this speed increase is that static type checking of ML programs greatly reduces the need for run-time tests.

Conservativity of Compile-Time Checking. A property of compile-time type checking is that the compiler must be conservative. This mean that compile-time type checking will find all statements and expressions that produce run-time type errors, but also may flag statements or expressions as errors even if they do not produce run-time errors. To be more specific about it, most checkers are both sound and conservative. A type checker is sound if no programs with errors are considered correct. A type checker is conservative if some programs without errors are still considered to have errors.

There is a reason why most type checkers are conservative: For any Turing-complete programming language, the set of programs that may produce a run-time type error is undecidable. This follows from the undecidability of the halting problem. To see why, consider the following form of program expression:

```
if (complicated-expression-that-could-run-forever)
    then (expression-with-type-error)
    else (expression-with-type-error)
```

It is undecidable whether this expression causes a run-time type error, as the only way for expression-with-type-error to be evaluated is for complicated-expression-that-could-run-forever to halt. Therefore, deciding whether this expression causes a run-time type error involves deciding whether complicated-expression-that-could-run-forever halts.

Because the set of programs that have run-time type errors is undecidable, no compile-time type checker can find type errors exactly. Because the purpose of type checking is to prevent errors, type checkers for type-safe languages are conservative. It is useful that type checkers find type errors, and a consequence of the undecidability of the halting problem is that some programs that could execute without run-time error will fail the compile-time type-checking tests.

The main trade-offs between compile-time and run-time checking are summarized in the following table.

Form of Type Checking	Advantages	Disadvantages
Run-time	Prevents type errors	Slows program execution
Compile-time	Prevents type errors Eliminates run-time tests Finds type errors *before* execution and run-time tests	May restrict programming because tests are *conservative*.

Combining Compile-Time and Run-Time Checking. Most programming languages actually use some combination of compile-time and run-time type checking. In Java, for example, static type checking is used to distinguish arrays from integers, but array bounds errors (which are a form of type error) are checked at run time.

Team-Fly

6.3 TYPE INFERENCE

Type inference is the process of determining the types of expressions based on the known types of some symbols that appear in them. The difference between type inference and compile-time type checking is really a matter of degree. A *type-checking* algorithm goes through the program to check that the types declared by the programmer agree with the language requirements. In *type inference*, the idea is that some information is not specified, and some form of logical inference is required for determining the types of identifiers from the way they are used. For example, identifiers in ML are not usually declared to have a specific type. The type system *infers* the types of ML identifiers and expressions that contain them from the operations that are used. Type inference was invented by Robin Milner (see the biographical sketch) for the ML programming language. Similar ideas were developed independently by Curry and Hindley in connection with the study of lambda calculus.

Although practical type inference was developed for ML, type inference can be applied to a variety of programming languages. For example, type inference could, in principle, be applied to C or other programming languages. We study type inference in some detail because it illustrates the central issues in type checking and because type inference illustrates some of the central issues in algorithms that find any kind of program errors.

In addition to providing a flexible form of compile-time type checking, ML type inference supports polymorphism. As we will see when we subsequently look at the type-inference algorithm, the type-inference algorithm uses *type variables* as place-holders for types that are not known. In some cases, the type-inference algorithm resolves all type variables and determines that they must be equal to specific types such as int, bool, or string. In other cases, the type of a function may contain type variables that are not constrained by the way the function is defined. In these cases, the function may be applied to any arguments whose types match the form given by a type expression containing type variables.

Although type inference and polymorphism are independent concepts, we discuss polymorphism in the context of type inference because polymorphism arises naturally from the way type variables are used in type inference.

6.3.1 First Examples of Type Inference

Here are two ML type-inference examples to give you some feel for how ML type inference works. The behavior of the type-inference algorithm is explained only superficially in these examples, just to give some of the main ideas. We will go through the type inference process in detail in Subsection 6.3.2.

6.1 Example

```
- fun f1(x) = x+2;
val f1 = fn : int → int
```

The function f1 adds 2 to its argument. In ML, 2 is an integer constant; the real number 2 would be written as 2.0. The operator + is overloaded; it can be either integer addition or real addition. In this function, however, it must be integer addition because 2 is an integer. Therefore, the function argument x must be an integer. Putting these observations

together, we can see that f1 must have type int → int.

Example 6.2

```
- fun f2(g,h) = g(h(0));
val f2 = fn : ('a → 'b) * (int → 'a) → 'b
```

The type (‘a → ‘b) * (int → ‘a) → ‘b inferred by the compiler is parsed as ((‘a → ‘b)* (int → ‘a)) → ‘b.

The type-inference algorithm figures out that, because h is applied to an integer argument, h must be a function from int to something. The algorithm represents "something" by introducing a type variable, which is written as ‘a. (This is unrelated to Lisp ‘a, which would be syntax for a Lisp atom, not a variable.) The type-inference algorithm then deduces that g must be a function that takes whatever h returns (something of type ‘a) and then returns something else. Because g is not constrained to return the same type of value as h, the algorithm represents this second something by a new type variable, ‘b. Putting the types of h and g together, we can see that the first argument to f2 has type (‘a → ‘b) and the second has type (int → ‘a). Function f2 takes the pair of these two functions as an argument and returns the same type of value as g returns. Therefore, the type of f2 is ((‘a → ‘b) * (int → ‘a)) → ‘b, as shown in the preceding compiler output.

6.3.2 Type-Inference Algorithm

The ML type-inference algorithm uses the following three steps:

1. A assign a type to the expression and each subexpression. For any compound expression or variable, use a type variable. For known operations or constants, such as + or 3, use the type that is known for this symbol.
2. Generates a set of constraints on types, using the parse tree of the expression. These constraints reflect the fact that if a function is applied to an argument, for example, then the type of the argument must equal the type of the domain of the function.
3. Solve these constraints by means of unification, which is a substitution-based algorithm for solving systems of equations. (More information on unification appears in the chapter on logic programming.)

The type-inference algorithm is explained by a series of examples. These examples present the following issues:

- explanation of the algorithm
- a polymorphic function definition
- application of a polymorphic function
- a recursive function
- a function with multiple clauses
- type inference indicates a program error

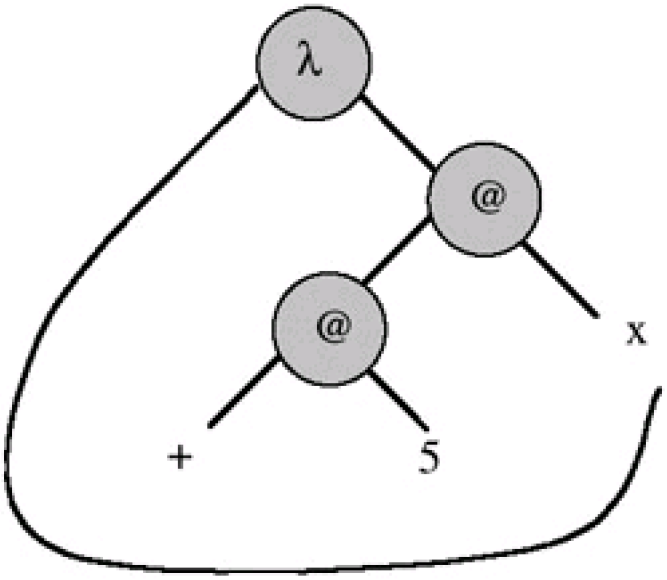
Altogether, these six examples should give you a good understanding of the type-inference algorithm, except for the interaction between type inference and overloading. The interaction between overloading and type inference is not covered in this book.

Example 6.3 Explanation of the Algorithm

We can see how the type-inference algorithm works by considering this example function:

```
- fun g(x) = 5 + x;
val g = fn : int → int
```

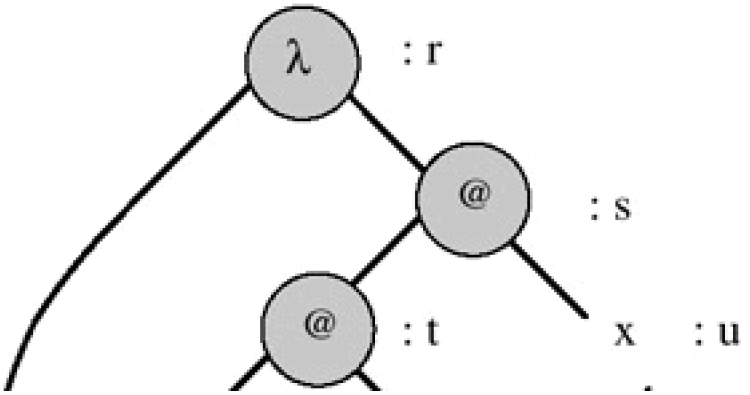
The easiest way to see how the algorithm works is by drawing the parse tree of the expression. We use an abbreviated form of parse tree that lists only the symbols that occur in the expression, together with the symbol @ for an application of a function to an argument. For the preceding expression we use the following graph. This is a form of parse tree, together with a special edge indicating the binding lambda for each bound variable. Here, the link from x to λ indicates that x is lambda bound at the beginning of the expression:

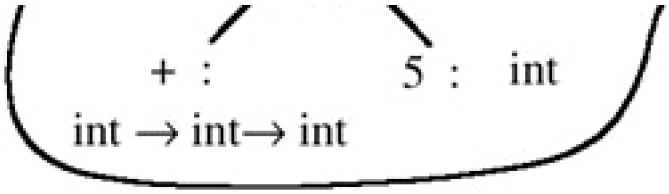


We use this graph to follow our type-inference steps:

1. *We assign a type to the expression and each subexpression:*

   We illustrate this step by redrawing the graph, writing a type next to each node. To simplify notation, we use single letters *r, s, t, u, v,* …, for type variables instead of ML syntax ‘a, ‘b, and so on:





Recall that each node in a parse tree represents a subexpression. Repeating the information in the picture, the following table lists the subexpressions and their types:

Subexpression	Type
λx. ((+ 5) x)	r
((+ 5) x)	s
(+ 5)	t
+	int → (int → int)
5	int
x	u

Here we have written addition (+) as a Curried function and have chosen type int→ int → int for this operation. The prefix notation for addition is not ML syntax, of course, but it is used here to make the pictures simpler. As we saw earlier, + can either be integer addition or real-number addition. Here we can see from context that integer addition is needed. The actual ML type-inference algorithm will require a few steps to figure this out, but we are not concerned with the mechanics of overloading resolution here.

2. *We generate a set of constraints on types, using the parse tree of the expression*.

Constraints are equations between type expressions that must be solved. The way we generate them depends on the form of each subexpression. For each function application, constraints are generated according to the following rule.

> *Function Application:* If the type of f is *a*, the type of *e* is b, and the type of fe is *c*, then we must have $a = b \rightarrow c$.

This typing rule for function application can be used twice in our expression.

> Subexpression (+5), Constraint int → (int → int) = int → t,
>
> Subexpression (+5) *x*, Constraint $t = u \rightarrow s$.

In the subexpression (+ 5), the type of the function + is int → (int → int), the type of the argument 5 is int and the type of the application is t. Therefore, we must have int → (int → int), = int → *t*. The reasoning for subexpression (+ 5) *x* is similar: In the subexpression (+ 5) *x*, the type of the function (+ 5) is *t*, the type of the argument *x* is *u*, and the type of the application is *s*. Therefore, we must have $t = u \rightarrow s$.

> *Lambda Abstraction (Function Expression):* If the type of *x* is *a* and the type of *e* is *b*, then the type of λ *x*. *e* must equal $a \rightarrow b$.

For our example expression, we have one lambda abstraction. This gives us the following constraint:

> Subexpression λ *x*. ((+5) *x*), Constraint $r = u \rightarrow s$.

In words, the type of the whole expression is *r*, the type of *x* is *u*, and the type of the subexpression ((+ 5) *x*) is *s*. This gives us the equation $r = u \rightarrow s$.

3. *We solve these constraints by means of unification*.

Unification is a standard algorithm for solving systems of equations by substitution. The general properties of this algorithm are not discussed here. Instead, the process is shown by example in enough detail that you should be able to figure out the types of simple expressions on your own.

For our example expression, we have the following constraints.

$$\text{int} \to (\text{int} \to \text{int}) = \text{int} \to t,$$

$$t = u \to s,$$

$$r = u \to s.$$

If there is a way of associating type expression to type variables that makes all of these equations true, then the expression is well typed. In this case, the type of the expression will be the type expression equal to the type variable *r*. If there is no way of associating type expression to type variables that makes all of these equations true, then there is no type for this expression. In this case, the type-inference algorithm will fail, resulting in an error message that says the expression is not well typed.

We can process these equations one at a time. The order is not very important, although it is convenient to put the equation involving the type of the entire expression last, as this is the output of the type-inference algorithm.

The first equation is true if $t = int \to int$. Because we need $t = int \to int$, we substitute $int \to int$ for = in the remaining equations. This gives us two equations to solve:

$$\text{int} \to \text{int} = u \to s,$$

$$r = u \to s.$$

The only way to have $int \to int = u \to s$ is if $u = s = int$. Proceeding as before, we substitute int for both *u* and *s* in the remaining equation. This gives us

$$r = \text{int} \to \text{int},$$

which tells us that the type *r* of the whole expression is int → int. Because every constraint is solved, we know that the expression is type able and we have computed a type for the expression.

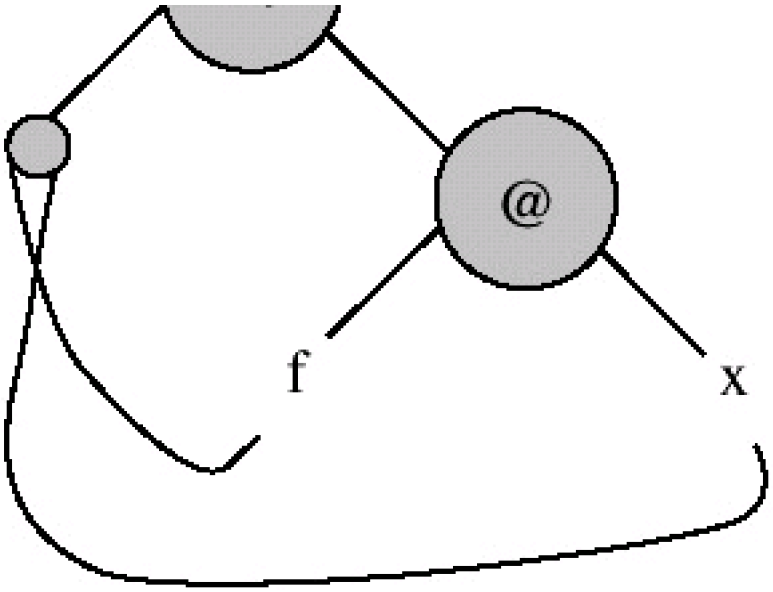
Example 6.4 A Polymorphic Function Definition

```
- fun apply(f,x) = f(x);
val apply = fn : ('a → 'b) * 'a → 'b
```

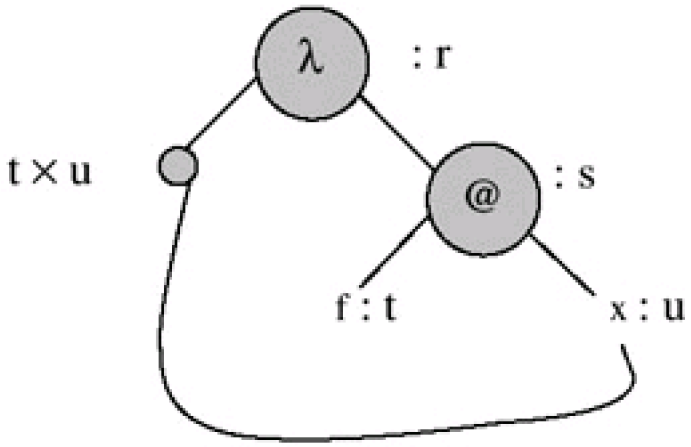
This is an example of a function whose type involves type variables. The type-inference algorithm begins by assigning a type to each subexpression. Because this makes it easiest to understand the algorithm, we write the function as a lambda expression with a pair ?f,x? instead of a variable as a formal parameter: apply is defined by the lambda expression λ ?f,x?. f x that maps a pair ?f,x? to the result f x of applying f to x.

Here is the parse graph of λ ?f,x?. f x, in which a pairing node is used on the left to indicate that the argument ?f,x? of the function is a pair, with links to f and x.





The first step of the algorithm is to assign types to each node in the graph, as shown here:



The same information is repeated in the following table, showing the subexpressions represented by each node and their types.

Subexpression	Type
λ *?f,x?*. *fx*	r
?f,x?.	t ×*u*
fx	s
f	t
x	u

The second step of the algorithm is to generate a set of constraints. For this example, there is one constraint for the application and one for the lambda abstraction. The application gives us

$$t = u \rightarrow s.$$

In words, the type of the application f x has type s, provided that the type of the function f is equal to ?*type of argument*? $\rightarrow s$. Because the type of the argument is *u*, this gives us the constraint $t = u \rightarrow s$.

The second constraint, from the lambda abstraction, is

$$r = t * u \rightarrow s.$$

In words, the type of λ ?f,x?. fx is *r*, where *r* must be equal to ?*type of argument*? → *s*,as *s* is the type of the subtree representing the function result. Because the argument is the pair ?f,x?, the type of the argument is t * u.

The constraints can be solved in order. The first requires t = u → s, which we solve by substituting u → s for t in the remaining constraint. This gives us

$$r = (u \rightarrow s)*u \rightarrow s.$$

This tells us the type of the function. If we rewrite (*u* → *s*) * *u* → *s* with 'a and 'b in place of *u* and *s*, then we get the compiler output previously shown. Because there are type variables in the type of the expression, the function may be used for many types of arguments. This is illustrated in the following example, which uses the type we have just computed for apply.

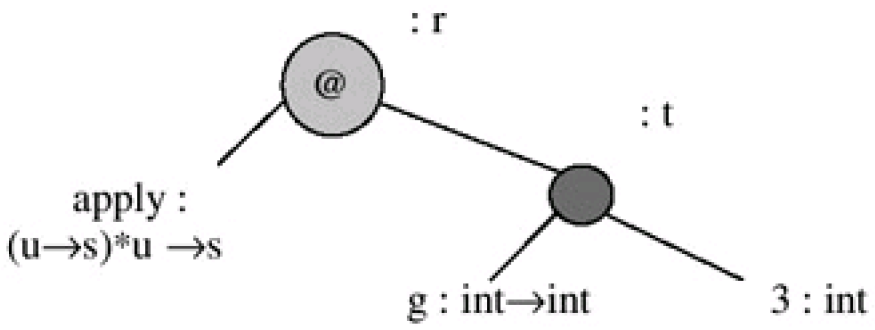
6.5 Example Application of a Polymorphic Function

In the last example, we calculated the type of apply. The type of apply is ('a → 'b)* 'a → 'b, which contains type variables. The type variables in this type mean that apply is a *polymorphic* function, a function that may be applied to different types of arguments. In the case of apply, the type ('a → 'b) * 'a → 'b means that apply may be applied to a pair of arguments of type ('a → 'b) * 'a for any types 'a and 'b. In particular, recall that function fun g(x) = 5 + x from Example 6.3 has type int → int. Therefore, the pair (g,3) has type int → int.* int, which matches the form ('a → 'b) * 'a for function apply. In this example, we calculate the type of the application

---

```
apply(g,3);
```

Following the three steps of the type inference algorithm, we begin by assigning types to the nodes in the parse tree of this expression:



In this illustration, the smaller unlabeled circle is a pairing node. This node and the two below it represent the pair (g,3). In the previous example, we associated a product type with the pairing node. Here, to show that it is equivalent to use a type variable and constraint, we associate a type variable *t* with the pairing node and generate the constraint t = (int → int) * int.

There are two constraints, one for the pairing node and one for the application node. For pairing, we have

$$t = (\text{int} \rightarrow \text{int}) * \text{int}.$$

For the application, we have

$$(u \to s) * u \to s = t \to r.$$

In words, the type $(u \to s) * u \to s$ of the function must equal ?*type of argument*?$\to r$, where r is the type of the application.

Now we must solve the constraints, The first constraint gives a type expression for t, which we can substitute for t in the second constraint. This gives us

$$(u \to s) * u \to s = (\text{int} \to \text{int}) * \text{int} \to r.$$

This constraint has an expression on each side of the equal sign. To solve this constraint, corresponding parts of each expression must be equal. In other words, we can solve this constraint precisely by solving the following four constraints:

$$u = \text{int}, s = \text{int}, u = \text{int}, s = r.$$

Because these require $u = s = \text{int}$, we have $r = \text{int}$. Because all of the constraints are solved, the expression apply(g,3) is typeable in the ML type system. The type of apply(g,3) is the solution for type variable r, namely int.

We can also apply to other types of arguments. If not : bool → bool, then

---

```
apply(not, false)
```

---

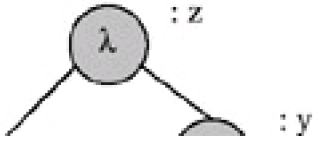
is a well-typed expression with type bool by exactly the same type-inference processes as those for apply(g,3). This illustrates the polymorphism of apply: Because the type (‘a → ‘b) * ‘a → ‘b of apply contains type variables, the function may be applied to any type of arguments that can be obtained if the type variables in (‘a → ‘b) * ‘a → ‘b are replaced with type names or type expressions.

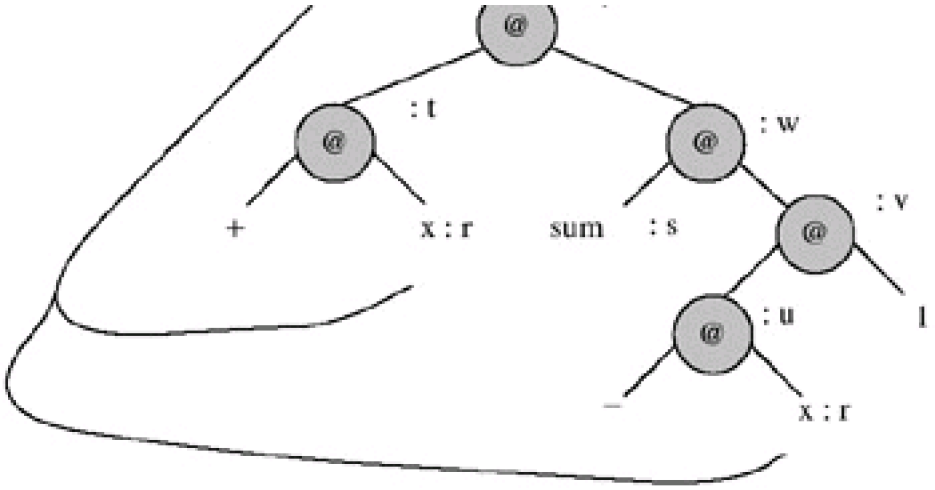
6.6 Example A Recursive Function

When a function is defined recursively, we must determine the type of the function body without knowing the type of recursive function calls. To see how this works, consider this simple recursive function that sums the integers up to a given integer. This function does not terminate, but it does type check:

```
- fun sum(x) = x+sum(x-1);
val sum = fn : int -> int
```

Here is a parse graph of the function, with type variables associated with each of the nodes except for +, –, and 1, as we ignore overloading and treat these as integer operations and integer constant. Because we are trying to determine the type of sum, we associate a type variable with sum and proceed:





Starting with the applications of + and – and proceeding from the lower right up, the constraints associated with the function applications and lambda abstraction in this expression are

$$\text{int} \to (\text{int} \to \text{int}) = r \to t,$$

$$\text{int} \to (\text{int} \to \text{int}) = r \to u,$$

$$u = \text{int} \to v,$$

$$s = v \to w,$$

$$t = w \to y,$$

$$z = r \to y.$$

To this list we add one more because the type of sum must also be the type of the entire expression:

$$s = z.$$

The constraint $s = z$ is the one additional constraint associated with the fact that this is a recursive declaration of a function. Solving these in order, we have

$$r = \text{int}, t = \text{int} \to \text{int},$$

$$u = \text{int} \to \text{int},$$

$$v = \text{int},$$

$$s = \text{int} \to w,$$

$$t = w \to y,$$

$$z = r \to y,$$

$$w = \text{int}, y = \text{int},$$

$$z = \text{int} \to \text{int},$$

$$s = \text{int} \to \text{int}.$$

Because the constraints can be solved, the function is typeable. In the process of solving the constraints, we have calculated that the type of sum is int→int.

6.7 Example A Function with Multiple Clauses

Type inference for functions with several clauses may be done by a type check of each clause separately. Then, because all clauses define the same function, we impose the constraint that the types of all clauses must be equal. For example, consider the append function on lists, defined as follows:

```
- fun append(nil, l) = l
  | append(x::xs, l) = x:: append(xs, l);
val append = fn : 'a list * 'a list → 'a list.
```

As the type : ‘a list * ‘a list → ‘a list indicates, append can be applied to any pair of lists, as long as both lists contain the same type of list elements. Thus, append is a polymorphic function on lists.

We begin type inference for append by following the three-step algorithm for the first clause of the definition, then repeating the steps for the second clause. This gives us two types:

```
append : 'a list * 'b → 'b
append : 'a list * 'b → 'a list
```

Intuitively, the first clause has type ‘a list * ‘b → ‘b because the first argument must match nil, but the second argument may be anything. The second clause has type ‘a list * ‘b → ‘a list because the return result is a list containing one element from the list passed as the first argument.

If we impose the constraint

```
'a list * 'b → 'b = 'a list * 'b -> 'a list
```

then we must have ‘b = a list. This gives us the final type for append:

```
append : 'a list * 'a list → 'a list
```

6.8 Example Type Inference Indicates a Program Error

Here is an example that shows how type inference may produce output that indicates a programming error, even though the program may type correctly. Here is a sample (incorrect) declaration of a reverse function on lists:

```
- fun reverse (nil) = nil
  | reverse (x::lst) = reverse (lst);
val reverse = fn : 'a list → 'b list
```

As the compiler output shows, this function is typeable; there is no type error in this declaration. However, look carefully at the type of reverse. The type 'a list → 'b list means that we can apply reverse to any type of list and obtain any type of list as a result. However, the type of the "reversed" list is not the same as the type of the list we started with!

Because it does not make sense for reverse to return a list that is a different type from its argument, there must be something wrong with this code. The problem is that, in the second clause, the first element x of the input list is not used as part of the output. Therefore, reverse always returns the empty list.

As this example illustrates, the type-inference algorithm may sometimes return a type that is more general than the one we expect. This does not indicate a type error. In this example, the faulty reverse can be used anywhere that a correct reverse function could be used. However, the type of reverse is useful because it tells the programmer that there is an error in the program.

Team-Fly

6.4 POLYMORPHISM AND OVERLOADING

Polymorphism, which literally means "having multiple forms," refers to constructs that can take on different types as needed. For example, a function that can compute the length of any type of list is polymorphic because it has type 'a list → int for every type 'a.

There are three forms of polymorphism in contemporary programming languages:

- *parametric polymorphism*, in which a function may be applied to any arguments whose types match a type expression involving type variables;
- *ad hoc polymorphism*, another term for overloading, in which two or more implementations with different types are referred to by the same name;
- *subtype polymorphism*, in which the subtype relation between types allows an expression to have many possible types.

Parametric and ad hoc polymorphism (overloading) are discussed in this section. Subtype polymorphism is considered in later chapters in connection with object-oriented programming.

6.4.1 Parametric Polymorphism

The main characteristic of parametric polymorphism is that the set of types associated with a function or other value is given by a type expression that contains type variables. For example, an ML function that sorts lists might have the ML type

```
sort:('a* 'a → bool) * 'a list → 'a list
```

In words, sort can be applied to any pair consisting of a function and a list, as long as the function has a type of the form 'a* 'a -> bool, in which the type 'a must also be the type of the elements of the list. The function argument is a less-than operation used to determine the order of elements in the sorted list.

In parametric polymorphism, a function may have infinitely many types, as there are infinitely many ways of replacing type variables with actual types. The sort function, for example, may be used to sort lists of integers, lists of lists of integers, lists of lists of lists of integers, and so on.

Parametric polymorphism may be implicit or explicit. In *explicit parametric polymorphism*, the program text contains type variables that determine the way that a function or other value may be treated polymorphically. In addition, explicit polymorphism often involves explicit instantiation or type application to indicate how type variables are replaced with specific types in the use of a polymorphic value. C++ templates are a well-known example of explicit polymorphism. ML polymorphism is called *implicit parametric polymorphism* because programs that declare and use polymorphic functions do not need to contain types - the type-inference algorithm computes when a function is polymorphic and computes the instantiation of type variables as needed.

C++ Function Templates

For many readers, the most familiar type parameterization mechanism is the C++ template mechanism. Although some C++ programmers associate templates with classes and object-oriented programming, function templates are also useful for programs that do not declare any classes.

As an illustrative example, suppose you write a simple function to swap the values of two integer variables:

```
void swap(int& x, int& y){
   int tmp = x; x = y; y = tmp;
}
```

Although this code is useful for exchanging values of integer variables, the sequence of instructions also works for other types of variables. If we wish to swap values of variables of other types, then we can define a function template that uses a type variable `T` in place of the type name `int`:

```
template <typename T>
void swap(T& x, T& y){
   T tmp = x; x = y; y = tmp;
}
```

For those who are not familiar with templates, the main idea is to think of the type name `T` as a parameter to a function from types to functions. When applied to, or *instantiated* to, a specific type, the result is a version of `swap` that has int replaced with another type. In other words, `swap` is a general function that would work perfectly well for many types of arguments. Templates allow us to treat `swap` as a function with a type argument.

In C++, function templates are instantiated automatically as needed, with the types of the function arguments used to determine which instantiation is needed. This is illustrated in the following example lines of code.

```
int i,j;  . . .    swap(i,j); // Use swap with T replaced with int
float a,b;  . . .  swap(a,b); // Use swap with T replaced with float
String s,t;  . . . swap(s,t); // Use swap with T replaced with String
```

Comparison with ML Polymorphism

In ML polymorphism, the type-inference algorithm infers the type of a function and the type of a function application (as explained in Section 6.3). When a function is polymorphic, the actions of the type-inference algorithm can be

understood as automatically inserting "template declarations" and "template instantiation" into the program. We can see how this works by considering an ML sorting function that is analogous to the C++ sort function previously declared:

```
fun insert(less, x, nil) = [x]
| insert(less, x, y::ys) = if less(x,y) then x::y::ys
                    else y::insert(less,x,ys);
fun sort(less, nil) = nil
| sort(less, x::xs) = insert(less, x, sort(less,xs));
```

For sort to be polymorphic, a less-than operation must be passed as a function argument to sort.

The types of insert and sort, as inferred by the type-inference algorithm, are

```
val insert = fn : ('a * 'a -> bool) * 'a * 'a list -> 'a list
val sort = fn : ('a * 'a -> bool) * 'a list -> 'a list
```

In these types, the type variable ‘a can be instantiated to any type, as needed. In effect, the functions are treated as if they were "templates." By use of a combination of C++ template, ML function, and type syntax, the functions previously defined could also be written as

```
template <type 'a>
fun insert(less : 'a * 'a -> bool, x : 'a, nil : 'a list) = [x]
| insert(less, x, y::ys) = if less(x,y) then x::y::ys
                    else y::insert(less,x,ys);
template <type 'a>
fun sort(less : 'a* 'a -> bool, nil : 'a list) = nil
| sort(less, x::xs) = insert(less, x, sort(less,xs));
```

These declarations are the explicitly typed versions of the implicitly polymorphic ML functions. In other words, the ML type-inference algorithm may be understood as a program preprocessor that converts ML expressions without type information into expressions in some explicitly typed intermediate language with templates. From this point of view, the difference between explicit and implicit polymorphism is that a programming language processor (such as the ML compiler) takes the simpler implicit syntax and automatically inserts explicit type information, converting from implicit to explicit form, before programs are compiled and executed.

Finishing this example, suppose we declare a less-than function on integers:

```
- fun less(x,y) = x < y;
val less = fn : int * int -> bool
```

In the following application of the polymorphic sort function, the sort template is automatically instantiated to type int, so sort can be used to sort an integer list:

```
- sort (less, [1,4,5,3,2]);
val it = [1,2,3,4,5] : int list
```

6.4.2 Implementation of Parametric Polymorphism

C++ templates and ML polymorphic functions are implemented differently. The reason for the difference is not related to the difference between explicitly polymorphic syntax and implicitly polymorphic syntax. The need for different implementation techniques arises from the difference between data representation in C and data representation in ML.

C++ Implementation

C++ templates are instantiated at program link time. More specifically, suppose that the swap function template is stored in one file and compiled and a program calling swap is stored in another file and compiled separately. The so-called relocatable object files produced by compilation of the calling program will include information indicating that the compiled code calls a function swap of a certain type. The program linker is designed to combine the two program parts by linking the calls to swap in the calling program to the definition of swap in a separate compilation unit. It does so by instantiating the compiled code for swap in a form that produces code appropriate for the calls to swap.

If a program calls swap with several different types, then several different instantiated copies of swap will be produced. One reason that a different copy is needed for each type of call is that function swap declares a local variable tmp of type T. Space for tmp must be allocated in the activation record for swap. Therefore the compiled code for swap must be modified according to the size of a variable of type T. If T is a structure or object, for example, then the size might be fairly large. On the other hand, if T is int, the size will be small. In either case, the compiled code for swap must "know" the size of the datum so that addressing into the activation record can be done properly.

The linking process for C++ is relatively complex. We will not study it in detail. However, it is worth noting that if < is an overloaded operator, then the correct version of < must be identified when the compiled code for sort is linked with a calling program. For example, consider the following generic sort function:

```
template <typename T>
void sort( int count, T * A[count] ) {
  for (int i=0; i <count-1; i++)
        for (int j=i+1; j<count-1; j++)
```

```
        if (A[j] < A[i]) swap(A[i],A[j]);
}
```

If A is an array of type T, then sort(n, A) will work only if operator < is defined on type T. This requirement of sort is not declared anywhere in the C++ code. However, when the function template is instantiated, the actual type T must have an operator < defined or a link-time error will be reported and no executable object code will be produced.

ML Implementation

In ML, there is one sequence of compiled instructions for each polymorphic function. There is no need to produce different copies of the code for different types of arguments because related types of data are represented in similar ways. More specifically, pointers are used in parameter passing and in the representation of data structures such as lists so that when a function is polymorphic, it can access all necessary data in the same way, regardless of its type. This property of ML is called *uniform data representation*.

The simplest example of uniform data representation is the ML version of the polymorphic swap function:

```
- fun swap(x,y) = let val tmp = x in x: = !y;y: = !tmp end;
val swap = fn : 'a ref * 'a ref -> unit
```

As the type indicates, this swap function can be applied to any two references of the same type. Although ML references generally work like assignable variables in other languages, the value of a reference is a pointer to a cell that contains a value. Therefore, when a pair of references is passed to the swap function, the swap function receives a pair of pointers. The two pointers are the same size (typically 32 bits), regardless of what type of value is contained in the locations to which they point. In fact, we can complete the entire computation by doing only pointer assignment. As a result, none of the compiled code for swap depends on the size of the data referred to by arguments x and y.

Uniform data representation has its advantages and disadvantages. Because there is no need to duplicate code for different argument types, uniform data representation leads to smaller code size and avoids complications associated with C++-style linking. On the other hand, the resulting code can be less efficient, as uniform data representation often involves using pointers to data instead of storing data directly in structures.

For polymorphic list functions to work properly, all lists must be represented in exactly the same way. Because of this uniformity requirement, small values that would fit directly into the car part of a list cons cell cannot be placed there because large values do not fit. Hence we must store pointers to small values in lists, just as we store pointers to large values. ML programmers and compiler writers call the process of making all data look the same by means of pointers *boxing*.

Comparison

Two important points of comparison are efficiency and reporting of error messages. As far as efficiency, the C++ implementation requires more effort at link time and produces a larger code, as instantiating a template several times will result in several copies of the code. The ML implementation will run more slowly unless special optimizations are applied; uniform data representation involves more extensive use of pointers and these pointers must be stored and followed.

As a general programming principle, it is more convenient to have program errors reported at compile time than at link time. One reason is that separate program modules are compiled independently, but are linked together only when the

entire system is assembled. Therefore, compilation is a "local" process that can be carried out by the designer or implementer of a single component. In contrast, link-time errors represent global system properties that are not known until the entire system is assembled. For this reason, C++ link-time errors associated with operations in templates can be irritating and a source of frustration.

Somewhat better error reporting for C++ templates could be achieved if the template syntax included a description of the operations needed on type parameters. However, this is relatively complicated in C++, because of overloading and other properties of the language. In contrast, the ML has simpler overloading and includes more information in parameterized constructs, allowing all type errors to be reported as a program unit is compiled.

6.4.3 Overloading

Parametric polymorphism can be contrasted with overloading. A symbol is *overloaded* if it has two (or more) meanings, distinguished by type, and resolved at compile time. In an influential historical paper, Christopher Strachey referred to ML-style polymorphism as *parametric polymorphism* (although ML had not been invented yet) and overloading as *ad hoc polymorphism*.

Example. In standard ML, as in many other languages, the operator + has two distinct implementations associated with it, one of type int * int $\rightarrow$ int, the other of type real*real $\rightarrow$ real. The reason that both of these operations are given the name + is that both compute numeric addition. However, at the implementation level, the two operations are really very different. Because integers are represented in one way (as binary numbers) and real numbers in another (as exponent and mantissa, following scientific notation), the way that integer addition combines the bits of its arguments to produce the bits of its result is very different from the way this is done in real addition.

An important difference between parametric polymorphism and overloading is that parameter polymorphic functions use one algorithm to operate on arguments of many different types, whereas overloaded functions may use a different algorithm for each type of argument.

A characteristic of overloading is that overloading is *resolved* at compile time. If a function is overloaded, then the compiler must choose between the possible algorithms at compile time. Choosing one algorithm from among the possible algorithms associated with an overloaded function is called resolving the overloading. In many languages, if a function is overloaded, then only the function arguments are used to resolve overloading. For example, consider the following two expressions:

```
3 + 2;     /* add two integers */
3.0 + 2.0; /* add two real (floating point) numbers */
```

Here is how the compiler will produce code for evaluating each expression:

- 3+2: The parsing phase of the compiler will build the parse tree of this expression, and the type-checking phase will compute a type for each symbol. Because the type-checking phase will determine that + must have type int * int $\rightarrow$ int, the code-generation phase of the compiler will produce machine instructions that perform integer addition.
- 3.0 + 2.0: The parsing phase of the compiler will build the parse tree of this expression, and the type-checking phase will compute a type for each symbol. Because the type-checking phase will determine that + must have type real * real $\rightarrow$ real, the code-generation phase of the compiler will produce machine instructions that perform integer addition.

Automatic conversion is a separate mechanism that may be combined with overloading. However, it is possible for a language to have overloading and not to have automatic conversion. ML, for example, does not do automatic conversion.

6.6 CHAPTER SUMMARY

In this chapter, we studied reasons for using types in programming languages, methods for type checking, and some typing issues such as polymorphism, overloading, and type equality.

Reasons for Using Types

There are three main uses of types in programming languages:

- *Naming and organizing concepts:* Functions and data structures can be given types that reflect the way these computational constructs are used in a program. This helps the programmers and anyone else reading a program figure out how the program works and why it is written a certain way.
- *Making sure that bit sequences in computer memory are interpreted consistently:* Type checking keeps operations from being applied to operands in incorrect ways. This prevents a floating-point operation from being applied to a sequence of bits that represents a string, for example.
- *Providing information to the compiler about data manipulated by the program:* In languages in which the compiler can determine the type of a data structure, for example, the type information can be used to determine the relative location of a part of this structure. This compile-time type information can be used to generate efficient code for indexing into the data structure at run time.

Type Inference

Type inference is the process of determining the types of expressions based on the known types of some of the symbols that appear in them. For example, we saw how to infer that the function g declared by

```
fun g(x) = 5+x;
```

has type int $\rightarrow$ int. The difference between type inference and compile-time type checking is a matter of degree. A type-checking algorithm goes through the program to check that the types declared by the programmer agree with the language requirements. In type inference, the idea is that some information is not specified and some form of logical inference is required for determining the types of identifiers from the way they are used.

The following steps are used for type inference:

1. Assign a type to the expression and each subexpression by using the known type of a symbol of a type variable.
2. Generate a set of constraints on types by using the parse tree of the expression.
3. Solve these constraints by using unification, which is a substitution-based algorithm for solving systems of equations.

In a series of examples, we saw how to apply this algorithm to a variety of expressions. Type inference has many characteristics in common with the kind of algorithms that are used in compilers and programming environments to

determine properties of programs. For example, some useful alias analysis algorithms that try to determine whether two pointers might point to the same location have the same general outline as that of type inference.

Polymorphism and Overloading

There are three forms of polymorphism: parametric polymorphism, ad hoc polymorphism (another term for overloading), and subtype polymorphism. The first two were examined in this chapter, with subtype polymorphism left for later chapters on object-oriented languages. Parametric polymorphism can be either implicit, as in ML, or explicit, as with C++ templates. There are also two ways of implementing parametric polymorphism, one in which the same data representation is used for many types of data and the other in which explicit instantiation of parametric code is used to match each different data representation.

The difference between parametric polymorphism and overloading is that parametric polymorphism allows one algorithm to be given many types, whereas overloading involves different algorithms. For example, the function + is overloaded in many languages. In an expression adding two integers, the integer addition algorithm is used. In adding two floating-point numbers, a completely different algorithm is used for computing the sum.

Type Declarations and Type Equality

We discussed opaque and transparent type declarations. In opaque type declarations, the type name stands for a distinct type different from all other types. In transparent type declarations, the declared name is a synonym for another type. Both forms are used in many programming languages.

Team-Fly



Team-Fly

Chapter 7: Scope, Functions, and Storage Management

In this chapter storage management for block-structured languages is described by the run-time data structures that are used in a simple, reference implementation. The programming language features that make the association between program names and memory locations interesting are scope, which allows two syntactically identical names to refer to different locations, and function calls, which each require a new memory area in which to store function parameters and local variables. Some important topics in this chapter are parameter passing, access to global variables, and a storage optimization associated with a particular kind of function call called a *tail call.* We will see that storage management becomes more complicated in languages with nested function declarations that allow functions to be passed as arguments or returned as the result of function calls.

7.1 BLOCK-STRUCTURED LANGUAGES

Most modern programming languages provide some form of block. A *block* is a region of program text, identified by begin and end markers, that may contain declarations local to this region. Here are a few lines of C code to illustrate the idea:

```
outer block {int x = 2;
             { int y = 3;   } inner block
               x = y+2;     }
             }
}
```

In this section of code, there are two blocks. Each block begins with a left brace, {,and ends with a right brace,}. The outer block begins with the first left brace and ends with the last right brace. The inner block is nested inside the outer block. It begins with the second left brace and ends with the first right brace. The variable x is declared in the outer block and the variable y is declared in the inner block. A variable declared within a block is said to be *local* to that block. A variable declared in an enclosing block is said to be *global* to the block. In this example, x is local to the outer block, y is local to the inner block, and x is global to the inner block.

C, Pascal, and ML are all block-structured languages. In-line blocks are delineated by { … } in C, begin…end in Pascal, and let…in…end in ML. The body of a procedure or function is also a block in each of these languages.

Storage management mechanisms associated with block structure allow functions to be called recursively.

The versions of Fortran in widespread use during the 1960s and 1970s were not block structured. In historical Fortran, every variable, including every parameter of each procedure (called a subroutine in Fortran) was assigned a fixed-memory location. This made it *impossible* to call a procedure recursively, either directly orindirectly. If Fortran procedure P calls Q, Q calls R, and then R attempts to call P, the second call to P is not allowed. If P were called a second time in this call chain, the second call would write over the parameters and return address for the first call. This would make it impossible for the call to return properly.

Block-structured languages are characterized by the following properties:

- New variables may be declared at various points in a program.
- Each declaration is visible within a certain region of program text, called a block.

Blocks may be nested, but cannot partially overlap. In other words, if two blocks contain any expressions or statements in common, then one block must be entirely contained within the other.

- When a program begins executing the instructions contained in a block at runtime, memory is allocated for the variables declared in that block.
- When a program exits a block, some or all of the memory allocated to variables declared in that block will be deallocated.
- An identifier that is not declared in the current block is considered global to the block and refers to the entity with this name that is declared in the closest enclosing block.

Although most modern general-purpose programming languages are block structured, many important languages do not provide full support for all combinations of block-structured features. Most notably, standard C and C++ do not allow local function declarations within nested blocks and therefore do not address implementation issues associated with the return of functions from nested blocks.

In this chapter, we look at the memory management and access mechanisms for three classes of variables:

- *local variables*, which are stored on the stack in the activation record associated with the block
- *parameters* to function or procedure blocks, which are also stored in the activation record associated with the block
- *global variables*, which are declared in some enclosing block and therefore must be accessed from an activation record that was placed on the run-time stack before activation of the current block.

It may seem surprising that most complications arise in connection with access to global variables. However, this is really a consequence of stack-based memory management: The stack is used to make it easy to allocate and access local variables. In placing local variables close at hand, a global variable may be buried on the stack under any number of activation records.

Simplified Machine Model

We use the simplified machine model in Figure 7.1 to look at the memory management in block-structured languages.

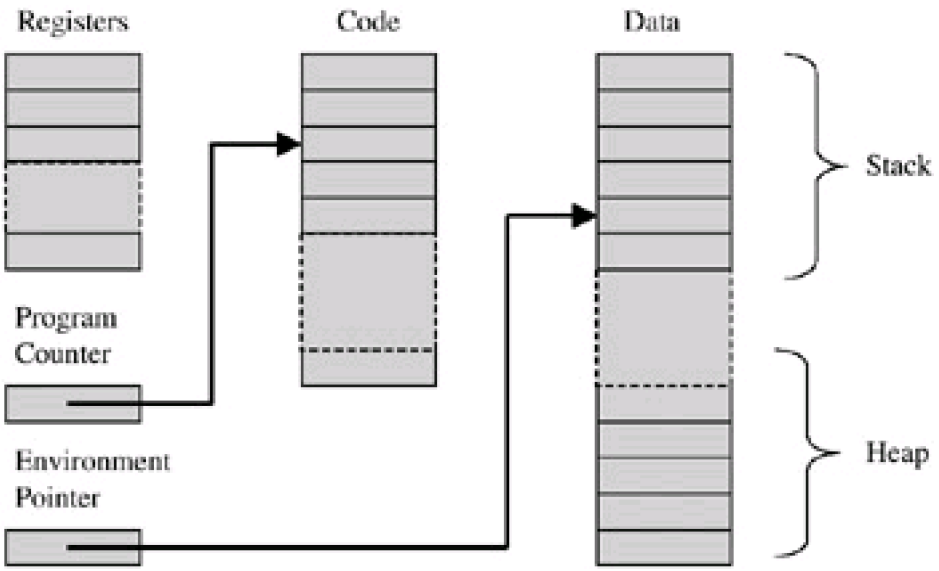


Figure 7.1: Program stack.

The machine model in Figure 7.1 separates code memory from data memory. The program counter stores the address of the current program instruction and is normally incremented after each instruction. When the program enters a new block, an *activation record* containing space for local variables declared in the block is added to the run-time stack (drawn here at the top of data memory), and the environment pointer is set to point to the new activation record. When the program exits the block, the activation record is removed from the stack and the

environment pointer is reset to its previous location. The program may store data that will exist longer than the execution of the current block on the heap. The fact that the most recently allocated activation record is the first to be deal located is sometimes called the *stack discipline*. Although most block-structured languages are implemented by a stack, higher-order functions may cause the stack discipline to fail.

Although Figure 7.1 includes some number of registers, generally used for short-term storage of addresses and data, we will not be concerned with registers or the instructions that may be stored in the code segment of memory.

Reference Implementation. A reference implementation is an implementation of a language that is designed to define the behavior of the language. It need not be an efficient implementation. The goal in this chapter is to give you enough information about how blocks are implemented in most programming languages so that you can understand when storage needs to be allocated, what kinds of data are stored on the run-time stack, and how an executing program accesses the data locations it needs. We do this by describing a reference implementation. Because our goal is to understand programming languages, not build a compiler, this reference implementation will be simple and direct. More efficient methods for doing many of the things described in this chapter, tailored for specific languages, may be found in compiler books.

A Note about C

The C programming language is designed to make C easy to compile and execute, avoiding several of the general scoping and memory management techniques described in this chapter. Understanding the general cases considered here will give C programmers some understanding of the specific ways in which C is simpler than other languages. In addition, C programmers who want the effect of passing functions and their environments to other functions may use the ideas described in this chapter in their programs.

Some commercial implementations of C and C++ actually do support function parameters and return values, with preservation of static scope by use of closures.(We will discuss closures in Section 7.4.) In addition, the C++ Standard Template Library (covered in Subsection 9.4.3) provides a form of function closure as many programmers find function arguments and return values useful.

Team-Fly

7.2 IN-LINE BLOCKS

An in-line block is a block that is not the body of a function or procedure. We study in-line blocks first, as these are simpler than blocks associated with function calls.

7.2.1 Activation Records and Local Variables

When a running program enters an in-line block, space must be allocated for variables that are declared in the block. We do this by allocating a set of memory locations called an *activation record* on the run-time stack. An activation record is also sometimes called a *stack frame*.

To see how this works, consider the following code example. If this code is part of a larger program, the stack may contain space for other variables before this block is executed. When the outer block is entered, an activation record containing space for x and y is pushed onto the stack. Then the statements that set values of x and y will be executed, causing values of x and y to be stored in the activation record. On entry into the inner block, a separate activation record containing space for z will be added to the stack. After the value of z is set, the activation record containing this value will be popped off the stack. Finally, on exiting the outer block, the activation record containing space for x and y will be popped off the stack:

```
{ int x=0;
  int y=x+1;
     { int z=(x+y)*(x-y);
       };
};
```

We can visualize this by using a sequence of figures of the stack. As in Figure 7.1, the stack is shown growing downward in memory in Figure 7.2.

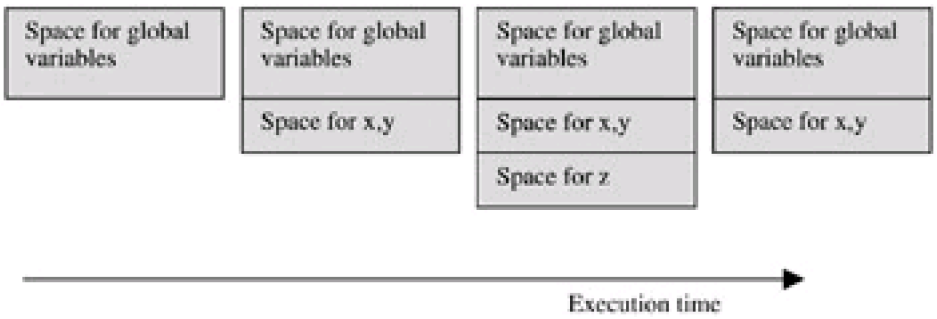


Figure 7.2: Stack grows and shrinks during program execution.

A simple optimization involves combining small nested blocks into a single block. For the preceding example program, this would save the run time spent in pushing and popping the inner block for z, as z could be stored in the same activation record as that of x and y. However, because we plan to illustrate the general case by using small examples, we do not use this optimization in further discussion of stack storage allocation. In all of the program examples we

consider, we assume that a new activation record is allocated on the run-time stack each time the program enters a block.

The number of locations that need to be allocated at run time depends on the number of variables declared in the block and their types. Because these quantities are known at compile time, the compiler can determine the format of each activation record and store this information as part of the compiled code.

Intermediate Results

In general, an activation record may also contain space for intermediate results. These are values that are not given names in the code, but that may need to be saved temporarily. For example, the activation record for this block,

```
{ int z = (x+y)*(x-y);
}
```

may have the form

Space for z
Space for x+y
Space for x–y

because the values of subexpressions x+y and x–y may have to be evaluated and stored some where before they are multiplied.

On modern computers, there are enough registers that many intermediate results are stored in registers and not placed on the stack. However, because register allocation is an implementation technique that does not affect programming language design, we do not discuss registers or register allocation.

Scope and Lifetime

It is important to distinguish the scope of a declaration from the lifetime of a location:

> *Scope*: a region of text in which a declaration is visible.
>
> *Lifetime*: the duration, during a run of a program, during which a location is allocated as the result of a specific declaration.

We may compare lifetime and scope by using the following example, with vertical lines used to indicate matching block entries and exits.

```
{ int x = . . . ;
|   { int y = . . . ;
|   |   { int x = . . . ;
|   |   |   . . . .
|   |   };
|   };
};
```

In this example, the inner declaration of x hides the outer one. The inner block is called a *hole in the scope* of the outer declaration of x, as the outer x cannot be accessed within the inner block. This example shows that lifetime does not coincide with scope because the life time of the outer x includes time when inner block is being executed, but the scope of the outer x does not include the scope of the inner one.

Blocks and Activation Records for ML

Throughout our discussion of blocks and activation records, we follow the convention that, whenever the program enters a new block, a new activation record is allocated on the run-time stack. In ML code that has sequences of declarations, we treat each declaration as a separate block. For example, in the code

```
fun f(x) = x+1;
fun g(y) = f(y) +2;
g(3);
```

we consider the declaration of f one block and the declaration of g another block inside the outer block. If this code is not inside some other construct, then these blocks will both end at the end of the program.

When an ML expression contains declarations as part of the *let-in-end* construct, we consider the declarations to be part of the same block. For example, consider this example expression:

```
let fun g(y) = y+3
    fun h(z) = g(g(z))
in
    h(3)
end;
```

This expression contains a block, beginning with let and ending with end. This block contains two declarations, functions g and h, and one expression, h(x), calling one of these functions. The construct let … in … end is approximately equivalent to { … ; … } in C. The main syntactic difference is that declarations appear between the keywords let and in, and expressions using these declarations appear between keywords in and end. Because the declarations of functions g and h appear in the same block, the names g and h will be given values in the same activation record.

7.2.2 Global Variables and Control Links

Because different activation records have different sizes, operations that push and pop activation records from the run-time stack store a pointer in each activation record to the top of the preceding activation record. The pointer to the top of the previous activation record is called the *control link*, as it is the link that is followed when control returns to the instructions in the preceding block. This gives us a structure shown in Figure 7.3. Some authors call the control link the *dynamic link* because the control links mark the dynamic sequence of function calls created during program execution.

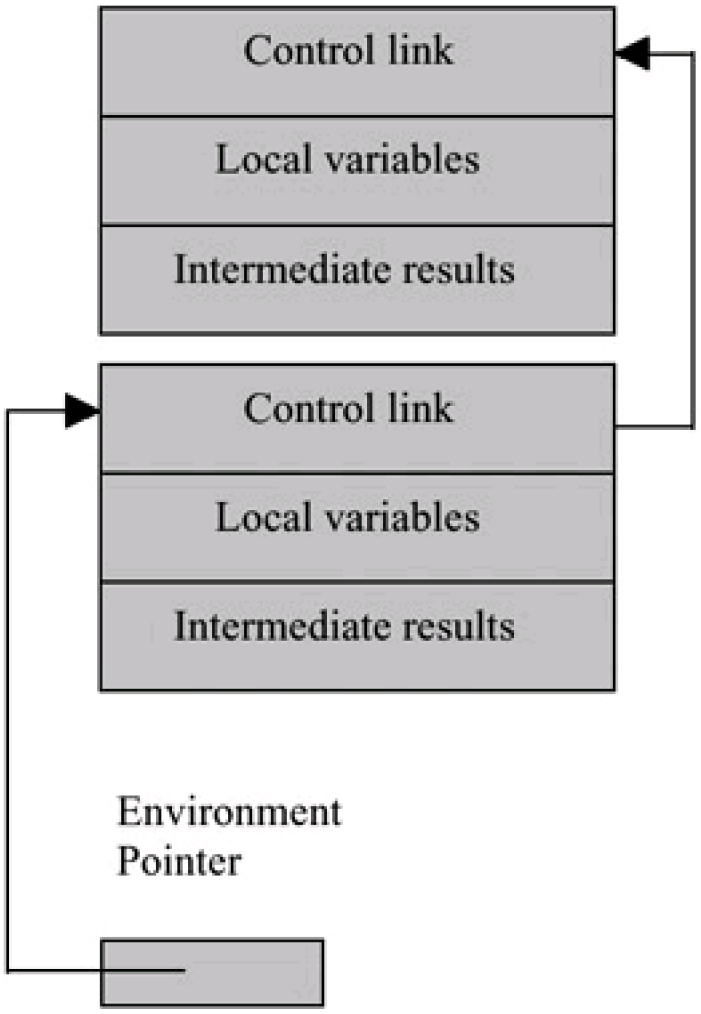


Figure 7.3: Activation records with control links.

When a new activation record is added to the stack, the control link of the new activation record is set to the previous value of the environment pointer, and the environment pointer is updated to point to the new activation record. When an activation record is popped off the stack, the environment pointer is reset by following the control link from the activation record.

The code for pushing and popping activation records from the stack is generated by the compiler at compile time and becomes part of the compiled code for the program. Because the size of an activation record can be determined from the text of the block, the compiler can generate code for block entry that is tailored to the size of the activation record.

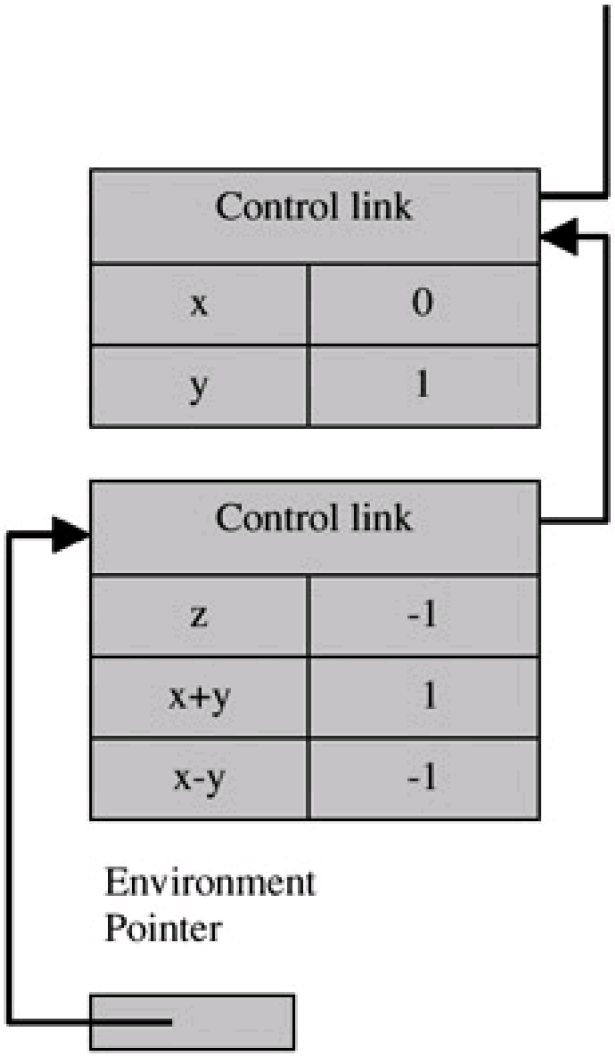
When a global variable occurs in an expression, the compiler must generate code that will find the location of that variable at run time. However, the compiler can compute the number of blocks between the current block and the block where the variable is declared; this is easily determined from the program text. In addition, the relative position of each variable within its block is known at compile time. Therefore, the compiler can generate lookup code that follows a predetermined number of links

Example 7.1

```
{ int x=0;
  int y=x+1;
     { int z=(x+y)*(x-y);
     };
     };
```

When the expressions x+y and x–y are evaluated during execution of this code, the run-time stack will have activation records for the inner and outer blocks as shown below:

Control link Control link z -1x+y



On a register-based machine, the machine code generated by the compiler will find the variables x and y, load each into registers, and then add the two values. The code for loading x uses the environment pointer to find the top of the current activation, then computes the location of x by adding 1 to the location stored in the control link of the current activation record. The compiler generates this code by analyzing the program text at compile time: The variable x is declared one block out from the current block, and x is the first variable declared in the block. Therefore, the control link from the current activation record will lead to the activation record containing x, and the location of x will be one location down from the top of that block. Similar steps can be used to find y at the second location down from the top of its activation record. Although the details may vary from one compiler to the next, the main point is that the compiler can determine the number of control links to follow and the relative location of the variable within the correct block from the source code. In particular, it is *not* necessary to store variable names in activation records.

Team-Fly

Team-Fly

7.3 FUNCTIONS AND PROCEDURES

Most block-structured languages have procedures or functions that include parameters, local variables, and a body consisting of an arbitrary expression or sequence of statements. For example, here are representative Algol-like and C-like forms:

```
procedure P(<parameters>)        <type> f(<parameters>)
begin                            {
  <local variables>;               <local variables>;
  <procedure body>;              <function body>;
end;                             };
```

The difference between a *procedure* and a *function* is that a function has a return value but a procedure does not. In most languages, functions and procedures may have side effects. However, a procedure has only side effects; a procedure call is a statement and not an expression. Because functions and procedures have many characteristics in common, we use the terms almost interchangeably in the rest of this chapter. For example, the text may discuss some properties of functions, and then a code example may illustrate these properties with a procedure. This should remind you that the discussion applies to functions and procedures in many programming languages, whether or not the language treats procedures as different from functions.

7.3.1 Activation Records for Functions

The activation record of a function or procedure block must include space for parameters and return values. Because a procedure may be called from different call sites, it is also necessary to save the return address, which is the location of the next instruction to execute after the procedure terminates. For functions, the activation record must also contain the location that the calling routine expects to have filled with the return value of the function.

The activation record associated with a function (see Figure 7.4) must contain space for the following information:

- *control link*, pointing to the previous activation record on the stack,
- *access link*, which we will discuss in Subsection 7.3.3,
- *return address*, giving the address of the first instruction to execute when the function terminates,
- *return-result address*, the location in which to store the function return value,
- *actual parameters* of the function,
- *local variables* declared within the function,
- *temporary storage* for intermediate results computed with the function executes.

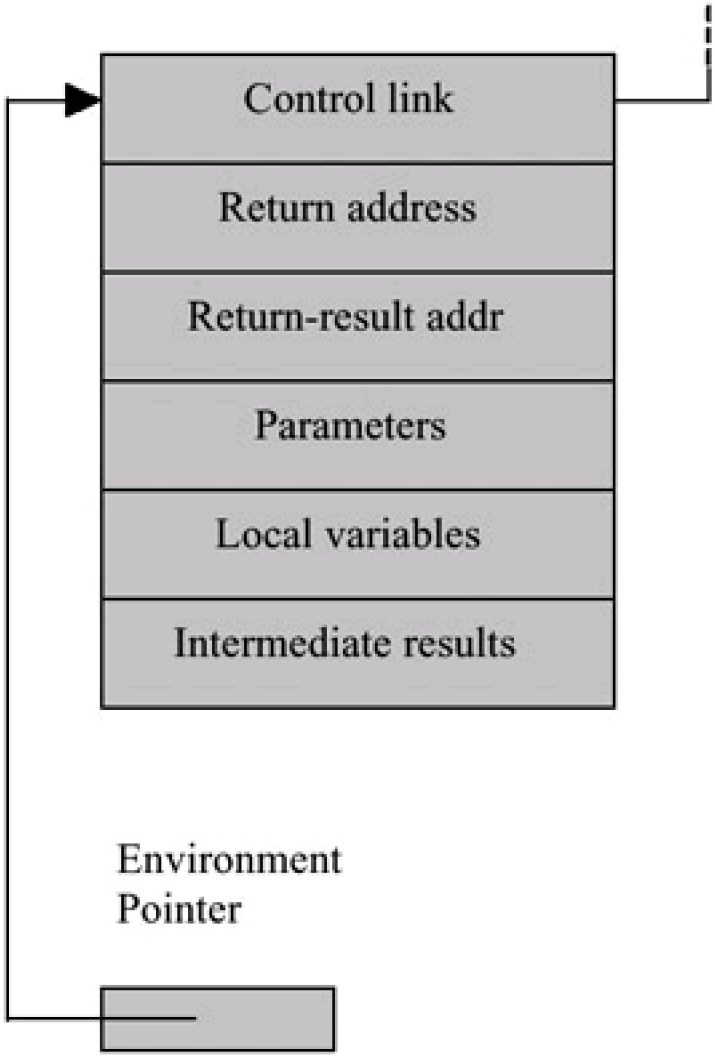


Figure 7.4: Activation record associated with function call.

This information may be stored in different orders and in different ways in different language implementations. Also, as mentioned earlier, many compilers perform optimizations that place some of these values in registers. For concreteness, we assume that no registers are used and that the six components of an activation record are stored in the order previously listed.

Although the names of variables are eliminated during compilation, we often draw activation records with the names of variables in the stack. This is just to make it possible for us to understand the figures.

Example 7.2

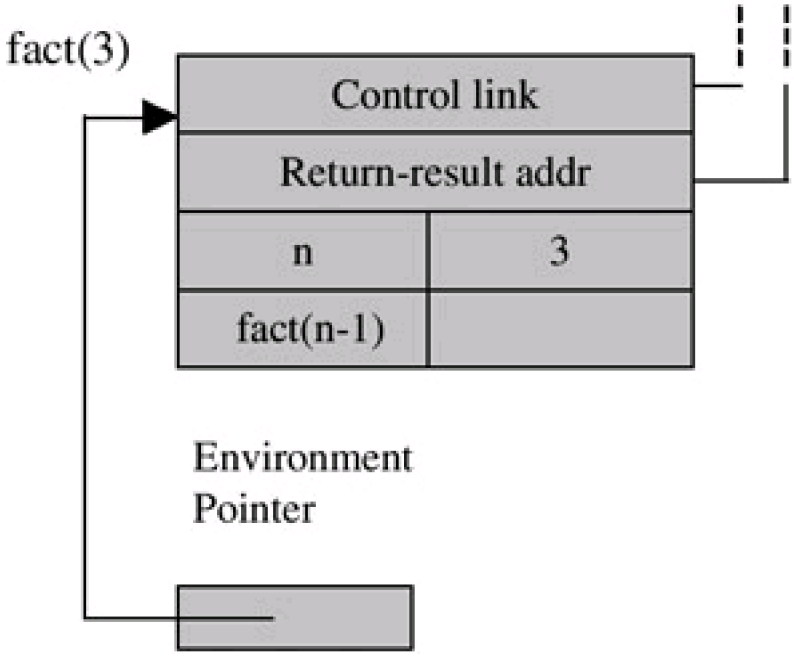
We can see how function activation records are added and removed from the run time stack by tracing the execution of the familiar factorial function:

```
fun fact(n) = if n <= 1 then 1 else n * fact(n-1);
```

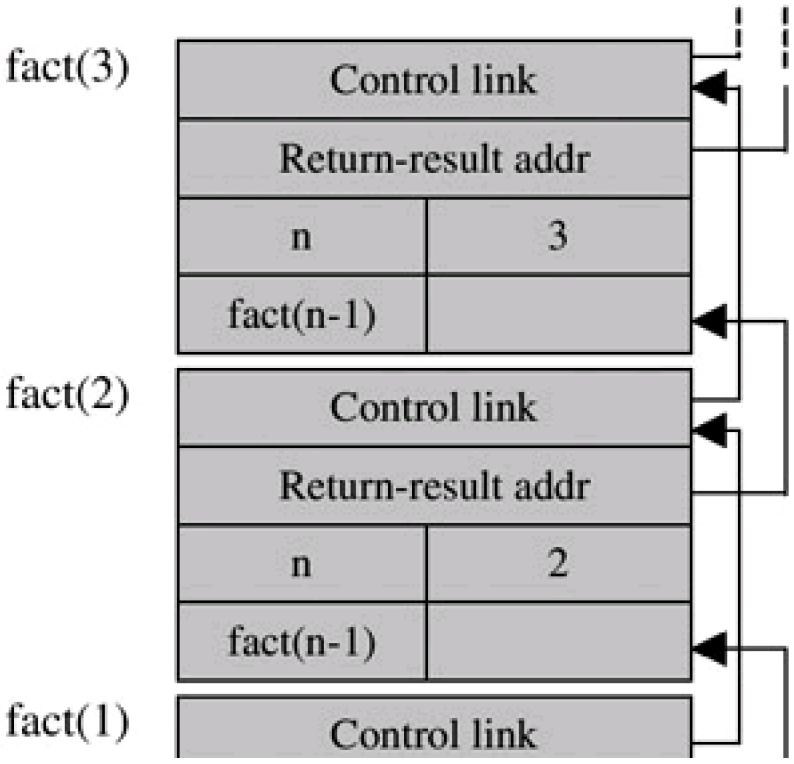
Suppose that some block contains the expression fact(3)+1, leading to a call of fact(3). Let us assume that the activation record of the calling block contains a location that will be used to store the value of fact(3) before computing fact(3)+1.

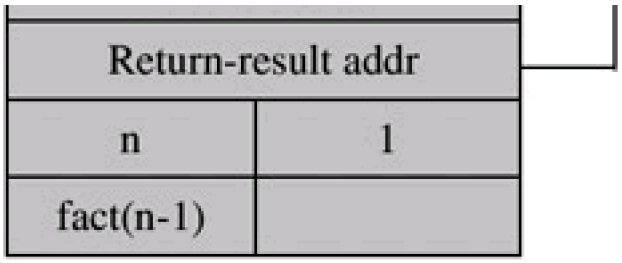
The next activation record that is placed on the stack will be an activation record for the call fact(3). In this activation record, shown after the list,

- the control link points to the activation record of the block containing the call fact(3),
- the return-result link points to the location in the activation record of the calling block that has been allocated for the intermediate value fact(3) of the calling expression fact(3)+1,
- the actual parameter value 3 is placed in the location allocated for the formal parameter n,
- a location is allocated for the intermediate value fact(n−1) that will be needed when n>0.



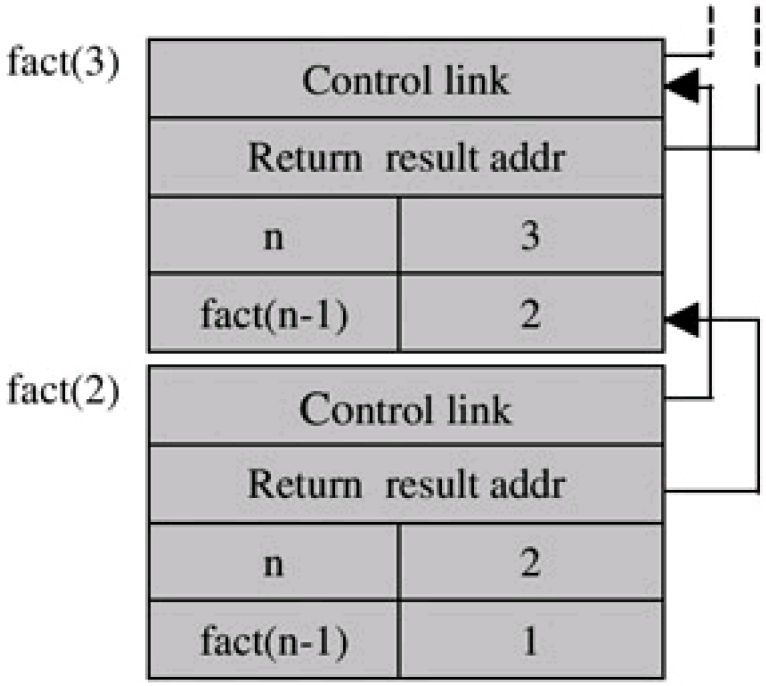
After this activation record is allocated on the stack, the code for factorial is executed. Because n>0, there is a recursive call fact(2). This leads to a recursive call fact(1), which results in a series of activation records, as shown in the subsequent figure.





Note that in each of the lower activation records, the return-result address points to the space allocated in the activation record above it. This is so that, on return from fact(1), for example, the return result of this call can be stored in the activation record for fact(2). At that point, the final instruction from the calculation of fact(2) will be executed, multiplying local variable n by the intermediate result fact(1).

The final illustration of this example shows the situation during return from fact(2) when the return result of fact(2) has been placed in the activation record of fact(3), but the activation record for fact(2) has not yet been popped off the stack.



7.3.2 Parameter Passing

The parameter names used in a function declaration are called *formal parameters*. When a function is called, expressions called *actual parameters* are used to compute the parameter values for that call. The distinction between formal and actual parameters is illustrated in the following code:

```
proc p (int x, int y) {
   if (x > y) then ... else ... ;
   ...
   x:= y*2 + 3;
   ...
}
p (z, 4*z+1);
```

The identifiers x and y are formal parameters of the procedure p. The actual parameters in the call to P are z and 4*z+1.

The way that actual parameters are evaluated and passed to the function depends on the programming language and the kind of parameter-passing mechanisms it uses. The main distinctions between different parameter-passing mechanisms are

- the time that the actual parameter is evaluated
- the location used to store the parameter value.

Most current programming languages evaluate the actual parameters before executing the function body, but there are some exceptions. (One reason that a language or program optimization might wish to delay evaluation of an actual parameter is that evaluation might be expensive and the actual parameter might not be used in some calls.) Among mechanisms that evaluate the actual parameter before executing the function body, the most common are

- *Pass-by-reference*: pass the L-value (address) of the actual parameter
- *Pass-by-value*: pass the R-value (contents of address) of the actual parameter

Recall that we discussed L-values and R-values in Subsection 5.4.5 in connection with ML reference cells (assignable locations) and assignment. We will discuss how pass-by-value and pass-by-reference work in more detail below. Other mechanisms such as *pass-by-value-result* are covered in the exercises.

The difference between pass-by-value and pass-by-reference is important to the programmer in several ways:

Side Effects. Assignments inside the function body may have different effects under pass-by-value and pass-by-reference.

Aliasing. Aliasing occurs when two names refer to the same object or location. Aliasing may occur when two parameters are passed by reference or one parameter passed by reference has the same location as the global variable of the procedure.

Efficiency. Pass-by-value may be inefficient for large structures if the value of the large structure must be copied. Pass-by-reference may be less efficient than pass-by-value for small structures that would fit directly on stack, because when parameters are passed by reference we must dereference a pointer to get their value.

There are two ways of explaining the semantics of call-by-reference and call-by-value. One is to draw pictures of computer memory and the run-time program stack, showing whether the stack contains a copy of the actual parameter or a reference to it. The other explanation proceeds by translating code into a language that distinguishes between L- and R-values. We use the second approach here because the rest of the chapter gives you ample opportunity to work with pictures of the run-time stack.

Semantics of Pass-by-Value

In pass-by-value, the actual parameter is evaluated. The value of the actual parameter is then stored in a new location allocated for the function parameter. For example, consider this function definition and call:

```
function f (x) = { x := x+1; return x };
. . . .f(y)...;
```

If the parameter is passed by value and y is an integer variable, then this code has the same meaning as the following ML code:

```
fun f (z : int) = let x = ref z in x := !x+1; !x end;
. . .f(!y)...;
```

As you can see from the type, the value passed to the function f is an integer. The integer is the R-value of the actual parameter y, as indicated by the expression !y in the call. In the body of f, a new integer location is allocated and initialized to the R-value of y.

If the value of y is 0 before the call, then the value of f(!y) is 1 because the function f increments the parameter and returns its value. However, the value of y is still 0 after the call, because the assignment inside the body of f changes the contents of only a temporary location.

Semantics of Pass-by-Reference

In pass-by-reference, the actual parameter must have an L-value. The L-value of the actual parameter is then bound to the formal parameter. Consider the same function definition and call used in the explanation of pass-by-value:

```
function f (x) = { x := x+1; return x };
. . . .f(y) . . .;
```

If the parameter is passed by reference and y is an integer variable, then this code has the same meaning as the following ML code:

```
fun f (x : int ref) = ( x := !x+1; !x );
. . .f(y)
```

As you can see from the type, the value passed to the function f is an integer reference(L-value).

If the value of y is 0 before the call, then the value of f(!y) is 1 because the function f increments the parameter and returns its value. However, unlike the situation for pass-by-value, the value of y is 1 after the call because the assignment inside the body of f changes the value of the actual parameter.

Example 7.3

Here is an example, written in an Algol-like notation, that combines pass-by-reference and pass-by-value:

```
fun f(pass-by-ref x : int, pass-by-value y : int)
   begin
      x:= 2;
      y:= 1;
      if x = 1 then return 1 else return 2;
   end;
var z : int;
z := 0;
print f(z,z);
```

Translating the preceding pseudo-Algol example into ML gives us

```
fun f(x : int ref, y : int) =
   let val yy = ref y in
      x := 2;
      yy := 1;
      if (!x = 1) then 1 else 2
   end;
val z = ref 0;
f(z,!z);
```

This code, which treats L-and R-values explicitly, shows that for pass-by-reference we pass an L-value, the integer reference `z`. For pass-by-value, we pass an R-value, the contents `!z` of `z`. The pass-by-value is assigned a new temporary location.

With `y` passed by value as written, `z` is assigned the value 2. If `y` is instead passed by reference, then `x` and `y` are aliases and `z` is assigned the value 1.

Example 7.4

Here is a function that tests whether its two parameters are aliases:

```
function (y,z){
   y:= 0;
   z :=0;
   y:= 1;
   if z = 1 then y :=0; return 1 else y :=0; return 0
}
```

If y and z are aliases, then setting y to 1 will set z to 1 and the function will return 1. Otherwise, the function will return 0. Therefore, a call f(x,x) will behave differently if the parameters are pass-by-value than if the parameters are pass-by-reference.

7.3.3 Global Variables (First-Order Case)

If an identifier x appears in the body of a function, but x is not declared inside the function, then the value of x depends on some declaration outside the function. In this situation, the location of x is outside the activation record for the function. Because x must be declared in some other block, access to a global x involves finding an appropriate activation record elsewhere on the stack.

There are two main rules for finding the declaration of a global identifier:

- *Static Scope*: A global identifier refers to the identifier with that name that is declared in the closest enclosing scope of the program text.
- *Dynamic Scope*: A global identifier refers to the identifier associated with the most recent activation record.

These definitions can be tricky to understand, so be sure to read the examples below carefully. One important difference between static and dynamic scope is that finding a declaration under static scope uses the static (unchanging) relationship between blocks in the program text. In contrast, dynamic scope uses the actual sequence of calls that are executed in the dynamic (changing) execution of the program.

Although most current general-purpose programming languages use static scope for declarations of variables and functions, dynamic scoping is an important concept that is used in special-purpose languages and for specialized constructs such as exceptions.

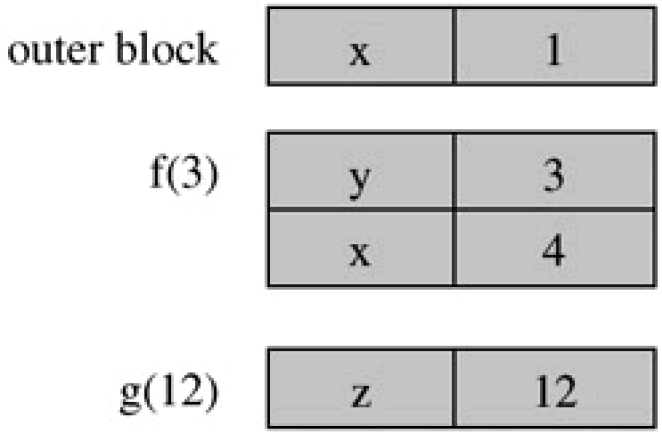
Dynamically Scoped	Statically Scoped
Older Lisps	Newer Lisps, Scheme
TeX/LaTeX document languages	Algol and Pascal
Exceptions in many languages	C
Macros	ML
	Other current languages

Example 7.5

The difference between static and dynamic scope is illustrated by the following code, which contains two declarations of x:

```
int x=1;
function g(z) = x+z;
function f(y) = {
    int x = y+1;
    return g(y*x)
};
f(3);
```

The call f(3) leads to a call g(12) inside the function f. This causes the expression x+z in the body of g to be evaluated. After the call to g, the run-time stack will contain activation records for the outer declaration of x, the invocation of f, and the invocation of g, as shown in the following illustration.



At this point, two integers named x are stored on the stack, one from the outer block declaring x and one from the local declaration of x inside f. Under dynamic scope, the identifier x in the expression x+z will be interpreted as the one from the most recently created activation record, namely x=4. Under static scope, the identifier x in x+z will refer to the declaration of x from the closest program block, looking upward from the place that x+z appears in the program text. Under static scope, the relevant declaration of x is the one in the outer block, namely x=1.

Access Links are Used to Maintain Static Scope

The *access link* of an activation record points to the activation record of the closest enclosing block in the program. In-line blocks do not need an access link, as the closest enclosing block will be the most recently entered block - for in-line blocks, the control link points to the closest enclosing block. For functions, however, the closest enclosing block is determined by where the function is declared. Because the point of declaration is often different from the point at which a function is called, the access link will generally point to a different activation record than the control link. Some authors call the access link the *static link*, as the access links represent the static nesting structure of blocks in the source program.

The general format for activation records with an access link is shown in Figure 7.5.

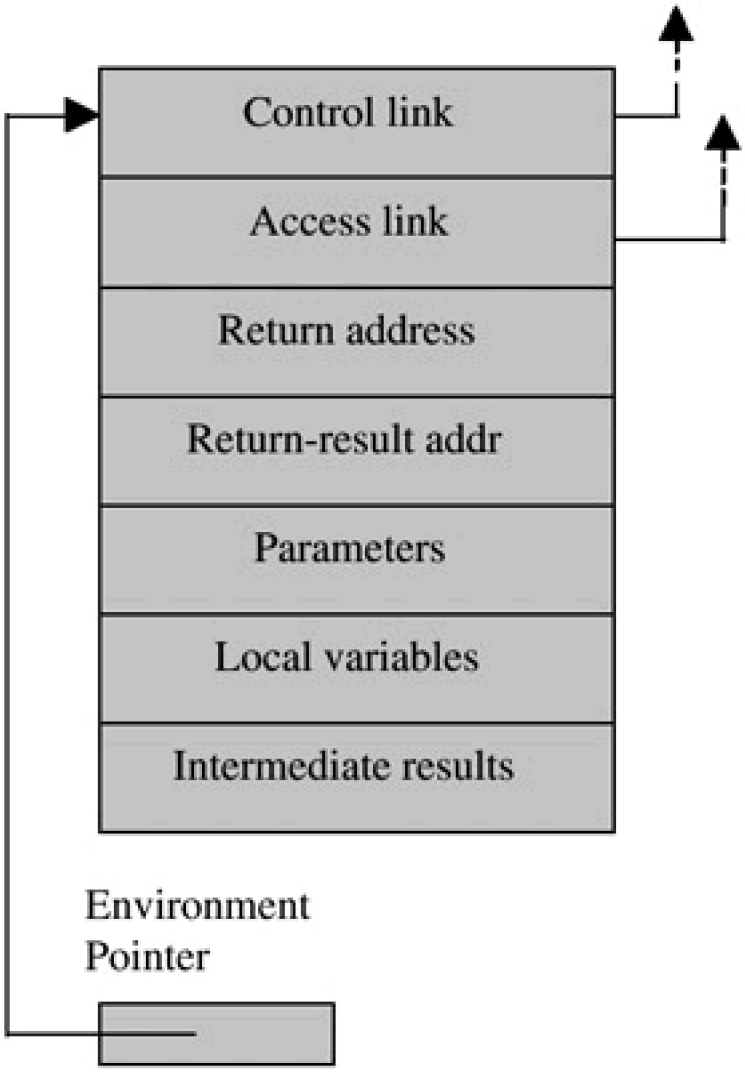


Figure 7.5: Activation record with access link for functions call with static scope.

Example 7.6

Let us look at the activation records and access links for the example code from Example 7.5, treating each ML declaration as a separate block.

Figure 7.6 shows the run-time stack after the call to g inside the body of f. As always, the control links each point to the preceding activation record on the stack.

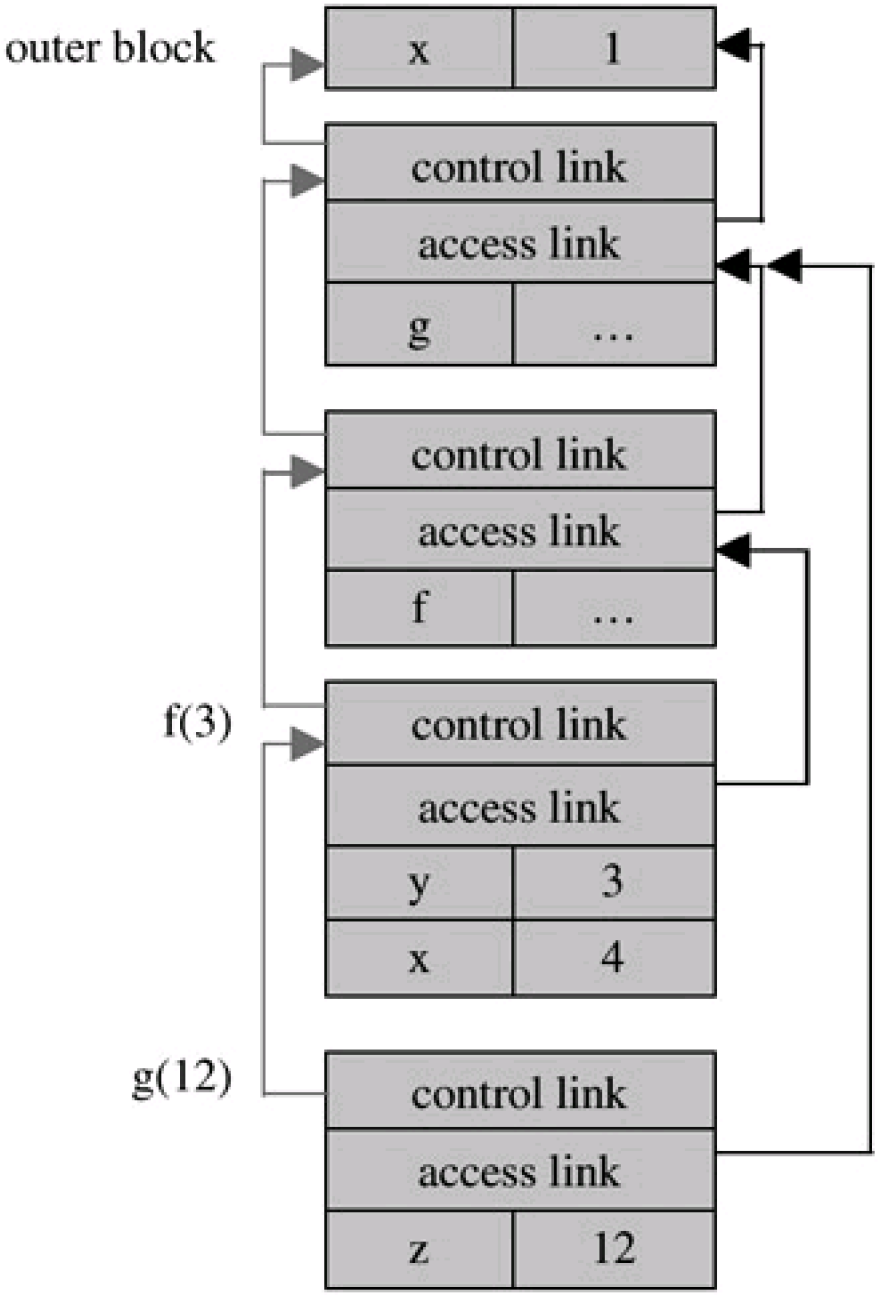


Figure 7.6: Run-time stack after call to g inside f.

The control links are drawn on the left here to leave room for the access links on the right. The access link for each block points to the activation record of the closest enclosing block in the program text. Here are some important points about this illustration, which follows our convention that we begin a new block for each ML declaration.

- The declaration of g occurs inside the scope of the declaration of x. Therefore, the access link for the declaration of g points to the activation record for the declaration of x.
- The declaration of f is similarly inside the scope of the declaration of g. Therefore, the access link for the declaration of f points to the activation record for the declaration of g.
- The call f(3) causes an activation record to be allocated for the scope associated with the body of f. The body of f occurs inside the scope of the declaration of f. Therefore, the access link for f(3) points to the activation record for the declaration of f.
- The call g(12) similarly causes an activation record to be allocated for the scope associated with the body of g. The body of g occurs inside the scope of the declaration of g. Therefore, the access link for g(12) points to the activation record for the declaration of g.
- We evaluate the expression x+z by adding the value of the parameter z, stored locally in the activation

record of g, to the value of the global variable x. We find the location of the global variable x by following the access link of the activation of g> to the activation record associated with the declaration of g. We then follow the access link in that activation record to find the activation record containing the variable x.

As for in-line blocks, the compiler can determine how many access links to follow and where to find a variable within an activation record at compile time. These properties are easily determined from the structure of the source code.

To summarize, the *control* link is a link to the activation record of the previous(calling) block. The *access link* is a link to the activation record of the closest enclosing block in program text. The control link depends on the dynamic behavior of program whereas the access link depends on only the static form of the program text. Access links are used to find the location of global variables in statically scoped languages with nested blocks at run time.

Access links are needed only in programming languages in which functions maybe declared inside functions or other nested blocks. In C, in which all functions are declared in the outermost global scope, access links are not needed.

7.3.4 Tail Recursion (First-Order Case)

In this subsection we look at a useful compiler optimization called tail recursion elimination. For tail recursive functions, which are subsequently described, it is possible to reuse an activation record for a recursive call to the function. This reduces the amount of space used by a recursive function.

The main programming language concept we need is the concept of tail call. Suppose function f calls function g. Functions f and g might be different functions or f and g could be the same function. A call to f in the body of g is a *tail call* if g returns the result of calling f without any further computation. For example, in the function

```
fun g(x) = if x=0 then f(x) else f(x)*2
```

the first call to f in the body of g is a tail call, as the return value of g is exactly the return value of the call to f. The second call to f in the body of g is not a tail call because g performs a computation involving the return value of f before g returns.

A function f is *tail recursive* if all recursive calls in the body of f are tail calls to f.

Example 7.7

Here is a tail recursive function that computes factorial:

```
fun tlfact(n,a) = if n <= 1 then a else tlfact(n-1,n*a);
```

More specifically, for any positive integer n, tlfact(n,a) returns n!.. We can see that tlfact is a tail recursive function because the only recursive call in the body of tlfact is a tail call.

The advantage of tail recursion is that we can use the same activation record for all recursive calls. Consider the call

tlfact(3,1). Figure 7.7 shows the parts of each activation record in the computation that are relevant to the discussion.

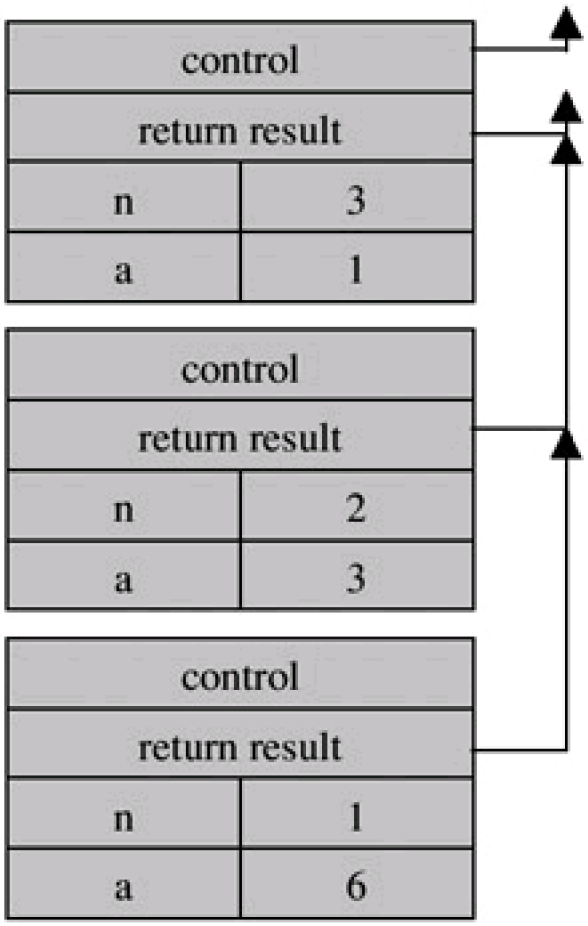


Figure 7.7: Three calls to tail recursive tlfact without optimization.

After the third call terminates, it passes its return result back to the second call, which then passes the return result back to the first call. We can simplify this process by letting the third call return its result to the activation that made the original call, tlfact(3,1). Under this alternative execution, when the third call finishes, we can pop the activation record for the second call as well, as we will no longer need it. In fact, because the activation record for the first call is no longer needed when the second call begins, we can pop the first activation record from the stack before allocating the second. Even better, instead of deallocating the first and then allocating a second activation record identical to the first, we can use one activation record for all three calls.

Figure 7.8 shows how the same activation record can be used used three times for three successive calls to tail recursive tlfact. The figure shows the contents of this activation record for each call. When the first call, tlfact(3,1), begins, an activation record with parameters (1,3) is created. When the second call, tlfact(2,3), begins, the parameter values are changed to (2,3). Tail recursion elimination reuses a single activation record for all recursive calls, using assignment to change the values of the function parameters on each call.

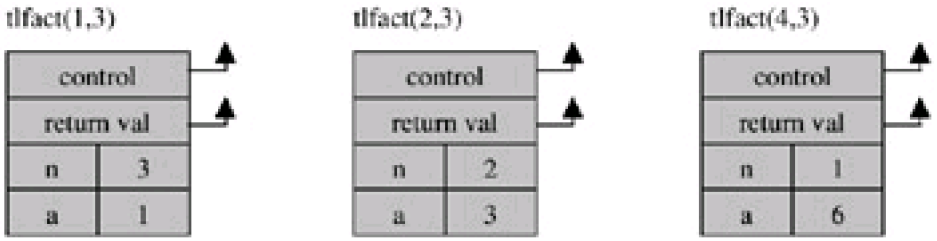


Figure 7.8: Three calls to tail recursive tlfact *with* optimization.

7.4 Tail Recursion as Iteration

When a tail recursive function is compiled, it is possible to generate code that computes the function value by use of

an iterative loop. When this optimization is used, tail recursive functions can be evaluated by only one activation record for a first call and all resulting recursive calls, not one additional activation record per recursive call. For example, the preceding tail recursive factorial function may be compiled into the same code as that of the following iterative function:

```
fun itfact(n,a) = ( while !n>0do(a:=!n * !a;n:=!n-1);!a);
```

An activation record for itfact looks just like an activation record for tlfact. If we look at the values of n and a on each iteration of the loop, we find that they change in exactly the same way is for tail recursive calls to tlfact. The two functions compute the same result by exactly the same sequence of instructions. Put another way, tail recursion elimination compiles tail recursive functions into iterative loops.

Team-Fly

7.4 HIGHER-ORDER FUNCTIONS

7.4.1 First-Class Functions

A language has *first-class functions* if functions can be

- declared within any scope,
- passed as arguments to other functions, and
- returned as results of functions.

In a language with first-class functions and static scope, a function value is generally represented by a *closure*, which is a pair consisting of a pointer to function code and a pointer to an activation record.

Here is an example ML function that requires a function argument:

```
fun map (f, nil) = nil
| map(f, x::xs) = f(x) :: map(f, xs)
```

The map function take a function f and a list m as arguments, applying f to each element of m in turn. The result of map(f, m) is the list of results f(x) for elements x of the list m. This function is useful in many programming situations in which lists are used. For example, if we have a list of expiration times for a sequence of events and we want to increment each expiration time, we can do this by passing an increment function to map.

We will see why closures are necessary by considering interactions between static scoping and function arguments and return values. C and C++ do not support closures because of the implementation costs involved. However, the implementation of objects in C++ and other languages is related to the implementation of function values discussed in this chapter. The reason is that a closure and an object both combine data with code for functions.

Although some C programmers may not have much experience with passing functions as arguments to other functions, there are many situations in which this can be a useful programming method. One recognized use for functions as function arguments comes from an influential software organization concept. In systems programming, the term *upcall* refers to a function call up the stack. In an important paper called "The Structuring of Systems Using Upcalls," (ACM Symp. Operating Systems Principles, 1985) David Clark describes a method for arranging the functions of a system into layers. This method makes it easier to code, modularize, and reason about the system. As in the network protocol stack, higher layers are clients of the services provided by lower layers. In a layered file system, the file hierarchy layer is built on the vnode, which is in turn built over the inode and disk block layers. In Clark' method, which has been widely adopted and used, higher levels pass handler functions into lower levels. These handler functions are called when the lower level needs to notify the higher level of something. These calls to a higher layer are called upcalls. This influential system design method shows the value of language support for passing functions as arguments.

7.4.2 Passing Functions to Functions

We will see that when a function f is passed to a function g, we may need to pass the closure for f, which contains a pointer to its activation record. When f is called within the body of g, the environment pointer of the closure is used to set the access link in the activation record for the call to f correctly. The need for closures in this situation has sometimes been called the *downward funarg problem*, because it results from passing function as arguments downward into nested scopes.

Example 7.8

An example program with two declarations of a variable x and a function f passed to another function g is used to illustrate the main issues:

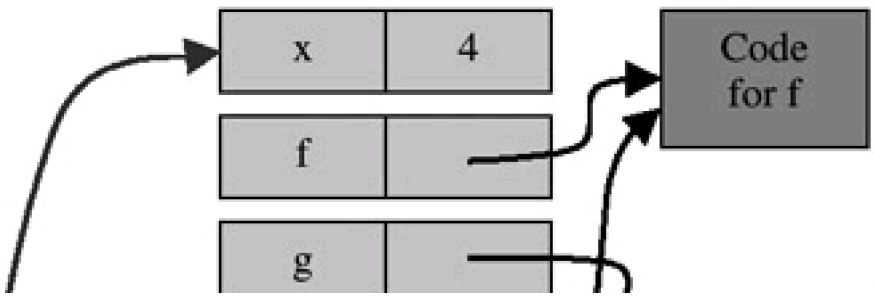
```
val x = 4;
fun f(y) = x*y;
fun g(h) = let val x=7 in h(3) + x;
g(f);
```

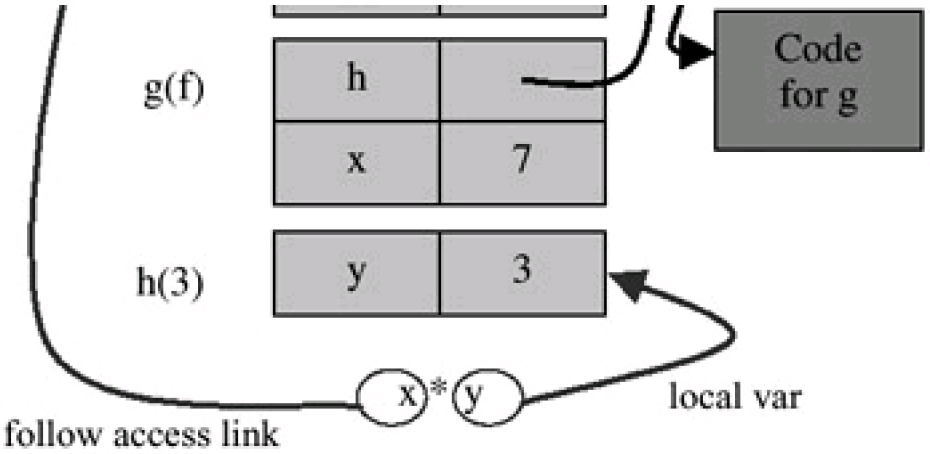
In this program, the body of f contains a global variable x that is declared outside the body of f. When f is called inside g, the value of x must be retrieved from the activation record associated with the outer block. Otherwise the body of f would be evaluated with the local variable x declared inside g, which would violate static scope. Here is the same program written with C-like syntax (except for the type expression int →int) for those who find this easier to read:

```
{ int x=4;
    {int f(int y) {return x*y;}
      {int g(int → int h){
         int x=7;
         return h(3) + x;
         }
       g(f);
}}}
```

The C-like version of the code reflects a decision, used for simplicity throughout this book, to treat each top-level ML declaration as the beginning of a separate block.

We can see the variable-lookup problem by looking at the run-time stack after the call to f from the invocation of g.





This simplified illustration shows only the data contained in each activation record. In this illustration, the expression x*y from the body of f is shown at the bottom, the activation record associated with the invocation of f (through formal parameter h of g). As the illustration shows, the variable y is local to the function and can therefore be found in the current activation record. The variable x is global, however, and located several activation records above the current one. Because we find global variables by following access links, the access link of the bottom activation record should allow us to reach the activation record at the top of the illustration.

When functions are passed to functions, we must set the access link for the activation record of each function so that we can find the global variables of that function correctly. We cannot solve this problem easily for our example program without extending some run-time data structure in some way.

Use of Closures

The standard solution for maintaining static scope when functions are passed to functions or returned as results is to use a data structure called a closure. A closure is a pair consisting of a pointer to function code and a pointer to an activation record. Because each activation record contains an access link pointing to the record for the closest enclosing scope, a pointer to the scope in which a function is declared also provides links to activation records for enclosing blocks.

When a function is passed to another function, the actual value that is passed is a pointer to a closure. The following steps are used for calling a function, given a closure:

- Allocate an activation record for the function that is called, as usual.
- Set the access link in the activation record by using the activation record pointer from the closure.

We can see how this solves the variable-access problem for functions passed to functions by drawing the activation recorrs on the run-time stack when the program in Figure 7.8 is executed. These are shown in Figure 7.9.

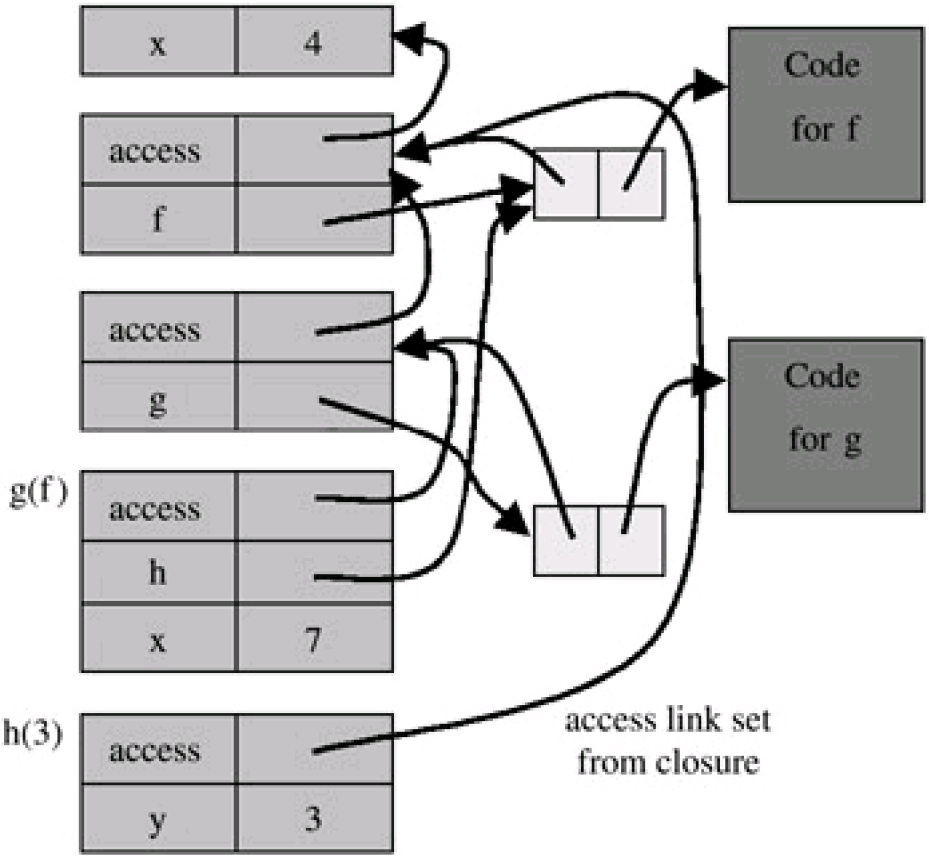


Figure 7.9: Access link set from clouser.

We can understand Figure 7.9 by walking through the sequence of run-time steps that lead to the configuration shown in the figure.

1. *Declaration of* x: An activation record for the block where x is declared is pushed onto the stack. The activation record contains a value for x and a control link (not shown).

2. *Declaration of* f: An activation record for the block where f is declared is pushed onto the stack. This activation record contains a pointer to the runtime representation of f, which is a closure with two pointers. The first pointer in the closure points to the activation record for the static scope of f, which is the activation record for the declaration of f, The second closure pointer points to the code for f, which was produced during compilation and placed at some location that is known to the compiler when code for this program is generated.

3. *Declaration of* g: As with the declaration of f, an activation record for the block where g is declared is pushed onto the stack. This activation record contains a pointer to the run-time representation of g, which is a closure.

4. *Call to* g(f): The call causes an activation record for the function g to be allocated on the stack. The size and the layout of this record are determined by the code for g. The access link is set to the activation record for the scope where g is declared; the access link points to the same activation record as the activation record in the closure for g. The activation record contains space for the parameter h and local variable x. Because the actual parameter is the closure for f, the parameter value for h is a pointer to the closure for f. The local variable x has value 7, as given in the source code.

5. *Call to* h(3): The mechanism for executing this call is the main point of this example. Because h is a formal parameter to g, the code for g is compiled without knowledge of where the function h is declared. As a result, the compiler cannot insert any instructions telling how to set the access link for the activation record for the call h(3). However, the use of closures provides a value for the access link - the access link for this activation record is set by the activation record pointer from the closure of h. Because the actual parameter is f, the access link points to the activation record for the scope where f is declared. When the code for f is executed, the access link is used to find x.

Specifically, the code will follow the access link up to the second activation record of the illustration, follow one additional access link because the compiler knew when generating code for f that the declaration of x lies one scope above the declaration of f, and find the value 4 for the global x in the body of f.

As described in step 5, the closure for f allows the code executed at run time to find the activation record containing the global declaration of x.

When we can pass functions as arguments, the access links within the stack form a tree. The structure is not linear, as the activation record corresponding to the function call h(3) has to skip the intervening activation record for g(f) to find the necessary x. However, all the access links point up. Therefore, it remains possible to allocate and deallocate activation records by use of a stack (last allocated, first deallocated)discipline.

7.4.3 Returning Functions from Nested Scope

A related but more complex problem is sometimes called the *upward funarg problem*, although it might be more accurate to call it the *upward fun-result problem* because it occurs when returning a function value from a nested scope, generally as the return value of a function.

A simple example of a function that returns a function is this ML code for function composition:

```
fun compose(f,g) = (fn x => g(f x));
```

Given two function arguments f and g, function compose returns the function composition of f and g. The body of compose is code that requires a function parameter x and then computes g(f(x)). This code is useful only if it is associated with some mechanism for finding values of f and g. Therefore, a closure is used to represent compose (f,g). The code pointer of this closure points to compiled code for "get function parameter x and then computes g(f(x))" and the activation record pointer of this closure points to the activation record of the call compose (f,g) because this activation record allows the code to find the actual functions f and g needed to compute their composition.

We will use a slightly more exciting program example to see how closures solve the variable-lookup problem when functions are returned from nested scopes. The following example may give some intuition for the similarity between closures and objects.

Example 7.9

In this example code, a "counter" is a function that has a stored, private integer value. When called with a specific integer increment, the counter increments its internal value and returns the new value. This new value becomes the stored private value for the next call. The following ML function, make_counter, takes an integer argument and returns a counter initialized to this integer value:

```
fun make_counter (init : int) =
   let val count = ref init (* private variable count *)
      fun counter(inc:int) = (count := !count + inc; !count)
   in
      counter (* return function counter from make counter *)
   end;
val c = make_counter(1); (* c is a new counter *)
c(2) + c(2);          (* call counter c twice *)
```

Function make_counter allocates a local variable count, initialized to the value of the parameter init. Function make_counter then returns a function that, when called, increments count's value by parameter inc, and then returns the new value of count.

The types and values associated with these declarations, as printed by the compiler, are

```
val make counter = fn : int  → (int  → int)
val c = fn : int → int
8 : int
```

Here is the same program example written in a C-like notation for those who prefer this syntax:

```
{int → int mk_counter (int init)
                    {int count = init;

      int counter(int inc){ return count += inc;}
      return counter
   }
   int → int c = mk counter(1);
   print c(2) + c(2);xs
 }
```

If we trace the storage allocation associated with this compilation and execution, we can see that the stack discipline fails. Specifically, it is necessary to save an activation record that would be popped off the stack if we follow the standard last allocated-first deallocated policy.

Figure 7.10 shows that the records are allocated and deallocated in execution of the code from Example 7.9. Here are the sequence of steps involved in producing the activation records shown in Figure 7.10:

1. *Declaration of* make_counter: An activation record for the block consisting of the declaration of function make_counter is allocated on the run-time stack. The function name make counter is abbreviated here as make_c so that the name fits easily into the figure. The value of make_counter is a closure. The activation record pointer of this closure points to the activation record for make_counter. The code pointer of this closure points to code for make_counter produced at compile time.

2. *Declaration of* c: An activation record for the block consisting of the declaration of function c is allocated on the run-time stack. The access pointer of this activation record points to the first activation record, as the declaration of make_counter is the previous block. The value of c is a function, represented by a closure. However, the expression defining function c is not a function declaration. It is an expression that requires a call to make_counter. Therefore, after this activation record is set up as the program executes, it is necessary to call make_counter.

3. *Call to* make_counter: An activation record is allocated in order to execute the function make_counter. This activation record contains space for the formal parameter init of make_counter, local variable count and function counter that will be the return result of the call. The code pointer of the closure for counter points to code that was generated and stored at compile time. The activation record pointer points to the third activation record, because the global variables of the function reside here. The program still needs activation record three after the call to make_counter returns because the function counter returned from the call to make_counter refers to variables init and count that are stored in (or reachable through) the activation record for make_counter. If activation record three (usedxs for the call to make_counter) is popped off the stack, the counter function will not work properly.

4. *First call to* c(2): When the first expression c(2) is evaluated, an activation record is created for the function call. This activation record has an access pointer that is set from the closure for c. Because the closure points to activation record three, so does the access pointer for activation record four. This is important because the code for counter refers to variable count, and count is located through the pointer in activation record three.

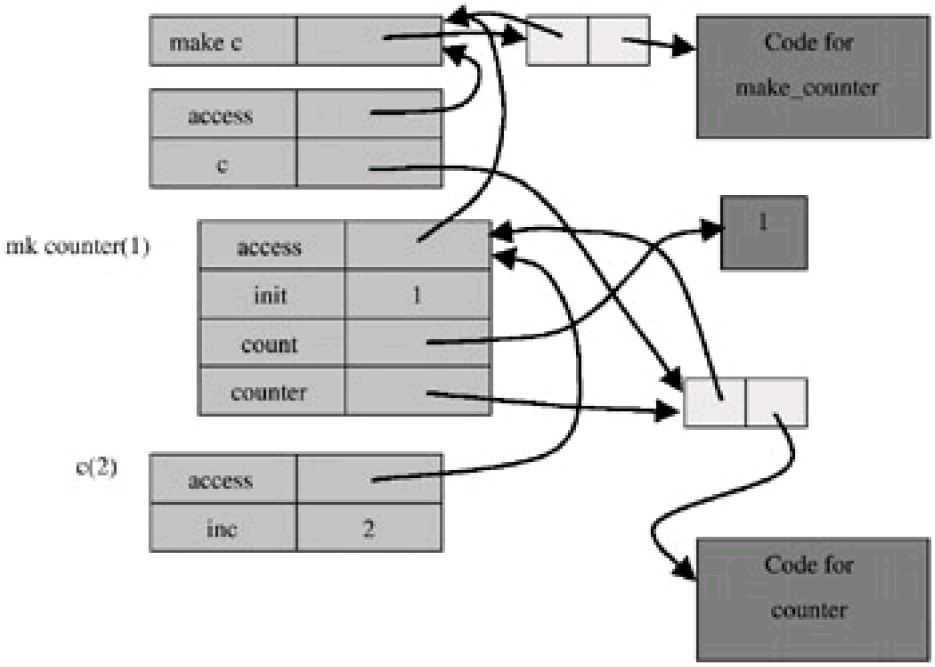


Figure 7.10: Activation records for function closure returned from function.

There are two main points to remember from this example:

- Closures are used to set the access pointer when a function returned from a nested scope is called.
- When functions are returned from nested scopes, activation records do *not* obey a stack discipline. The activation record associated with a function call cannot be deallocated when the function returns if the return value of the function is another function that may require this activation record.

The second point here is illustrated by the preceding example: The activation record for the call to make_counter can not be deallocated when make_counter returns, as this activation record is needed by the function count returned from make_counter.

Solution to Storage Management Problem

You may have noted that, after the code in Example 7.9 is executed, following the steps just described, we have several activation records left on the stack that might or might not be used in the rest of the program. Specifically, if the function c is called again in another expression, then we will need the activation records that maintain its static scope. However, if the function c is not used again, then we do not need these activation records. How, you may ask, does the compiler or run-time system determine when an activation record can be deallocated?

There are a number of methods for handling this problem. However, a full discussion is complicated and not necessary for understanding the basic language design trade-offs that are the subject of this chapter. One solution that is relatively straightforward and not as inefficient as it sounds is simply to use a garbage-collection algorithm to find activation records that are no longer needed. In this approach, a garbage collector follows pointers to activation records and collects unreachable activation records when there are no more closure pointers pointing to them.

Team-Fly



7.5 CHAPTER SUMMARY

A *block* is a region of program text, identified by begin and end markers, that may contain declarations local to this region. Blocks may occur inside other blocks or as the body of a function or procedure. Block-structured languages are implemented by activation records that contain space for local variables and other block-related information. Because the only way for one block to overlap another is for one to contain the other, activation records are generally managed on a run-time stack, with the most recently allocated activation record deallocated first (last allocated-first deallocated).

Parameters passed to functions and procedures are stored in the activation record, just like local variables. The activation record may contain the actual parameter value (in pass-by-value) or its address (in pass-by-reference). Tail calls may be optimized to avoid returning to the calling procedure. In the case of tail recursive functions that do not pass function arguments, this means that the same activation record may be used for all recursive calls, eliminating the need to allocate, initialize, and then deallocate an activation record for each call. The result is that a tail recursive function may be executed with the same efficiency as an iterative loop.

Correct access to statically scoped global variables involves three implementation concepts:

- Activation records of functions and procedures contain *access* (or *static scoping*) *links* that point to the activation record associated with the immediately enclosing block.
- Functions passed as parameters or returned as results must be represented as *closures*, consisting of function code together with a pointer to the correct lexical environment.
- When a function returns a function that relies on variables declared in a nested scope, static scoping requires a deviation from stack discipline: An activation record may need to be retained until the function value (closure) is no longer in use by the program.

Each of the implementation concepts discussed in this chapter may be tied to a specific language feature, as summarized in the following table.

Language Feature	Implementation construct
Block	Activation record with local memory
Nested blocks	Control link
Functions and procedures	Return address, return-result address
Functions with static scope	Access link
Function as arguments and results	Closures
Functions returned from nested scope	Failure of stack deallocation order for activation records

Team-Fly

Chapter 8: Control in Sequential Languages

After looking briefly at the history of jumps and structured control, we will study exceptions and continuations. Exceptions are a form of jump that exits a block or function call, returning to some previously established point for handling the exception. Continuations are a more general form of "return" based on calling a function that is passed into a block for this purpose. The chapter concludes with a discussion of *force* and *delay,* complimentary techniques for delaying computation by placing it inside a function and forcing delayed computation with a function call.

8.1 STRUCTURED CONTROL

8.1.1 Spaghetti Code

In Fortran or assembly code, it is easy to write programs with incomprehensible control structure. Here is a short code fragment that illustrates a few of the possibilities. The fragment includes a Fortran CONTINUE statement, which is an instruction that does nothing but is used for the purpose of placing a label between two instructions. If you scan the code from top to bottom, you might get the idea that the instructions between labels 10 and 20 act together to perform some meaningful task. However, then as you scan downward, you can see that it is possible later to jump to instruction11, which is in the middle of this set of instructions:

```
10 IF (X .GT. 0.000001) GO TO 20
   X = -X
11 Y = X*X - SIN(Y)/(X+1)
   IF (X .LT. 0.000001) GO TO 50
20 IF (X*Y .LT. 0.00001) GO TO 30
   X = X-Y-Y
30 X = X+Y

   ...
50 CONTINUE
```





An energetic man with sincere conviction and a sense of humor, Guy Lewis Steele, Jr., is a Distinguished Engineer at Sun Microsystems, Inc. He received his A.B. in applied mathematics from Harvard College (1975) and his S.M. and Ph.D. in computer science and artificial intelligence from MIT (1977, 1980). He is known for his seminal work on Scheme, including the revolutionary continuation-based Rabbit compiler described in his S.M. thesis, and subsequent years of work on Common Lisp, parallel computing, and Java. In addition to reference books on Common Lisp, Fortran, C, and Java, Guy was the original lead author of *The Hacker's Dictionary*, now revised as *The New Hacker's Dictionary*.

Guy Steele's interests also include chess and music. A Life Member of the United States Chess Federation, he has sung in the bass section of the MIT Choral Society, the Masterworks Chorale, and in choruses with the Pittsburgh Symphony Orchestra and the Boston Concert Opera. Guy composed The Telnet Song, published in the April 1984 ACM, which pokes lighthearted fun at the exponential explosion of ctrl-Q keystrokes needed to exit cascaded telnet sessions.

In a masterful 1998 plenary address at the Object-Oriented Programming: Systems, Languages, and Applications (OOPSLA) conference, Guy illustrated the difficulty of growing a language by starting with words of one syllable and defining every longer word in his talk from one-syllable words and previously defined words of two or more syllables. You can find the text of this speech on the web. Guy Steele was awarded the ACM Grace Murray Hopper Award in 1988.

```
X=A
Y = B-A + C*C
GO TO 11
...
```

Although no short sequence can begin to approximate the Byzantine control flow of many archaic Fortran programs, this example may give you some feel for the kind of confusing control flow that was used in many programs in the early days of computing.

8.1.2 Structured Control

In the 1960s, programmers began to understand that unstructured jumps could make it difficult to understand a program. This was partly a realization about programming style and partly a realization about programming languages: If incomprehensible control flow is bad programming style, then programming languages should provide mechanisms that make it easy to organize the control structure of programs. This led to the development of some constructs that structure jumps:

```
if ... then ... else ... end
while ... do ... end
for ... {...}
case ...
```

These are now adopted in virtually all modern languages.

In modern programming style, we group code in logical blocks, avoid explicit jumps except for function returns, and cannot jump *into* the middle of a block or function body.

The restriction on jumps into blocks illustrates the value of leaving a construct out of a programming language. If a label is placed in the middle of a function body and a program executes a jump to this label, what should happen? Should an activation record be created for the function call? If not, then local variables will not be meaningful. If so, then how will function parameters stored in the activation record be set? Without executing a call to the function, there are no parameter values to use in the call. Because these questions have no good, convincing, clear answers, it is better to design the compiler to reject programs that might jump into the middle of a function body.

Although most introductory programming books and courses today emphasize the importance of clean control structure and discourage the use of go to (if the language even allows it), it took many years of discussion and debate to reach this modern point of view. One reason for the change in perspective over the years is the decreasing importance of instruction-level efficiency. In 1960 and even 1970, there were many applications in which it was useful to save the cost of a test, even if it meant complicating the control structure of the program. Therefore, programmers considered it important to be able to jump out of the middle of a loop, avoiding another test at the top of the loop.

In the 1980s and 1990s, as computer speed increased, the number of applications in which a small change in efficiency would truly matter decreased significantly, to the point at which, in the 1990s, Java was introduced without any go to statement. Those interested in history may enjoy reading E.W. Dijkstra's March 1968 letter to the editor of *Communications of the ACM*, "Go To Considered Harmful," later posted on the Association for Computing Machinery web site as the October 1995 "Classic of the Month." The letter begins with these sentences:

> For a number of years I have been familiar with the observation that the quality of programmers is a decreasing function of the density of **go to** statements in the programs they produce. More recently I discovered why the use of the **go to** statement has such disastrous effects, and I became convinced that the **go to** statement should be abolished from all "higher level" programming languages….

Simple control structures such as if-then-else have now been in common use since the rise of Pascal in the late 1970s. Exceptions and continuations, discussed in the remainder of this chapter, are more recent innovations in programming languages.

Team-Fly

8.2 EXCEPTIONS

8.2.1 Purpose of an Exception Mechanism

Exceptions provide a structured form of jump that may be used to exit a construct such as a block or function invocation. The name *exception* suggests that exceptions are to be used in exceptional circumstances. However, programming languages cannot enforce any sort of intention like this. Exceptions are a basic mechanism that can be used to achieve the following effects:

- jump out of a block or function invocation
- pass data as part of the jump
- return to a program point that was set up to continue the computation.

In addition to jumping from one program point to another, there is also some memory management associated with exceptions. Specifically, unnecessary activation records may be deallocated as the result of the jump. We subsequently see how this works.

Exception mechanisms may be found in many programming languages, including Ada, C++, Clu, Java, Mesa, ML, and PL/1. Every exception mechanism includes two constructs:

- a statement or expression form for *raising* an exception, which aborts part of the current computation and causes a jump (transfer of control),
- a *handler* mechanism, which allows certain statements, expressions, or function calls to be equipped with code to respond to exceptions raised during their execution.

Another term for raising an exception is *throwing* an exception; another term for handling an exception is *catching* an exception.

There are several reasons why exceptions have become an accepted language construct. Many languages do not otherwise have a clean mechanism for jumping out of a function call, for example, aborting the call. Rather than using go tos, which can be used in unstructured ways, many programmers prefer exceptions, which can be used to jump only out of some part of a program, not into some part of the program that has not been entered yet. Exceptions also allow a programmer to pass data as part of the jump, which is useful if the program tries to recover from some kind of error condition. Exceptions provide a useful dynamic way of determining to where a jump goes. If more than one handler is declared, the correct handler is determined according to dynamic scoping rules. This effect would not be easy to achieve with other forms of control jumps.

The importance of dynamic scoping is illustrated in the following example.

Example 8.1

This example involves computing the inverse of a matrix. For the purpose of this example, a matrix is an array of real numbers. We compute matrix multiplication, used in linear algebra, by multiplying the entries of two matrices together in a certain way. If A is a matrix and I is the identity matrix (with ones along the diagonal and zeros elsewhere), then the inverse A^{-1} of A is a matrix such that the product $AA^{-1} = I$. A square matrix has an inverse if and only if something called the *determinant* of A is not zero. It is approximately as much computational work to find the determinant of a matrix as it is to invert a matrix.

Suppose we write a function for inverting matrices that have real-number entries. Because only matrices with nonzero determinants can be inverted, we cannot correctly compute the inverse of a matrix if the determinant turns out to be

zero. One way of handling this difficulty might be to check the determinant before calling the invert function. This is not a good idea, however, as computing the determinant of a matrix is approximately as much work as inverting it. Because we can easily check the determinant as we try to convert the inverse, it makes more sense to try to invert the matrix and then raise an exception in the event that we detect a zero determinant. This leads to code structured as follows:

```
exception Determinant; (* declare Determinant exception *)
fun invert(aMatrix) =
 ...
   if ...
   then raise Determinant (* if matrix determinant is zero *)
      else ...
   end;
invert(myMatrix) handle Determinant ...;
```

In this example, the function invert raises an exception if the determinant of aMatrix turns out to be zero. If the function raises this exception as a result of the call invert(myMatrix), then because of the associated handler declaration, the call to invert is aborted and control is transferred to the code immediately following handle Determinant.

In this example, the exception mechanism is used to handle a condition that makes it impossible to continue the computation. Specifically, if the determinant is zero, the matrix has no inverse, and it is impossible for the invert function to return the inverse of the matrix.

If you think about it, this example illustrates the need for some kind of dynamicscoping mechanism. More specifically, suppose that there are several calls to invert in a program that does matrix calculations. Each of these calls could cause the Determinant exception to be raised. When a call raises this exception, where would we like the exception to be handled?

The main idea behind dynamic scoping of exception handlers is that the code in which the invert function is called is the best place to decide what to do if the determinant is zero. If exceptions were statically scoped, then raising an exception would jump to the exception handler that is declared lexically before the definition of the invert function. This handler would be in the program text before the invert function. If invert is in a program library, then the handler would have to be in the library. However, the person who wrote the library has no idea what to do in the cases in which the exception occurs. For this reason, exceptions are dynamically scoped: When an exception is raised, control jumps to the handler that was most recently established in the dynamic call history of the program. We will see how this works in more detail in after looking at some specific exception mechanisms.

8.2.2 ML Exceptions

The ML mechanism has three parts:

- Exceptions *declarations*, which declare the name of an exception and specify the type of data passed when this exception is raised,
- A *raise* expression, which raises an exception and passes data to the handler,
- *Handler* declarations, which establish handlers.

An ML exception declaration has the form

```
exception ?name? of ?type?
```

where ?name? is the name of the exception and ?type? is the type of data that are passed to an exception handler when this exception is raised. The last part of this declaration, of ?type?, is optional. For example, the exception Ovflw below does not pass data, but the exception Signal requires an integer value that is passed to the receiving handler:

```
exception Ovflw;
exception Signal of int;
```

An ML raise expression has the form

```
raise ?name? ?arguments?
```

where ?name? is a previously declared exception name and ?arguments? have a type that matches the exception declaration. For example, the exceptions previously declared can be raised with the following expressions:

```
raise Ovflw;
raise Signal (x+4);
```

An ML handler is part of an expression form that allows a handler to be specified. An expression with handler has the form

```
(exp1) handle (pattern) => (exp2);
```

We evaluate this expression by evaluating ?exp1?. If the evaluation of ?exp1? terminates normally, then this is the value of the larger expression. However, if ?exp1? raises an exception that matches ?pattern?, then any values passed in raising the exception are bound according to ?pattern? and ?exp2? is evaluated. In this case, the value of ?exp2? becomes the value of the entire expression. If ?exp2? raises an exception or ?exp2? raises an exception that does not match ?pattern?, then ?exp1? handle ?pattern?=> ?exp2? has an uncaught exception that can be caught by the handler established by an enclosing expression or function call. A more general form involving multiple patterns is described below.

Examples

Here is a simple example that uses the "overflow" exception previously mentioned:

```
exception Ovflw;          (* Declare exception name *)
fun f(x) = if x <= Min (* Function with exception *)
           then raise Ovflw
           else 1/x;
(* - Expression that handles Ovflw in two ways - *)
( f(x) handle Ovflw => 0 ) / (f(x) handle Ovflw => 1)
```

Note that the final expression has two different handlers for the Ovflw exception. In the numerator, the handler returns value 0, making the fraction zero if overflow occurs. In the denominator, it would cause division by zero if the handler returns zero; the handler therefore returns 1 if the Ovflw exception is raised. This example shows how the choice of handler for an exception raised inside a function depends on how the function is called.

Here is another example, illustrating the way that data may be passed and used:

```
exception Signal of int;
fun f(x) = if x=0 then raise Signal(0)
       else if x=1 then raise Signal(1)
       else if x 10 then raise Signal(x-8)
       else (x-2) mod 4;
f(10) handle Signal(0) => 0
     | Signal(1) => 1
     | Signal(x) => x+8;
```

The handler in this expression uses pattern matching, which follows the form established for ML function declarations. More specifically, the meaning of an expression of the form

```
<exp> handle <pattern1> => <exp1>
      | <pattern2> = <exp2>
                ...
      | <patternn> => <expn>
```

is determined as follows:

1. The expression to the left of the handle keyword is evaluated.
2. If this expression terminates normally, its value is the value of the entire expression with handler declaration; the handler is never invoked. (If evaluation of this expression does not terminate, then evaluation of the enclosing expression cannot terminate either.) If the expression raises an exception that matches <patterni> (and there is no matching handler declared within <exp>),then the handler is invoked.
3. If the handler is invoked, pattern matching works just as an ordinary ML function call. The value passed by exception is matched against <pattern1>, <pattern2>, …, <patternn> in order until a match is found. If the value matches <patterni>, this causes any variables in <patterni> to be bound to values. The corresponding expression <expi> is evaluated with the bindings created by pattern matching.

8.2.3 C++ Exceptions

C++ exceptions are similar in spirit to exceptions in ML and other languages. The C++ syntax involves try blocks, a throw statement, and a catch block to handle exceptions that have been thrown within the associated try block. C++ exceptions are slightly less elegant than ML exceptions because C++ exceptions are not a separate kind of entity recognized by the type system.

The try block surrounds statements in which exceptions may be thrown. Here is a code fragment showing the form of a try block:

```
try {
    // statements that may throw exceptions
}
```

A throw statement may be executed within a try block, either directly by a statement in the try block or from a function called directly or indirectly from the block. A throw statement contains an expression and passes the value of the expression. Here is an example in which a character value is thrown:

```
throw "This generates a char * exception";
```

A catch block may immediately follow a try block and receive any thrown exceptions. Here is an example a catch-block receiving char * exceptions:

```
catch (char *message) {
      // statements that process the thrown char * exception
}
```

C++ uses types to distinguish between different kinds of exceptions. The throw statement may be used to throw different types of values:

```
throw "Hello world";        // throws a char *
throw 18;                   // throws an int
throw new String("hello"); // throws a String *
```

In a block that may throw more than one type of exceptions, multiple catch blocks may be used:

```
try {
    // code may throw char pointers and other pointers
}
catch (char *message) {
    // code processing the char pointers thrown as exceptions
}
catch (void *whatever) {
    // code processing all other pointers thrown as exceptions
}
```

Although it is possible to throw objects, some care is required because local objects are deallocated when an exception is thrown. More discussion of objects and storage allocation in C++ appears in Chapter 12.

Here is a more complete C++ example, combining try, throw, and catch:

```
void f(char * c) {
  ... if (c == 0) throw exception("Empty string argument");
```

```
}
main() {
try {
... f(x); ...
}catch (exception) {
exit(1);
}
}
```

As in ML and other languages, throwing a C++ exception causes a control transfer to the most recently established handler that is appropriate for the exception. Whereas ML uses pattern matching to determine whether a handler is appropriate for an exception, C++ uses type matching.

As is generally the case for C/C++, the type-matching issues are a little more complicated that we would like. To be specific, a handler of the form

---

```
catch(T t)
catch(const T t)
catch(T& t)
catch(const T& t)
```

---

can catch exception objects of type E if

- T and E are the same type, or
- T is an accessible base class of E at the throw point, or
- T and E are pointer types and there exists a standard pointer conversion from E to T at the throw point.

The rules for standard pointer conversion are also a little complicated, but we do not discuss them here because they are not critical for understanding the basic idea of exceptions.

One significant difference between ML and C++ that is important when programming with exceptions is that ML is garbage collected and C++ is not. In both languages, raising or throwing an exception will cause all run-time stack activation records between the point of the throw and the point of the catch to be deallocated. However, storage that is reachable from activation records may no longer be reachable after the exception, as the activation records with pointers no longer exist. This problem is discussed at the end of Subsection 8.2.4. There are also some details regarding the way exceptions work in constructors and destructors of objects that will be of interest to C++ programmers.

8.2.4 More about Exceptions

This section contains some additional examples of exceptions, with ML used as an illustrative syntax, and we discuss the interaction between exceptions, storage management, and static type checking. If you have used exceptions in a language different from ML, you might think about how the exception mechanism you have used is different and whether the difference is a consequence of some basic difference between the underlying programming languages.

Exceptions for Error Conditions

Exceptions arose as a mechanism for handling errors that occur when a program is running. One common form of run-time error occurs when an operation is not defined on some particular arguments. For example, division by zero raises an exception in many languages that have exception mechanisms. Here is a simple ML example, involving the left-subtree function that is not meaningful for trees that have only one node:

```
datatype 'a tree = Leaf of 'a | Node of 'a tree * 'a tree;
exception No_Subtree;
fun lsub ( Leaf x ) = raise No_Subtree
| lsub (Node(x,y)) = x;
```

In this example, a function lsub(t) returns the left subtree of t if the tree has two subtrees (left and right). However, if there is no left subtree, then the No_Subtree exception is raised.

Exceptions for Efficiency

Sometimes it is useful to terminate a computation when the answer is evident. Exceptions can be useful for this purpose, as an exception terminates a computation. Consider the following code for computing the product of the integers stored at the leaves of a tree. This is written for the tree data type previously defined:

```
fun prod (Leaf x)=x:int
| prod (Node(x,y)) = prod(x) * prod(y);
```

This correctly computes the product of all the integers stored at the leaves of a tree, but is inefficient in the case that some of the leaves are zero. If we are frequently computing the product for large trees that have zero at one or more leaves, then we might want to optimize this function as follows:

```
fun prod(aTree) =
   let exception Zero
      fun p(Leaf x) = if x = 0 then raise Zero else x
      | p(Node(x,y)) = p(x) * p(y)
   in
      p(aTree) handle Zero => 0
   end;
```

In this function, a test is performed at each leaf to see if the value is zero. If it is, then an exception is raised and no other leaves of the tree are examined. This function is less efficient than the preceding one for trees without a zero, but is more efficient if zero is found. Even if the last leaf is zero, raising an exception avoids all the multiplications that would otherwise be performed.

Static and Dynamic Scope

We look at two ML code fragments that use identifier X in analogous ways. In the first example, X is an expression variable and hence is accessed according to static scoping rules. In the second, X is the name of an exception, and so its handlers are used according to dynamic scoping rules. The point of these two code fragments is to see the differences between the two scoping rules and to clearly illustrate that exception handlers are determined by dynamic scope.

The following code illustrates static scoping:

```
val x = 6;
let fun f(y) = x
and g(h)=let val x = 2 in h(1) end
in
   let val x = 4 in g(f) end
end;
```

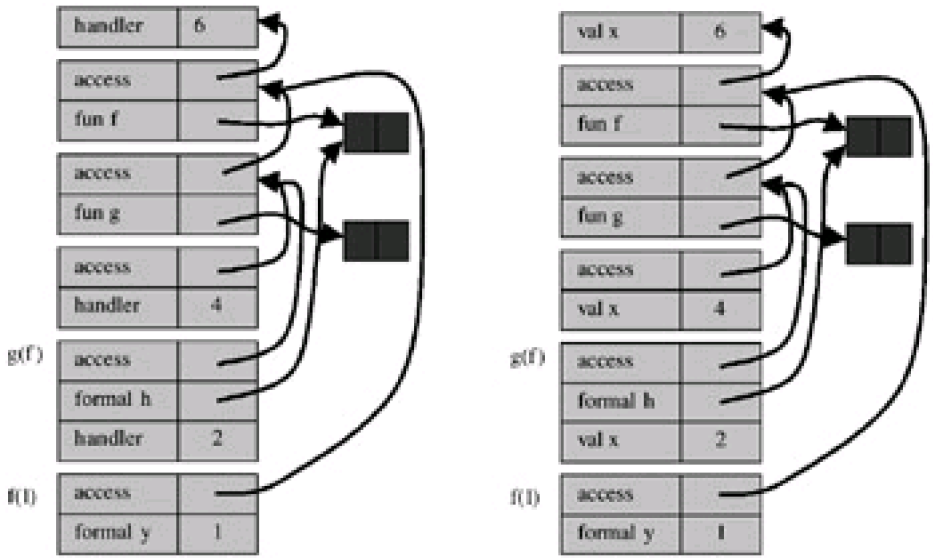
Under static scoping, the value returned by the call to g(f) is the value of x in the scope where f is declared, which is 6.

If we rewrite this code making X an exception, we will see what value we get under dynamic scoping. One thing to remember when looking at the following code is that ML exceptions use a postfix notation, so although the code is structurally quite similar, it looks somewhat different at first glance:

```
exception X;
(let fun f(y) = raise X
and g(h) = h(1) handle X => 2
in
   g(f) handle X => 4
end ) handle X => 6;
```

Here, the value of the g(f) expression is 2. The handler X that is used is the latest one at the time the function raising the exception is called.

The following illustration shows the run-time stack for each code fragment, following the style explained in Chapter 7, with the exception code on the left and the corresponding code with values in place of exceptions on the right. For brevity, only the handler (or identifier) values and the access links are shown in each activation record, except that the parameter value is also shown in the activation record for each function call. If you begin at the bottom of the stack and search up the stack for the most recent handler, which is the rule for dynamic scope, this is the one with value 2. On the other hand, static scoping according to access links leads to value 6.



This illustration also clearly shows which activation records will be popped off the run-time stack when an exception is raised. Specifically, when the exception X is raised in the code associated with the stack on the left, control transfers to the handler identified in the top activation record. At this point, all of the activation records below the activation record with handler X and value 6 are removed from the run-time stack because they are not needed to continue the computation.

Typing and Exceptions

In ML, the expression

```
e1 handle A => e2;
```

returns either the value of e1 or the value of e2. This suggests that the types of e1 and e2 should be the same. To see why in more detail, look at this expression, which contains the preceding one:

```
1 + (e1 handle A => e2);
```

Here, the value of e1 handle A => e2 is added to 1, so both e1 and e2 must be integer expressions. More generally, the type system can assign only one type to the expression (e1 handle A => e2). Because (e1 handle A => e2) can return either the value of e1 or the value of e2, both e1 and e2 must have the same type.

The situation for raise is very different, as raise <exception> does not have a value. To see how this works, consider the expression

```
1 + raise No_Value
```

where No_Value is a previously declared exception. In this expression, the addition will never be performed because raising the exception jumps to the nearest handler, which is outside the expression shown. In ML, the type of raise <exception> is a type variable 'a, which allows the type-inference algorithm to give raise <exception> any type.

Exceptions and Resource Allocation

Raising an exception may cause a program to jump out of any number of in-line blocks and function invocations. In each of these blocks, the program may have allocated data on the stack or heap. In ML, C++, and other contemporary languages, all data allocated on the run-time stack will be reclaimed after an exception is raised. This occurs when the handler is located and intervening activation records are popped off the run-time stack.

Data allocated on the heap are treated differently in different languages. In ML, data on the heap that is no longer be reachable after the exception will become garbage. Because ML is garbage collected, the garbage collector will reclaim this unreachable data. The situation is illustrated by the following code:

```
exception X;
(let
   val y = ref [1,2,3]
in
  ... raise X
end) handle X => ... ;
```

The local declaration let val x= … in … end causes a list [1,2,3] to be built in the heap. When raise X raises an exception inside the scope of this declaration, control is transferred to the handler outside this scope. The reference y, stored in the activation record associated with the local declaration, is popped off the stack. The list [1,2,3] remains in the program heap and is later collected whenever the garbage collector happens to run.

In C++, storage that is reachable from activation records may no longer be reachable after an exception is thrown. Because C++ is not garbage collected, it is up to the programmer to make sure that any data stored on the heap are explicitly deallocated. However, it may be impossible to explicitly deallocate memory that was previously allocated by the program if the only pointers to the data were those on the run-time stack. In particular, in the C++ version of the preceding program, with a linked list created in the heap, the only pointer to the list is the pointer y on the run-time stack. After the exception is thrown, the pointer y is gone, and there is no way to reach the data. There are two general solutions: Either make sure there is some way of reaching all heap data from the handler or accept the phenomenon and live with the memory leak. The C++ implementation provides some assistance for this problem by invoking the destructor of each object on the run-time stack as the containing activation record is popped off the stack. This solves the list problem if we build the list by placing one list object on the stack and by including code in the list destructor that follows the list pointer and invokes the destructor of any reachable list nodes.

The general problem with managing unreachable memory also occurs with other resources. For example, if a file is opened between the point of the handler and the point where an exception is thrown, there may be no way to close the open file. The same problem may occur with synchronization locks on concurrently accessible data areas. There is no systematic language solution that seems effective for handling these situations cleanly. Resource management is simply a complication that must be handled with care when one is programming with exceptions.

8.5 CHAPTER SUMMARY

We looked at several ways of controlling the order of execution and evaluation in sequential (nonconcurrent) programs. Here are the main topics of the chapter:

- structured programming without go to,
- exceptions,
- continuations,
- Delay and Force.

Control and Go to. Because structured programming is commonly accepted and taught, we did not look at the entire historical controversy surrounding go to statements. The main conclusion in the "Go to considered harmful" debate is that, in the end, program clarity is often more important than absolute efficiency. This has always been true to some degree, but as computer speed has increased, the relation between programmer time and program execution time has shifted. In modern software development, it is not worth several days of programmer time to reduce the execution time of a large application by one or two instruction cycles. Programmer time is expensive, and clever programming can lead to costly mistakes and increased debugging time. An instruction or two takes so little time that for most applications noone will notice the difference.

Exceptions. Exceptions are a structured form of jump that may be used to exit a block or function call and pass a return value in the process. Every exception mechanism includes a statement or expression form for *raising* an exception and a mechanism for defining *handlers* that respond to exceptions. When an exception is raised and several handlers have been established, control is transferred to the handler associated with the most recent activation record on the run-time stack. In other words, handlers are selected according to dynamic scope, not the static scope rules used for most other declarations in modern general-purpose programming languages.

Continuations. Continuation is a programming technique based on higher-order functions that may be used directly in programming or in program transformations in an optimizing compiler. Some mostly functional languages, such as Scheme and ML, provide direct support for capturing and invoking continuations.

Intuitively, the continuation of a statement or expression is a function representing the computation remaining to be performed after this statement or expression is evaluated. There is a general, systematic method for transforming any program into continuation-passing form. A function in continuation-passing form does not return, but calls a continuation (passed as an argument to the function) in order to continue after the function is complete. Continuation-passing form is related to tail recursion (see Subsection 7.3.4). Formally, a continuation-passing function appears to be tail recursive, as all calls are tail calls. However, continuation-passing functions may pass functions are arguments.

This makes it impossible to perform tail recursion elimination - the function created and passed out of a function call may need the activation record of the function in order to maintain the value of statically scoped global variables. However, a clever programmer can sometimes use the continuation-passing version of a function to devise a tail recursive function that will be compiled as efficiently as an iterative loop.

Continuation-passing form is used in a number of contemporary compilers for languages with higher-order functions.

Delay and Force. Delay and Force may be used to delay a computation until it is needed. When the delayed computation is needed, Force is used. Delay and Force may be implemented in conventional programming languages by use of functions: The delayed computation is placed inside a function and Force is implemented by calling this function. This technique is simplest to apply if functions can be declared anywhere in a program. If a delayed value may be needed more than once, this value may be stored in a location that will be used in every subsequent call to Force.

Team-Fly

Chapter 10: Concepts in Object-Oriented Languages

OVERVIEW

Over the past 30 years, object-oriented programming has become a prominent software design and implementation strategy. The topics covered in this chapter are object-oriented design, four key concepts in object-oriented languages, and the way these language concepts support object-oriented design and implementation.

An *object* consists of a set of operations on some hidden data. An important characteristic of objects is that they provide a uniform way of encapsulating almost any combination of data and functionality. An object can be as small as a single integer or as large as a file system or database. Regardless of its size, all interactions with an object occur by means of simple operations that are called *messages* or *member-function calls.*

If you look in magazines or research journals, you will find the adjective *object-oriented* applied to a variety of languages. As object orientation has become more popular and gained wider commercial acceptance, advocates of specific languages have decided that their favorite language is now object oriented. This has created some amount of confusion about the meaning of object oriented. In this book, we are interested in making meaningful distinctions between different language features and understanding how specific features support different kinds of programming. Therefore, the term *object-oriented language* is used to refer to programming languages that have objects and the four features highlighted in this chapter: dynamic lookup, abstraction, subtyping, and inheritance.

Team-Fly

10.1 OBJECT-ORIENTED DESIGN

Object-oriented design involves identifying important concepts and using objects to structure the way that these concepts are embodied in a software system. The following list of steps is taken from one overview of object-oriented design, written by object-oriented design proponent Grady Booch (Object-Oriented Design with Applications, Benjamin/Cummings, 1991):

- Identify the objects at a given level of abstraction.
- Identify the semantics (intended behavior) of these objects.
- Identify the relationships among the objects.
- Implement the objects.

Object-oriented design is an iterative process based on associating objects with components or concepts in a system. The process is iterative because typically we implement an object by using a number of subobjects, just as we typically implement a procedure by calling a number of finer-grained procedures. Therefore, after the important objects in a system are identified and implemented at one level of abstraction, the next iteration will involve identifying additional objects and implementing them. The "relationships among objects" mentioned here might be relationships between their interfaces or relationships between their implementations. Modern object-oriented languages provide mechanisms for using relationships between interfaces and relationships between implementations in the design and implementation process.

The data structures used in the early examples of top-down programming (see Section 9.1) were very simple and remained invariant under successive refinements of the program. When refinement involves replacing a procedure with more detailed procedures, older forms of structured programming languages such as Algol, Pascal, and C are adequate. For more complex tasks, however, both the procedures and the data structures of a program need to be refined together. Because objects are a combination of functions and data, object-oriented languages support the joint refinement of functions and data more effectively than do procedure-oriented languages.

Team-Fly

Team-Fly

10.2 FOUR BASIC CONCEPTS IN OBJECT-ORIENTED LANGUAGES

All object-oriented languages have some form of object. As mentioned in the preceding section, an object consists of functions and data, accessible only through a specific interface. In common object-oriented languages, including Smalltalk, Modula-3,C++, and Java, the implementation of an object is determined by its *class*. In these languages, we create objects by creating an *instance* of their classes.

The function parts of an object are called *methods* or *member functions*, and the data parts of an object are called *instance variables*, fields, or data members.

Programming languages with objects and classes typically provide dynamic lookup, abstraction, subtyping, and inheritance. These are the four main language concepts for object-oriented programming. They may be summarized in the following manner:

- *Dynamic lookup* means that when a message is sent to an object, the function code (or *method*) to be executed is determined by the way that the object is implemented, not some static property of the pointer or variable used to name the object. In other words, the object "chooses" how to respond to a message, and different objects may respond to the same message in different ways.
- *Abstraction* means that implementation details are hidden inside a program unit with a specific interface. For objects, the interface usually consists of a set of public functions (or *public methods*) that manipulate hidden data.
- *Subtyping* means that if some object `a` has all of the functionality of another object `b`, then we may use `a` in any context expecting `b`.
- *Inheritance* is the ability to reuse the definition of one kind of object to define another kind of object.

These terms are defined and these features are explored in more detail in the following subsections.

There are several forms of object-oriented languages that are not covered directly in this book. One form is the *delegation-based language*. Two delegation-based languages are Dylan, originally designed to program Apple Newton personal digital assistants, and Self, a general-purpose language evolving out of research on implementation of object-oriented languages. In delegation-based languages, objects are defined directly from other objects when new methods are added by means of *method addition* and old methods are replaced by means of *method override*. Although delegation-based languages do not have classes, they do have the four essential characteristics required for object-oriented languages.

10.2.1 Dynamic Lookup

In any object-oriented language, there is some way to invoke the operations associated with an object. A general syntax for invoking an operation on an object, possibly with additional arguments, is

```
object --> operation (arguments)
```

In Smalltalk, this is called "sending a message to an object," whereas in C++ it is called "calling a member function of an object." To avoid switching back and forth between different choices of terminology, we will use the Smalltalk terminology for the remainder of this section. In Smalltalk terminology, a *message* consists of an operation name and set of additional arguments. When a message is sent to an object, the object responds to the message by executing a function called a *method*.

Dynamic lookup means that a method is selected dynamically, at run time, according to the implementation of the object that receives a message. The important property of dynamic lookup is that different objects may implement the same operation differently. For example, the statement

```
x --> add(y)
```

sends the message add(y) to the object x. If x is an integer, then the method (code implementing this operation) may add integer y to x. If x is a set, then the add method may insert y into the set x. These operations have different effects and are implemented differently. However, a single line of code x → add(y) inside a loop could cause integer addition the first time it is executed and set insertion the second time if the value of the variable x changes from an integer to a set between one pass through the loop and another.

Dynamic lookup is sometimes confused with overloading, which is a mechanism based on static types of operands. However, the two are very different, as we will see.

Dynamic lookup is a very useful language feature and an important part of object-oriented programming. Consider, for example, a simple graphics program that manipulates pictures containing shapes such as squares, circles, and triangles. Each square object may contain a draw method with code to draw a square, each circle a draw method that contains code to draw a circle, and so on. When the program wants to display a given picture, sending a draw message to each shape in the picture can do this. The part of the program that sends the draw message does not have to know which kind of shape will receive the message. Instead, each shape receiving a draw message will know how to draw that shape. This makes sense because the implementer of a specific shape is in the best position to figure out how to draw that kind of shape.

We can understand some aspects of dynamic lookup and scoping by using a brief comparison with abstract data types. Using an abstract data-type mechanism, we might define matrices as follows:

```
abstype matrix = . . .
  with
     create(. . . ) = . . .
     update(m, i, j, x) = . . . set m(i, j) = x. . .
     add(m1, m2) = . . .
     . . .
  end;
```

A characteristic of this implementation of matrices is that the add function takes two matrices as arguments, with call of the form

```
add(x, y)
```

The declaration of type matrix and associated operations has a specific scope. Within this scope, `add` refers specifically to the function declared for matrices. Therefore, in an expression `add(x,y)`, both `x` and `y` must be matrices. If `add` were defined for complex numbers in some outer scope, then either the inner declaration hides the outer one or the language must provide some static overloading mechanism.

With objects in a class-based language, we might instead declare matrices as follows:

```
class matrix
  . . . (representation)
  update(i, j, x) = . . . set (i, j) of *this* matrix . . .
  add(m) = . . . add m to *this* matrix . . .
end
```

The `add` method of a matrix requires one matrix as an argument. The method might be invoked by an expression such as

```
x --> add(y)
```

In this expression, the operation `add` appears to have only one argument, the matrix that is to be added to the matrix `x` receiving the message `add(y)`.

There are several ways that dynamic lookup may be implemented. In one implementation, each object contains a pointer to a method lookup table that associates a method body with each message defined for that object. When a message is sent to an object at run time, the corresponding method is retrieved from that object's method table. Because different objects may have different method lookup tables, sending the same message to different objects may result in the execution of different code.

It is also possible to think of dynamic lookup as a run-time form of overloading. More specifically, we can think of each method name as the name of an overloaded function. When a message m is sent to an object named by variable x, then x is treated as the first argument of an overloaded function named m. Unlike traditional overloading, though, the code to execute must be chosen according to the run-time value of x. In contrast, traditional overloading uses the static type of a variable x to decide which code to use.

Dynamic lookup is an important part of Smalltalk, C++, and Java. In Smalltalk and Java, method lookup is done

dynamically by default. In C++, only *virtual* member functions are selected dynamically.

There is a family of object-oriented languages that is based on the "run-times overloading" view of dynamic lookup. The most prominent design of this form is the common Lisp object system, sometimes referred to by the acronym CLOS. In CLOS, an expression corresponding to

```
x --> f(y,z)
```

is treated as a call f(x, y, z) to an overloaded function with three arguments. Although ordinary dynamic lookup would select a function body for f based on the implementation of x alone, CLOS method lookup uses all three arguments. This feature is sometimes called *multiple dispatch* to distinguish it from more conventional single dispatch languages in which only one of the arguments of a function (the object receiving the message) determines the function body that is called at run time.

Multiple dispatch is useful for implementing operations such as equality, in which the appropriate comparisons to use depend on the dynamic type of both the receiver object and the argument object. Although multiple dispatch is in some ways more general than the single dispatch found in Smalltalk, C++, and Java, there is also some loss of encapsulation. Specifically, to define a function on different kinds of arguments, that function must have access to the internal data of each function argument.

Because single-dispatch languages are the object-oriented mainstream, we focus on single-dispatch languages in this book.

10.2.2 Abstraction

As discussed in Chapter 9, abstraction involves restricting access to a program component according to its specified interface. In most modern object-oriented languages, access to an object is restricted to a set of public operations that are chosen by the designer and implementer of the object. For example, in a program that manipulates geometric shapes, each shape could be represented by an object. We could implement an object representing a circle by storing the center and radius of the circle. The designer of circle objects could choose to make a function that changes the center of the circle part of the interface or choose not to put such a function in the interface. If there is no public function for changing the center of a circle, then no client code could change the center of a circle, as client code can manipulate objects only through their interface.

Abstraction based on objects is similar in many ways to abstraction based on abstract data types: Objects and abstract data types both combine functions and data, and abstraction in both cases involves distinguishing between a public interface and private implementation. However, other features of object-oriented languages make abstraction in object-oriented languages more flexible than abstraction in which abstract data types are used. One way of understanding the flexibility of object-oriented languages is by looking at the way that relationships between similar abstractions can be used to advantage.

Consider the following two abstract data types, written in ML syntax. The first is an abstract data type of queues, the second an abstract data type of priority queues. For simplicity, both queues and priority queues are defined for only integer data:

```
exception Empty;
abstype queue=Q of int list
  with
```

```
    fun mk_Queue() = Q(nil)
    and is_empty(Q(l)) = l=nil
    and add(x,Q(l)) = Q(l @ [x])
    and first (Q(nil)) = raise Empty | first (Q(x::l)) = x
    and rest (Q(nil)) = raise Empty | rest (Q(x::l)) = Q(l)
    and length (Q(nil)) = 0 | length (Q(x::l))= 1 + length (Q(l))
  end;
```

In this abstract data type, a queue is represented by a list. The add operation uses the ML append operator @ to add a new element to the end of a list. The first and the rest operations read and remove an element from the front of a list. Because client code cannot manipulate the representation of a queue directly, the implementation maintains an invariant: List elements appear in first-in/first-out order, regardless of how queues are used in client programs.

A priority queue is similar to a queue, except that elements are removed according to some preference ordering. More specifically, some priority is given to elements, and the first and the remove operations read and remove the queue elements that have highest priority:

```
abstype pqueue = Q of int list
  with
    fun mk_PQueue() = Q(nil)
    and is_empty(Q(l)) = l=nil
    and add(x,Q(l)) =
        let fun insert(x,nil) = [x:int]
        | insert(x,y::l) = if x<y then x::y::l else y::insert(x,l)
        in Q(insert(x,l)) end
    and first (Q(nil)) = raise Empty | first (Q(x::l)) = x
    and rest (Q(nil)) = raise Empty | rest (Q(x::l)) = Q(l)
    and length (Q(nil)) = 0 | length (Q(x::l))= 1 + length (Q(l))
  end;
```

The interface of an abstract data type is the list of public functions and their types. Queues and priority queues have the same interface: Both have the same number of operations, the operations have the same names, and each operation has the same type in both cases, except for the difference between the type names pqueue and queue. The point of this example is that, although the interfaces of queues and priority queues are identical, this correspondence is not used in traditional languages with abstract data types. In contrast, if we implement queues and priority queues in an object-oriented language, then we can take advantage of the similarity between the interfaces of these data structures.

A drawback to the kind of abstract data types used in ML and similar languages becomes apparent when we consider a program that uses both queues and priority queues. For example, suppose that we build a system with several wait queues, such as a hospital. In a hospital billing department, customers are served on a first-come, first-serve basis. However, in a hospital emergency room, patients are treated in order of the severity of their injuries or ailments. In a hospital program, we might like to treat priority queues and ordinary queues uniformly. For example, we might wish to count the total number of people waiting in any line in the hospital. To write this code, we would like to have a list of all the queues (both priority and ordinary) in the hospital and sum the lengths of all the queues in the list. However, if the length operation is different for queues and priority queues, we have to decide whether to call q-length or pq-length, even though the correct operation is uniquely determined by the data. This shortcoming of ordinary abstract data types is eliminated in object-oriented programming languages by a combination of subtyping (see Subsection 10.2.3) and

dynamic lookup.

The implementation of priority queues shows us another drawback of traditional abstract data types. Although the priority queue add function is different from the queue add function, the other five functions have identical implementations. In an object-oriented language, we may use inheritance (see Subsection 10.2.4) to define priority_queue from queue (or vice versa), giving only the new add function and reusing the other functions.

10.2.3 Subtyping

Subtyping is a relation on types that allows values of one type to be used in place of values of another. Although it is simplest to describe subtyping in the context of statically typed programming languages, there is also an implicit subtyping relation in untyped languages. We will discuss subtyping assuming that we are in a typed language, in most of this section, turning to untyped languages in the final paragraphs.

In most typed languages, the application of a function f to an argument x requires some relation between the type of f and the type of x. The most common case is that f must be a function from type A to type B, for some A and B, and x must be a variable or expression of type A. We can think of this type comparison as a comparison for equality: The type checker finds a type $A \to B$ for function f and a type C for x, and checks that A=C.

In languages with subtyping, there is a subtype relation on types. The basic principle associated with subtyping is *substitutivity*: If A is a subtype of B, then any expression of type A may be used without type error in any context that requires an expression of type B. We write A<:B to indicate that A is a subtype of B.

With subtyping, the subtype relation is used in place of equality in type checking. Specifically, to type the application of f to argument x, the type checker finds a type $A \to B$ for function f and a type C for x, and checks that C is a subtype of A.

The primary advantage of subtyping is that it permits uniform operations over various types of data. For example, subtyping makes it possible to have heterogeneous data structures that contain objects that belong to different subtypes of some common type. Consider as an example a queue containing various bank accounts to be balanced. These accounts could be savings accounts, checking accounts, investment accounts, and so on. However, if each type of account is a subtype of bank_account, then a queue of elements of type bank_account can contain all types of accounts.

Subtyping in an object-oriented language also allows functionality to be added without modifying general parts of a system. If objects of a type B lack some desired behavior, then we may wish to replace objects of type B with objects of another type A that have the desired behavior. In many cases, the type A will be a subtype of B. By designing the language so that substitutivity is allowed, one may add functionality in this way without any other modification to the original program.

This use of subtyping helps in building a series of prototypes of an airport scheduling system. In an early prototype, one would define a class airplane with methods such as position, orientation, and acceleration that would allow a control-tower object to affect the approach of an airplane. In a later prototype, it is likely that different types of airplanes would be modeled. If we add classes for Boeing 757s and Beechcrafts, these would be subtypes of airplane, containing extra methods and fields reflecting features specific to these aircraft. By virtue of the subtyping relation, Beechcrafts and Boeings are subtypes of airplane, and the general control algorithms that apply to all airplanes can be used for Beechcrafts and Boeings without modification.

10.2.4 Inheritance

Inheritance is a language feature that allows new objects to be defined from existing ones. We discuss the form of inheritance that appears in most class-based object-oriented languages by using a neutral notation. The following class A defines objects with private data v and public methods f and g. We define the class B by inheriting the declarations of A, redefining g, and adding a private variable w:

---

class A =

```
    private
      val v = . . .
    public
      fun f(x) = . . . g(. . .) . . .
      fun g(y) = . . . original definition . . .
    end;
class B = extend A with
    private
      val w=. . .
    public
      fun g(y) = . . . new definition . . .
    end;
```

In principle, inheritance can be implemented by code duplication. For every object or class of objects defined by inheritance, there is a corresponding definition that does not use inheritance; it is obtained by expansion of the definition so that inherited code is duplicated. The importance of inheritance is that it saves the effort of duplicating (or reading duplicated) code and that, when one class is implemented by inheriting from another, changes to one affect the other. This has a significant impact on code maintenance and modification.

A straightforward implementation of inheritance that avoids code duplication is to build linked data structures of method lookup tables. More specifically, for each class, a lookup table can contain the list of operations associated with the class. When one class inherits from another, then the second class can contain a pointer to the lookup table of the first. If a method is inherited, it can be found in the method lookup table of the class in which it was originally defined. A scheme of this form is used in the implementation of Smalltalk, as we will see in Subsection 11.5.3.

A significant optimization may be made in statically typed languages such as C++, in which the set of possible messages to each object can be determined at compile time. If lookup tables can be constructed so that all subclasses of a given class store repeated pointers in the same relative positions, then the offset of a method within a lookup table can be computed at compile time. This reduces the cost of method lookup to a simple indirection without a search, followed by an ordinary function call. We will look at this implementation of inheritance in more detail in Subsection 12.3.3.

Inheritance and Abstraction

In ordinary modules or abstract data types, there are two views of an abstraction: the client view and the implementation view. With inheritance, there are three views of classes: the implementation view, the client view, and the inheritance view. The inheritance view is the view of classes that inherit from a class. Because object definitions have two external clients, there are two interfaces to the outside: The *public* interface lists what the general client may see, whereas the *protected* interface lists what inheritors may see. (This terminology comes from C++.) In most languages, the public interface is a subset of the protected one. In Smalltalk, these interfaces are generated automatically: The public interface includes all the methods of an object, whereas the protected interface is all methods and all instance variables (data). In C++, the programmer explicitly declares which components of an object are public, which are protected, and which are *private* and visible only in the class definition itself. We will discuss this in more detail in Section 12.3.

10.2.5 Closures as Objects

The first characteristic of objects, dynamic lookup , is also provided by records (in Pascal or ML terminology) or structs (in C terminology). In a language with closures, we can simulate objects by using records that have function components. If an object has private data (or functions), then we can hide them by using static scoping. This leads us to look at record closures as a simple first model of objects. It turns out to be instructive to see how useful this notion of object is and where it falls short.

To see the similarities, consider the following ML code:

```
exception Empty;
fun newStack(x) =
  let val store = ref [x]
  in
    {push = fn(y) => store := y ::(!store),
    pop = fn() => case !store of
                nil = raise Empty
              | (y::ys) = (store := ys; y)
    }
    end;
val myStack = newStack(0);
#push(myStack)(1);
#pop(myStack)( );
```

The notation #field_name(record_value) is ML notation for field selection. In Pascal-like syntax, this expression would be written as record_value.field_name.

The function newStack returns a record with two function components, the first called push, the second called pop. Because the fields of this record contain functions, they are represented at run time as closures. The environment pointers for these closures point to the activation record for the newStack function, which stores the local data store. The initial value of store is a list containing only the initial element passed as an argument to newStack. If you draw out the activation records and closures, you will obtain a diagram that is very similar to the ones we will be drawing to represent

objects. However, most object-oriented languages optimize the representation in one or more ways.

Because closures and objects have essentially the same functionality, it is reasonable to wonder why we talk about "object-oriented" programming, instead of "closure-oriented" programming. In other words, what do object-oriented programming languages have that languages like ML lack? The answer is subtyping and inheritance. If you try to translate an object-oriented program into a non-object-oriented language, you will appreciate the language support for subtyping and inheritance.

10.2.6 Inheritance Is Not Subtyping

Perhaps the most common confusion surrounding object-oriented languages is the difference between subtyping and inheritance. The simplest distinction between subtyping and inheritance is this: *Subtyping is a relation on interfaces, inheritance is a relation on implementations.*

One reason subtyping and inheritance are often confused is that some class mechanisms combine the two. A typical example is C++, in which A will be recognized by the compiler as a subtype of B only if B is a public base class of A. Combining subtyping and inheritance is an elective design decision, however; C++ could have been designed differently without linking subtyping and public base classes in this way.

We may see that, in principle, subtyping and inheritance do not always go hand-in-hand by considering an example suggested by object-oriented researcher, Alan Snyder. Suppose we are interested in writing a program that requires dequeues, stacks, and queues. These are three similar kinds of data structures, with the following basic characteristics:

- *Queues*: Data structures with insert and delete operations, such that the first element inserted is the first one removed (first-in, first-out),
- *Stacks*: Data structures with insert and delete operations, such that the first element inserted is the last one removed (last-in, first-out),

- *Dequeues*: Data structures with two insert and two delete operations. A dequeue, or doubly ended queue, is essentially a list that allows insertion and deletion from each end. If an element is inserted at one end, then it will be the first one returned by a series of removes from that end and the last one returned by a series of removes from the opposite end.

An important part of the relationship among stacks, queues, and dequeues is that a dequeue can serve as both a stack and a queue. Specifically, suppose a dequeue d has insert operations insert_front and insert_rear and delete operations delete_front and delete_rear. If we use only insert_front and delete_rear, we have a queue. However, if we use insert_front and delete_front, we have a stack.

One way to implement these three classes is first to implement dequeue and then implement stack and queue by appropriately restricting (and perhaps renaming) the operations of dequeue. For example, we may obtain stack from dequeue by limiting access to those operations that add and remove elements from one end of a dequeue. Similarly, we may obtain queue from dequeue by restricting access to those operations that add elements at one end and remove them from the other. This method of defining stack and queue by inheriting from dequeue is possible in C++ through the use of private inheritance. (This is not a recommended style of implementation; this example is used simply to illustrate the differences between subtyping and inheritance.)

Although stack and queue may be implemented from dequeue, they are not subtypes of dequeue. Consider a function f that takes a dequeue d as an argument and then adds an element to both ends of d. If stack or queue were a subtype of dequeue, then function f should work equally well when given a stack s or a queue q. However, adding elements to both ends of either a stack or a queue is not legal; hence, neither stack nor queue is a subtype of dequeue. In fact, the reverse is true. Dequeue is a subtype of both stack and queue, as any operation valid for either a stack or a queue would be a legal operation on a dequeue. Thus, inheritance and subtyping are different relations in principle: it makes perfect sense to define stack and queue by inheriting from dequeue, but dequeue is a subtype of stack and queue, not the other way around.

A more detailed comparison of the two mechanisms appears in Section 11.7, in which the inheritance and subtyping relationships among Smalltalk collection classes are analyzed.

Team-Fly

Team-Fly

10.3 PROGRAM STRUCTURE

There are some systematic differences between the structure of function-oriented (or procedure-oriented) programs and object-oriented programs. One of the main differences is in the organization of functions and data. In a function-oriented program, data structures and functions are declared separately. If a function will be applied to many types of data, then it is common to use some form of case or switch statement within the function body. In an object-oriented program, functions are associated with the data they are designed to manipulate. Using dynamic lookup, the programming language implementation will select the correct function for each kind of data. This basic difference between function-oriented and object-oriented programs is illustrated by a comparison of Example 10.1 and Example 10.2. A longer example illustrating this point, written in C and C++, appears in Appendix B.1.

In both Examples 10.1 and 10.2, we consider a hospital simulation. The data in these examples represent doctors, nurses, and orderlies. The functions that will be applied to these data include a function to display information about a hospital employee and a function to set or determine the pay of an employee.

Example 10.1 Conventional Function-Oriented Organization

In a conventional function-oriented program, operations are grouped into function. If we want a single function to display information about all types of hospital employees, then we may use run-time tests to determine how to apply each operation to the given data. In outline, codes for display and pay functions might look like this:

```
display(x) =
 case type(x) of
      Doctor : [ "display Doctor" ]
      Nurse : [ "display Nurse" ]
      Orderly : [ "display Orderly" ]
 end;
 end;
 pay(x) =
    case type(x) of
       Doctor : [ "pay Doctor a lot" ]
       Nurse : [ "pay Nurse less" ]
       Orderly : [ "pay Orderly less than that" ]
    end;
 end;
```

Example 10.2 Object-Oriented Organization

In an object-oriented program, functions are grouped with the data they are designed to manipulate. For the hospital example, the doctor, nurse, and orderly classes will contain the code for the two functions. In outline, this produces the following program organization:

```
class Doctor =
    display = "Display Doctor";
    pay = "pay Doctor a lot ";
end;
class Nurse =
      display = "display Nurse";
      pay = "pay Nurse less";
end;
class Orderly =
      display = "display Orderly";
      pay = "pay Orderly less than that";
end;
```

Comparison of Examples 10.1 and 10.2

The data and operations used in Examples 10.1 and 10.2 may be arranged into the following matrix. In the conventional function-oriented organization, the code is arranged by row into functions that work for all kinds of data. In the object-oriented organization, the code is arranged by column, grouping each function case with the data it is designed for.

Operation	Doctor	Nurse	Orderly
Display	Display Doctor	Display Nurse	Display Orderly
Pay	Pay Doctor	Pay Nurse	Pay Orderly

In the function-oriented organization, it is relatively easy to add a new operation, such as PayBonus or Promote, but difficult to add a new kind of data, such as Administrator or Intern. In the object-oriented organization, it is easy to add new data, such as Administrator or Intern, but more cumbersome to add new operations such as PayBonus or Promote because this involves changes to every class.

Team-Fly

Team-Fly

10.4 DESIGN PATTERNS

The design pattern method is a popular approach to software design that has developed along with the rise in popularity of object-oriented programming. In basic terms, a *design pattern* is a general solution that has come from the repeated addressing of similar problems. Design patterns are not solutions developed from first principles or generic code that can simply be instantiated for a variety of purposes. Instead, a design pattern is a guideline or approach to solving a kind of problem that occurs in a number of specific forms. Solutions based on a design pattern can be similar; applying a design pattern to a specific situation can require some thought.

The concept of a design pattern can be used in any design discipline, such as mechanical design or architecture. The work of architect Christopher Alexander is often cited as an inspiration for software design patterns. Here is an architectural example, excerpted from one of Alexander's books (*A Pattern Language: Towns, Buildings, Construction, Oxford Univ. Press, 1977*). This passage includes both a description of the problem context and a solution developed as a result of experience:

> Sitting Circle
>
> … A group of chairs, a sofa and a chair, a pile of cushions - these are the most obvious things in everybody's life - and yet to make them work, so people become animated and alive in them, is a very subtle business. Most seating arrangements are sterile, people avoid them, nothing ever happens there. Others seem somehow to gather life around them, to concentrate and liberate energy. What is the difference between the two?
>
> … Therefore, place each sitting space in a position which is protected, not cut by paths or movements, roughly circular, made so that the room itself helps suggest the circle - not too strongly - with paths and activities around it, so that people naturally gravitate toward the chairs when they get into the mood to sit. Place the chairs and cushions loosely in the circle, and have a few too many.

When programmers find that they have solved the same kind of problem over and over again in slightly different ways but using essentially the same design ideas, they may try to identify the general design pattern of their solutions. The popularity of this process has led to the identification of a large number of software design patterns. To quote pattern advocate Jim Coplien, a *good* pattern does the following:

- *It solves a problem*: Patterns capture solutions, not just abstract principles or strategies.
- *It is a proven concept*: Patterns capture solutions with a track record, not theories or speculation.
- *The solution isn't obvious*: Many problem-solving techniques (such as software design paradigms or methods) try to derive solutions from first principles. The best patterns *generate* a solution to a problem indirectly - a necessary approach for the most difficult problems of design.
- *It describes a relationship*: Patterns don't just describe modules, but describe deeper system structures and mechanisms.
- *The pattern has a significant human component (minimize human intervention).* All software serves human comfort or quality of life; the best patterns explicitly appeal to aesthetics and utility.

Beyond reading about general principles of design patterns, the best way to learn about patterns is to study some examples and use patterns in your programming. Here are a couple of examples. You can find many more in the books and web pages devoted to design patterns. A widely used book is *Design Patterns: Elements of Reusable Object-Oriented Software* by E. Gamma, R. Helm, R. Johnson, and J. Vlissides (Addison-Wesley, 1994).

Example 10.3 Singleton Design Pattern

The singleton design pattern is a *creational* design pattern, meaning that it is a pattern that is used for creating objects in a certain way. Here is a brief overview of the singleton pattern, that uses the kind of subject headings that are commonly used in books and other presentations of design patterns.

Motivation

The singleton pattern is useful in situations in which there should be a single instance (object) of a class. This pattern gives a class direct control over how many instances can be created. This is better than making the programmer responsible for creating only one instance, as the restriction is built into the program.

Implementation

Only one class needs to be written to implement the singleton pattern. The class uses encapsulation to keep the class constructor (the function that returns new objects of the class) hidden from client code. The class has a public method that calls the constructor only if an object of the class has not already been created. If an object has been created, then the public function returns a pointer to this object and does not create a new object.

Sample Code

Here is how a generic singleton might be written in C++. Readers who are not familiar with C++ may wish to scan the explanation and return to this example after reading Chapter 12. The interface to class Singleton provides a public method that lets client code ask for an instance of the class:

```
class Singleton {
   public:
      static Singleton* instance(); // function that returns an instance
   protected:
      Singleton(); // constructor is not made public
   private:
      static Singleton*_instance; // private pointer to single object
};
```

Here is the implementation. Initially, the private pointer_instance is set to 0. In the implementation of public method instance(), a new object is created and assigned to_instance only if a previous call has not already created an object of this class:

```
Singleton* Singleton::_singleton = 0
Singleton* Singleton::instance() {
   if (_instance = = 0){_instance = new Singleton; }
   return_instance;
}
```

Example 10.4 Façade

Façade is a *structural object* pattern, which means it is a pattern related to composing objects into larger structures containing many objects.

Motivation

The *façade* pattern provides a single object for accessing a set of objects that have been combined to form a structure. In effect, the façade provides a higher-level interface to a collection of objects, making the collection easier to use.

Implementation

There is a façade class, defined for a set of classes that are used to make up a structure "behind" the facade. In a typical use, a façade object has relatively little actual code, passing most calls to objects in the structure behind the façade.

Example of Façade Pattern

Façade is a very common pattern when a task is accomplished by a combination of the results of a number of subtasks. For example, a compiler might be constructed by implementation of a lexical scanner, parser, semantic analyzer, and other phases indicated in the figure in Subsection 4.1.1. If each phase is implemented as an object with methods that perform its main functions, then the compiler itself will be a façade object that takes a program as input and uses the separate objects that are implementing each phase to compile the program. A user of the compiler may see the interface presented by the compiler object. This is a more useful interface than the more detailed interfaces to the constituent objects that are hidden behind the façade.

Team-Fly

Team-Fly

10.5 CHAPTER SUMMARY

This chapter contains a short overview of object-oriented design and summarizes the four basic concepts associated with object-oriented languages: dynamic lookup, abstraction, subtyping, and inheritance.

- *Dynamic lookup* means that when a message is sent to an object, the function code (or *method*) that is executed is determined by the way that the object is implemented. Different objects may respond to the same message in different ways.
- *Abstraction* means that implementation details are hidden inside a program unit with a specific interface. The interface of an object is usually a set of public functions (or *public methods*) that manipulate hidden data.
- *Subtyping* means that if some object a has all of the functionality of another object b, then we may use a in any context expecting b.
- *Inheritance* is the ability to reuse the definition of one kind of object to define another kind of object.

In conventional languages that implement closures and allow records to contain functions, records provide a form dynamic lookup and abstraction. Subtyping and inheritance, in the form needed to support object-oriented programming, are generally not found in conventional languages.

Many people confuse subtyping and inheritance. As the term is used in this book, subtyping is a relation on types that allows values of one type to be used in place of values of another. (In Section 11.7, Smalltalk is used to discuss subtyping in a language that does not have a static type system.) As the term is used in this book, inheritance allows new objects to be defined from existing ones. In class-based languages, inheritance allows the implementation of one class to be reused as part of the implementation of another. The simplest way to keep subtyping and inheritance straight is to remember this: *Subtyping is a relation on interfaces* and *inheritance is a relation on implementations.*

In Section 10.3, the difference between the organizational structure of object-oriented programs and the organizational structure of conventional programs is summarized. In conventional languages, functions are designed to operate on many types of data. In object-oriented programs, functions can be written to operate on a single type of data, with dynamic lookup finding the right function at run time.

In Section 10.4, we looked at the basic idea behind design patterns and saw two examples, singleton and façade. A design pattern is a general solution that has come from the repeated addressing of similar problems. The design pattern method is a popular approach to software design that has evolved along with object-oriented programming.

Team-Fly

Team-Fly

10.6 LOOKING FORWARD: SIMULA, SMALLTALK, C++, JAVA

In the next three chapters, we will look at four object-oriented languages:

- *Simula*, the first object-oriented language. The object model in Simula was based on procedure activation records, with objects originally described as procedures that return a pointer to their own activation record. There was no abstraction in Simula 67, but a later version incorporated abstraction into the object system. Simula was an important inspiration for C++.
- *Smalltalk*, a dynamically typed object-oriented language. Many object-oriented ideas originated or were popularized by the Smalltalk group, which built on Alan Kay's then-futuristic idea of the Dynabook. The Dynabook, which was never built by this group, was intended to be a small portable computer capable of running a user-friendly programming language. We will look at the Smalltalk implementation of method lookup and later compare this with C++.
- C++, a widely used statically typed object-oriented language. This language is designed for efficiency around the principle that programs that do not use a certain feature should run as efficiently as programs written in a language without that feature. A significant design constraint was backward compatibility with C.
- *Java*, a modern language design in which security and portability are valued as much as efficiency. Some interesting features are interfaces, which provide explicit support for abstract base classes, and run-time class loading, intended for use in a distributed environment.

Because there is not enough time to study all aspects of each language, we will concentrate on a few important or distinctive features of each one. One general theme in our investigation of these languages is the trade-off between language features and implementation complexity.

Simula is primarily important as a historical language and for the way it illustrates the relationship between objects and activation records. Of the remaining three languages, Smalltalk represents one extreme and C++ the other. Smalltalk is extremely flexible and based on the notion that everything is an object. C++, on the other hand, is defined to favor efficiency over conceptual simplicity. Although C++ provides objects, many features of C++ are inherited from C and are not based on objects. Java is a compromise between Smalltalk and C++ in the sense that the flexibility of the implementation and organization around objects are closer to Smalltalk than to C++. Java also contains features not found in either of the other languages, such as dynamic class loading and a typed intermediate language.

Team-Fly

EXERCISES

10.1 Expression Objects

We can represent expressions given by the grammar

e ::= *num* | *e* + *e*

by using objects from a class called expression. We begin with an "abstract class" called expression. Although this class has no instances, it lists the operations common to all kinds of expressions. These are a predicate telling whether there are subexpressions, the left and right subexpressions (if the expression is not atomic),and a method computing the value of the expression:

```
class expression() =
        private fields:
                (* none appear in the _interface_ *)
        public methods:
                atomic?()  (* returns true if no subexpressions *)
                lsub()      (* returns "left" subexpression if not atomic *)
                rsub()      (* returns "right" subexpression if not atomic *)
                value()      (* compute value of expression *)
end
```

Because the grammar gives two cases, we have two subclasses of expression, one for numbers and one for sums:

```
class number(n) = extend expression() with
        private fields:
                val num = n
        public methods:
                atomic?() = true
                lsub () = none (* not allowed to call this, *)
                rsub () = none (* because atomic?() returns true *)
                value () =num
end
class sum(e1, e2) = extend expression() with
        private fields:
                val left = e1
                val right = e2
        public methods:
                atomic?() = false
                lsub () = left
```

```
rsub () = right

value () = ( left.value() ) + ( right.value() )
```

end

a. *Product Class:* Extend this class hierarchy by writing a prod class to represent product expressions of the form

*e ::=… | e * e.*

b. *Method Calls:* Suppose we construct a compound expression by

```
val a = number(3);

val b = number(5);

val c = number(7);

val d = sum(a,b);

val e = prod(d,c);
```

and send the message value to e. Explain the sequence of calls that are used to compute the value of this expression: e.value(). What value is returned?

c. *Unary Expressions:* Extend this class hierarchy by writing a square class to represent squaring expressions of the form

e ::= … | e 2.

What changes will be required in the expression interface? What changes will be required in subclasses of expression? What changes will be required in functions that use expressions?[*] What changes will be required in functions that do not use expressions? (Try to make as few changes as possible to the program.)

d. *Ternary Expressions:* Extend this class hierarchy by writing a cond class to represent conditionals[?] of the form

e ::=… | e?e :e

What changes will be required if we wish to add this ternary operator? [As inpart (c), try to make as few changes as possible to the program.]

e. *N-Ary Expressions:* Explain what kind of interface to expressions we would need if we would like to support atomic, unary, binary, ternary and *n* –ary operators without making further changes to the interface. In this part of the problem, we are not concerned with minimizing the changes to the program; instead, we are interested in minimizing the changes that may be needed in the future.

10.2 Objects vs. Type Case

With object-oriented programming, classes and objects can be used to avoid "type-case" statements. Here is a program in which a form of case statement is used that inspects a user-defined type tag to distinguish between different classes of shape objects. This program would not statically type check in most typed languages because the correspondence between the tag field of an object and the class of the object is not statically guaranteed and visible to the type checker. However, in an untyped language such as Smalltalk, a program like this could behave in a computationally reasonable way:

```
enum shape_tag {s_point, s_circle, s_rectangle};

class point{
        shape_tag tag;

        int x;

        int y;
```

```
                point (int xval, int yval)
                        { x = xval; y = yval; tag = s point;}
                int x_coord (){ return x;}
                int y_coord (){ return y;}
                void move (int dx, int dy){ x += dy; y += dy;}
};
class circle{
                shape_tag tag;
                point c;
                int r;
                circle (point center, int radius)
                        { c = center; r = radius; tag = s_circle}
                point center (){ return c;}
                int radius (){ return radius;}
                void move (int dx, int dy){ c.move (dx, dy);}
                void stretch (int dr){ r += dr;}
};
class rectangle{
                shape_tag tag;
                point tl;
                point br;
                rectangle (point topleft, point botright)
                        { tl = topleft; br = botright; tag = s_rectangle;}
                point top_left (){ return tl;}
                point bot_right (){ return br;}
                void move (int dx, int dy){ tl.move (dx, dy); br.move (dx, dy);}
                void stretch (int dx, int dy){ br.move (dx, dy);}
 };
/* Rotate shape 90 degrees. */
void rotate (void *shape){
                switch ((shape_tag *) shape){
                        case s_point:
                        case s_circle:
                                break;
                        case s_rectangle:
                                {
                                        rectangle *rect = (rectangle *)
                                        shape;
```

```
                int d = ((rect->bot_right
                ().x_coord ()
                                -
                                rect->top_left
                                ().x_coord
                                ()) -
                                (rect->top_left
                                ().y_coord ()
                                -
                                rect->bot_right
                                ().y_coord
                                ()));
                rect->move (d, d);
                rect->stretch (-2.0 * d, -2.0 * d);
            }
        }
    }
```

a. Rewrite this so that, instead of rotate being a function, each class has a rotate method and the classes do not have a tag.

b. Discuss, from the point of view of someone maintaining and modifying code, the differences between adding a triangle class to the first version (as previously written) and adding a triangle class to the second [produced in part (a) of this question].

c. Discuss the differences between changing the definition of rotate (say, from 90 ? to the left to 90 ? to the right) in the first and the second versions. Assume you have added a triangle class so that there is more than one class with a nontrivial rotate method.

10.3 Visitor Design Pattern

The extension and maintenance of an object hierarchy can be greatly simplified (or greatly complicated) by design decisions made early in the life of the hierarchy. This question explores various design possibilities for an object hierarchy representing arithmetic expressions.

The designers of the hierarchy have already decided to structure it as subsequently shown, with a base class Expression and derived classes IntegerExp, AddExp, MultExp, and so on. They are now contemplating how to implement various operations on Expressions, such as printing the expression in parenthesized form or evaluating the expression. They are asking you, a freshly minted language expert, to help.

The obvious way of implementing such operations is by adding a method to each class for each operation. The Expression hierarchy would then look like:

```
class Expression
{
        virtual void parenPrint();
        virtual void evaluate();
        //…
 }
class IntegerExp : public Expression
{
```

```
                virtual void parenPrint();

                virtual void evaluate();

                //…

}

class AddExp : public Expression

{

                virtual void parenPrint();

                virtual void evaluate();

                //…

}
```

Suppose there are *n* subclasses of Expression altogether, each similar to IntegerExp and AddExp shown here. How many classes would have to be added or changed to add each of the following things?

a. A new class to represent product expressions.

b. A new operation to graphically draw the expression parse tree.

Another way of implementing expression classes and operations uses a pattern called the visitor design pattern. In this pattern, each operation is represented by a visitor class. Each visitor class has a visitCLS method for each expression class CLS in the hierarchy. The expression class CLS is set up to call the visit CLS method to perform the operation for that particular class. Each class in the expression hierarchy has an accept method that accepts a visitor as an argument and "allows the visitor to visit the class and perform its operation." The expression class does not need to know what operation the visitor is performing.

If you write a visitor class ParenPrintVisitor to print an expression tree, it would be used as follows:

```
Expression *expTree =…some code that builds the expression tree…;

Visitor *printer = new ParenPrintVisitor();

expTree->accept(printer);
```

The first line defines an expression, the second defines an instance of your ParenPrintVisitor class, and the third passes your visitor object to the accept method of the expression object.

The expression class hierarchy that uses the visitor design pattern has this form, with an accept method in each class and possibly other methods:

```
class Expression

{

                virtual void accept(Visitor *vis) = 0; //Abstract class

                //…

}

class IntegerExp : public Expression

{

                virtual void accept(Visitor *vis){vis->visitIntExp(this);};

                //…

}

  class AddExp : public Expression
```

```
{
        virtual void accept(Visitor *vis)
        { lhs->accept(vis); vis->visitAddExp(this); rhs->accept(vis);}
        //…
}
```

The associated Visitor abstract class, naming the methods that must be included in each visitor and some example subclasses, have this form:

```
class Visitor
{
        virtual void visitIntExp(IntegerExp *exp) = 0;
        virtual void visitAddExp(AddExp *exp) = 0; // Abstract class
}
class ParenPrintVisitor : public Visitor
{
        virtual void visitIntExp(IntegerExp *exp) { // IntExp print code};
        virtual void visitAddExp(AddExp *exp) { // AddExp print code};
}
class EvaluateVisitor : public Visitor
{
        virtual void visitIntExp(IntegerExp *exp) { // IntExp eval code};
        virtual void visitAddExp(IntegerExp *exp){ // AddExp eval code};
}
```

Suppose there are *n* subclasses of Expression and *m* subclasses of Visitor. How many classes would have to be added or changed to add each of the following things by use of the visitor design pattern?

c. A new class to represent product expressions.

d. A new operation to graphically draw the expression parse tree.

   The designers want your advice.

e. Under what circumstances would you recommend using the standard design?

f. Under what circumstances would you recommend using the visitor design pattern?

[*] Keep in mind that not all functions simply want to evaluate entire expressions. They may call the other methods as well.

[?] In C, conditional expressions a?b:c evaluate a and then return the value of b if a is nonzero or return the value of c if a is zero.

Team-Fly

Chapter 11: History of Objects-Simula and Smalltalk

Objects were invented in the design of Simula and refined in the evolution of Smalltalk. In this chapter, we look at the origin of object-oriented programming in Simula, based on the concept of a procedure that returns a pointer to its activation record, and the development of a purely object-oriented paradigm in the Smalltalk project and programming language. Twenty years after its development, Smalltalk provides an important contrast with C++ and Java both in simplicity of concept and in the way that its implementation provides maximal programming flexibility.

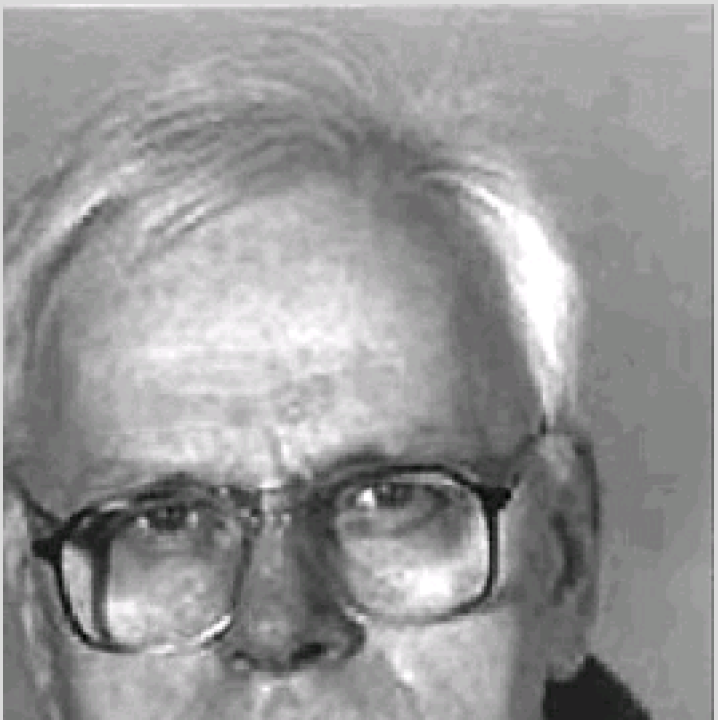
11.1 ORIGIN OF OBJECTS IN SIMULA

As the name suggests, the *Simula* programming language was originally designed for the purpose of simulation. The language was designed by O.-J. Dahl and K. Nygaard at the Norwegian Computing Center, Oslo, in the 1960s. Although the designers began with a specific interest in simulation, they eventually produced a general-purpose programming language with widespread impact on the field of computing.

Simula has been extremely influential as the first language with classes, objects, dynamic lookup, subtyping, and inheritance. It was an inspiration to the Xerox Palo Alto Research Center (PARC) group that developed Smalltalk and to Bjarne Stroustrop in his development of C++. Although Simula 67 had important object-oriented concepts, much of the popular mystique surrounding objects and object-oriented design developed later as a result of other efforts, most notably the Smalltalk work of Alan Kay and his collaborators at Xerox PARC.

11.1.1 Object and Simulation

You may wonder what objects have to do with simulation. A partial answer may be seen in the outline of an event-based simulation program. An event-based simulation is one in which the operation of a system is represented as a sequence of events. Here is pseudocode representing a generic event-based simulation program:





A driving force behind Simula was Kristen Nygaard, an operations research specialist and a political activist. He wanted a general modeling language that could be used to describe complex dynamic social and industrial systems in a simple way. Nygaard's interest in modeling motivated the development of innovative computing techniques, including objects, classes, inheritance, and quasi-parallel program execution allowing every object to have an optional action thread.

Nygaard was concerned by the social consequences of computing and expressed some of his reservations as follows:

I could see that SIMULA was being used to organize work for people and I could see that it would contribute to major changes in this area: More routine work, less demand for knowledge and a skilled labor force, less flexibility at the work place, more pressure. … I gradually came to face a moral dilemma. … I realized that the technology I had helped to develop had serious consequences for other people… The question was what to do about it? I had no desire to uninvent SIMULA.. because I was convinced that in the society I wanted to help build, computers would come to play an immensely important role. [Translation by J.R. Holmevik]

In addition to the ACM Turing Award and the IEEE John von Neumann Medal, Kristen Nygaard, received the 1990 Norbert Wiener Award for Professional and Social Responsibility from Computer Professionals for Social Responsibility (CPSR), "For his pioneering work in Norway to develop ‘participatory design,' which seeks the direct involvement of workers in the development of the computer-based tools they use." Historical information about Nygaard and the development of Simula can be found in an article by J.R. Holmevik, (*Annals of the History of Computing* Vol. 16, Number 4,1994; pp. 25-37). More recent information may be found on Nygaard's home page.

```
Q := make_queue(first_event);
repeat
   remove next event e from Q
   simulate event e
   place all events generated by e on Q
until Q is empty
```

This form of simulation requires a data structure (some form of queue or priority queue) that may contain a variety of kinds of events. In a typed language, the most natural way to obtain such a structure is through subtyping. In addition, the operation simulate e must be written in some generic way or involve case statements that branch according to the kind of event that e actually represents. Objects help because dynamic lookup for simulate e can determine the correct code automatically. Inheritance arises when we consider ways of implementing related kinds of events.

As this quick example illustrates, event-based simulations can be programmed in a general-purpose object-oriented language. The designers of Simula discovered this when they tried to make a special-purpose language, one tailored to simulation only. Apparently, when someone once asserted that Simula was not a general-purpose language, Nygaard's response was that, because Simula had all of the features of Fortran and Algol, and more, what would he need to remove before Simula could be called general purpose?

11.1.2 Main Concepts in Simula

Simula was designed as an extension and modification of Algol 60. Here are short lists of the main features that were added to and removed from Algol 60:

- *Added to Algol 60:*

  class concepts and reference variables (pointers to objects),

  pass-by-reference,

  char, text, and input-output features,

  coroutines, a mechanism for writing concurrent programs.

- *Removed from Algol 60:*

  changed default parameter passing mechanism from pass-by-name to a combination of pass-by-value and pass-by-result,

  some initialization requirements on variables,

  own variables (which are analogous to C static variables),

  the Algol 60 string type (in favor of a text type).

In addition to objects, concurrency was an important development in Simula. This arose from an interest in simulations that had several independent parts, each defining a sequence of events. Before representing events by objects, the designers experimented with representing independent sequences of events by independent processes, with the main simulation loop alternating between these processes to allow the simulation to progress. Here is a short quote from Nygaard on the incorporation of processes into the language:

> In the spring of 1963, we were almost suffocated by the single-stack structure of Algol. Then Ole-Johan developed a new storage management scheme [the multistack scheme] in the summer and autumn of 1963. The preprocessor idea was dropped, and we got a new freedom of choice. In Feb, 1964 the process concept was created, which [led to] Simula 67's class and object concept.

This quote appears in "The Development of the Simula Languages" by K. Nygaard and O.-J. Dahl (published in *History of Programming Languages*, R. L. Wexelblat, ed., Academic, New York, 1981.)

Team-Fly

Team-Fly

11.2 OBJECTS IN SIMULA

Objects arise from a very simple idea: After a procedure call is executed, it is possible to leave the procedure activation record on the run-time stack and return a pointer to it. A procedure of this modified form is called a *class* in Simula and an activation record left on the stack is an *object*:

> *Class*: A procedure returning a pointer to its activation record.
>
> *Object*: An activation record produced by call to a class, called an instance of the class.

Because a Simula activation record contains pointers to the functions declared in the block and their local variables, a Simula object *is* a closure! Pointers, missing from Algol 60, were needed in Simula because a class returns a pointer to an activation record. In Simula terminology, a pointer is called a ref.

Although the concept of object begins with the idea of leaving activation records on the stack, returning a pointer to an activation record means that the activation record cannot be deallocated until the activation record (object) is no longer used by the program. Therefore, Simula implementations place objects on the heap, not the run-time stack used for procedure calls. Simula objects are deallocated by the garbage collector, which deallocates objects only when they are no longer reachable from the program that created them.

11.2.1 Basic Object-Oriented Features in Simula

Simula contains most of the main object-oriented features that we use today. In addition to classes and objects, mentioned in the preceding subsection, Simula has the following features:

- *Dynamic lookup*, as operations on an object are selected from the activation record of that object,
- *Abstraction* in later versions of Simula, although not in Simula 67,
- *Subtyping*, arising from the way types were associated with classes,
- *Inheritance*, in the form of class prefixing, including the ability to redefine parts of a class in a subclass.

Although Simula 67 did not distinguish between public and private members of classes, a later version of the language allowed attributes to be made "protected," which means that they are accessible for subclasses (but not other classes), or "hidden," in which case they are not accessible to subclasses either.

In addition to the features just listed, Simula contains a few object-related features that are not found in most object-oriented languages:

- Inner, which indicates that the method of a subclass should be called in combination with execution of superclass code that contains the inner keyword,
- Inspect and qua, which provide the ability to test the type of an object at run time and to execute appropriate code accordingly. Inspect is a class (type) test, and qua is a form of type cast that is checked for correctness at run time.

All of these features are discussed in the following subsections. Some features that are found in other languages but not in early Simula are multiple inheritance, class variables (as in Smalltalk), and the self / super mechanism found in Smalltalk.

11.2.2 An Example: Points, Lines, Circles

Here is a short program example, taken from a classic book describing early Simula (Birtwistle, Dahl, Myhrhaug, and Nygaard, *Simula Begin*, Auerbach, 1973). This example illustrates some characteristics of Simula and shows how closely early object-oriented programming in Simula resembles object-oriented programming today.

Problem

Given three distinct points *p*, *q*, and *r* in the plane, find the center and the radius of the circle passing through *p*, *q*, and *r*. The situation is drawn in Figure 11.1.

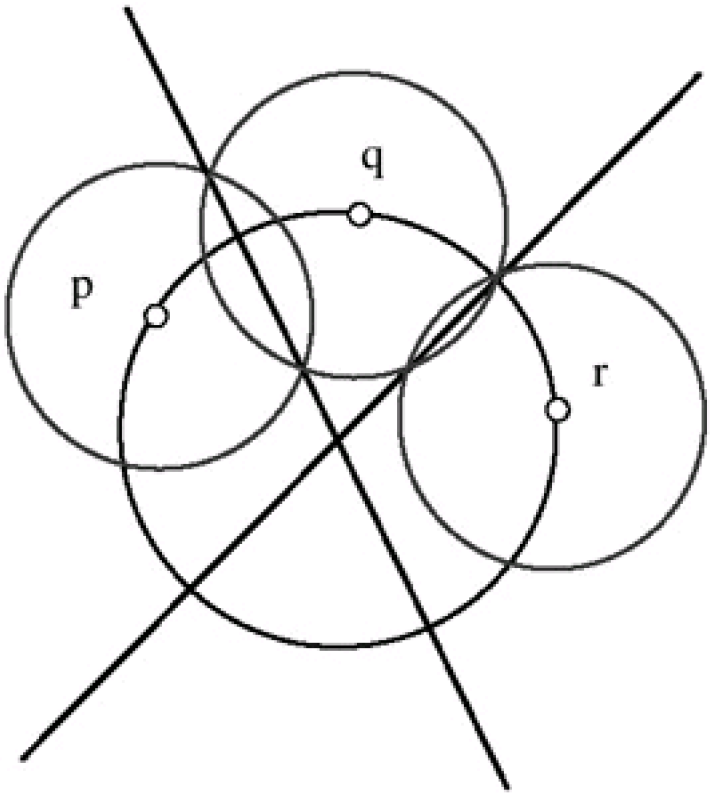


Figure 11.1: Points, circles, and their lines of intersection.

Algorithm

An algorithm for solving this problem is suggested by 11.1. The algorithm has the following steps:

1. Draw intersecting circles *Cp* and *Cq*, centered at points *p* and *q*, respectively.
2. Draw intersecting circles *Cq′* and *Cr*, centered at *q* and *r*, respectively. For simplicity, we assume that *Cq* and *Cq′* are the same circle.
3. Draw line *L*1 through the points of intersection of *Cp* and *Cq*.
4. Draw line *L*2 through the points of intersection of *Cq* and *Cr*.
5. The intersection of *L*1 and *L*2 is the center of the desired circle.

This method will fail if the three points are colinear, as there is no circle passing through three colinear points.

Methodology

We can code this algorithm by representing points, lines, and circles as objects and equipping each class of objects with the necessary operations. Here is a sketch of the classes and operations we need:

Point

Representation
x, y coordinates

Operations

---

equality(anotherPoint) : boolean
distance(anotherPoint) : real

Line

Representation

All lines have the form $ax + by + c = 0$. We may store a, b, and c, normalized so that all three numbers are not too large. When we call the Line class to build a Line object, we will normalize the values of a, b, and c.

Operations

---

parallelto(anotherLine) : boolean
meets(anotherLine) : ref(Point)

The parallelto operation is used to see if two lines will intersect. The meets operation is used to find the intersection of two lines that are not parallel.

Circle

Representation

---

center : ref(Point)

Operations

---

intersects(anotherCircle) : ref(Line)

It should be clear how to solve the problem with these classes of objects. Given two points, we can pass one to the distance function of the other and obtain the distance between the two points. This lets us calculate the radius for intersecting circles centered at the two points. Given two circles, the intersects function finds the line passing through the two points of intersection, and so on.

11.2.3 Sample Code and Representation of Objects

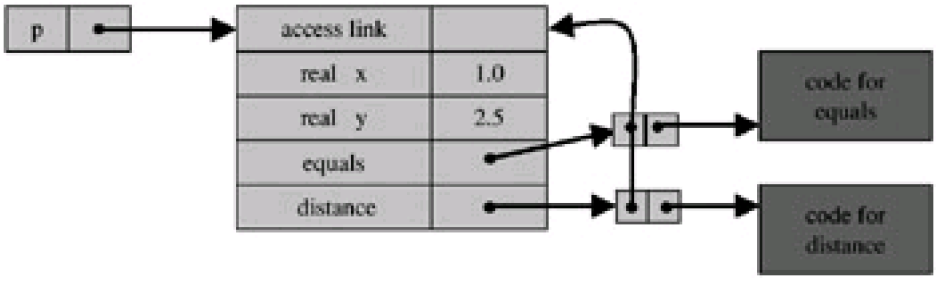
In Simula, the Point class can be written and used to create a new point, as follows:

```
class Point(x,y); real x,y;
begin
   boolean procedure equals(p); ref(Point) p;
      if p =/= none then
         equals := abs(x - p.x) + abs(y - p.y) < 0.00001;
   real procedure distance(p); ref(Point) p;
      if p == none then error else
         distance := sqrt(( x - p.x )**2 + (y - p.y) ** 2);
end ***Point***
p :- new Point(1.0, 2.5);
```

Because all objects are manipulated by refs (the Simula term for pointers), the type of an object variable has the form ref(Class), where Class is the class of the object. Because the equals procedure requires another point as an argument, the formal parameter p is declared to have type ref(Point). Simula references are initialized to a special value none and :- is used for pointer assignment. The test p =/= none (read "p not-equal none") tests whether the pointer p refers to an object.

In the body of the Point class, we access parts of the object, which are locally declared variables and procedures, simply by naming them. Parts of other objects, such as the object passed as a parameter to distance, are accessed with a dot notation, as in p.x and p.y.

When the statement at the bottom of the code example above is executed, it produces a run-time structure that we may draw as follows:



This Point object contains pointers to the closures for Point class procedures and the environment pointer of each closure points to the activation record that is the object. (This means the codes for equals and distance can be shared among all points, but the closure pairs must be different for each object.) When one of these procedures is called, as in p.equals(q), an activation record for the call is created and its access link is set according to the closure for the procedure. This way, when the code for equals refers to x, the x that will be used is the x stored inside the object p.

There are several ways that this representation of objects may be optimized; you should assume only that this representation captures the behavior of Simula objects, not that every Simula compiler actually uses precisely this storage layout. Some useful optimizations are shown in the chapters on Smalltalk and C++, in which each object

stores only a pointer to a table storing pointers to the methods.

To give a little more sample code, the line class may be written in Simula as follows:

```
class Line(a,b,c); real a,b,c;
begin
  boolean procedure parallelto(l); ref(Line) l;
    if l =/= none then
        parallelto := abs(a*l.b - b* l.a) < 0.00001;
  ref(Point) procedure meets(l); ref(Line) l;
  begin real t;
    if l =/= none and ~parallelto(l) then
        begin
        t := 1/(l.a * b - l.b * a);
        meets :- new Point(..., ...);
        end;
  end; ***meets***
  real d;
  d := sqrt(a**2 + b**2);
  if d = 0.0 then error else
    begin
    d := 1/d;
    a:=a*d;b:=b*d;c:=c*d;
    end;
end *** Line***
```

The procedure meets invokes another procedure of the same object, parallelto. The code following the procedure meets is initialization code; it is executed whenever a Line object is instantiated. You might want to think about how this initialization code, which is written as if a class were an ordinary procedure, corresponds to constructor code in object-oriented languages you are familiar with.

Team-Fly

Team-Fly

11.3 SUBCLASSES AND SUBTYPES IN SIMULA

11.3.1 Subclasses and Inheritance

The classes in a Simula program are arranged in a hierarchy. One class is a *superclass* of a second if the second is defined by inheritance from the first. We also say that a class is a *subclass* of its superclass.

Simula syntax for a class C1 with subclasses C2 and C3 is

```
class C1
    <declarations_1> ;
C1 class C2
    <declarations_2>
C1 class C3
    <declarations_3>
```

When we create a C2 object, for example, we do this by first creating a C1 object (activation record) and then appending a C2 object (activation record). In a picture, a C2 object looks like this:

C1 parts of object

C2 parts of object

The structure is essentially the same as if procedure called C2 was declared and called with in C1: The access links of the second activation record refer to the first.

Here is an example code that uses Simula class prefixing to define a colored point subclass of the Point class defined in Subsection 11.2.3:

```
Point class ColorPt(c); color c;      ! List new parameter only
begin
   boolean procedure equals(q); ref(ColorPt) q;
     ...;
end ***ColorPt***
ref (Point) p;                        ! Class reference variables
ref (ColorPt) cp;
p :- new Point(2.7, 4.2);
cp :- new ColorPt(3.6, 4.9, red) ;    ! Include parent class parameters
```

The ColorPt class adds a color field c to points. Because Simula 67 did not hide fields, the c field of a ColorPt object can be accessed and changed directly by use of the dot notation. For example, cp.c := green changes to color of the point named by cp. The ColorPt class redefines equals so that cp.equals can compare color as well as x and y.

The statement p :- New Point(2.7, 4.2) causes an activation record to be created with locations for parameters x and y. These are set to 2.7 and 4.2, respectively and then the body of the Point class is executed. Because the body of the Point} class is empty, nothing happens at this stage for points. After the body is executed, a pointer to the activation record is returned. The activation record contains pointers to function values (closures) equals and distance.

A prefixed class object is created by a similar sequence of steps that involves calls to the parent class before the child class. More specifically, an activation record is created for the parent class and an activation record is created for the child class. Then parameter values are copied to the activation records, parent class first, and the class bodies are executed, parent class first. Some additional details are considered in the exercises.

11.3.2 Object Types and Subtypes

All instances of a class are given the same type. The name of this type is the same as the name of the class. For example, if p and q are variables referring to objects created by the Point class, then they will have type ref(Point). As mentioned in Section 11.2, ref arises because all Simula objects are manipulated through pointers.

The class names (types of objects) are arranged in a *subtype* hierarchy corresponding exactly to the subclass hierarchy. In other words, the only subtype relations that are recognized in Simula 67 are exactly those that arise from inheritance: If class A is derived from class B, then the Simula type checker treats type A as a subtype of type B.

There are some interesting subtleties regarding assignment and subtyping that also apply to other languages. For example, look carefully at the following legal Simula code, in which Simula syntax :- is used for reference (pointer) assignment:

```
class A;
A class B;   /* B is a subclass of A */
ref (A) a;
ref (B) b;
a :- b;      /* legal since B is a subclass of A */
...
b :- a;      /* also legal but checked at run-time to make sure a points to a B object*/
```

Both assignments are accepted at compile time and satisfy Simula's notion of type compatibility. However, a run-time check is needed for the second assignment to guarantee type safety. The same run-time checking also appears in Beta, a cultural descendant of Simula, and in Java array array assignment. The reason for the run-time test is that the object reference a might point to an object of class B or it might point to an object of class A that is not an object of class B. In The first case, the assignment is OK; in the second case, it is not. If an assignment causes an object reference b with static type ref(B) to point to an object that is *not* from class B or some subclass of B, this is a type error.

It is possible to rewrite the assignment to b in the code we just looked at by using Simula inspect:

```
inspect a
   when B do b:-a
   otherwise . . . /* some appropriate action */
```

If a does not refer to a B object, then the otherwise clause will catch the error and let the programmer take appropriate action. However, in the case in which the programmer knows that a refers to a B object, the syntax with an implicit run-time test is obviously simpler.

There was an error in the original Simula type checker surrounding the relationship between subtyping and inheritance. This is illustrated in the following code, extracted from a running DEC-20 Simula program written by Alan Borning. (The DEC-20 version of Simula uses := instead of :- for pointer assignment.)

```
class A;
A class B; /* B is a subclass of A */
ref (A) a;
ref (B) b;
proc assignA (ref (A) x)
    begin
    x:=a
    end;
assignA(b);
```

In the terminology of Chapter 10, Simula subclassing produces the subtype relation B<: A. Simula also uses the semantically incorrect principle that, if B < : A, then ref (B) <: ref(A). Therefore, this code will statically type check, but will cause a type error at run time. The same type error occurs in the original implementation of Eiffel, a language designed many years later.

Team-Fly

Team-Fly

11.4 DEVELOPMENT OF SMALLTALK

The Smalltalk project at Xerox PARC in the 1970s streamlined the object metaphor and drew attention to object-oriented programming. Although the central object-oriented concepts in Smalltalk resemble their Simula ancestors, Smalltalk was not an incremental modification of any previously existing language. It was a completely new language, with new terminology and an original syntax.

This is a list of some of the main advances of Smalltalk over Simula:

- The object metaphor was extended and refined

  In Smalltalk, everything is an object, even a class.

  All operations are therefore messages to objects.

  Objects and classes were shown to be useful organizing concepts for building an entire programming environment and system.

- Abstraction was added, using the distinction between

  private *instance variables* (data associated with an object) and

  public *methods* (code for performing operations)

These short lists, however, do not do justice to the Smalltalk project. Smalltalk was conceived as part of an innovative and forward-looking effort that designed an entire computing system around a new programming language.

Smalltalk is more flexible and more powerful than Simula or other languages that preceded it, and many Smalltalk fans consider it a better language than most that followed. Smalltalk is untyped, allowing greater flexibility in the modification of programs and greater flexibility in the use of general-purpose algorithms on a variety of data objects. In addition, the importance of treating everything as an object cannot be underestimated. This is the source of much of the flexibility of Smalltalk. Smalltalk has many interesting and powerful facilities, including a mechanism that allows objects to detect a message they do not understand and to respond to the message.

Dynabook

Part of the inspiration for the Smalltalk language and system came from Alan Kay's concept of a *Dynabook*. The Dynabook was to be a small portable computer that was capable of storing useful and interesting personal information and running programs for a variety of applications. It was a laptop computer with business and entertainment software, and sufficient storage space to store telephone numbers, business data, or a copy of a popular novel. It is almost impossible to explain, in the present day, how utterly improbable this sounded in the 1970s. At the end of the 1970s, a minicomputer, typically shared by 15 or 20 people, required the same floor space as a small laundromat and had roughly the same appearance - rows of machines, some with visible spinning drums. Very-large-scale integrated (VLSI) circuits were not a reality until the late 1970s, so there was really no way the Dynabook could conceivably have been manufactured at the time it was imagined. To give another historical data point, the Xerox Alto of the 1970s was an advanced personal computer that cost $32,000 at the time, more than the annual salary of a computer professional. It had removable disks, which each person would use to store personal working data. Each disk was 2ft in diameter, a couple inches thick, and weighed 5-10 lb! Hardly the sort of thing you could carry around in your pocket or even imagine inserting into a portable computer. Not only was Kay's prediction of hardware miniaturization revolutionary, but the idea that people would use a computer for a wide range of personal activities was shocking.

Smalltalk was intended to be the operating system interface as well as the standard programming language for Dynabook; all of a user's interaction with the Dynabook would be through Smalltalk. With this in mind, the syntax of Smalltalk was designed to be used with a special-purpose Smalltalk editor. Because the Xerox group believed that hardware would eventually catch up with the need for computing power, the Smalltalk implementation emphasized

flexibility and ease of use over efficiency.

Team-Fly



11.5 SMALLTALK LANGUAGE FEATURES

11.5.1 Terminology

The Smalltalk project developed precise terminology for the main Smalltalk concepts and constructs. Here is a list of the main object-related terms and brief descriptions:

- *Object*: A combination of private data and functions. Each object is an *instance* of some class.
- *Class*: A template defining the implementation of a set of objects.
- *Subclass*: Class defined by inheriting from its superclass.
- *Selector*: The name of a message (analogous to a function name).
- *Message:* A selector together with actual parameter values (analogous to a function call).
- *Method*: The code in a class for responding to a message.
- *Instance Variable*: Data stored in an individual object (instance of a class).

The way these terms were used in the Smalltalk language and documentation is close to current informal use among many object-oriented professionals, but not entirely universal. For example, some people use "method" to refer to both the name of a method (which Smalltalk would call a selector) and the code used to respond to a message.

11.5.2 Classes and Objects

The Smalltalk environment included a special editor designed for the Smalltalk language. With this editor, classes were defined by filling in a table of a certain form, illustrated with the definition of a Point class in Figure 11.2.

class name	Point
super class	Object
class var	pi
instance var	x y
class messages and methods	
?…names and code for methods.?	
instance messages and methods	
?…names and code for methods.?	

Figure 11.2: Definition of Point class.

In Smalltalk terminology, a class variable is a variable that is defined in the class and shared among all objects of the class. In this example, the value of pi may be used in geometric calculations involving points. Therefore, the class includes a variable called pi. An instance variable is a variable that is repeated for each object. Each point object, for example, will have two instance variables, x and y, as each point may be located at a different position in the *x-y* plane. Methods are functions that may be used to manipulate objects, and messages contain the names and parameters of methods.

Here are some example class messages and methods for point objects (subsequently explained):

```
newX:xvalue Y:yvalue ||
        ^ self new x: xvalue y: yvalue
newOrigin ||
        ^ self new x: 0 y: 0
        initialize ||
            pi <- 3.14159
```

and some example instance messages and methods:

```
x: xcoord y: ycoord ||
    x <- xcoord
    y <- ycoord
moveDx: dx Dy: dy ||
    x<-dx + x
    y<-dy + y
x || ^x
y || ^y
draw ||< < . . . code to draw point ...> >
```

These are written with Smalltalk syntax, which includes the following:

```
|| --place to list local declarations for method body
^ - unary prefix operator giving return value of method
<- - assignment
```

Smalltalk message names include positions for message arguments, indicated by colons.

The first class method, `new X:Y:`, is a two-argument method that creates a new point. The two arguments are the *x* coordinate and the *y* coordinate of the new point. A new point at coordinates (3, 2) is created when the message

```
newX: 3 Y:2
```

is sent to the Point class, with *x* coordinate 3 following the first colon and *y* coordinate 2 following the second coordinate. When we look at inheritance and other situations in which the name of a method is important, it will be useful to remember that the official Smalltalk name of a message contains all the words and colons, as in "new X:Y:."

The newX:Y: method is a class method of the Point class, which means that if you want to create a new point with specific *x* and *y* coordinates, you may send a newX:Y: message to the Point class (which is itself an object). The newX:Y: method has no local variables, as any local declarations would appear between the vertical bars following the name. The effect of the method is indicated by the expression

```
^ self new x: xvalue y: yvalue
```

As previously mentioned, the first symbol, ∧, means "return." The value that is returned is the object produced when the message new x: xvalue y: yvalue is sent to the Point class. (Self refers to the object that contains the method; this is discussed in Subsection 11.6.3.) This message is handled by a predefined class method, provided by the system, that allows any class to create instances of it.

To continue the example, suppose we execute the following code:

```
p <- Point newOrigin
p moveDx: 3 Dy: 4
```

This will create a point at the origin, (0, 0), and then move the point to position (3, 4). Thereafter, the value of expression

```
px
```

that sends the message x to the object p will be the object 3 and the value of

that sends the message y to the object p will be the object 4.

Run-Time Representation of Objects

The run-time representation of a Smalltalk object has a number of parts. Each point object, for example, may have a different *x* coordinate and *y* coordinate. Therefore, each object must contain space for an *x* coordinate and a *y* coordinate. The methods, on the other hand, are shared among all point objects. To save space and take advantage of the similarity between objects of the same class, each object contains a pointer to a data structure containing information associated with its class. The basic run-time structure for points is illustrated in Figure 11.3. This illustration is intended to show the basic functionality of Smalltalk classes and objects; actual implementations might store things differently or perform various optimizations to improve performance.

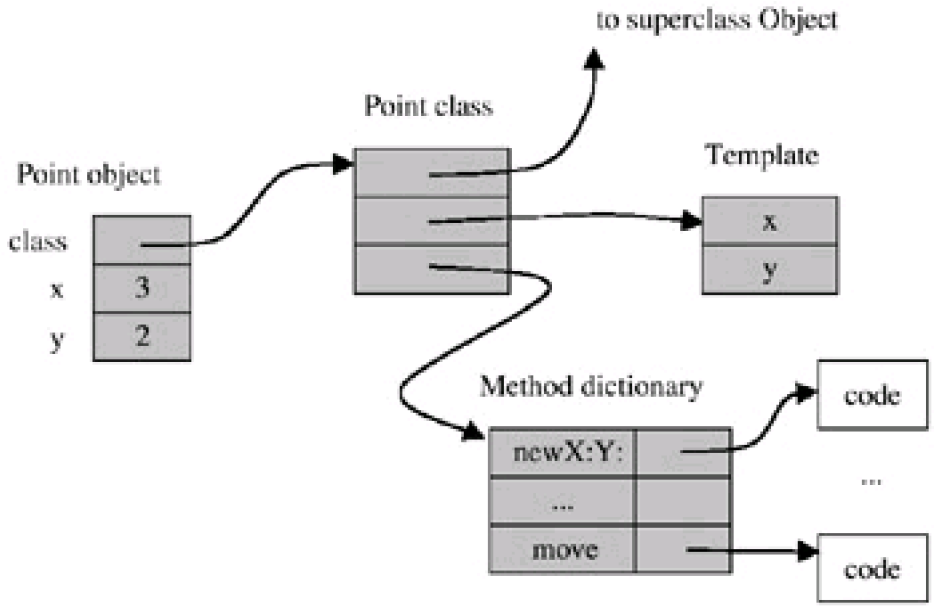


Figure 11.3: Run-time representation of Point object and class.

When a message, such as move, is sent to a point object, the code for move is located by following a pointer to the class of the object, following another pointer to the method dictionary, and then looking up the code in this data structure. If the code for a method is not found in the method dictionary for the class, the method dictionaries of superclasses are searched, as described in the next subsection on inheritance.

11.5.3 Inheritance

We look at Smalltalk inheritance by considering a subclass of Point that implements colored points. The main difference between ColoredPoint and Point is that each colored point has an associated color. If we want to provide methods for the same geometric calculations, then we can make colored points a subclass of points. In Smalltalk, the ColoredPoint class can be defined as shown in Figure 11.4.

class name	ColoredPoint
super class	Point
class var	

instance var	color
class messages and methods	
newX:xv Y:yv C:cv	?…code..?
instance messages and methods	
color	? °color
draw	?…code..?

Figure 11.4: Definition of ColoredPoint class.

The second line of Figure 11.4 shows that Point is the superclass of ColoredPoint. As a result, all instance variables and methods of Point become instance variables and methods of ColoredPoint by default. In Smalltalk terminology, ColoredPoint *inherits* instance variables *x* and *y*, methods x:y:, moveDx:Dy:, and so on. In addition, ColoredPoint adds an instance variable color and a method color to return the color of a ColoredPoint.

Although ColoredPoint inherits the draw method from Point, the draw method for ColoredPoint must be implemented differently. When we draw a ColoredPoint, the point should be drawn in color. Therefore, Figure 7.1 shows a draw method, overriding the one inherited from Point. (Redefinition of a method is sometimes called *overriding* the method.) An option that is available in Smalltalk, but not in most other object-oriented languages, is to specify that a superclass method should be undefined on a subclass.

Because colored points should have a color specified when they are created, the ColoredPoint class will have a class method that sets the color of a colored point:

```
newX: xvalue Y:yvalue C:cvalue ||
    ^ self new x: xvalue y:yvalue color:cvalue
```

As an example use of ColoredPoint, suppose we execute the following code:

```
cp <- ColoredPoint newX:1 Y:2 C:red
cp moveDx: 3 Dy: 4
```

Then the value of the expression

```
cp x
```

that sends the message x to object cp will be the object 4, and the value of the expression

```
cp color
```

that sends the message color to the object cp will be the object red. Note that even though move is an inherited method, defined originally for points without color, the result of moving a ColoredPoint is a ColoredPoint.

Run-Time Structure to Support Inheritance

The run-time structures for a ColoredPoint object and ColoredPoint class are illustrated in Figure 11.5. The illustration shows a ColoredPoint object, with the object's class pointer pointing to the ColoredPoint class object. The ColoredPoint class contains a superclass pointer that points to the Point class object. Like the Point class, the ColoredPoint class also has a pointer to a template, which contains the names of the instance variables, and a method dictionary, which contains pointers to code that implements each of the methods. Like Figure 11.3, this figure is schematic. An actual compiler could store things slightly differently or perform optimizations to improve performance.

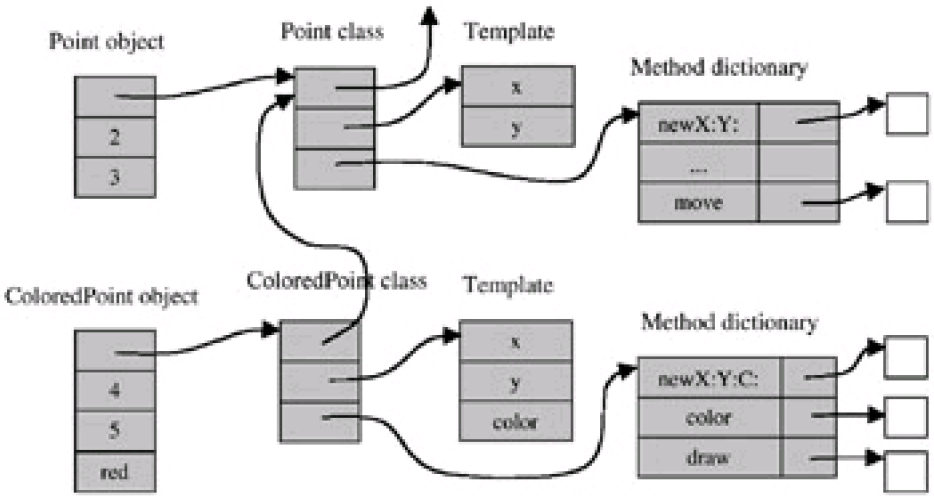


Figure 11.5: Run-time representation of ColoredPoint object and class.

If a draw message is sent to a ColoredPoint object, then we find the code for executing this method by following the pointer in the object to the ColoredPoint class structure and by following the pointer there to the method dictionary for the ColoredPoint class. The dictionary is then searched for a method with the right name. If a message other than new, color, and draw is sent, then the method name will not be found in the ColoredPoint method dictionary. In this case, the superclass pointer in the ColoredPoint class is used to locate the Point class and the Point method dictionary. Then the search continues as if the object were an instance of its superclass.

It should be clear from these data structures that looking up a method at run time can involve a large number of steps. If class A inherits from class B and class B inherits from class C, for example, then searching for an inherited method could involve searching all three method dictionaries at run time. If the operation performed by this method is simple, then it might take more time to find a method than to execute it.

This run-time cost of Smalltalk method lookup illustrates one of the basic principles of programming language design: *You can't get something for nothing*. Dynamic lookup is a powerful programming feature, but it has its costs.

Fortunately, there are optimizations that reduce the cost of method lookup in practice. One common optimization (also used in Java) is to cache recently found methods at the method invocation site in a program. If this is done, then a second invocation of the same method can be executed by a jump directly to the code location stored in the cache. As discussed in the next chapter, static type information can also be used to significantly reduce the cost of method lookup. This makes lookup more efficient in C++. However, optimizations based on static type information do not work in Smalltalk because Smalltalk has no static type system.

There are some interesting details surrounding the way that instance variables are accessed by methods. Because the code for the draw method in the ColoredPoint class is compiled at the same time that the layout of ColoredPoint objects is determined, the ColoredPoint method can be compiled so that if it needs to access instance variables of a ColoredPoint object, it can locate them easily (given a pointer to the object). Specifically, the off set of the *x* coordinate, *y* coordinate, and color, relative to the starting address of the ColoredPoint object, is known at compile time and can be used in any ColoredPoint method. The same is true for Point methods. However, because a Point method can be inherited by ColoredPoint and invoked on a ColoredPoint object, it is necessary for Point instance variables to appear at the same relative offset in both Point and ColoredPoint objects. More generally, accessing instance variables by use of offsets known at compile time requires the layout of a subclass object to resemble the layout of a superclass object. You might want to think about the advantages and disadvantages of compiling methods in this way compared with run-time search for instance variables by using the class templates. One consequence of using compile-time offsets is that, when a superclass is changed and a new instance variable is added, all subclasses must be recompiled. (This was the case for the Xerox PARC Smalltalk compiler.)

11.5.4 Abstraction in Smalltalk

In most object-oriented languages, a programmer can decide which parts of an object are visible to clients of the class and which parts are not. Smalltalk makes this decision for the programmer by using particularly simple rules:

- *Methods are Public:* Any code with a pointer to an object may send any message to that object. If the corresponding method is defined in the class of the object, or any superclass, the method will be invoked. This makes all methods of an object visible to any code that can access the object.
- *Instance Variables are Protected:* The instance variables of an object are accessible only to methods of the class of the object and to methods of its subclasses.

This is not a fully general approach to abstraction. In particular, there are practical situations in which it would be useful to have private methods, accessible only to other methods of the same class. However, the Smalltalk visibility rules are simple, easy to remember, and sufficient in many situations.

Team-Fly

Team-Fly

11.6 SMALLTALK FLEXIBILITY

11.6.1 Dynamic Lookup and Polymorphism

Advocates of object-oriented languages often speak about the "polymorphism" inherent in object-oriented code. When they say this, they mean that the same message name invokes different code, depending on the object that receives the message. In this book, we refer to this phenomenon as dynamic lookup, as explained in Chapter 10.

Here is an example that illustrates the value of dynamic lookup. Consider a list of objects, some of which are points and others of which are colored points. If we want to display every point on the list, we may traverse the list, sending each object the draw method. Because of dynamic lookup, the Point objects will execute the Point draw method and the ColoredPoint objects will execute the ColoredPoint draw method. Dynamic lookup supports code reuse, as one routine will display lists of Points, lists of ColoredPoint, and lists that have both Points and ColoredPoints.

The run-time structures used for Smalltalk classes and objects support dynamic lookup in two ways. First, methods are selected through the receiver object. Second, method lookup starts with the method dictionary of the class of the receiver and then proceeds upward. The first method found with the appropriate name is selected.

The following code sends a draw message to a Point and a ColoredPoint:

```
p <- Point newOrigin
p draw
p <- ColoredPoint newX:1 Y:2 C:red
p draw
```

Be sure you understand how and why different code is used in lines two and four to draw the two different kinds of objects.

11.6.2 Booleans and Blocks

An assumption that many people make when learning about Smalltalk is that it has the same kind of syntax, with specific reserved words and special forms, as other languages.

However, many constructs that would be handled with a special-purpose statement form in other languages can be handled uniformly in Smalltalk with the principle that *everything is an object*. A good example is the Smalltalk treatment of Booleans, which uses the built-in construct of blocks.

A Smalltalk Boolean object (true or false) has a selector named ifTrue:ifFalse:. The method associated with this selector requires two parameters, one to be executed if the object is true and the other if the object is false. The arguments to this method are generally Smalltalk *blocks*, which are objects that have a message, "execute yourself." Although blocks may be defined by special in-line syntax, for convenience, they are not essentially different from other Smalltalk objects.

The reason for using blocks in Smalltalk Booleans is easy to understand if we think about conditionals in other languages. In general, the construct

```
if . . . then . . . else . . .
```

cannot be defined as an ordinary function, as an ordinary function call would involve evaluating all arguments before calling the function. If we evaluated A, B, and C first in

```
if A then B else C
```

this is inefficient and incorrect if B and C have side effects or fail to terminate. We want to evaluate B only in the case that A is true and evaluate C only in the case that A is false.

For example, consider the following expression:

```
i< j ifTrue:[i add: 1] ifFalse: [j subtract: 1]
```

This is the Smalltalk way of writing

```
if i < j then i+1 else j-1
```

The first part of the expression, i<j, is a<j message to object i, asking it to compare itself with j. The result will be either the object true or the object false, in either case an object that understands an ifTrue:ifFalse: message. Therefore, after the comparison, i<j, one of the two subsequent blocks will be executed, depending on the result of the comparison.

The way Smalltalk actually works is that the class Boolean has two subclasses, True and False, each with one instance, called true and false, respectively. In these classes, the ifTrue:ifFalse: method is implemented as follows:

```
True class ...
   ifTrue: trueBlock ifFalse: falseBlock
      trueBlock value
False class ...
    ifTrue: trueBlock ifFalse: falseBlock
       falseBlock value
```

---

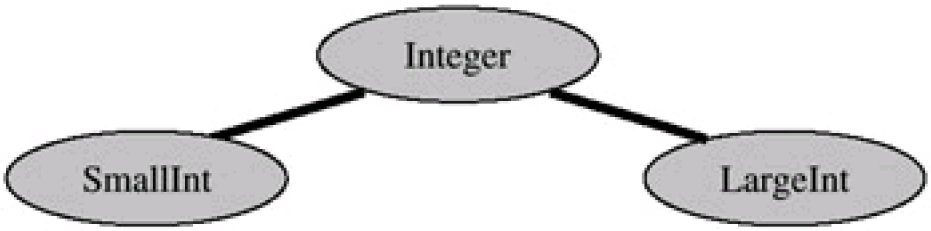
where value is the message you send to a block if you want it to evaluate itself and return its value.

There is a little bit of "special treatment" here in that the system will not allow more instances of classes True and False to be created. This is to guarantee correctness of an optimization. For efficiency, the standard idiom ifTrue: […] ifFalse: […] is optimized by the compiler in most Smalltalk implementation. However, this is again simply an optimization for efficiency. In principle, Booleans could be defined within the language. In addition, the unoptimized methods previously given are called if any arguments are variables that might hold blocks.

11.6.3 Self and Super

The special symbol self may be used in the body of a Smalltalk method. The special property of self is that it always refers to the object that contains this method, whether directly or by inheritance. The way this works is most easily explained by example.

There are several classes of numbers in Smalltalk. Three are arranged in the following hierarchy:



Even if the Integer class does not create any objects, we can define the following method in the Integer class:

```
factorial ||
    self <= 1
    ifTrue: [^ 1]
    ifFalse: [^(self - 1) factorial * self]
```

This method computes the factorial of the integer object to which it belongs. The method first asks the object to compare itself with the predefined number object 1. If it is less-than-or-equal, the result will be the object true and the block [∧1] will be evaluated, giving the result 1. If the object is greater than 1, then the second block[∧(self - 1) factorial * self] will be evaluated, causing a recursive call to the factorial method on a smaller integer.

There are two operations in the ifFalse case of the factorial method that could be implemented in different ways in the two subclasses of Integer. The first is - (minus). Because small integers may have a different representation from large integers, subtraction may be implemented differently in the two classes. Multiplication, *, may also be implemented

differently in the two classes. These methods do not have to return objects of the same class. For example, n*m could be a SmallInt if n and m are both small, or could be a LargeInt if both are SmallInts and their product is larger than the maximum size for SmallInt.

In Smalltalk, we begin a message to self by beginning the search for each method at the object where the first message is received. For example, suppose we want to compute the factorial of a small integer n. The computation will begin by sending the factorial message to n. Because the first step of factorial is self <= 1, the message <= 1 will be sent to the object n. It is important that dynamic lookup is used here, as the correct method for comparing n with 1 is the <= method defined in the SmallInt class. In fact, the Integer class does not need to have a <= method defined in order for the calculation of factorial to work properly. To appreciate the value of self, you might think about whether you could write a factorial method in class Integer that would work properly on SmallInt and LargeInt without using self.

The special symbol super is similar to self, except that, when a message is sent to super, the search for the appropriate method body starts with the superclass of the object instead of the class of the object. This mechanism provides a way of accessing a superclass version of a method that has been overidden in a subclass.

11.6.4 System Extensibility: The Ingalls Test

Dan Ingalls, a leader of the Smalltalk group, proposed the following test for determining whether a programming language is truly object oriented:

> Can you define a new kind of integer, put your new integers into rectangles, ask the system to blacken a rectangle, and have everything work?

This test is probably the most extreme test of an object-oriented language. Essentially he is asking "Is *everything* an object?" Or, more specifically, "Is every operation determined by dynamic lookup through objects?"

Why is This Important?

The main issue addressed by this test is the extensibility of programs and systems. If we build a graphic-oriented system that manipulates and displays various kinds of objects, then the design and implementation of basic objects like points and coordinates might be determined very early in the design process. If a complex system is built around one basic construct, like standard language-supplied integers, then how easy will it be to modify the system to use another form of integers?

The answer, of course, is that it depends on how you build the system. In Smalltalk, however, the only way of building a system is by representing basic concepts as objects. Because all operations in Smalltalk are done by sending messages and every message results in run-time lookup of the appropriate method body, a lot of flexibility is inherently built into any system. In C++ and Java, by contrast, it is possible to build a system with integers that are not objects. As a result, it is possible to build systems that will not immediately accept other forms of integer without some change to the code. In particular, most Java or C++-based systems will not be able to handle a rectangle that has some coordinates given by built-in integers that are not objects and some coordinates given by objects.

What Are the Implementation Costs?

If a system is built so that every operation on integers is accompanied by method lookup, then an expression x add y sending a message to an integer cannot be safely optimized to a static function call such as integer_add(x,y). If the compiler or programmer replaces x add y with integer_add(x,y), then we cannot replace a built-in integer x with a user-defined integer object x.

If a program uses integers repeatedly and each integer operation involves a run-time search for an appropriate method, this is a *substantial run-time cost*, even if the best-known optimizations are used to reduce running time as much as possible. In fact, for essentially this reason, the efficiency of many Smalltalk programs is 5 to 10 times less than corresponding programs in C. Of course, program running time is not everything. If a system will be used in many ways, and flexibility and adaptability are important, then flexibility and adaptability may be more important than running speed.

Team-Fly

11.7 RELATIONSHIP BETWEEN SUBTYPING AND INHERITANCE

11.7.1 Interfaces as Object Types

Although the Smalltalk language does not use any static type checking, there is an implicit form of type that every Smalltalk programmer uses in some way. The type of an object in Smalltalk is its interface, the set of messages that can be sent to the object without receiving the error "message not understood." The interface of an object is determined by its class, as a class lists the messages that each object will answer. However, different classes may implement the same messages, as there are no Smalltalk rules to keep different classes from using the same selector names.

The Smalltalk library contains a number of implemented classes for a variety of purposes. One part of the library that provides interesting examples of interfaces and inheritance is the collection-class library. The collection classes implement general collections like Set, Bag, Dictionary, LinkedList, and a number of more specialized kinds of collections. Some example interfaces from the collection classes are the following sets of selector names:

```
Set_Interface =  {isEmpty, size, includes:, removeEvery:}
Bag_Interface = {isEmpty, size, includes:, occurrencesOf:, add:, remove:}
Dictionary_Interface = {at:put:, add:, values, remove:}
Array_Interface = {at:put:, atAllPut:, replaceFrom:to:with:}
```

Remember that a colon indicates a position for a parameter. For example, if s is a set object, then s includes:3 sends the message includes:3 to s. The result will be true or false, depending on whether the set s contains the element 3.

Here is a short explanation of the intended meaning of each of the selectors:

isEmpty: return true if the collection is empty, else false

size: return number of elements in the collection

includes:x: return true if x is in the collection, false otherwise

removeEvery:x: remove all occurrences of x from the collection

occurrencesOf:x: return a number telling how many occurrences of x are in the collection

add:x: add x to the collection

remove:x: remove x from the collection

at:x put:y: insert value y into an indexed collection at position indexed by x

values: return the set of indexed values in the indexed collection (e.g., list all the words in a dictionary)

atAllPut:x: in finite updateable collection such as an array, set every value to x

replaceFrom:x t:y with:v: in range from x to y, set every indexable value to v (e.g.,for array A, set A[x]=A[x+1]= … =A[y]=v)

11.7.2 Subtyping

Because Smalltalk is an untyped language, it may not seem relevant to compare subtyping and inheritance. However, if we recall the definition of a subtype,

> Type A is a subtype of a type B if any context expecting an expression of type B may take any expression of type A without introducing a type error,

we can see that subtyping is about substitutivity. If we build a Smalltalk program by using one class of objects and then try to use another class in its place, this can work if the interface of the second class contains all of the methods of the first, but not necessarily otherwise. Therefore, it makes sense to associate subtyping with the *superset* relation on Smalltalk class interfaces. Subtyping is sometimes called conformance when discussing interfaces instead of types of some typed programming language.

An example of subtyping arises if we consider extensible collections. To be specific, let us define ExtensibleCollection_Interface by

---

ExtensibleCollection_Interface = {isEmpty, size, includes:, add:, removeEvery:}

This is the same as Set_Interface, with the method add: added. Using <: for subtyping, we have

---

ExtensibleCollection Interface <: Set_Interface

In words, this means that every object that implements the ExtensibleCollection interface also implements the Set interface. Semantically, this makes sense because a program that uses Set objects can send any of the messages listed in the Set interface to Set objects. Because each one of these messages is also implemented in every ExtensibleCollection object, we can substitute ExtensibleCollection objects for Set objects without obtaining "message not understood" errors.

11.7.3 Subtyping and Inheritance

In Smalltalk, the interface of a subclass is often a subtype of the interface of its superclass. The reason is that a subclass will ordinarily inherit all of the methods of its superclass, possibly adding more methods. This is certainly the case for Point and ColoredPoint, as described earlier in this chapter. Points have the five methods x:y:, moveDx:Dy:, x,y, and draw. Colored points have these five methods and a color method, so every colored point has all of the point methods.

In general, however, subclassing does not always lead to subtyping in Smalltalk. The simplest way that subtyping may fail is when a subclass does not implement all of the methods of the superclass. In Smalltalk, there is simply a way of saying, in a subclass, that one or more superclass methods should not be defined on subclass objects. Because it is

possible to delete a method from a superclass, a subclass may not produce a subtype.

On the other hand, it is easy to have subtyping without inheritance. Because subtyping, as we have defined it, is based only on the names of methods, two classes could be defined independently (neither a subclass of the other) but have exactly the same methods. Then, according to our definitions, each is a subtype of the other. But there is no inheritance.

William Cook, a researcher at Hewlett-Packard Labs and later a developer at Apple Computer, did an empirical study of the collection classes in the Smalltalk-80 class library to determine which relationships between subtyping and inheritance arise in Smalltalk practice (Interfaces and Specifications for the Smalltalk-80 Collection Classes, ACM Conf. Object. Oriented Programming: Systems, Languages, and Applications, ACM SIGPLAN Notices 27(10), 1992). Among a relatively small number of classes, he found all possible relationships between subtyping and inheritance. A diagram from his study is shown in Figure 11.6. In this diagram, solid lines sindicate subtyping between interfaces, and dotted curves show actual places where inheritance is used in the construction of the collection classes of the Smalltalk-80 library. In studying this diagram, it is important to keep in mind that there are more interfaces listed here than were actually implemented in the library. To show the structural relationship between classes more clearly, Cook included some interfaces that are intersections of the interfaces of classes in the library.

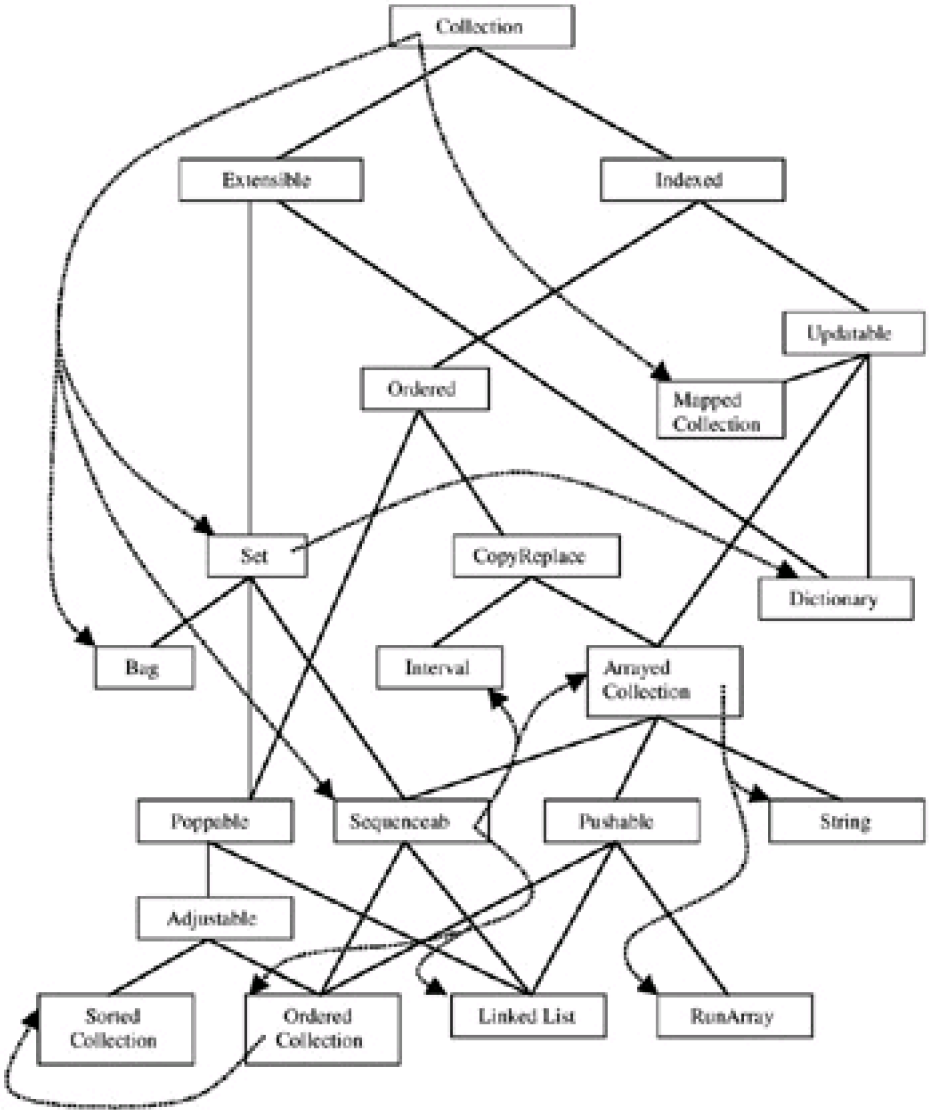


Figure 11.6: Smalltalk collection class hierarchy

Generally speaking, the subtyping relation between a set of classes is independent of how the classes are implemented, but the inheritance relation is not. Subtyping is a relation between types, which is a property that is important to users of an object, whereas inheritance is a property of implementations. Imagine a group of people sitting down to design the Smalltalk collection classes. Once they have agreed on the names of the classes and the operations on each class, they have agreed on the subtyping relation between the classes. However, the inheritance

relation is not determined at this point. If each person implemented one of the classes independently, then all of the classes could be built without using inheritance at all. At the other extreme, it is often possible to factor the functionality of a set of related classes so that inheritance is used extensively. To give a specific example, arrays and dictionaries are similar kinds of collections. Abstractly, both are a form of function from a finite set to an arbitrary set. Therefore, it is conceivable that a programmer could build a general finite-function class and use this as the dictionary class. This programmer can inherit from dictionary to produce the special case of arrays in which the index set is always a range of integers. On the other hand, a different programmer could first build arrays and then use arrays to implement dictionaries by hashing each index word to an integer value and using this to store the indexed value in a specific array location. This illustrates the independence of subtyping and inheritance in Smalltalk.

Team-Fly



Previous Next

Team-Fly

11.8 CHAPTER SUMMARY

In this chapter, we studied Simula, the first object-oriented language, and Smalltalk, an influential language that generalized and popularized object-oriented programming.

Simula was designed in Norway, beginning as part of a project to develop a simulation language. Smalltalk was designed at Xerox PARC (in Palo Alto, California) and was intended to accompany a futuristic concept called the Dynabook. An important unifying concept in Smalltalk is that everything should be an object.

Simula

- *Objects:* A Simula object is an activation record produced by call to a class.
- *Classes:* A Simula class is a procedure that returns a pointer to its activation record. The body of a class may initialize the object it creates.
- *Abstraction:* Hiding was not provided in 1967, but was added later and used as the basis for C++.
- *Subtyping:* Objects are typed according to the classes that create them. Subtyping is determined by class hierarchy.
- *Inheritance:* A Simula class may be defined as an extension of a class that has already been defined.

Smalltalk

- *Objects:* A Smalltalk object is created by a class. At run time, an object stores its instance variables and a pointer to the instantiating class.
- *Classes:* A Smalltalk class defines class variables, class methods, and the instance methods that are shared by all objects of the class. At run time, the class data structure contains pointers to an instance variable template, a method dictionary, and the superclass.
- *Abstraction:* Abstraction is provided through protected instance variables. All methods are public but instance variables may be accessed only by the methods of the class and methods of subclasses.
- *Subtyping:* Smalltalk does not have a compile-time type system. Subtyping arise simplicitly through relations between the interfaces of objects. Subtyping depends on the set of messages that are understood by an object, not the representation of objects or whether inheritance is used.
- *Inheritance:* Smalltalk subclasses inherit all instance variables and methods of their superclasses. Methods defined in a superclass may be redefined in a subclass or deleted.

Because Smalltalk has no compile-time type system, a variable may point to any object of any type at run time. This makes it more difficult to implement efficient method lookup algorithms for Smalltalk than for some other object-oriented languages. You may wish to review the data structure and algorithms for locating a method of an object as illustrated in Subsection 11.5.3. Be sure you understand how search for a method proceeds from subclass to superclass in the event that the method is not defined in the subclass.

Team-Fly

EXERCISES

11.1 Simula Inheritance and Access Links

In Simula, a class is a procedure that returns a pointer to its activation record. Simula prefixed classes were a precursor to C++ -derived classes, providing a form of inheritance. This question asks about how inheritance might work in an early version of Simula, assuming that the standard static scoping mechanism associated with activation records is used to link the derived class part of an object with the base class part of the object.

Sample Point and ColorPt classes are given in the text. For the purpose of this problem, assume that if cp is a ColorPt object, consisting of a Point activation record followed by a ColorPt activation record, the access link of the parent class (Point) activation record points to the activation record of the scope in which the class declaration occurs, and the access link of the child-class (ColorPt) activation record points to activation record of the parent class.

a. Fill in the missing information in the following activation records, created by execution of the following code:

```
ref(Point) p;

ref(ColorPt) cp;

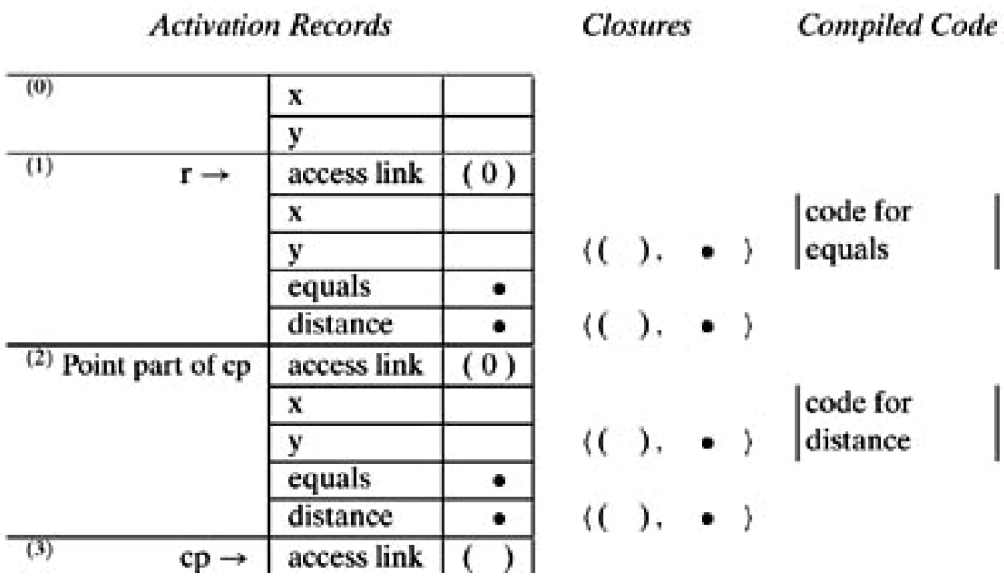
r :- new Point(2.7, 4.2);

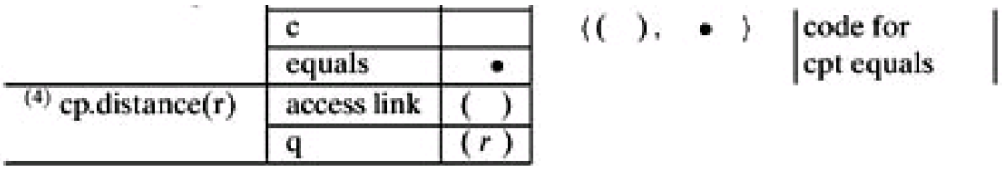
cp :- new ColorPt(3.6, 4.9, red);

cp.distance(r);
```

Remember that function values are represented by closures and that a closure is a pair consisting of an environment (pointer to an activation record) and a compiled code.

In the following illustration, a bullet (*) indicates that a pointer should be drawn from this slot to the appropriate closure or compiled code. Because the pointers to activation records cross and could become difficult to read, each activation record is numbered at the far left. In each activation record, place the number of the activation record of the statically enclosing scope in the slot labeled "access link." The first two are done for you. Also use activation record numbers for the environment pointer part of each closure pair. Write the values of local variables and function parameters directly in the activation records.





b. The body of distance contains the expression

```
sqrt(( x - q.x )**2 + (y - q.y) ** 2)
```

that compares the coordinates of the point containing this distance procedure to the coordinate of the point q passed as an argument. Explain how the value of x is found when cp.distance(r) is executed. Mention specific pointers in your diagram. What value of x is used?

c. This illustration shows that a reference cp to a colored point object points to the ColorPt part of the object. Assuming this implementation, explain how the expression cp.x can be evaluated. Explain the steps used to find the right x value on the stack, starting by following the pointer cp to activation record (3).

d. Explain why the call cp.distance(r) needs access to only the Point part of cp and not the ColorPt part of cp.

e. If you were implementing Simula, would you place the activation records representing objects r and cp on the stack, as shown here? Explain briefly why you might consider allocating memory for them elsewhere.

11.2 Loophole in Encapsulation

A priority queue (also known as a heap) is a form of queue that allows the minimum element to be retrieved. Assuming that Simula has a list type, a Simula pq class might look something like this:

```
class pq();

begin
        (int list) contents;

        bool procedure isempty();
                < … return true if pq empty, else false …>

        procedure insert(x); int x;
                < … put x at appropriate place in list …>

        int procedure deletemin();
                < … delete first list elt and return it …>

        contents := nil

end
```

where the sections <…> would be filled in with appropriate executable Simula code. We would create a priority queue object by writing

```
h :- new pq();
```

and access the components by writing something like

```
h.contents;

h.isempty;

h.insert(x);

h.deletemin();
```

In ML we can write a function that produces a closure with essentially the same behavior as that of a Simula priority

queue object. This is subsequently written, together with the associated exception declaration:

```
exception Empty;

fun new_pq() =
        let val contents = (ref nil) : int list ref
                fun insert(x,nil) = [x]
                | insert(x, y::l) = if x<y then x::(y::l)
                else y : : insert(x, l)
        in
                {
                emp_meth = fn () => !contents = nil,
                ins_meth = fn x => (contents := insert(x, !contents)),
                del_meth = fn () => case !contents of
                        nil => raise Empty
                        | y::l => (contents := l; y),
        }
end;
```

This question asks you to think about a "loophole" in the Simula object system that allows a client program to interfere with the correct behavior of a priority queue object.

a. The main property of a priority queue is that if *n* elements are inserted and *k* are removed (for any $k < n$), then these *k* elements are returned in increasing order. Explain in two or three sentences why both the Simula and ML forms of priority queue objects should exhibit this behavior in a "reasonable" program.

b. Explain in a few sentences how a "devious" program that uses a Simula priority queue object `h` could cause the following behavior: After `0` and `1` are inserted into the priority queue by calls `h.insert(0)` and `h.insert(1)`, the first `deletemin` operation on the priority queue returns some number other than `0`. (This will not be possible for an ML priority queue object.) Do not use any pointer arithmetic or other operations that would not be allowed in a language like Pascal.

c. Write a short sequence of priority queue operations of the form

```
h :- new pq();
h.insert(0);
h.insert(1);
…
h.deletemin();
```

where the ellipsis (…) may contain `h` operations, but not `insert` or `deletemin`, demonstrating the behavior you described in part (b).

11.3 Subtyping of Refs in Simula

In Simula, the procedure call `assignA(b)` in the following context is considered statically type correct:

```
class A … ; /* A is a class */
A class B …; /* B is a subclass of A */
ref (A) a; /* a is a variable pointing to an A object */
```

```
ref (B) b; /* b is a variable pointing to a B object */

proc assignA (ref (A) x)
                    begin
                                        x:=a

                    end;

assignA(b);
```

a. Assume that *if B* <: *A then ref (B)* <: *ref (A)*. Using this principle, explain why both the procedure assignA and the call assignA(b) can be considered statically type correct.

b. Explain why actually executing the call assignA(b) and performing the assignment given in the procedure may lead to a type error at run time.

c. The problem is that the "principle," *if B* <: *A then ref (B)* <: *ref (A)*, is not semantically sound. However, we can make type checking by using this principle sound by inserting run-time tests. Explain the run-time test you think a Simula compiler should insert in the compiled code for procedure assignA. Can you think of a reason why the designers of Simula might have decided to use run-time tests instead of disallowing ref subtyping in this situation? (You do not have to agree with them; just try to imagine what rationale might have been used at the time.)

11.4 Smalltalk Run-Time Structures

Here is a Smalltalk Point class whose instances represents points in the two-dimensional Cartesian plane. In addition to accessing instance variables, an instance method allows point objects to be added together.

Class name	Point
Superclass	Object
Class variables	*Comment: none*
Instance variables	*x y*
Class messages and methods	*Comment: instance creation* newX: xValue Y: yValue || ↑ self new x: xValue y: yValue
Instance messages and methods	*Comment: accessing instance vars* x: xCoordinate y: yCoordinate || x ← xCoordinate y ← yCoordinate x || ↑ x y || ↑ y *comment: arithmetic* + aPoint || ↑ Point newX: (x + aPoint x) Y: (y + aPoint y)

a. Complete the top half of the illustration of the Smalltalk run-time structure shown in Figure P.11.4.1 for a point object with coordinates (3, 4) and its class. Label each of the parts of the top half of the figure, adding to the drawing as needed.

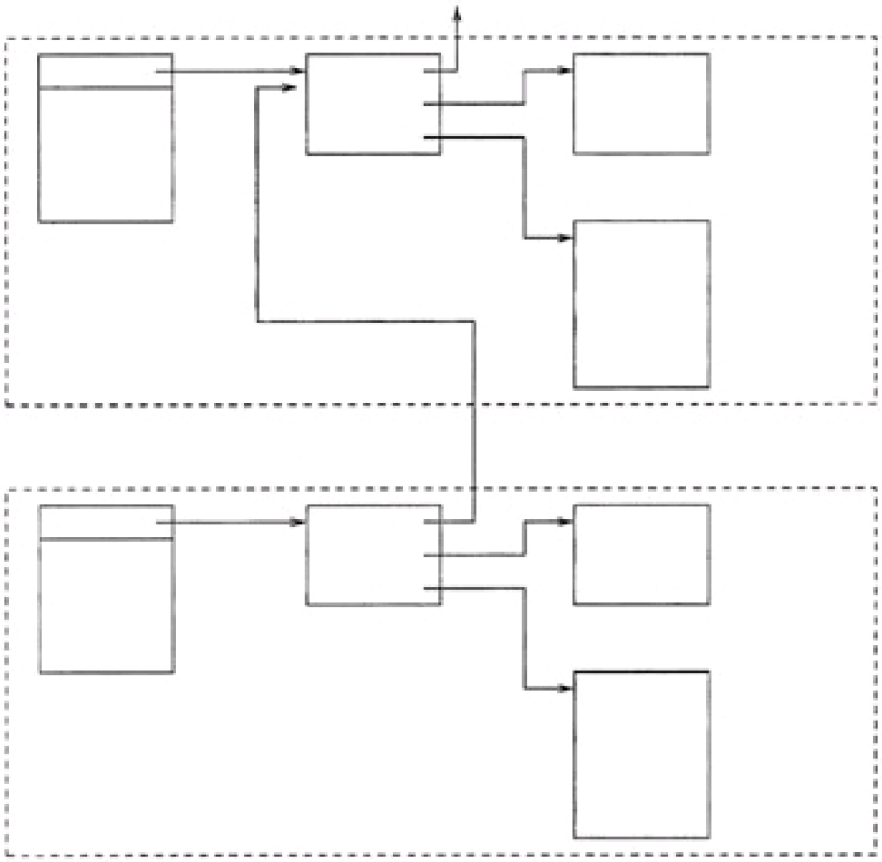


Figure P.11.4.1: Smalltalk run-time structures for Point and PolarPoint

b. A Smalltalk programmer has access to a library containing the Point class, but she cannot modify the Point class code. In her program, she wants to be able to create points by using either Cartesian or polar coordinates, and she wants to calculate both the polar coordinates (radius and angle) and the Cartesian coordinates of points. Given a point (*x* , *y*) in Cartesian coordinates, the radius is ((x * x) + (y * y)) squareRoot and the angle is (x/y) arctan. Given a point (*r* ,θ) in polar coordinates, the *x* coordinate is r*(θ cos) and the *y* coordinate is r*(θ sin).

   i. Write out a subclass PolarPoint, of Point and explain how this solves the programming problem.

   ii. Which parts of Point could you reuse and which would you have to define differently for PolarPoint?

c. Complete the drawing of the Smalltalk run-time structure by adding a Polar Point object and its class to the bottom half of the figure you already filled in with Point structures. Label each of the parts and add to the drawing as needed.

11.5 Smalltalk Implementation Decisions

In Smalltalk, each class contains a pointer to the class template. This template stores the names of all the instance variables that belong to objects created by the class.

a. The names of the methods are stored next to the method pointers. Why are the names of instance variables stored in the class, instead of in the objects (next to the values for the instance variables)?

b. Each class's method dictionary stores the names of only the methods explicitly written for that class; inherited methods are found by searching up the super-class pointers at run time. What optimization could be done if each subclass contained all of the methods of its superclasses in its method dictionary? What are some of the advantages and disadvantages of this optimization?

c. The class template stores the names of all the instance variables, even those inherited from parent classes. These are used to compile the methods of the class. Specifically, when a subclass is

added to a Smalltalk system, the methods of the new class are compiled so that, when an instance variable is accessed, the method can access this directly without searching through the method dictionary and without searching through superclasses. Can you see some advantages and disadvantages of this implementation decision compared with looking up the relative position of an instance variable in the appropriate class template each time the variable is accessed? Keep in mind that in Smalltalk, a set of classes could remain running for days, while new classes are added incrementally and, conceivably, existing classes could be rewritten and recompiled.

11.6 Protocol Conformance

We can compare Smalltalk interfaces to classes that use *protocols*, which are lists of operation names (selectors). When a selector allows parameters, as in at: put:, the selector name includes the colons but not the spaces. More specifically, if dict is an updatable collection object, such as a dictionary, then we could send dict a message by writing dict at:'cross' put:'angry'. (This makes our dictionary definition of "cross" the single word "angry.") The protocol for updatable collections will therefore contain the seven-character selector name at:put:. Here are some example protocols:

stack : {isEmpty, push:, pop }

queue : {isEmpty, insert:, remove }

priority_queue : {isEmpty, insert:, remove }

dequeue : {isEmpty, insert:,insertFront:, remove, removeLast }

simple collection : {isEmpty }

Briefly, a stack can be sent the message isEmpty, returning true if empty, false otherwise; push: requires an argument (the object to be pushed onto the stack), pop removes the top element from the stack and returns it. Queues work similarly, except they are first-in, first-out instead of first-in, last-out. Priority queues are first-in, minimum-out and dequeues are doubly ended queues with the possibility of adding and removing from either end. The simple_collection class just collects methods that are common to all the other classes. We say that the protocol for A *conforms* to the protocol for B if the set of A selector names contains the set of B selector names.

a. Draw a diagram of these classes, ordered by protocol conformance. You should end up with a graph that looks like William Cook's drawing shown in Figure 11.6.

b. Describe briefly, in words, a way of implementing each class so that you make B a subclass of A only if the protocol for B conforms to the protocol set for A.

c. For some classes A and B that are unrelated in the graph, describe a strategy for implementing A as a subclass of B in a way that keeps them unrelated.

d. Describe implementation strategies for two classes A and B (from the preceding set of classes) so that B is a subclass of A, but A conforms to B, not the other way around.

11.7 Removing a Method

Smalltalk has a mechanism for "undefining" a method. Specifically, if a class A has method m, then a programmer may cancel m in subclass B by writing

```
m:
        self should Not Implement
```

With this declaration of m in subclass B, any invocation of m on a B object will result in a special error indicating that the method should not be used.

a. What effect does this feature of Smalltalk have on the relationship between inheritance and subtyping?

b. Suppose class A has methods m and n, and method m is canceled in subclass B. Method n is inherited and not changed, but method n sends the message m to self. What do you think happens if a B object b is sent a message n? There are two possible outcomes. See if you can identify both, and explain which one you think the designers of Smalltalk would have chosen and why.

11.8 Subtyping and Binary Methods

This question is about the relationship between subtyping and inheritance. Recall that the main principle associated with subtyping is substitutivity: If A is a subtype of B, then wherever a B object is required in a program, an A object may be used instead without producing a type error. For the purpose of this question, we will use *Message not understood* as our Smalltalk type error. This is the most common error message resulting from a dynamic type failure in Smalltalk; it is the object-oriented analog of the error *Cannot take car of an atom* that every Lisp programmer has generated at some time. Remember that Smalltalk is a dynamically typed language. This question asks you to show how substitutivity can fail, using the fact that a method given in a superclass can be redefined in a subclass.

If there are nor estrictions on how a method (member function) may be redefined in a subclass, then it is easy to redefine a method so that it requires a different number of arguments. This will make it impossible to meaningfully substitute a subclass object for a superclass object. A more subtle fact is that subtyping may fail when a method is redefined in a way that appears natural and (unless you've seen this before) unproblematic. This is illustrated by the following Point class and ColoredPoint subclass:

Class name	Point
Class variables	
Instance variables	xval yval
Instance messages and methods	xcoord ↑ xval ycoord ↑ yval origin xval ← 0 yval ← 0 movex: dx movey: dy xval ← xval + dx. yval ← yval + dy equal: pt ↑ (xval = pt xcoord & yval = pt ycoord)
Class name	ColoredPoint
Class variables	
Instance variables	color
Instance messages and methods	color ↑ color changecolor: newc color ← newc equal: cpt ↑ (xval = cpt xcoord & yval = cpt ycoord & color = cpt color)

The important part here is the way that equal is redefined in the colored point class. This change would not be allowed in Simula or C++, but is allowed in Smalltalk.(The C++ compiler does not consider this an error, but it would not treat it as redefinition of a member function either.) The intuitive reason for redefining equal is that two colored points are equal only if they have the same coordinates and are the same color.

Problem: Consider the expression p1 equal:p2, where p1 and p2 are either Point objects or ColoredPoint objects. It is guaranteed not to produce *Message not understood* if both p1 and p2 are either Points or ColoredPoints, but may produce an error if one is a Point and the other a ColoredPoint. Consider all four combinations of p1 and p2 as Points and ColoredPoints, and explain briefly how each message is interpreted.

11.9 Delegation-Based Object-Oriented Languages

In this problem, we explore a delegation-based object-oriented language, SELF, in which objects can be defined directly from other objects. Classes are not needed and not supported. SELF has run-time type checking and garbage collection.

Here is the Self language description: (extracted from D. Ungar and R. B. Smith, Self: The power of simplicity, ACM SIGPLAN Notices 22(12), 1987, pp. 227-242.)

> In Smalltalk, … everything is an object and every object contains a pointer to its class, an object that describes its format and holds its behavior. In SELF too, everything is an object. But, instead of a class pointer, a SELF object contains named slots with may store either state or behavior. If an object receives a message and it has no matching slot, the search continues via a *parent* pointer. This is how SELF implements inheritance. Inheritance in SELF allows objects to share behavior, which in turn allows the programmer to alter the behavior of many objects with a single change. For instance, a [Cartesian] point object would have slots for its non-shared characteristics: x and y. Its parent would be an object that held the behavior shared among all points: +, etc.
>
> In SELF, there is no direct way to access a variable: instead, objects send messages to access data residing in name slots. So to access its "x" value, a point sends itself the "x" message. The message finds the "x" slot, and evaluates the object found therein…. In order to change contents of the "x" slot to, say, 17, instead of performing an assignment like "x←17," the point must send itself the "x:" message with 17 as the argument. The point object must contain a slot named "x:" containing the assignment [function].

a. Using this description, draw the run-time data structure of the SELF version of a (3,4) Point object, as described in the last two sentences of the first paragraph, and it parent object. Assume that assignments to the x and y characteristics are permitted.

b. SELF's lack of classes and instance variables make inheritance more powerful. For example, to create two points sharing the same x coordinate, the x and x: slots can be put in a separate object that is a parent of each of the two points. Draw a picture of the run-time data structures for two points sharing the same x coordinate.

c. To create a new Point object, the clone message is sent to an existing point. The language description continues:

> > Creating new objects … is accomplished by a simple operation, copying…. [In Smalltalk,] creating new objects from classes is accomplished by instantiation, which includes the interpretation of format information in a class. Instantiation is similar to building a house from a plan.
>
> Cloning an object does not clone its parent, however.
>
> If a *point* object contains fields x and y and methods x:, y:, move, then cloning the object will create another object with two fields and three methods. Each point will have a parent pointer, two fields, and three methods - six entries in all. The SELF point will be twice the size of the corresponding Smalltalk point.
>
> Explain how you would structure your SELF program so that each point can be cloned without cloning its methods.

d. SELF also allows a *change-parent* operation on an object. The parent pointer of an object can be set to point to any other object. (An exception is that the first object must not be an ancestor of the second - we do not want a cycle.)

The change-parent operation is useful for objects that change behavior in different states. For example, a window can be in the *visible* or *iconified* (minimized) state. When iconified, mouse clicks and window display work differently than when the window is visible. A window's parent pointer can be set to VisibleWindow initially, then changed to IconifiedWindow when the "minimized" button is pressed. Another example is a file object that can be in the *open* or *closed* state. The open method changes the parent pointer from Closed File to OpenFile.

The change-parent operation adds a lot of flexibility to SELF. Can you think of disadvantages of this feature?

Team-Fly



Team-Fly

Chapter 12: Objects and Run-Time Efficiency- C++

OVERVIEW

C++ is an object-oriented extension of the C language. It was originally called *C with classes*, with the name C++ originating around 1984. A Bell Laboratories researcher interested in simulation, Bjarne Stroustrup began the C++ project in the early 1980s. His goal was to add objects and classes to C, using his experience with Simula as the basis for the design. The design and implementation of C++ was originally a one person effort, with no apparent intent to produce a commercial product. However, as interest in objects and program structure grew over the course of the 1980s, C++ became popular and widely used. In the 1990s, C++ became the most widely used object-oriented language, with good compilers and development environments available for the Macintosh, PC, and Unix-based workstations.

C, discussed in Section 5.2, was originally designed by Dennis Ritchie. The C programming language was used for writing the Unix operating system at Bell Laboratories. The original implementation of C++ was a preprocessor that converted C++ to C.

Team-Fly

Previous Next

12.1 DESIGN GOALS AND CONSTRAINTS

The main goal of C++ is to provide object-oriented features in a C-based language without compromising the efficiency of C. In the process of adding objects to C, some other improvements were also made. In more detail, the main design goals of C++ may be summarized as follows:

- data abstraction and object-oriented features,
- better static type checking,
- backwards compatibility with C. In other words, most C code should compile as legal C++, without requiring significant changes to the code,
- efficiency of compiled code, according to the principle "If you do not use a feature, you should not pay for it."

The principle stated in the final goal is significant and may require some thought to appreciate. This principle suggests that C programs should compile as efficiently under the C++ compiler as under the C compiler. It would violate this principle to implement C integers as objects, for example, and use dynamic method lookup to find integer functions at run time as in Smalltalk, as this would significantly reduce the performance of C integer calculations. The principle does not mean that C++ statements that also appear in C must be implemented in exactly the same way in both languages, but whatever changes are adopted in C++, they must not slow down execution of compiled code unless some slower features of C++ are also used in the program.



Bjarne Stroustrup is the principal designer of C++. Bjarne came to Bell Laboratories in 1979 after finishing his M.S. degree in Aarhus, Denmark, and his Ph.D. on design of distributed systems in Cambridge, U.K.

Working on distributed computing, in the same Computing Science Center as C and Unix designers Ritchie and Thompson, Stroustrup decided to add object-oriented features from Simula to C. His goal was to further his

personal research objectives, which involved simulation.

An occasionally reserved but essentially gregarious and friendly person, Stroustrup was catapulted into the technological limelight by the success of C++, first within AT&T and later in the wider community of practicing software developers. He has written several books on C++, including *The Design and Evolution of C++* (Addison-Wesley, 1994), which contains an interesting history, explanation, and commentary on the language. Bjarne's nonresearch interests include reading books that are not about computer science, photography, hiking and running, travel, and music.

12.1.1 Compatibility with C

The decision to maintain compatibility with C has a pervasive effect on the design of C++. Those familiar with C know that C has a specific machine model, revealing much of the structure of the underlying computer architecture. In particular, C operations that return the address of a variable and place any bit pattern in any location make it possible for C programs to rely on the exact representation of data. Therefore C++ must adhere to the same data representations as C.

Most other object-oriented languages, including those designed before C++ and those designed after, use garbage collection to relieve programmers from the task of identifying inaccessible objects and deallocating the associated memory locations. However, there is no inherent reason why objects must be garbage collected. The strongest connection is that with the increased emphasis on abstraction and type correctness, it would have been consistent with other goals of C++ to offer some form of garbage collection where feasible in a form that does not affect the running time of programs that do not use garbage-collected objects. However, because there are features of C that make garbage collection extremely difficult, it would have been difficult to base C++ objects on garbage collection. Not only is garbage collection counter to the C philosophy of providing programmer control over memory, but the similarity between pointers and integers makes it effectively impossible to build a garbage collector that works on C++ programs that use pointer arithmetic or cast integers to pointers.

An important specific decision is to treat C++ objects as a generalization of C structs. This makes it necessary to allow objects to be declared and manipulated in the same way as structs. In particular, objects can be allocated in the activation records of functions or local blocks, as well as on the heap, and can be manipulated directly (i.e.,not through pointers). This is one place where C++ deviates from Simula, as Simula objects can be manipulated only through pointers. More specifically, C++ allows a form of object assignment that copies one object into the space previously occupied by another, whereas most other object-oriented languages allow only pointer assignment for objects. Some aspects of C++ objects on the stack are explored in the homework exercises.

Team-Fly



12.1.2 Success of C++

C++ is a very carefully designed language that has succeeded admirably, in spite of difficult design constraints. Measured by the number of users, C++ is without question the most successful language of the decade from its development in the mid-1980s until the release of Java in the mid-1990s. However, the design goals and backward compatibility with C do not allow much room for additional aesthetic consideration. Some aspects of C++ have become complex and difficult for many programmers to understand. On the other hand, many C programmers use the C++ compiler and appreciate the benefits of better type checking. Perhaps a fair summary of the success of C++ is that it is widely used, with most users choosing to program in some subset of the language that they feel they understand and that they find suitable for their programming tasks. In other words, C++ is a useful programming tool that allows designers to craft good object-oriented programs, but it does not enforce good programming style in the way that other language designs have attempted to do. This is intended to be a statement of fact and not a count against C++. In fact, much of its success seems attributable to the way that C++ is designed to give programmers choices and not to restrict programmers to a particular style.

There are many published style guides that advocate use of certain features of C++ and caution against the use of others. Those interested in serious C++ programming or interested in understanding how many programmers view the language may wish to visit their library or bookstore and take a look at some of the current guides.

Team-Fly

12.2 OVERVIEW OF C++

Before looking at the object-oriented features of C++ in Subsection 12.2.2, we will take a quick tour of some of the additions to C that are not related to objects. Evaluation and commentary on C++ appear in Subsection 12.2.3.

12.2.1 Additions to C Not Related to Objects

There are a number of differences between C++ and C that are not related to objects. Although we are primarily interested in the C++ object system, it is worth considering a few of the more significant changes. The most interesting additions are

- type bool
- reference types and pass-by-reference
- user-defined overloading
- function templates
- exceptions

There are also changes in memory management calls (new and delete instead of malloc and free), changes in stream and file input and output, addition of default parameter values in function declarations, and more minor changes such as the addition of end-of-line comments and elimination of the need for type def keyword with struct/union/enum declarations.

The first three additions, bool, pass-by-reference, and overloading, are discussed in the remainder of this subsection. Function templates are discussed in Subsection 9.4.1 and a general discussion of exceptions appears in Subsection 8.2.

Type bool

In C, the value of a logical test is an integer. For example, the *C Reference Manual* defines the comparison operator < and logical operator && as follows:

- The operator < (less than) returns 1 if the first operand is less than the second and 0 otherwise.
- The && operator returns 1 if both its operands are nonzero and 0 otherwise.

This makes it possible to write C statements with expressions such as

```
if ((5 < 3) + 1 && 2 ==3) { ... }
```

which combine comparison operations and arithmetic.

To make a syntactic distinction between Booleans and integers, C++ has a separate type: bool with values true and false. Because of conversions, this does not completely separate integers and Booleans. In particular, integer values

may be assigned to variables of type bool, with implicit conversion of nonzero numbers to true and 0 to false. However, the type bool does improve the readability of many programs by indicating that a variable or return value of a function will be used as a Boolean value rather than an integer.

The C++ changes can be mimicked in C by the declarations

```
typedef int bool;
#define false 0
#define true 1
```

which define the type bool and values true and false. One difference between built-in Booleans of C++ and Booleans in C is that when C++ Booleans are printed, they print as true and false, not 1 and 0.

Because of all the implicit conversions, a separate Boolean type does not help with one of the most common simple programmer mistakes in C. At least for beginners, conditional statements such as

```
if (a=b) c;
```

are often the result of typographic error; the programmer meant to write

```
if (a==b) c;
```

The reason why the first statement type checks and compiles in C is that an integer assignment a=b has type int and has the value of b. Because a C conditional statement requires an integer (not a Boolean), there is no warning associated with this statement. In most languages with bool different from integer, it would be a type error to write if (a=b) c. However, because C++ Booleans are automatically converted to integers, the statement if (a=b) c remains legal in C++.

Because the introduction of bool into C++ has very little effect in the end, you might wonder why this was done. Before a Boolean type was added, C/C++ often contained definitions of bool, true, and false by macro, as previously illustrated. However, bool could be defined in different ways, with the resulting types having slightly different semantics. For example, bool could be defined as int, unsigned int, or short int. This caused problems when libraries that use slightly different definitions were combined. Therefore it was useful to standardize code by adding a built-in Boolean type.

Reference Type and Pass-By Reference

In C, all parameters are passed by value. If you wish to modify a value that is passed as a parameter, you must pass a pointer to the value. For example, here is C code for a function that increments an integer:

```
void increment(int *p) { /* parameter is pointer to int */
(*p)++; }               /* modify value pointed to by p */
main()
{intk=0;
increment(&k);          /* pass address of variable k */
k;                      /* value of k is now 1 */
...
}
```

In C++, it is possible to pass-by-reference, as shown here:

```
void increment(int & n) { /* parameter is reference to int */
 n++; }                  /* modify reference param */
main(){
 intk=0;
 increment(k);           /* no need to explicitly compute address */
 k;                      /* value of k is now 1 */
 ...
}
```

The effect is similar to C's pointer argument, but the calling function does not have to provide the address of an argument and the called function does not have to dereference pointers. A mild in convenience with C pointer arguments is that the programmer must remember whether a given function uses pass-by-value or the pointer in its parameter list. If pointer arguments are changed to C++ pass-by-reference, then no address calculation needs to be written as part of the call and a programmer using functions from a library can avoid this issue entirely.

A benefit of explicit references is sometimes called *pass-by-constant-reference.* If a function argument is not to be changed by the function, then it is possible to specify that the argument is to be a constant, as in `void f (const int &x)`. In the body of a function with parameter `x` passed this way, it is illegal to assign to `x`.

User-Defined Overloading

As discussed in Chapter 6, overloading allows one name to be used for more than one value. In C++, it is possible to declare several functions with the same name, as long as the functions have a different number and type of parameters. For example, here is a C++ program with three types of print function, each called `show`:

```
#include <stdio.h>
void show(int val) {
```

```
    printf("Integer: %d\n", val);
}
void show(double val) {
    printf("Double: %lfn", val);
}
void show(char *val)   {
   printf("String: %s\n", val);
}
int main( )  {

show(12);
show(3.1415);
show("Hello World\n!");
```

Because the three functions have different types of arguments, the compiler can determine which function to call by the type of the actual parameter used in the function call.

C++ does not allow overloaded functions that have the same number and types of arguments but differ only in their return value because C and C++ functions can be called as statements. When a function is called as a statement, ignoring the return value of the function, the compiler would not have any way of determining which function to call.

One source of potential confusion in C++ arises from combinations of overloading and automatic conversion. In particular, if the actual parameters in a function call do not match any version of a function exactly, the compiler will try to produce a match by promoting and/or converting types. In this example code,

```
void f(int)
void f(int *)
...
f('a')
```

the call f('a') will result in f(int) instead of f(int *) because a char can be promoted to be an int. When a function is overloaded and there are many possible implicit conversions, it can be difficult for a programmer to understand which conversions and call will be used.

12.2.2 Object-Oriented Features

The most significant part of C++ is the set of object-oriented concepts added to C; these are the main concepts:

- *Classes*, which declare the type associated with all objects constructed from the class, the data members of each object, and the member functions of the class.
- *Objects*, which consist of private data and public functions for accessing the hidden data, as in other object-oriented languages.
- *Dynamic lookup*, for member functions that are declared virtual. A virtual function in a derived class (subclass) may be implemented differently from virtual functions of the same name in a base class (superclass).
- *Encapsulation*, based on programmer designations public, private, and protected that determine

whether data and functions declared in a class are visible outside the class definition.

- *Inheritance*, using subclassing: One class may be defined by inheriting the data and functions declared in another. C++ allows single inheritance, in which a class has a single base class (superclass) or multiple inheritance in which a class has more than one base class.
- *Subtyping*, based on subclassing. For one class to define a subtype of the type defined by another class, inheritance must be used. However, the programmer may decide whether inheritance results in a subtype or not.

This is only a brief summary; there are many more features, and it would take an entire book to explain all of the interactions between features of C++. Further description of classes, inheritance, and objects appear in Section 12.3.

C++ Terminology. Although C++ terminology differs from Smalltalk (and Java) terminology, there is a fairly close correspondence. The terms class and object are used similarly. Smalltalk instance variables are called member data and methods are called member functions. The term subclass is not usually used in connection with C++. Instead, a superclass is called a base class and a subclass a derived class. The term inheritance has the same meaning in both languages.

12.2.3 Good Decisions and Problem Areas

C++ is the result of an extensive effort involving criticism and suggestions from many experienced programmers. In many respects, the language is as good a design as possible, given the goals of adding objects and better compile-time type checking to C, without sacrificing efficiency or backwards compatibility. Some particularly successful parts of the design are

- *encapsulation:* careful attention to visibility and hiding, including *public, protected,* and *private* levels of visibility and *friend* functions and classes,
- *separation of subtyping and inheritance:* classes may have public or private base classes, giving the programmer some explicit control over the resulting subtype hierarchy,
- *templates* (described in Section 9.4),
- *exceptions* (described in Section 8.2)
- *better static type checking* than C.

There are also a number of smaller successful design decisions in C++. One example is the way the scope resolution operator (written as ::) is used in connection with inheritance and multiple inheritance to resolve ambiguities that are problematic in other languages.

Problem Areas

There are a number of aspects of C++ that programmers occasionally find difficult. Some of the main problem areas are

- *casts and conversions*, which can be complex and unpredictable in certain situations,
- *objects allocated on the stack* and other aspects of object memory management,
- *overloading*, a complex code selection mechanism in C++ that can interact unpredicatably with dynamic lookup (virtual function lookup),
- *multiple inheritance*, which is more complex in C++ than in other languages because of the way objects and virtual function tables are configured and accessed.

These problem areas exist not because of oversight but because the goals of C++, followed to their logical conclusion, led to a design with these properties. In other words, these problems were not the result of carelessness or inattention, but a consequence of decisions that were made with other objectives in mind. To be fair, most of them have their roots in C, not the C++ extensions to C. None the less, these features might reasonably lead programmers to prefer other languages when compatibility with C and absolute efficiency of compiled code are not essential to an

application.

Some programmers might also say that lack of garbage collection, or lack of a standard interface for writing concurrent programs, is a problem in C++. However, these are facilities that simply lie outside the scope of the language design.

Casts and Conversions. A cast instructs the compiler to treat an expression of one type as if it is an expression of another. For example, (float) i instructs the compiler to treat the variable i as a float, regardless of its type otherwise, and (int *)x causes x to be treated as pointer to an integer. In certain situations, C and C++ compilers perform implicit type conversions. For example, when a variable is assigned the value of an expression, the expression may be converted to the type associated with the variable, if needed. In most object-oriented languages, the automatic conversion of an object from one type to another does not change the representation of the object. For example, we have seen from the description of subtyping in Smalltalk in Chapter 11 that Point objects can be treated as Colored Point objects without changing the representation of objects because Colored Point objects are represented in a way that is compatible with the representation of Point objects. If multiple inheritance is used in C++, however, converting an object from a subtype to a supertype may require some change in the value of a pointer to the object. This happens in a way that may introduce hard-to-find bugs and forces programmers to understand the underlying representation of objects. More generally, Stroustrup himself says, "Syntactically and semantically, casts are one of the ugliest features of C and C++" in *The Design and Evolution of C++* (Addison-Wesley, 1994).

Objects Allocated on the Stack. Simula, Smalltalk, and Java allow objects to be created only on the heap, not on the run-time stack. In these other languages, objects can be accessed through pointers, but not through ordinary stack variables that contain space for an object of a certain size. Although C++ on-the-stack objects are efficiently allocated and deallocated as part of entry and exit from local blocks, there are also some disadvantages. The most glaring rough spot is the way that assignment works in combination with subtyping, truncating an object to the size that fits when an assignment is performed. This can change the behavior of an object, as eliminating some of its member data forces the compiler to change the way that virtual functions are selected. Although the C++ language definition explains what happens and why, the behavior of object assignment can be confusing to programmers.

Overloading. Overloading in itself is not a bad programming language feature, but C++ user-defined overloading can be complex. In addition, the interaction with dynamic lookup (virtual functions) can be unpredictable. Because overloading is a compile-time code selection mechanism and dynamic lookup is a run-time code selection mechanism, the two behave very differently. This causes confusion for many programmers.

Multiple Inheritance. There are some inherent complications associated with multiple inheritance that are difficult to avoid. These are discussed in Section 12.5. The C++ language design compounds these problems with a tricky object and method lookup table (vtable) format. Unfortunately, the details of the implementation of multiple inheritance seem to intrude on ordinary programming, sometimes forcing a programmer who is interested in using multiple inheritance to learn some of the implementation details. Even a programmer not using multiple inheritance may be affected if a derived class is defined using multiple inheritance.

Team-Fly

12.3 CLASSES, INHERITANCE, AND VIRTUAL FUNCTIONS

The main object-oriented features of C++ are illustrated by a repeat of the point and colored-point classes from Chapter 11.

12.3.1 C++ Classes and Objects

We can define point objects in C++ by declaring the point class, here called Pt, and separately declaring the member functions of class Pt. The class declaration without implementation defines both the interface of point objects and the data used to implement points, but need not include the code for member functions. The declarations that appear in the Pt class are divided into three parts, public members, protected members, and the private members of the class.

For simplicity, Pt is a class of one-dimensional points, each point having only an *x* coordinate:

```
/* ---------- Point Class Interface -------------------*/
class Pt  {
  public:
    Pt(int xv);
    Pt(Pt* pv);
    int getX();
    virtual void move(int dx);
  protected:
    void setX(int xv);
  private:
  int x;
  } ;
  /* ---------- Declarations of Constructors --------*/
   Pt::Pt(int xv)  {  x = xv;  }
   Pt::Pt(Pt* pv)  {  x = pv->x;  }
 /* ---------- Declarations of Member Functions --------*/
  int Pt::getX()   {  return x;  }
  void Pt::setX(int xv)  {  x = xv;  }
```

Constructors. A constructor is used to initialize the member data of an object when a program contains a statement or expression indicating that a new object is to be created. When a new point is created, space is allocated, either on the heap or in an activation record on the stack, depending on the statement creating the object. A constructor is then called to initialize the locations allocated for the object. Constructors are declared with the same syntax as that of member functions, except that the "function" name is the same as the class name. In the point class, two constructors are declared. The result is an overloaded function Pt, with one constructor called if the argument is an integer and the other called if the argument is a Pt.

In Smalltalk terminology, constructors are "class methods," not "instance methods," as a constructor is called without referring to any object that is an instance of the class. In particular, constructors are not member functions.

Visibility. As previously mentioned, the declarations that appear in the Pt class are divided into three parts, public

members, protected members, and private members of the class. These designations, which affect the visibility of a declaration, may be summarized as follows:

- *public:* members that are visible in any scope in which an object of the class can be created or accessed,
- *protected:* members that are visible within the class and in any derived classes,
- *private:* members visible only within the class in which these members are declared.

The Pt class is written following one standard visibility convention that is used by many C++ programmers. Member data is made private, so that if a programmer wishes to change the way that objects of the class are represented, this may be done without affecting the way that other classes (including derived classes) depend on this class. Members that modify private data are made protected, so that derived classes may change the value of member data, but external code is not allowed to do so. Finally, member functions that read the value of member data and provide useful operations on objects are declared public, so that any code with access to an object of this class can manipulate these objects in useful ways.

A feature of C++ that is not illustrated in this example is the *friend* designation, which is used to allow visibility to the private part of a class. A class may declare friend functions and friend classes. If the Pt class contained the declaration friend class A, then code written as part of class A (such as member functions of class A) would have access to the private part of class Pt. The friend mechanism is used when a pair of classes is closely related, such as matrices and vectors.

Virtual Functions. Member functions may be designated *virtual* or left *nonvirtual*, which is the default for member functions that are not preceded by the keyword virtual. If a method is virtual, it may be redefined in derived classes. Because different objects will then have different member functions of the same name, selection of virtual member functions is by dynamic lookup: There is a run-time code selection mechanism that is used to find and invoke the correct function. These extra steps make calls to virtual functions less efficient than calls to nonvirtual functions.

Nonvirtual methods may not be redefined in derived classes. As a result, calls to nonvirtual methods are compiled and executed in the same way as ordinary function calls not associated with objects or classes.

A common point of confusion is that, syntactically, a redeclaration of a nonvirtual function may appear in a derived class. However, this produces an overloaded function, with code selection done at compile time. We will discuss this phenomenon in Subsection 12.3.3.

12.3.2 C++ Derived Classes (Inheritance)

The following ColorPt class defines an extension of the one-dimensional points defined in the Pt class. As the name suggests, ColorPt objects have a color added. For simplicity, we assume that colors are represented by integers. In addition, to illustrate virtual function definition, moving a colored point causes the color to darken a little:

```
/* ---------- Color Point Class Interface -------------------*/
class ColorPt: public Pt  {
public:
  ColorPt(int xv,int cv);
  ColorPt(Pt* pv,int cv);
  ColorPt(ColorPt* cp);
  int getColor();
  virtual void darken(int tint);
  virtual void move(int dx);
protected:
 void setColor(int cv);
private:
 int color;
};
```

```
/* ---------- Declarations of Constructors --------*/
 ColorPt::ColorPt(int xv,int cv)
  : Pt(xv)      /* call parent's constructor */
  { color = cv; }  /* then initialize color */
ColorPt::ColorPt(Pt* pv,int cv): Pt(pv) { color = cv; }
/* ---------- Declarations of Member Functions --------*/
 int ColorPt::getColor() { return color; }
 void ColorPt::darken(int tint) { color += tint; }
 void ColorPt::move(int dx) {Pt::Move(dx); this->darken(1); }
 void ColorPt::setColor(int cv) { color=cv; }
```

Inheritance. The first line of code declares that the ColorPt class has Pt as a public base class; this is the meaning of the clause : public Pt following the name of the ColorPt class. If the keyword public is omitted, then the base class is called a private base class.

When a class has a base class, the class inherits all of the members of the base class. This means that ColorPt objects have all of the public, protected, and private members of the Pt class. In particular, although the ColorPt class declares only member data color, ColorPt objects also have member data x, inherited from Pt.

The difference between public base classes and private base classes is that, when a base class is public, then the declared (derived) class is declared to be a subtype of the base class. Otherwise, the C++ compiler does not treat the class as a subtype, even though it has all the members of the base class. This is discussed in more detail in Section 12.4.

Constructors. The ColorPt class has three constructors. As in the Pt class, the result is an overloaded function ColorPt, with selection between the three functions completed at compile time according to the type of arguments to the constructor.

The two constructor bodies previously discussed illustrate how a derived-class constructor may call the constructor of a base class. As with points, when a new colored point is created, space is allocated, either on the heap or in an activation record on the stack, depending on the statement creating the object, and a constructor is called to initialize the locations allocated for the object. Because the derived class has all of the data members of the base class, most derived-class constructors will call the base class constructor to initialize the inherited data members. If the base class has private data members, which is the case for Pt, then the only way for the derived class ColorPt to initialize the private members is to call the base-class constructor.

Visibility. When one class inherits from another, the members have essentially the same visibility in the derived class as in the base class. More specifically, public members of the base class become public members of the derived class, protected members of the base class are accessible in the derived class and its derived classes, which is the same visibility as if the member were declared protected in the derived class. Inherited private members exist in the derived class, but cannot be named directly in code written as part of the derived class. In particular, every ColorPt object has an integer member x, but the only way to assign to or read the value of this member is by calling the protected and public functions from the Pt class.

Virtual Functions. As previously mentioned, virtual functions in a base class may be redefined in a derived class. The member function move is declared virtual in the Pt class and redefined above for ColorPt. In this example, the move function for points just changes the *x* coordinate of the point, whereas the move function on colored points changes the *x* coordinate and the color. If the implementation of ColorPt::move were omitted, then the move function from Pt would be inherited on ColorPt. The implementation of virtual functions is discussed in the next subsection.

12.3.3 Virtual Functions

Dynamic lookup is used for C++ virtual functions. A virtual function f defined on an object o is called by the syntax o.f(…) or p->f(…) if p is a pointer to object o. When a virtual function is called, the code for that function is located by a sequence of run-time steps. These steps are similar to the lookup algorithm for Smalltalk, but simpler because of some optimizations that are made possible by the C++ static type system. Each object has a pointer to a data

structure associated with its class, called the *virtual function table*, or *vtable* for short (sometimes written as vtbl). The relationship among an object, the vtable of its class, and code for virtual functions is shown in Figure 12-1 with points and colored points.

Virtual Function on Base Class. Suppose that p is a pointer to the Pt object in Figure 12-1. When an expression of the form p->move(…) is evaluated, the code

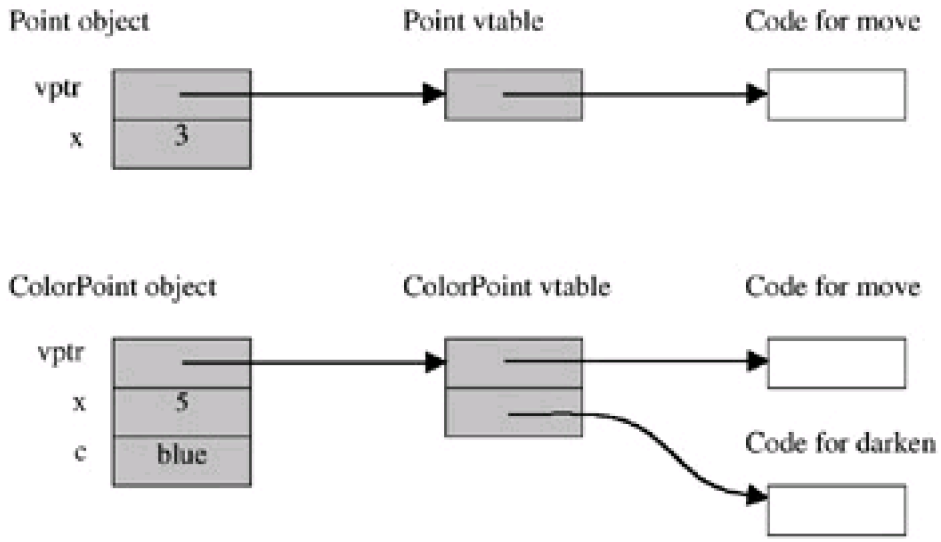


Figure 12.1: Representation of Point and ColoredPoint in C++.

for move must be found and executed. The process begins by following the vtable pointer in p to the vtable for the Pt class. The vtable for Pt is an array of pointers to functions. Because move is the first (and only) virtual function of the Pt class, the compiler can determine that the first location in this array is a pointer to the code for move. Therefore, the run-time code for finding move simply follows the first pointer in the Pt vtable and calls the function reached by this pointer. Unlike in Smalltalk, here there is no need to search the vtable at run time to determine which pointer is the one for move. The C++ type system lets the compiler determine the type of the object pointer at compile time, and this allows the compiler to find the relative position of the virtual function pointer in the vtable at compile time, eliminating the need for run-time search of the vtable.

Virtual Function on Derived Class. Suppose that cp is a pointer to the ColorPt object in Figure 12-1. When an expression of the form cp->move(…) is evaluated, the algorithm for finding the code for move is exactly the same as for p->move(…):The vtable for ColorPt is an array containing two pointers, one for move and the other for darken. The compiler can determine at compile time that cp is a pointer to a ColorPt object and that move is the first virtual function of the class, so the run-time algorithm follows the first pointer in the vtable without Smalltalk-style run-time search.

Correspondence Between Base- and Derived-Class Vtables. As a consequence of subtyping, a program may assign a colored point to a pointer to a point and call move through the base-class pointer, as in

```
p = cp;
p-> move(...);
```

Because p now points to a colored point, the call p->move(…) should call the move function from the ColorPt class. However, the compiler does not know at compile time that p will point to a ColorPt object. In fact, the same statement can be executed many times (if this is inside a loop, for example) and p may point to a Pt object on some executions and a ColorPt object on others. *Therefore, the code that the compiler produces for finding* move *when* p *points to a* Pt

must also work correctly when p *points to a* ColorPt. The important issue is that because move is first in the Pt vtable, move must also be first in the ColorPt vtable. However, the compiler can easily arrange this, as the compiler will first compile the base class Pt before compiling the derived-class ColorPt. In summary, when a derived class is compiled, the virtual functions are ordered in the same way as the base class. This makes the first part of a derived-class vtable look like a base class vtable. Because member data can be accessed by inherited functions, member data in a derived class are also arranged in the same order as member data in a base class.

Unlike in Smalltalk, here there is no link from ColorPt to Point; the derived-class vtable contains a copy of the base-class vtable. This causes vtables to be slightly larger than Smalltalk method dictionaries might be for corresponding programs, but the space cost is small compared with the savings in running time.

Note that nonvirtual functions do not appear in a vtable. Because nonvirtual functions cannot be redefined from base class to derived class, the compiler can determine the location of a nonvirtual function at compile time (just like normal function calls in C).

12.3.4 Why is C++ Lookup Simpler than Smalltalk Lookup?

At run-time, C++ lookup uses indirection through the vtable of the class, with an offset (position in the vtable) that is known at compile time. In contrast, Smalltalk method lookup does a run-time search of one or more method dictionaries. The C++ lookup procedure is much faster than the Smalltalk procedure. However, the C++ lookup procedure would not work for Smalltalk. Let's find out why.

Smalltalk has no static type system. If a Smalltalk program contains the line

```
p selector : parameters
```

sending a message to an object p, then the compiler has no compile-time information about the class of p. Because any object can be assigned to any Smalltalk pointer, p could point to any object in the system. The compiler knows that selector must refer to some method defined in the class of p, but different classes could put the same selector (method name) in different positions of their method dictionaries. As a consequence, the compiler must generate code to perform a run-time search for the method.

The C++ static type system makes all the difference. When a call such as p->move(…) is compiled, the compiler can determine a static type for the pointer p. This static type must be a class, and that class must declare or inherit a function called move. The compiler can examine the class hierarchy to see what location a virtual function move will occupy in the vtable for this class. A call

```
p->move(x)
```

compiles to the equivalent of the C code

```
(*(p->vptr[1]))(p,x)
```

where the index 1 in the array reference vptr[1] indicates that move is first function in the vtable (represented by the array vptr) for the class of p. The reason why p is passed as an argument to the function (*(p->vptr[1])) is explained in the next subsection.

Arguments to Member Functions and this

There are several issues related to calls to member function, function parameters, and the this pointer. Consider the following code, in which one virtual function calls another:

```
class A {
  public:
   virtual int f (int x);
   virtual int g(int y);
};
int A::f(int x) { . . . g(i) ...;}
int A::g(int y) { . . . f(j) ...;}
```

If virtual function f is redefined in a derived class B, then a call to the inherited g on class B objects should invoke the new function B::f, not the original function A::f defined for class A. Therefore, calls to one virtual function inside another must use a vtable. However, the call to f inside A::g does not have the form p->f(…), and it is not clear at first glance what object p we would use if we wanted to change the call from the simple f(…) that appears above to p->f(…).

One way of understanding the solution to this problem is to rewrite the code as it is compiled. In other words, the preceding function A::g is compiled as if it were written as

```
int A::g(A* this, int x) { . . . this->f(j) . . . ;}
```

with a new first parameter this to the function, called this, and the call f(j) replaced with this->f(j). Now the call this->f(j) can be compiled in the usual way, by use of the vtable pointer of this to find the code for f, provided that when g is called, the appropriate object is passed as the value of this.

The calling sequence for a member function, whether it is virtual or not, passes the object itself as the this pointer. For example, returning to the Pt and ColorPt example, in code such as

```
ColorPt* q = new ColorPt(3,4);
q->darken(5);
```

the call to darken is compiled as if it were a C function call:

```
(*(q->vptr[2]))(q,5);
```

This call shows the offset of darken in the vtable (assumed here to be 2) and the call with the pointer to q passed as the first argument to the compiled code for the member function.

There are several ways that the this pointer is used. As previously illustrated, the this pointer is used to call virtual functions on the object. The this pointer is also used to access data members of the object, whether there are virtual functions or not. The this pointer is also used to resolve overloading, as described in the next subsection.

Scope Qualifiers

Because some of the calling conventions may be confusing, it is worth saying clearly how names are interpreted in C++. There are three *scope qualifiers*. They are :: (double colons), -> (right arrow, consisting of two ascii characters - and >), and . (period or dot). These are used to qualify a member name with a class name, a pointer to an object, or an object name, respectively.

The following rules are for resolving names:

- A name outside a function or class, not prefixed by :: and not qualified, refers to global object, function, enumerator, or type.
- Suppose C is a class, p is a pointer to an object of class C, and o is an object of class C. These might be declared as follows:

```
ClassC:... { .... }; /* C is a class */
C *p = new C( ....); /* p is a pointer to an object of class C */
C o( . . . )              /* o is an object of class C */
```

Then the following qualified names can be formed with ::, ->, and .:

```
C::n    /* Class name C followed by member name n */
```

```
p->n /* pointer p to object of class C followed by member name n */
o.n      /* pointer p to object of class C followed by member name n */
```

These refer to a member n of class C, or a base class of C if n is not declared in C but is inherited from a base class.

Nonvirtual and Overloaded Functions

The C++ virtual function mechanism does not affect the way that the address of a nonvirtual or overloaded function is determined by the C++ compiler. For nonvirtual functions, the C++ calls work in exactly the same way as in C, except for the way that the object itself is passed as the this pointer, as described in the preceding subsection.

There are also some situations in which overloading and virtual function lookup may interact or be confusing.

Recall that, if a function is not a virtual function, the compiler will know the address of the function at compile time. (Those familiar with linking may realize that this is not strictly true for separately compiled program units. However, linkers effectively make it possible for compilers to be written as if the location of a function is known at compile time.) Therefore, if a C++ program contains a call f(x), the compiler can generate code that jumps to an address associated with the function f.

If f is an overloaded function and a program contains a call f(x), then the compile-time type associated with the parameter x will be used to decide, at compile time, which function code for f will be called when this expression is executed at run time.

The difference between overloading and virtual function lookup is illustrated by the following code. Here, we have two classes, a parent and a child class, with two member functions in each class. One function, called printclass, is overloaded. The other, called printvirtual, is a virtual function that is redefined in the derived class:

```
class parent {
public:
  void printclass() {printf("parent");};
  virtual void printvirtual() {printf("parent");};
};
class child : public parent {
public:
  void printclass() {printf("child");};
  void printvirtual() {printf("child");}; };
main() {
  parent p; child c; parent *q;
  p.printclass(); p.printvirtual(); c.printclass(); c.printvirtual();
  q = &p; q->printclass(); q->printvirtual();
  q = &c; q->printclass(); q->printvirtual();
```

The program creates two objects, one of each class. When we invoke the member functions of each class directly through the object identifiers c and p, we get the expected output: The parent class functions print "parent" and the child class functions print "child." When we refer to the parent class object through a pointer of type *parent, then we get the same behavior. However, something else happens when we refer to the child class object through the pointer of type *parent. The call q->printclass() always causes the parent class member to be called; the print class function is not virtual so the type of the pointer q is used. The static type of q is *parent, so overloading resolution leads to the parent class

function. On the other hand, the call q->printvirtual() will invoke a virtual function. Therefore, the output of this program is

---

parent parent child child parent parent parent child.

The call q->printclass() is effectively compiled as a call printclass(q), passing the object q as the this pointer to printclass. Although the printclass function does not need the this pointer in order to print a string, the argument is used to resolve overloading. More specifically, q->printclass() calls the parent class function because this call is compiled as printclass(q) and the type of the implicit argument q is used by the compiler to choose which version of the overloaded printclass function to call.

Team-Fly

Team-Fly

12.6 CHAPTER SUMMARY

The main goal of C++ is to provide object-oriented features in a C-based language without compromising the efficiency of C. C++ is a very carefully designed language that has succeeded admirably in spite of difficult design constraints.

Some features of C++ that are not part of the object system are type bool, reference types and pass-by-reference, user-defined overloading, function templates, and exceptions. C++ also has better static type checking than C.

The focus on C++ efficiency is articulated in the design principle, "If you do not use a feature, you should not pay for it." This principle suggests that C programs should compile as efficiently under the C++ compiler as under the C compiler. It would violate this principle to implement C integers as objects, for example, and use dynamic method lookup to find integer functions at run-time as in Smalltalk, as this would significantly reduce the performance of C integer calculations.

The most significant part of C++ is the set of object-oriented concepts added to C; these are the main concepts:

- *Classes*, which declare the type associated with all objects constructed from the class, the data members of each object, and the member functions of the class.
- *Objects*, which consist of private data and public functions for accessing the hidden data, as in other object-oriented languages.
- *Dynamic lookup*, for member functions that are declared virtual. A virtual function in a derived class (subclass) may be implemented differently from virtual functions of the same name in a base class (superclass).
- *Encapsulation*, based on programmer designations public, private, and protected that determine whether data and functions declared in a class are visible outside the class definition.
- *Inheritance*, using subclassing, in which one class may be defined by inheriting the data and functions declared in another. C++ allows single inheritance, in which a class has a single base class (superclass) or multiple inheritance in which a class has more than one base class.
- *Subtyping*, based on subclassing. For one class to define a subtype of the type defined by another class, inheritance must be used. However, the programmer may decide whether inheritance results in a subtype or not by specifying that the base class is public or not.

At run time, C++ virtual function lookup uses indirection through the virtual function table (vtable) of the class. The C++ static type system makes it possible to determine the offset (position of a pointer in the vtable) of virtual function at compile time. When a call like p->move(…) is compiled, the compiler can determine a static type for the pointer p. This static type must be a class, and this class must declare or inherit a function called move. The compiler can examine the class hierarchy to see what location the virtual function move will occupy in the vtable for the class.

A fundamental property of C++ class implementation is that if class B has class A as a public base class, then the initial segment of every B object must look like an A object, and similarly for the B and A virtual function tables. This property makes it possible to access an inherited A member of a B object in exactly the same way we would access the same member of an A object. Although it is fairly easily to guarantee this property with only single inheritance, the implementation of objects and classes with multiple inheritance is more complex.

There are a number of aspects of C++ that programmers occasionally find difficult. Some of the main problem areas are

- *casts and conversions* that can be complex and unpredictable in certain situations,

- *objects allocated on the stack* and other aspects of object memory management,
- *overloading*, a complex code selection mechanism in C++ that can interact unpredicatably with dynamic lookup (virtual function lookup),
- *multiple inheritance*, which is more complex in C++ than in other languages because of the way objects and virtual function tables are configured and accessed.

These problem areas exist not because of oversights but because the goals of C++, followed to their logical conclusion, led to a design with these properties.

Team-Fly



Team-Fly

Chapter 13: Portability and Safety-Java

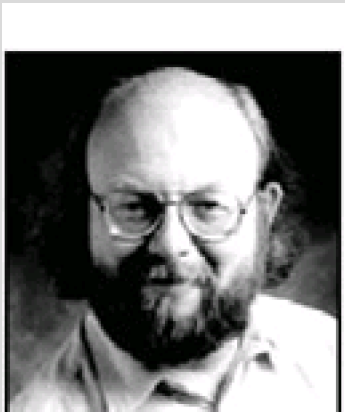
OVERVIEW

The Java programming language was designed by James Gosling and others at Sun Microsystems. The language, arising from a project that began in 1990, was originally called *Oak* and was intended for use in a device affectionately referred to as a *set-top box*. The set-top box was intended to be a small computational device, attached to a network of some kind, and placed on top of a television set. There are various features a set-top box might provide. You can imagine some of your own by supposing that a web browser is displayed on your television set and, instead of a keyboard, you click on icons by using some buttons on your remote control. You might want to select a television program or movie or download a small computer simulation that could be executed on the computational device and displayed on your screen. A television advertisement for an automobile might allow you to download an interactive visual tour of the automobile, giving each viewer a personalized simulation of driving the card own the road. Whatever scenario might appeal to you, the computing environment would involve graphics, execution of simple programs, and communication between a remote site and a program executed locally.

At some point in the development of Oak, engineers and managers at Sun Microsystems realized that there was an immediate need for an Internet-browser programming language, a language that could be used to write small applications that could be transmitted over the network and executed under the control of any standard browser on any standard platform. The need for a standard is a result of the intrinsic desire of companies and individuals with web sites to be able to reach as large an audience as possible. In addition to portability, there is also a need for security so that someone downloading a small application can execute the program without fear of computer viruses or other hazards.

The Oak language started as a reimplementation of C++. Although language design was not an end goal of the project, language design became an important focus of the group. Some of the reasons for designing a new language are given in this overly dramatic but still informative quote from "The Java Saga" by David Bank in *Hot Wired* (December 1995):

> Gosling quickly concluded that existing languages weren't up to the job. C++ had become a near-standard for programmers building specialized applications where speed is everything … But C++ wasn't reliable enough for what Gosling had in mind. It was fast, but its interfaces were inconsistent, and programs kept on breaking. However, in consumer electronics, reliability is more important than speed. Software interfaces had to be as dependable as a two-pronged plug fitting into an electrical wall socket. "I came to the conclusion that I needed a new programming language," Gosling says.







James Gosling was the lead engineer and key architect behind the Java programming language and platform. He is now back in Sun's research laboratories, working on software development tools. His first project at Sun was the NEWS window system, distributed for Sun workstations in the 1980s. Before joining Sun, Gosling built a multiprocessor version of UNIX; the original Andrew window system and toolkit; and several compilers and mail systems. He is known to many as the author of the original UNIX ‘Emacs.'

A techie with a sense of humor, Gosling is shown in the right-hand photograph about to put a pie in the face of a stagehand wearing a Bill Gates mask, in a picture from his partially politicized web page at http://java.sun.com/people/jag/.

James Gosling received a B.S. in Computer Science from the University of Calgary, Canada, and a Ph.D. in Computer Science from Carnegie-Mellon University for a dissertation entitled, "The Algebraic Manipulation of Constraints."

For a variety of reasons, including a tremendous marketing effort by Sun Microsystems, Java became surprisingly successful a short time after it was released as an Internet communication language in mid-1995.

The main parts of the Java system are

- the Java programming language,
- Java compilers and run-time systems (Java virtual machine),
- an extensive library, including a Java toolkit for graphic display and other applications, and sample Java applets.

Although the library and toolkit helped with early adoption, we will be primarily interested in the programming language, its implementation, and the way language design and implementation considerations influenced each other. Gosling, more modest in real life than the preceding quote might suggest, has this to say about languages that influenced Java: "One of the most important influences on the design of Java was a much earlier language called *Simula*. It is the first OO language I ever used (on a CDC 6400!). … [and] where the concept of a ‘class' was invented."

Team-Fly



13.1 JAVA LANGUAGE OVERVIEW

13.1.1 Java Language Goals

The Java programming language and execution environment were designed with the following goals in mind:

- *Portability:* It must be easy to transmit programs over the network and have them run correctly in the receiving environment, regardless of the hardware, operating system, or web browser used.
- *Reliability:* Because programs will be run remotely by users who did not write the code, error messages and program crashes should be avoided as much as possible.
- *Safety:* The computing environment receiving a program must be protected from programmer errors and malicious programming.
- *Dynamic Linking:* Programs are distributed in parts, with separate parts loaded into the Java run-time environment as needed.
- *Multithreaded Execution:* For concurrent programs to run on a variety of hardware and operating systems, the language must include explicit support and a standard interface for concurrent programming.
- *Simplicity and Familiarity:* The language should appeal to your average website designer, typically a C programmer or a programmer partially familiar with C/C++.
- *Efficiency:* This is important, but may be secondary to other considerations.

Generally speaking, the reduced emphasis on efficiency gave the Java designers more flexibility than the C++ designers had.

13.1.2 Design Decisions

Some of the design goals and global design decisions are listed in Table 13.1, in which + indicates that a decision contributes to this goal, – indicates that a decision detracts from this goal, and +/– indicates that there are advantages and disadvantages of the decision. Some squares are left blank, indicating that a design decision has little or no effect on the goal. We can see the relative importance of efficiency in the Java design process by looking down the right most column. This does not mean that efficiency was sacrificed needlessly, only that relative to other goals, efficiency was not the primary objective.

Table 13.1: Java design decisions

	Portability	Safety	Simplicity	Efficiency

Section 13.4

Garbage Collection. Subsection 6.2.1

Concurrency Support. Java has a concurrency model based on threads, which are independent concurrent processes. This is a significant part of the language, both because the design is substantial and because of the importance of having standardized concurrency primitives as part of the Java language. Clearly, if Java programs relied on operating system specific concurrency mechanisms, Java programs would not be portable across different operating system platforms.

Simplicity. Although Java has grown over the years, and features like reflection and inner classes may not seem "simple," the language is still smaller and simpler in design than most production-quality general-purpose programming languages. One way to see the relative simplicity of Java is to list the C++ features that do not appear in Java. These include the following features:

- *Structures and unions:* Structures are subsumed by objects, and some uses of unions can be replaced with classes that share a common superclass.
- *Functions* can be replaced with static methods.
- *Multiple inheritance* is complex and most cases can be avoided if the simpler interface concept of Java is used.
- *Goto* is not necessary.
- *Operator overloading* is complex and deemed unnecessary; Java functions can be overloaded.
- *Automatic coercions* are complex and deemed unnecessary.
- *Pointers* are the default for objects and are not needed for other types. As a result a separate pointer type is not needed.

Some of these features appear in C++ primarily because of the C++ design goal of backward compatibility with C. Others were omitted from Java after some discussion, because it was decided that complexity of including them was more significant than the functionality they would provide. The most significant omissions are multiple inheritance, automatic conversions, operator overloading, and pointer operations of the forms found in C and C++.

Team-Fly

Team-Fly

13.2 JAVA CLASSES AND INHERITANCE

13.2.1 Classes and Objects

Java is written in a C++-like syntax so that programming is more accessible to C and C++ programmers. This makes the C++ one-dimensional point class used in Subsection 12.3.1 look similar when translated into Java. Here is an abbreviated version of the class, with the move method omitted:

```
class Point {
    public int getX() { ... }
    protected void setX (int x){ ... }
    private int x;
    Point(int xval) {x = xval;}
};
```

Like other class-based languages, Java classes declare the data and functions associated with all objects created by this class. When a Java object is created, space is allocated to store the data fields of the object and the constructor of the class is called to initialize the data fields. As in C++, the constructor has the same name as the class. Also following C++, the Point class has public, private, and protected components. Although public, private, and protected are keywords of Java, these visibility specifications do not mean exactly the same thing in the two languages, as explained in Subsection 13.2.2.

Java terminology is slightly different from that of Simula, Smalltalk and C++. Here is a brief summary of the most important terms used to discuss Java:

- *Class* and *object* have essentially the same meaning as in other class-based object-oriented languages, *field:* data member
- *Method:* member function, *static member*: analogous to Smalltalk class field or class method, *this:* like C++ this or Smalltalk self, the identifier this in the body of a Java method refers to the object on which this method was invoked
- *Native Method:* method written in another language, such as C
- *Package:* set of classes in shared name space.

We will look at several characteristics of classes and objects in Java, including static fields and methods, overloading, finalize methods, main methods, toString methods used to produce a print representation of an object, and the possibility of defining native methods. We will discuss the Java run-time representation of objects and the implementation of method lookup in Section 13.4 in connection with other aspects of the architecture of the run-time system.

Initialization. Java guarantees that a constructor is called whenever an object is created. Because some interesting issues arise with inheritance, this is discussed in Subsection 13.2.3.

Static Fields and Methods. Java static fields and methods are similar to Smalltalk class variables and class methods. If a field is declared to be static, then there is one field for the entire class, instead of one per object. If a method is declared static, the method may be called without using an object of the class. In particular, static methods may be called before any objects of the class are created. Static methods can access only static fields and other static methods; they cannot refer to this because they are not part of any specific object of the class. Outside a class, a static member is usually accessed with the class name, as in class_name.static_method(args), rather than through an object reference.

Static fields may be initialized with initialization expressions or a *static initialization block*. Both are illustrated in the following code:

```
class ... {
 /* --- static variable with initial value --- */
 static int x = initial_value;
 /* --- static initialization block --- */
 static { /* code to be executed once, when class is loaded */
 }
}
```

As indicated in the program comment, the static initialization block of a class is executed once, when the class is loaded. Class loading is discussed in Section 13.4 in connection with the JVM. There are specific rules governing the order of static initialization, when a class contains both initialization expressions and a static initialization block. There are also restrictions on the form of static initialization blocks. For example, a static block cannot raise an exception, as it is not certain that a corresponding handler will be installed at class-loading time.

Overloading. Java overloading is based on the signature of a method, which consists of the method name, the number of parameters, and the type of each parameter. If two methods of a class (whether both declared in the same class, or both inherited, or one declared and one inherited) have the same name but different signatures, then the method name is overloaded. As in other languages, overloading is resolved at compile time.

Garbage Collection and Finalize***:*** Because Java is garbage collected, it is not necessary to explicitly free objects. In addition, programmers do not need to worry about dangling references created by premature deallocation of objects. However, garbage collection reclaims only the space used by an object. If an object holds access to another sort of resource, such as a lock on shared data, then this must be freed when the object is no longer accessible. For this reason, Java objects may have finalize methods, which are called under two conditions, by the garbage collector just before the space is reclaimed and by the virtual machine when the virtual machine exits. A useful convention in finalize methods is to call super.finalize, as subsequently illustrated, so that any termination code associated with the superclass is also executed:

```
class...{
 ...
 protected void finalize () {
  super.finalize();
  close(file);
 }
};
```

There is an interesting interaction between finalize methods and the Java exception mechanism. Any uncaught exceptions raised while a finalize method is executed are ignored.

A programming problem associated with finalize methods is that the programmer does not have explicit control over when a finalize method is called. This decision is left to the run-time system. This can create problems if an object holds a lock on a shared resource, for example, as the lock may not be freed until the garbage collector determines that the program needs more space. One solution is to put operations such as freeing all locks or other resources in a method that is explicitly called in the program. This works well, as long as all users of the class know the name of the method and remember to call this method when the object is no longer needed.

Some other interesting aspects of Java objects and classes are main methods used to start program execution, toString methods used to produce a print representation of an object, and the possibility of defining native methods:

- main: A Java application is invoked with the name of the class that drives the application. This class must have a main method, which must be public, static, must return void, and must accept a single argument of type String[]. The main method is called with the program arguments in a string array. Any class with a main method can be invoked directly as if it were a stand-alone application, which can be useful for testing.
- toString: A class may define a toString method, which is called when a conversion to string type is needed, as in printing an object.
- native methods: A native method is one written in another language, such as C. Portability and safety are reduced with native methods: Native code cannot be shipped over the network on demand, and controls incorporated into the JVM are ineffective because the method is not interpreted by the virtual machine. The reasons for using native methods are (1) efficiency of native object code and (2) access to utilities or programs that have already been written in another language.

13.2.2 Packages and Visibility

Java has four visibility distinctions for fields and methods, three corresponding to C++ visibility levels and a fourth arising from packages.

Java *packages* are an encapsulation mechanism similar to C++ name space that allows related declarations to be grouped together, with some declarations hidden from other packages. In a Java program, every field or method belongs to a specific class and every class is part of a package, as shown in Figure 13.1. A class can belong to the unnamed default package, or some other package if specified in the file containing the class.

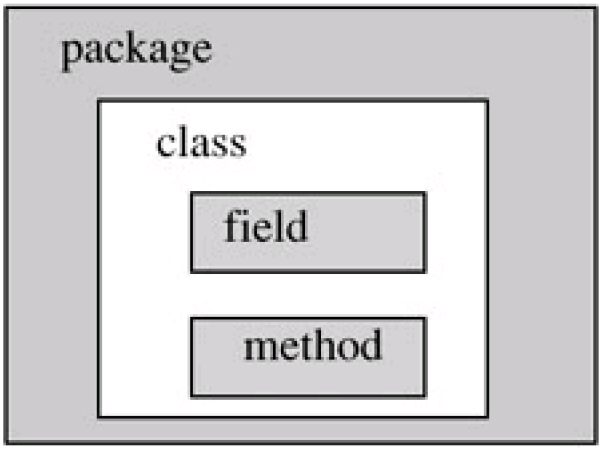


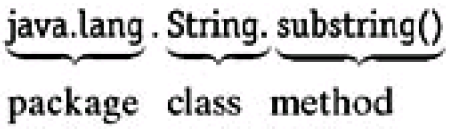
Figure 13.1: Java package and class visibility.

The visibility distinctions in Java are

- *public:* accessible anywhere the class is visible,
- *protected:* accessible to methods of the class and any subclasses, as well as to other classes in the same package,
- *private:* accessible only in the class itself,
- *package:* accessible only to code in the same package; not visible to subclasses in other packages. Members declared without an access modifier have package visibility.

Put another way, a method can refer to the private members of the class it belongs to, non private members of all classes in the same package, protected members of superclasses (including superclasses in a different package), and public members of all classes in any visible package.

Names declared in another package can be accessed with import, which imports declarations from another package, or with qualified names of the following form, which indicate the package containing the name explicitly:



13.2.3 Inheritance

In Java terminology, a *subclass* inherits from its *superclass.* The Java inheritance mechanism is essentially similar to that of Smalltalk, C++, and other class-based object-oriented languages. The syntax associated with inheritance is similar to C++, with the keyword extends, as shown in this example ColorPoint class extending the Point class from Subsection 13.2.1:

```
class ColorPoint extends Point {
  // Additional fields and methods
  private Color c;
  protected void set C (Color d) {c=d;}
  public Color getC() {return c;}
  // Define constructor
  ColorPoint(int xval, Color cval) {
    super(xval); // call Point constructor
    c = cval; }  // initialize ColorPoint field
};
```

Method Overriding and Field Hiding. As in other languages, a class inherits all the fields and methods of its superclass, except when a field or method of the same name is declared in the subclass. When a method name in the subclass is the same as a method name in the superclass, the subclass definition *overrides* the superclass method with the same signature. An overriding method must not conflict with the definition that it overrides by having a different return type. An overridden method of the superclass can be accessed with the keyword super. For fields, a field declaration in a subclass hides any superclass field with the same name. A hidden field can be accessed by use of a qualified name (if it is static) or by use of a field access expression that contains a cast to a superclass type or the keyword super.

Constructors. Java guarantees that a constructor is called whenever an object is created. In compiling the constructor of a subclass, the compiler checks to make sure that the superclass constructor is called. This is done in a specific way that programmers generally want to take into consideration. In particular, if the first statement of a subclass constructor is not a call to super, then the call super() is inserted automatically by the compiler. This does not always

work well, because, if the superclass does not have a constructor with no arguments, the call super() will not match a declared constructor, and a compiler error results. An exception to this check occurs if one constructor invokes another. In this case, the first constructor does not need to call the superclass constructor, but the second one must. For example, if the constructor declaration ColorPoint() {ColorPoint(0,blue);} is added to the preceding ColorPoint class, then this constructor is compiled without inserting a call to the superclass Point constructor.

A slight oddity of Java is that the inheritance conventions for finalize are different from the conventions for constructors. Although a call to the superclass is required for constructors, the compiler does not force a call to the superclass finalize method in a subclass finalize method.

Final Methods and Classes. Java contains an interesting mechanism for restricting subclasses of a class: A method or an entire class can be declared final. If a method is declared final, then the method cannot be overridden in any subclass. If a class is declared final, then the class cannot have any subclasses. The reason for this feature is that a programmer may wish to define the behavior of all objects of a certain type. Because subclasses produce subtypes, as discussed in Section 13.3, this requires some restriction on subclasses. To give an extreme example, the singleton pattern discussed in Section 10.4 shows how to design a class so that only one object of the class can be created. The pattern hides the constructor of the class and makes public only a function that will call the constructor once during program execution. This pattern solves the problem of restricting the number of objects of the class, but only if no subclass overrides the public method with a method that can create more than one object. If a programmer really wants to enforce the singleton pattern, there must be a way to keep other programmers from defining subclasses of the singleton class.

The Java class java.lang. System is another example of a final class. This class is final so that programmers do not override system methods.

In a loose sense, Java final is the opposite of C++ virtual: Java methods can be overridden until they are marked final, whereas C++ member functions can be overridden only if they are virtual. The analogy is not exact, though, as a C++ member function cannot be virtual in one class and nonvirtual in a base class or derived class, because that would violate the requirement that base- and derived-class vtables must have the same layout.

Class ***Object.*** In principle, every class declared in a Java program extends another class, as a class without an explicit superclass is interpreted as a subclass of the class Object. The class Object is the one class that has no superclasses. Class Object contains the following methods, which can be overridden in derived classes:

- GetClass, which returns the Class object that represents the class of the object. This can be used to discover the fully qualified name of a class, its members, its immediate superclass, and any interfaces that it implements.
- ToString, which returns a String representation of an object.
- equals, which defines a notion of object equality based on value, not reference, comparison.
- hashCode, which returns an integer that can be used to store the object in a hash table.
- clone, which is used to make a duplicate of an object.
- Methods wait, notify, and notifyAll used in concurrent programming.
- finalize, which is run just before an object is destroyed (discussed in Subsection 13.2.1 in connection with garbage collection),

Because all classes inherit the methods of class Object, every object has these methods.

13.2.4 Abstract Classes and Interfaces

The Java language has an *abstract-class* mechanism that is similar to C++. As we discussed in Chapter 12, an abstract class is a class that does not implement all of its methods and therefore cannot have any instances. Java uses the keyword abstract instead of the C++ "=0" syntax, as shown in the following code:

```
abstract class Shape {
 ...
 abstract point center();
 abstract void rotate(degrees d);
 ...
}
```

Java also has a "pure abstract" form of class called an interface. An interface is defined in a manner similar to that of a class, except that all interface members must be constants or abstract methods. An interface has no direct implementation, but classes may be declared to implement an interface. In addition, an interface may be declared as an extension of another, providing a form of interface inheritance.

One reason that Java programmers use interfaces instead of pure abstract classes when a concept is being defined but not implemented is that Java allows a single class to implement several interfaces, whereas a class can have only one superclass. The following interfaces and class illustrate this possibility. The Shape interface identifies some properties of simple geometric shapes, namely, each has a center point and a rotate method. Drawable similarly identifies properties of objects that can be displayed on a screen. If circles are geometric shapes that can be displayed on a screen, then the Circle class can be declared to implement both Shape and Drawable, as subsequently shown.

Interfaces are often used as the type of argument to a method. For example, if the windows system has a method for drawing items on a screen, then the argument type of this method could be Drawable, allowing every object that implements the Drawable interface to be displayed:

```
interface Shape {
 public Point center();
 public void rotate(float degrees);
}
interface Drawable {
 public void setColor(Color c);
 public void draw();
}
class Circle implements Shape, Drawable {
 // does not inherit any implementation
 // but must define Shape, Drawable methods
}
```

Unlike C++ multiple inheritance (discussed in Section 12.5), there is no name clash problem for Java interfaces. More specifically, suppose that the preceding two interfaces Shape and Circle also both define a Size method. If the two Size methods both have the same number of arguments and the same argument types, then class Circle must implement one Size method with this number of arguments and the argument types given in both interfaces. On the other hand, if the two Size methods have a different number of arguments or the arguments can be distinguished by type, then these are considered two different method names and Circle must define an implementation for each one. Because Java method lookup uses the method name and number and types of arguments to select the method code, the two methods with the same name will be treated separately at method lookup time.

13.3 JAVA TYPES AND SUBTYPING

13.3.1 Classification of Types

The Java types are divided into two categories: primitive types and reference types. The eight primitive types are the boolean type and seven numeric types. The seven numeric types are the forms of integers byte, short, int, long, and char and the floating-point types float and double. The three forms of reference types are class types, interface types (subsequently discussed), and array types. There is also a special null type. The values of a reference type are references to objects (which include arrays). All objects, including arrays, support the methods of class Object.

The subtyping relationships between the main families of types are illustrated in Figure 13.2, which includes a Shape interface and Circle and Square classes to show how user-defined classes and interfaces fit into the picture. Other predefined types such as String, ClassLoader, and Thread occupy positions similar to that of Exception in this figure. Object[] is the type of arrays of objects, and similarly for array types Shape[], Circle[], and Square[].

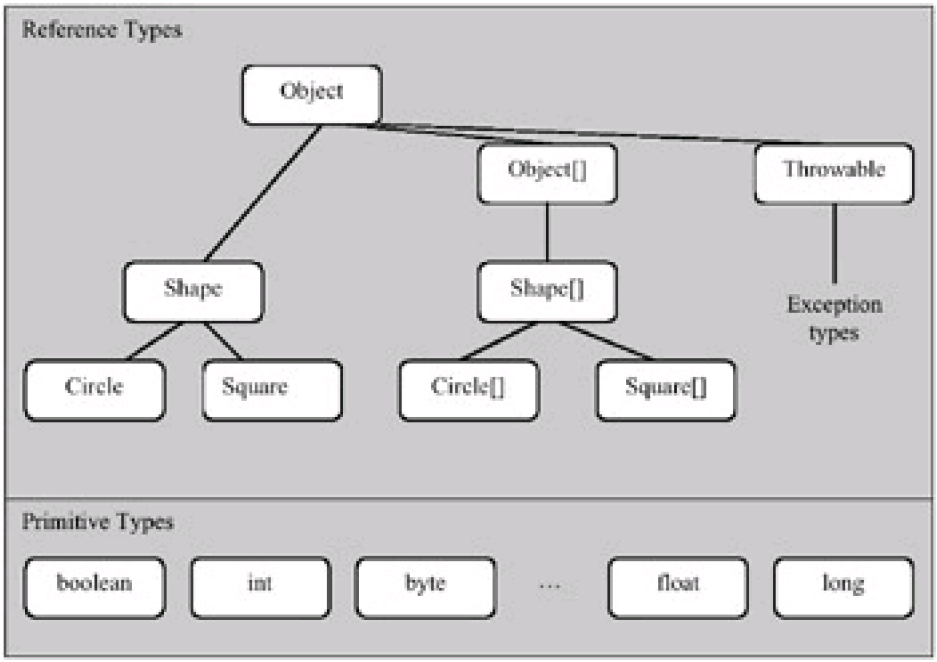


Figure 13.2: Classification of types in Java.

Although it is standard Java terminology to call Object and its subtypes reference types, this may be slightly confusing. Although C++ distinguishes Object from Object*, there are no explicit pointer types in Java. Instead, the difference between a pointer to an object and an object itself is implicit, with pointer dereferencing combined with operations like method invocation and field access. If T is a reference type, then a variable x of type T is a reference to T objects; in C++ the variable x would have type T*. Because there is no explicit way to dereference x to get a value of type T, Java does not have a separate type for objects not referred to by pointer.

Because every class is a subtype of Object, variables of type Object can refer to objects or arrays of any type.

13.3.2 Subtyping for Classes and Interfaces

Subtyping for classes is determined by the class hierarchy and the interface mechanism. Specifically, if a class A extends class B, then the type of A objects is a subtype of the type of B objects. There is no other way for one class to define a subtype of another, and no private base classes (as in C++) to allow inheritance without subtyping.

A class may be declared to implement one or more interfaces, meaning that any instance of the class implements all the abstract methods specified in the interface. This (multiple) interface subtyping allows objects to support (multiple) common behaviors without sharing any common implementation.

Run-Time Type Conversion. Java does not allow unchecked casts. However, objects of a supertype may be cast to a subtype by a mechanism that includes a run-time type test. If you want to make a list class in Java, you make it hold objects of type Object. Objects of any class can then be put onto a list, but, to take them off the list and use them nontrivially, they must be converted back to their original type (or some supertype). In Java, type conversion is checked at run time, raising an exception if the object does not have the designated type.

Implementation. Java uses different bytecode instructions for member lookup by interface and member lookup by class or subclass. In a high-performance compiler, lookup by class could be implemented as in C++, with offset known at compile time. However, lookup by interface cannot, because a class may implement many interfaces and the interfaces may list members in different orders. This is discussed in detail in Subsection 13.4.4.

13.3.3 Arrays, Covariance, and Contravariance

For any type T, Java has an *array* type T[ ] of arrays whose elements have type T. Although array types are grouped with classes and interfaces, it is not possible to inherit from an array type. In Java terminology, array types are final.

Array types are subtypes of Object, and therefore arrays support all of the methods associated with class Object. Like other reference types, an array variable is a pointer to an array and can be null. It is common to create arrays when an array reference is declared, as in

```
Circle[ ] x = new Circle[array_size]
```

However, it is also possible to create array objects "anonymously," in much the same way that we can create other Java objects. For example,

```
new int[ ] {1,2,3, . . . 10}
```

is an expression that creates an integer array of length 10, with values 1, 2, 3, …, 10. Because a variable of type T[ ] can be assigned an array of any length, the length of an array is not part of its static type.

There are some complications surrounding the way that Java array types are placed in the subtype hierarchy. The most significant decision is that if A <: B then the Java type checker also uses the subtyping A[]<:B[]. This introduces a problem often referred to as the *array covariance problem*. Consider the following class and array declarations:

```
class A { ... }
class B extends A{ ... }
```

```
B[ ] bArray = new B[10]
A[ ] aArray = bArray   // considered OK since B[ ] <: A[ ]
aArray[0] = new A()  // allowed but run-time type error; raises
ArrayStoreException
```

In this code, we have B<:A because class B extends class A. The array reference bArray refers to an array of B objects, initially all null, and the array reference aArray refers to the same array. The declaration A[ ] aArray = bArray is allowed by the Java type checker (although semantically it should not be allowed) because of the Java design decision that if B<:A then B[]<:A[]. The problem with allowing the A[ ] array reference aArray to refer to an array of B objects is illustrated by the last statement. The assignment aArray[0] = new A() looks perfectly reasonable: aArray is an array with static type A[ ], suggesting that it is acceptable to assign an A object to any location in the array. However, because a Array actually refers to an array of B objects, this assignment would violate the type of bArray. Because this assignment would cause a typing problem, the Java implementation does not allow the assignment to be executed. A run-time test will determine that the value being assigned to an array of B objects is not a B object and the ArrayStoreException exception will be raised.

Although the Java designers considered array covariance advantageous for some specific purposes (writing some binary copy routines), array covariance in Java leads to some confusion and many run-time tests. It does not seem to be a successful language design decision.

13.3.4 Java Exception Class Hierarchy

Java programs may declare, raise, and handle exceptions. The Java exception mechanism has the general features we discussed in Section 8.2. Java exceptions may be the result of a throw statement in a user program or the result of some error condition detected by the virtual machine such as an attempt to index outside the bounds of an array. In Java terminology, an exception is said to be *thrown* from the point where it occurred and is said to be *caught* at the point to which control is transferred. As in other languages, throwing an exception causes the Java implementation to halt every expression, statement, method or constructor invocation, initializer, and field initialization expression that has started but not completed execution. This process continues until a handler is found that matches the class of the exception that is thrown.

One interesting aspect of the Java exception mechanism is the way that it is integrated into the class and type hierarchy. Every Java exception is represented by an instance of the class Throwable or one of its subclasses. An advantage of representing exceptions as objects is that an exception object can be used to carry information from the point at which an exception occurs to the handler that catches it. Subtyping can also be used in handling an exception: A handler matches the class of an exception if it explicitly names the class of the exception that is thrown or names some superclass of the class of the exception.

The Java exception mechanism is designed to work well in multithreaded (concurrent) programs. When an exception is thrown, only the thread in which the throw occurs is affected. The effect of an exception on the concurrent synchronization mechanism is that locks are released as synchronized statements and invocations of synchronized methods complete abruptly. We will return to this topic in Chapter 14.

Java exceptions are caught inside a construct called a *try-finally block*. Here is an example outline, showing a block with two handlers, each identified by the catch keyword. Intuitively, a try-finally block tries to execute a sequence of statements. If the sequence of statements terminates normally, then that is the end result of the block. However, if an exception is raised, it may be caught inside the block. If an exception is raised and caught, then the sequence of statements following the keyword finally will be executed after the exception handler has finished:

```
try {
        ?statements?
```

```
} catch (?ex-type1??identifier1?) {
        ?statements?
   catch  (?ex-type2??dentifier2?) {
        ?statements?
} finally {
        ?statements?
}
```

Although it may not be apparent why, there is some complication in the JVM associated with try-finally blocks. Specifically, a significant fraction of the complexity of the Java bytecode verifier is a result of the way finally clauses are implemented as a form of "local subroutine" (called jsr) in the Java bytecode interpreter. We will discuss the JVM in Section 13.4.

The classes of exceptions are shown in Figure 13.3. Every exception is, by definition, an object of some subclass of Throwable. The class Throwable is a direct subclass of Object. Programs can use the preexisting exception classes in throw statements or define additional exception classes. Additional exceptions classes must be subclasses of Throwable or one of its subclasses. To take advantage of the Java platform's compile-time checking for exception handlers, it is typical to define most new exception classes as checked exception classes. These are subclasses of Exception that are not subclasses of RuntimeException.

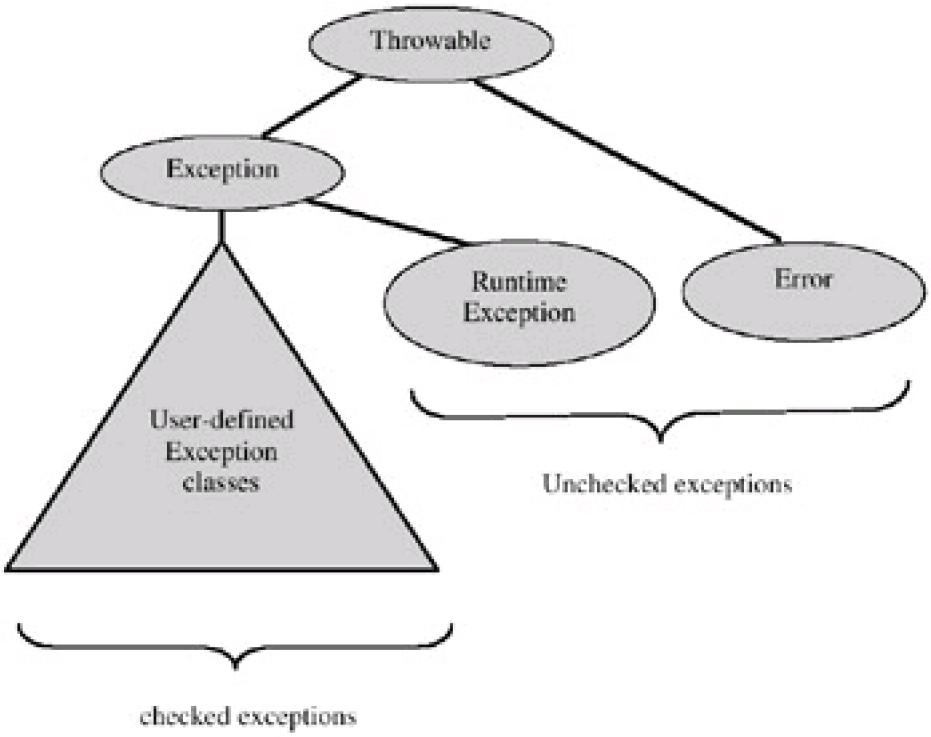


Figure 13.3: Java exception classes.

The phrases checked exceptions and "unchecked exceptions" refer to compile-time checking of the set of exceptions that may be thrown in a Java program. The Java compiler checks, at compile time, that a program contains handlers for every checked exception. We accomplish this by analyzing the checked exceptions that can be thrown during execution of a method or constructor. These checks are based on the declared set of exceptions that a Java method could throw, called the throws clause of the method. For each checked exception that is a possible result of a method call, the throws clause for the method must mention the class of that exception or one of the superclasses of the class of that exception. This compile-time checking for the presence of exception handlers is designed to reduce the number of exceptions that are not properly handled.

Because Error and RuntimeException exceptions are generally thrown by the Java run-time system and not by the user

code, it is not necessary for a programmer to declare these exceptions in the throws clause of a method. More specifically, the run-time exception classes (RuntimeException and its subclasses) are exempt from compile-time checking because, in the judgment of the designers of the Java programming language, declaring such exceptions would not aid significantly in establishing the correctness of programs. Many of the operations and constructs of the Java programming language can result in run-time exceptions. The information available to a compiler and the level of analysis the compiler performs are usually not sufficient to establish that such run-time exceptions cannot occur, even though this may be obvious to the programmer.

Ordinary programs do not usually recover from exceptions from the class Error or its subclasses. The class Error is a separate subclass of Throwable, distinct from Exception in the class hierarchy, to allow programs to use the idiom

---

```
} catch (Exception e) {
```

---

to catch all exceptions from which recovery may be possible without catching errors from which recovery is typically not possible.

13.3.5 Subtype Polymorphism and Generic Programming

In Section 9.4, we discussed some approaches to generic programming based on templates and related mechanisms such as generic modules and packages. The main idea is that we can define a generic data structure, such as a list or binary search tree, so that any type of data can be inserted into or retrieved from the data structure, provided that a few required operations, such as equality test or a linear ordering, are defined and implemented on the data. Generic algorithms such as sorting, applicable to any list or array of data, can be defined similarly by use of templates or related mechanisms. Templates and generic modules are useful in generic programming because they provide a form of polymorphism: The same code may be instantiated and executed on a variety of types of data.

In object-oriented programming, the term polymorphism is often used to refer to a specific kind of polymorphism that we call *subtype polymorphism*. In subtype polymorphism, a single piece of code, typically a set of functions or methods, can be applied to more than one type of argument. However, the language mechanism for allowing this is not through implicit type parameters (as in ML type-inference polymorphism) or explicit type parameters (as in C++ templates or Ada generic packages), but through subtyping. Specifically, in a language with subtyping, if a method m will accept any argument of type A, then m may also be applied to any argument from any subtype of A. This works almost as well as parametric polymorphism (polymorphism based on type parameters), except that typically the compile-time type checker has less information about the types of arguments. This either leads to run-time type checking or some sacrifice in type safety. The general phenomenon is illustrated in the following Java example.

Example 13.1 Java Subtype Polymorphism

Let us consider the problem of defining a general class of stacks. We would like to be able to build stacks of any type of object. We will implement stacks as linked lists of stack nodes, each node containing a stack element.

Because Object is a supertype of all reference types (including all references to objects and arrays), we can let stack cells contain references of type Object. Let us begin with a subsidiary class definition, the class of stack nodes:

```
class Node {
  Object element; // Any object can be placed in a stack
  Node next;
  Node (Node n, int e) {
```

```
  next=n;
  element=e;
 }
}
```

This class is intended to belong to the same package as that of the Stack class. Because there are no access qualifiers here (public, private, or protected) the fields and methods are accessible only to methods in the same package.

Here is the outline of a Stack class in which Node objects are used to store elements of a linked list of stack elements. The methods shown here are empty, which tells whether a stack is empty, push, which adds a new element to the top of a stack, and pop, which removes the top node and returns the object stored in that node. The class is declared public so that it is visible outside the package in which it is defined:

```
public class Stack {
  private Node top=null;         // top of the stack, starts as empty
  public Stack(){
  }
  public boolean empty(){        // determine whether stack is empty
    return top == null;
 }
  public void push(Object val) { // push element on top of stack
    top = new Node(top,val);
 }
  public Object pop() {          // Remove first node and return value.
    if (top==null) return -1;
    Node temp = top;
    top = top.next;
    return temp.element;
 }
}
```

Most of the code here is entirely straightforward. The important limitation, for the purpose of generic programming, is the type of the node elements and the return type of pop. Because the argument type of push is Object, any object can be pushed onto a stack. However, the return type of pop is also Object, meaning that, when an object is removed from a stack, the static type of the expression removing the object is Object. Therefore, if a program pushes strings onto a stack and then pops them off the stack, the Java type checker will not be able to determine that the objects removed from the stack are strings.

Here is an example of the way that stacks might be used in a Java program:

```
String s = "Hello";
Stack st = new Stack();
...
st.push(s);
...
s = (String) st.pop;
```

In this code fragment, a String object is pushed onto a stack. When the String object is popped off the stack, the static type of st.pop() is Object, not string. Therefore, to assign st.pop() to a string variable, it is necessary to cast the Object to String. This cast generates a run-time test. If the top element on the stack is not a string when this assignment is executed, the run-time test will throw an exception and prevent the assignment from occurring.

In comparison, a C++ stack class template may be defined by the following form:

```
template <typename t> class Stack {
  private: Node<t> top;
  public: boolean empty() {...};
          void push (t* x) {...}
          t* pop ( ){...}
};
```

If Java had a template mechanism similar to this, then we could define a generic stack class like this,

```
class Stack<A> {
  public Boolean empty(){...}
 public void push(A a){...}
  public A pop(){...}
}
```

and use generic stacks as follows:

```
String s = "Hello";
Stack<String> st = new Stack<String>();
st.push(s);
...
s = st.pop();
```

There are several important points of comparison between the two styles illustrated here. In the template-based code, the stack st has type Stack<string> instead of Stack. As a result, st.pop() has static type String. Two advantages are clarity and efficiency. The programmer's intent is clearer, as the stack st is explicitly declared to be a stack of strings.

The efficiency advantage is that there is no need for a cast and run-time test.

There have been several research projects aimed at adding a template mechanism to Java. Although the basic goals of such a mechanism seem clear, there are a number of details that need to be addressed. One issue is implementation. One approach is to simply translate Java with templates into Java without templates. This allows programmers to write code with templates and provides a relatively simple implementation. However, the result of the translation is code that contains many run-time-checked type conversions, so it may not be as efficient as other implementation techniques. An alternative is to compile a generic class such as Stack<t> into a form of class file that has type parameters and load class Stack<String> by instantiating this class file at class-load time. This produces more efficient code, but might involve loading more classes. Because class loading is slow, the overall program may run more slowly in some cases. There is also the problem of instantiating class templates for primitive types such as boolean, int, float, and so on. The first implementation method will not work for primitive types, as these are stored differently from object types. The second method, instantiating class files at load time, can be made to work, although handling primitive types adds some complexity to the mechanism. In either case, there are some additional subtleties in the design of the type-checking mechanism for generic classes that we have not discussed.

Team-Fly



Previous Next

Team-Fly

13.5 SECURITY FEATURES

Computer security involves preventing unauthorized use of computational resources. Some computer security topics are network security, which provides methods for preserving the secrecy and integrity of information transmitted on the network, and operating system security, which is concerned with preventing an attacker from gaining access to a system. In this section, we look at ways that programming languages can help and hinder our efforts to build secure systems, with particular attention to how Java is designed to improve the security of systems that use Java.

One way that attackers may try to gain access to a system is to send data over the network to a program that processes network input. For example, many attacks against the e-mail program Sendmail have been devised over the years. These attacks typically take advantage of some error or oversight in a particular implementation of Sendmail. Sendmail is a target because this program accepts network connections and runs with super user privileges. In Subsection 13.5.1 we will look at buffer overflow attacks, which are an important example of the kind of attack that is often mounted against network programs.

Because Java code is sometimes transmitted over the network, a computer user may run code that comes from another site, such as a web page. *Mobile code,* a general term for code transmitted over the network before it is executed, provides another way for an attacker to try to gain access to a system or cause damage to a system. The two main mechanisms for managing risks raised by mobile code are called *sandboxing* and *code signing*. The idea behind sandboxing, which was invented before the development of Java, is to give a program a restricted execution environment. This restricted environment, called a sandbox, may contain certain functions (analogous to certain toys you might give a child to play with in a sandbox), but the program cannot call functions or perform actions that are outside the sandbox.

Code signing is used to determine which person or company produced a class file. With digital signatures, it is possible for the producer of code to sign it in a way that any recipient can verify the signature. The JVM uses digital signatures to identify the producer of a file that contains Java bytecode. The user of a computer system can specify a set of trusted producers and allow code from these producers to execute on their computer. With the Java security manager, it is also possible to assign different access permissions to different code producers.

After looking at buffer overflow, we will discuss some of the main features of the Java sandbox. If you want to learn more about the Java security model, you can read one of the books on Java security or find additional information on the web.

13.5.1 Buffer Overflow Attack

In *a buffer overflow attack*, an attacker sends network messages that cause a program to read data into a buffer in memory. If the program does not check the number of bytes placed in the buffer, then the attacker may be able to write past the allocated memory. By writing over the return address that is stored in the activation record of a function, the attacker may cause the program to misbehave and "return" by jumping to instructions chosen by the attacker. This is a powerful form of attack. The vast majority of security advisories issued by the Computer Emergency Response Team (CERT) over the past decade or two have been security problems related to buffer overflow.

Java code is not vulnerable to buffer overflow attacks because the JVM check s array bounds at run time. This prevents a function from writing more data into an array than the array can hold. To show how important this feature is for Java security, we look at how a buffer overflow attack may be designed against C code.

Here is a very simple C function that illustrates the principle, taken from an article called "Smashing the Stack for Fun and Profit," written by someone called Aleph One (Phrack 49(14), 1996):

```
void f (char *str) {
char buffer[16];
   ...
  strcpy(buffer,str);
}
```

This function copies a character string, passed by pointer, into the array buffer on the run-time stack. The C function strcpy() copies the contents of *str into buffer[ ] until a null character is found in the string. Because the function does not check the length of the string, careful C programmers would consider the use of strcpy() a coding error. The similar function strncpy(), which has an additional parameter n and copies at most n characters, should be used instead. If the function f is called from this main program,

```
void_main() {
   char large_string[256];
   int i;
   for( i = 0; i < 255; i++)
       large_string[i] = 'A';

   large_string[255]=0;
   f(large_string);
}
```

then the call to strcpy in function f will write over locations on the stack. Because buffer[ ] is 16 bytes long and large_string is 256 bytes, the copy will write 240 bytes beyond the end of the array. This will change the return address of f, which is sitting in the activation record for f a few memory locations away from buffer[ ]. Because large_string contains the character 'A', with hex character value 0x41, the return address of f is set to 0x41414141. Because this address is outside of the process address space, a segmentation fault will occur when function f tries to "return" to location 0x41414141. However, if we change 'A' to a value that is the address of a meaningful instruction, the buffer overflow will cause function f to jump to that location.

Several steps are needed to turn this simple idea into an attack. First, the attacker needs to choose a function that reads data that the attacker can control. Next, the attacker must figure out what data will cause the function to misbehave in a way that is of interest to the attacker. On some computer architectures, an attacker can fill a buffer with code the attacker wants to execute and then cause the function to jump into the buffer. This allows the attacker to execute any instructions.

13.5.2 The Java Sandbox

The JVM executes Java bytecode in a restricted environment called the Java sandbox. The term sandbox is metaphorical - there is no literal data structure or code called the sandbox. The word sandbox is used to indicate that, when bytecode is executed, some operations that can be written in the Java language might not be allowed to proceed, just the way that, when a child plays in a sandbox, an adult supervisor may give the child only those toys that the supervisor considers safe.

The JVM restricts the operation of a Java bytecode program by using four interrelated mechanisms: the class loader, the Java bytecode verifier, run-time checks performed by the JVM, and the actions of the security manager. Although the class loader, bytecode verifier, and virtual machine run-time checks perform general functions related to type correctness and preserving the integrity of the run time system, the security manager is a specific object designed specifically for security functions. We take a quick look at how each of these mechanisms contributes to Java security. If you are interested in learning more, you may want to read *Securing Java* by Gary McGraw and Ed Felten (Wiley, 1999) or other books on Java security or the JVM.

Class Loader

The class loader, discussed in Subsection 13.4.2, contributes to the Java sandbox in three ways:

- The class-loader architecture separates trusted class libraries from untrusted packages by making it possible to load each with different class loaders.
- The class-loader architecture provides separate name spaces for classes loaded by different class loaders.
- The class loader places code into categories (called *protection domains*) that let the security manager restrict the actions that specific code will be allowed to take.

The Bytecode Verifier and Virtual Machine Run-Time Tests

The Java bytecode verifier checks Java bytecode programs before they are executed, as discussed in Subsection 13.4.3. Complimentary run-time checks, such as array-bounds checks, are performed by the JVM. Together, these checks provide the following guarantees:

- *No Stack Overflow or Underflow:* The verifier examines the way that bytecode manipulates data on the operand stack and guarantees that no method will overflow the operand stack allocated to it.
- *All Methods are Called with Parameters of the Correct Type:* This type-correctness guarantee prevents the kind of type-confusion attacks discussed in Subsection 13.5.3.
- *No Illegal Data Conversions (Casts) Occur:* For example, treating an integer as a pointer is not allowed. This is also essential to prevent the kind of type-confusion attacks discussed in Subsection 13.5.3.
- *Private, Public, Protected, and Default Accesses must be Legal:* In other words, no improper access to restricted classes, interfaces, variables, and methods will be allowed.

The Security Manager

The security manager is a single Java object that answers at run time. The job of the security manager is to keep track of which code is allowed to do which dangerous operations. The security manager does its job by examining the protection domain associated with a class. Each protection domain has two attributes - a signer and a location. The

signer is the person or organization that signed the code before it was loaded into the virtual machine. This will be null if the code is not signed by anyone. The location is the URL where the Java classes reside. These attributes are used to determine which operations the code is allowed to perform. A standard security manager will disallow most operations when they are requested by untrusted code and may allow trusted code to do whatever it wants.

A running virtual machine can have only one security manager installed at a time. In addition, once a security manager has been installed, it cannot be uninstalled without restarting the virtual machine. Java-enabled applications such as web browsers install a security manager as part of their initialization, thus locking in the security manager before any potentially untrusted code has a chance to run.

The security manager does not observe the operation of other running bytecode. Instead, the security manager uses the system policy to answer questions about access permissions. It is up to library code that has access to important resources to consult the security manager. In outline, the standard Java library uses the security manager according to the following steps:

- If a Java program makes a call to a potentially dangerous operation in the Java API, the associated Java API code asks the security manager whether the operation should be allowed.
- If the operation is permitted, the call to the security manager returns normally (without throwing an exception) and the Java API performs the requested operation.
- If the operation is not permitted, the security manager throws a SecurityException. This exception propagates back to the Java program.

One limitation of this mechanism is that a hostile applet or other untrusted code may catch a security exception by using a try-finally block. Therefore, although the security manager may prevent an action by raising an exception, this exception might not be noticed by the user of a system under attack.

13.5.3 Security and Type Safety

Type safety is the cornerstone of Java security, in the sense that all of the sandbox mechanisms rely on type safety. To understand the importance of type safety, we discuss some of the ways that type errors can allow code to perform arbitrary and potentially dangerous actions. We begin with a C example, as C has many type loop-holes. It is then shown how similar problems could occur in Java code if it were possible to create two pointers with different types that point to the same object.

One way for a C/C++ program to call an arbitrary function is through a function pointer. For example, suppose a C/C++ program contains lines of the following form:

```
int (*fp)() /* variable "fp" is a function pointer */
. . .
fp = addr; /* assign fp an address stored in an integer variable */
(*fp)(n);  /* call the function at this address */
```

Using casts, code like this can call a function at virtually any address. Therefore, if a program may cast integers to function pointers, it is impossible to tell in advance which functions the program calls.

Similar type confusion between object types would also allow a program to perform dangerous actions. Here is an example from *Securing Java* by McGraw and Felten. Suppose that it is possible to create a Java program with two pointers to the same object o, one pointer with type T and one with type U. Let us further suppose that classes T and U are defined as follows:

```
class T {
    SecurityManager x;
}
class U {
    MyObject x;
}
```

where MyObject is some class that we can write in whatever way we want. Here is a program that uses the two pointers to object o to change the security manager, even though the fields of the security manager are declared private and should not be subject to alteration by arbitrary code:

```
T t = (the pointer to object o with type T);
U u = (the pointer to object o with type U);
t.x = System.getSecurity(); // the Security Manager
MyObject m = u.x;
```

The first two lines of this code segment create two references to object o with types T and U, respectively. The next line assigns the security manager to the field x of object o. This assignment is allowed because the type of field x is SecurityManager. The final line then gives us a reference m to the security manager, where the type of m is MyObject, a class that we can declare in any way we wish. By changing the fields of m, the program can change the private fields of the security manager.

Although this example shows how type confusion can be used to corrupt the security manager, the basic tactic may be used to corrupt almost any part of the running Java system.

An early type-confusion attack on Java was made possible by a buggy class loader and linker. The incorrect system aborted class loading if an exception was raised but did not remove the class name and associated class methods. More specifically, the first step in class loading was to add the class name and constructor to the table of defined references, then load the bytecode for the class. If an exception was raised during the second step, the first step was not undone. This made it possible to then successfully load another class with a different interface and to use the first class name for objects of the second class.

Team-Fly

13.6 JAVA SUMMARY

The Java programming language and execution environment were designed with portability, reliability, and safety in mind. These goals took higher priority than efficiency, although many engineers and implementers have worked hard to make Java implementations run efficiently.

Java programs are compiled from Java source code (in the Java programming language) to Java bytecode, an interpreted language that is executed on a simple stack-based virtual machine. The run-time environment provides dynamic linking, allowing classes to be loaded into the virtual machine as they are needed. The Java language contains specific concurrency primitives (discussed in Chapter 14), and the virtual machine runs multiple threads simultaneously.

Java is a modern general-purpose object-oriented language that is used in many academic, educational, and commercial projects. We studied the object-oriented features of Java, the Java Virtual Machine and implementation of method lookup, and Java security features.

Objects and Classes

A Java object has fields, which are data members of the object, and methods, which are member functions. All objects are allocated on the run-time heap (not the run-time stack), accessible through pointers, and garbage collected.

Every Java object is an instance of a class. Classes can have static (class) fields that are initialized when the class is loaded into the virtual machine. A constructor of the class is used to create and initalize objects, and objects may have a finalize method that is called when the garbage collector reclaims the space occupied by the object.

Dynamic Lookup

Method lookup in Java is more efficient than in Smalltalk, because the static type system gives the compiler more information, but often is less efficient than in C++. When instructions that access a field or method are executed the first time, there is a search for the appropriate location in an object template or method lookup table. After the first execution of a reference inside a program, the virtual machine optimizes the search for object fields and methods by modifying bytecode. Bytecode instructions that refer to the constant pool are modified to an equivalent so-called *quick* bytecode, which refers to the absolute address of a field or method.

Encapsulation

Java has four visibility distinctions for fields and methods:

- *Public:* accessible anywhere the class is visible.
- *Protected:* accessible to methods of the class and any subclasses as well as to other classes in the same package.
- *Private:* accessible only in the class itself.
- *Package:* accessible only to code in the same package; not visible to subclasses in other packages. Members declared without an access modifier have package visibility.

Names declared in another package can be accessed by import or by qualified names.

Java type checking, applied when source code is compiled and again when byte-code is loaded into the virtual machine, together with run-time tests performed by the bytecode interpreter, guarantees that private fields are truly private and similarly for other visibility levels.

Inheritance

In Java terminology, a *subclass* inherits from its *superclass.* The Java inheritance mechanism is essentially similar to that of Smalltalk, C++, and other class-based object-oriented languages. Java provides single inheritance for classes and multiple inheritance for interfaces.

Java individual methods of a class or an entire class can be declared final. If a method is declared final, then the method cannot be overridden in any subclass. If a class is declared final, then the class cannot have any sub classes. This gives the designer of a class the ability to fix the implementation of a method (or the entire class) by keeping designers of subclass from overriding the method.

In principle, every class declared in a Java program inherits from another class. The class Object is the one class that has no superclasses.

Subtyping

Subtyping for classes is determined by the class hierarchy and the interface mechanism. Specifically, if class A extends class B, then the type of A objects is a subtype of the type of B objects. There is no other way for one class to define a subtype of another, and no private base classes (as in C++) to allow inheritance without subtyping.

A class may be declared to implement one or more interfaces, meaning that any instance of the class implements all the abstract methods specified in the interface. This (multiple) interface subtyping allows objects to support (multiple) common behaviors without sharing any common implementation.

The Java types are divided into two categories, called primitive types and reference types. The primitive types include boolean and seven numeric types. The three forms of reference types are class types, interface types, and array types. All class, interface, and array types are considered subtypes of Object. Java exception types are subclasses of Throwable, which is a subtype of Object.

Virtual Machine Architecture

The main parts of the Java Virtual Machine are the class loader, the bytecode verifier, the linker, and the bytecode interpreter. A Java source code file is compiled to produce a *class file*. Class files are in a specific format that contains bytecode instructions and some auxiliary data structures, including a symbol table called a *constant pool.* The class loader reads the class file and arranges the bytecode instructions in a form suitable for the verifier. The Java bytecode language contains type information, allowing the verifier to checks that bytecode is type correct before it is executed by the virtual machine. The linker resolves interfile references, and the bytecode interpreter executes the bytecode instructions.

When instructions that access a field or method are executed, there is a search for the appropriate location, based on the symbolic name of the field or method in the constant pool. The JVM implementation optimizes this search by modifying bytecode during program execution. Bytecode instructions that initially refer to the constant pool are modified to an equivalent so-called quick bytecode, which refers to the absolute address after the name has been resolved. Different degrees of optimization are possible for method invocation when the class is known and method invocation when the interface is guaranteed to be the same for all executions of the instruction.

Security

Java security is provided by type-correct compilation and execution and by access control through the security manager. The JVM restricts the operation of a Java bytecode program by using four interrelated mechanisms: the class loader, the Java bytecode verifier, run-time checks performed by the JVM, and the actions of the security manager. Whereas the class loader, bytecode verifier, and virtual machine run-time checks perform general functions related to type correctness and preserving the integrity of the run-time system, the security manager is a specific object designed specifically for security functions. The techniques used to restrict execution of a bytecode program are collectively called the Java sandbox.

Code signing is used to determine which person or company produced a class file. The security manager uses the producer of a class and the URL from which it was obtained to decide whether it can perform potentially hazardous operations such as creating, opening, or writing into a file.

Without type safety, Java programs could be susceptible to the buffer overflow attacks and could carry out the type-confusion attacks we discussed in Section 13.5.

Team-Fly



Team-Fly

Chapter 14: Concurrent and Distributed Programming

OVERVIEW

A concurrent program defines two or more sequences of actions that may be executed simultaneously. Concurrent programs may be executed in two general ways:

- *Multiprogramming.* A single physical processor may run several processes simultaneously by interleaving the steps of one process with steps of another. Each individual process will proceed sequentially, but actions of one process may occur between two adjacent steps of another.
- *Multiprocessing.* Two or more processors may share memory or be connected by a network, allowing processes on one processor to interact with processes running simultaneously on another.

Concurrency is important for a number of reasons. Concurrency allows different tasks to proceed at different speeds. For example, multiprogramming allows one program to do useful work while another is waiting for input. This makes more efficient use of a single processor. Concurrency also provides programming concepts that are important in user interfaces, such as window systems that display independent windows simultaneously and for networked systems that need to send and receive data to other computers at different times. Multiprocessing makes more raw processing power available to solve a computational problem and introduces additional issues such as unreliability in network communication and the possibility of one processor proceeding while another crashes. Interaction between sequential program segments, whether they are on the same processor or different processors, raises significant programming challenges.

Concurrent programming languages provide control and communication abstractions for writing concurrent programs. In this chapter, we look at some general issues in concurrent programming and three language examples: the actor model, Concurrent ML, and Java concurrency. Some constructs we consider are more appropriate for multiprogramming, some for multiprocessing, and some are useful for both.

Historically, many concurrent systems have been written in languages that do not support concurrency. For example, concurrent processes used to implement a network router can be written in a language like C, with system calls used in place of concurrency constructs. Concurrently executed sequential programs can communicate by reading and writing to a shared structure or by using abstractions provided by the operating system. Programs can send or receive data from a Unix pipe, for example, by function calls that are handled by the operating system. C programs can also create processes and terminate them by using system calls.





A buoyant, inquisitive person with a twinkle in his eye, Charles Anthony Richard Hoare began his career in computing in 1960 as a programmer for Elliott Brothers, a small scientific computer manufacturer. He became Professor of Computer Science at the Queen's University in Belfast in 1968 and moved to Oxford University in 1977. Professor Hoare retired from Oxford at the mandatory age in 1999 but continues to be active in computing at Microsoft Research Labs in Cambridge, U.K.

Midway through his career, Hoare received the ACM Turing Award in 1980, "For his fundamental contributions to the definition and design of programming languages." At the time he was recognized for his work on methods for reasoning about programs, clever algorithms such as Quicksort, data structuring techniques, and the study of monitors. In 1980, he was also beginning his study of concurrency through the development of Communicating Sequential Processes (CSP). Along with Milner's Calculus of Communicating Systems (CCS), CSP has been a highly influential system for specifying and analyzing certain types of concurrent systems. The standard method for proving properties of imperative programs by using loop in variants, defined in his 1969 journal article, is now commonly called Hoare Logic.

The picture on this page shows Tony Hoare and his wife Jill standing outside Buckingham Palace, holding the medal he received when he was knighted by Queen Elizabeth II.

There are several reasons why a concurrent programming language can be a better problem-solving tool for concurrent programming than a sequential language. One is that a concurrent programming language can provide abstractions and control structures that are designed specifically for concurrent programming. We look at several examples in this chapter. Another is that a programming language may provide "lighter-weight" processes. Operating systems generally give each process its own address space, with a separate run-time stack, program counter, and heap. Although this protects one process from another, there are significant costs associated with setting up a process and in switching the flow of control from one process to another. Programming language implementations may provide lightweight processes that run in the same operating system address space but are protected by programming language properties. With lightweight processes, there is less cost associated with concurrency, allowing a programmer to use concurrency more freely. Finally, there is the issue of portability across operating systems. Although it is possible to do concurrent programming with C and Unix system calls, for example, programs written for one version of Unix may not run correctly under other versions of Unix and may not be easy to port to other operating systems.

Team-Fly

Team-Fly

14.1 BASIC CONCEPTS IN CONCURRENCY

Before looking at some specific language designs, we discuss some of the basic issues in concurrent programming and some of the traditional operating system mechanisms used to allow concurrent processes to cooperate effectively.

A sequence of actions may be called a *process*, *thread*, or *task*. Although some authors use these terms interchangeably, they can have different connotations. The word process is often used to refer to an operating system process, which generally runs in its own address space. Threads, in specific languages like Concurrent ML and Java, may run under control of the language run-time system, sharing the same operating system address space. Some authors define thread to mean "lightweight process," which means a process that does not run in a separate operating system address space. The term process is used in the rest of Section 14.1, as many of the mechanisms we discuss are used in operating systems. In discussing Concurrent ML and Java, we use the term thread because this is the standard term for these languages and because threads defined in a Concurrent ML or Java program are lightweight processes that all run in the same operating system address space.

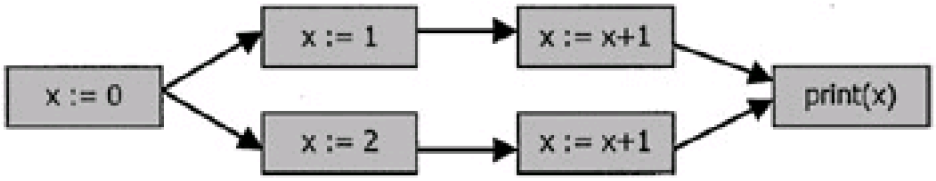
14.1.1 Execution Order and Nondeterminism

Concurrent programs may have many possible execution orders. An elementary and historical concurrent programming construct is the cobegin/coend form used in Concurrent Pascal. This language was developed by P. Brinch Hansen at Caltech in the 1970s. Here is an example Concurrent Pascal program that has several execution orders:

```
x:=0;
cobegin
   begin x := 1; x := x+1 end;
   begin x := 2; x := x+1 end;
coend;
print(x);
```

This program executes sequentially, as usual in Pascal, except that, within the cobegin/coend statement, all of the independent blocks are executed concurrently. Concurrent blocks may interact by reading or assigning to global variables. The cobegin/coend statement terminates when all of the concurrent blocks inside it terminate. After the cobegin/coend statement terminates, the program continues sequentially with the statement following coend.

Here is a diagram showing how our sample Concurrent Pascal program can be executed on a single processor. Each assignment statement is executed *atomically*, meaning that even though an assignment statement might be carried out with a sequence of more elementary machine-language steps, once the processor starts executing an assignment statement, the processor continues and completes the assignment before allowing another process to execute. In other words, only one assignment statement runs at a time. However, the order between assignments may be different on different executions of the program. Each arrow in this figure illustrates a constraint on execution order: The statement at the left end of the arrow must finish before the statement on the right can start. However, there is no necessary ordering between statements that are not connected by a sequence of arrows. In particular, the assignment x:=1 could be executed before or after the assignment x:=2. Here are some possible execution orders for this program:



---

x:=0; x:=1; x:=2; x:=x+1; x:=x+1; print(x) // Output 4
x:=0; x:=2; x:=1; x:=x+1; x:=x+1; print(x) // Output 3
x:=0; x:=1; x:=x+1; x:=2; x:=x+1; print(x) // Output 3
x:=0; x:=2; x:=x+1; x:=1; x:=x+1; print(x) // Output 2

This program illustrates one of the main difficulties in designing and testing concurrent systems: nondeterminism. A program is *deterministic* if, for each sequence of program inputs, there one sequence of program actions and resulting outputs. In contrast, a program is *nondeterministic* if there is more than one possible sequence of actions corresponding to a single input sequence. Our sample Concurrent Pascal program is nondeterministic because the order of execution of the statements inside the cobegin/coend construct is not determined. There are several possible execution orders and, if the program is run several times on the same computer, it is possible for the program to produce different outputs on different runs.

Nondeterminism makes it difficult to design and debug programs. In a complex software system, there may be many possible execution orders. It is difficult for program designers to think carefully about millions of possible execution orders and identify errors that may occur in only a small number of them. It also may be impractical to test program behavior under all possible execution orders. Even if there is some way to cause the system to try all possible orders (a difficult task for distributed systems involving many computers connected by a network), there could be so many orders that it would take years to try them all.

One example that illustrates the complexity of nondeterminism is the design of cache coherency protocols for multiprocessors. In a modern shared-memory multiprocessor, several processors share common memory. Because access to shared memory takes time and can be a bottleneck when one processor needs to wait for another process, each processor maintains a local cache. Each cache is a small amount of memory that duplicates values stored in main memory. The job of the cache coherence protocol is to maintain the processor caches and to guarantee some form of memory consistency, e.g., the value returned by every load/store sequence must be consistent with what could have happened if there were no caches and all load/store instructions operated directly on shared memory in some order. Cache coherence protocols are notoriously difficult to design and many careful designs contain flaws that surface only under certain conditions. Because of their subtlety, many cache coherence protocols are analyzed by automated tools that are designed to detect errors in nondeterministic systems. In Subsection 14.4.3 we look at the Java virtual machine memory model, which involves some of the same issues as cache coherency protocols for multiprocessors.

14.1.2 Communication, Coordination, and Atomicity

Every programming language for explicit concurrency provides some mechanism to initiate and terminate independent sequential processes. The programming languages we consider in this chapter also contain mechanisms for some or all of the following general purposes:

- *communication* between processes, achieved by mechanisms such as buffered or synchronous communication channels, broadcast, or shared variables or objects,
- *coordination* between processes, which may explicitly or implicitly cause one process to wait for another process before continuing,

- *atomicity*, which affects both interaction between processes and the handling of error conditions.

When we compare concurrent programming languages, we always look at the way processes are defined, how they communicate and coordinate activities, and how programmers may define atomic actions within the language (if the language provides explicit support for atomicity).

The most elementary form of inter process communication, as illustrated in the Concurrent Pascal example in Subsection 14.1.1, is through shared variables. Processes may also communicate through shared data structures or files. Another form of inter process communication is called *message passing.* Here are some of the main distinctions among various forms of message-passing mechanisms:

- *Buffering.* If communication is *buffered*, then every data item that is sent remains available until it is received. In *unbuffered* communication, a data item sent before the receiver is ready to accept that it may be lost.
- *Synchronicity.* In *synchronous* communication, the sender cannot transmit a data item unless the receiver is ready to receive it. With *asynchronous* communication, the sending process may transmit a data item and continue executing even if the receiver is not ready to receive the data.
- *Message Order*. A communication mechanism may preserve transmission order or it may not. If a mechanism preserves transmission order, then a sequence of messages will be received in the order that they are sent.

Coordination mechanisms allow one process to wait for another or notify a waiting process that it may proceed. Concurrent Pascal cobegin/coend provides a rudimentary form of process coordination as all processes started at the same cobegin must finish before the statement following coend may proceed. Locking and semaphores, discussed in Subsections 14.1.3 and 14.1.4, provide more sophisticated forms of concurrency control.

An action is *atomic* if every execution will either complete successfully or terminate in a state that is indistinguishable from the state in which the action was initiated. A non atomic action may involve intermediate states that are observable by other processes. A nonatomic action may also halt in error before the entire action is complete, leaving some trace of its partial progress. Generally speaking, any concurrent programming language must provide some atomic actions, because, without some guarantee of atomicity, it is extremely difficult to predict the behavior of any program.

14.1.3 Mutual Exclusion and Locking

There are many programming situations in which two or more processes share some data structure, and some coordination is needed to maintain consistency of the data structure. Here is a simple example that illustrates some of the issues.

Example 14.1

Suppose we have a system that allows people to sign-up for mailing lists. This system may involve a number of lists of people, and some of these lists may be accessed by more than one process. Here is a procedure to add a person to a list:

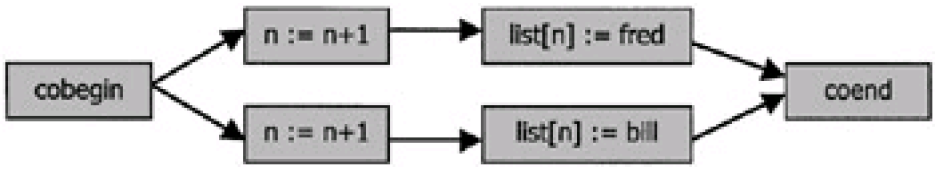
```
procedure sign-up(person)
   begin
     n:=n + 1;
     list[n] := person;
   end;
```

This procedure uses an array to store the names in the list and an integer variable to indicate the position of the last person added to the list. There may be other procedures to read names from the list or remove them.

The sign-up procedure works correctly if only one process adds to the list at a time, but not if two processes add to the list concurrently. For example, consider the following program section:

```
cobegin
   sign-up(fred);
   sign-up(bill);
end;
```

As discussed in Subsection 14.1.1, there are several possible execution orders for the assignments to number and list. The ordering between assignments is shown in this illustration:



One possible execution order increments n twice, then writes names fred and bill into the same location. This is not correct: Instead of adding both names to the list, we end up with an empty location, and only one of the two names in the list. The state resulting from the sequence of assignments

```
n := n+1; n := n+1; list[n] := fred; list[n] := bill;
```

is called an *inconsistent state* because it is not consistent with the intended behavior of the operations on lists. An invariant for list and integer variable n is that for every integer i up to n there are data stored in list[i]. This invariant is preserved by sign-up, if it is called sequentially. However, if it is called concurrently, the data structure invariant may be destroyed.

Mutual Exclusion

Example 14.1 shows that, for certain operations, it is important to restrict concurrency so that only one process proceeds at a time. This leads to the notion of *critical section*. A critical section is a section of a program that accesses shared resources. Because a process in a critical section may disrupt other processes, it may be important to allow only one process into a critical section at a time. Mechanisms for preventing conflicting actions in independent processes are called *mutual-exclusion* mechanisms.

Mutual-exclusion mechanisms may be designed to satisfy some or all of the following criteria:

- *Mutual exclusion*: Only one process at a time may be in its critical section.

- *Progress:* If no processes are in their critical section and some process wants to enter a critical section, it becomes possible for one waiting process to enter its critical section.
- *Bounded waiting:* If one process is waiting, there must be a bound on the number of times that other processes are allowed to enter the critical section before this process is allowed to enter. In other words, no waiting process should have to wait indefinitely.

It is also desirable to ensure that, if one process halts in a critical section, other processes will eventually be able to enter the critical section.

Locks and Busy Waiting

A general approach to mutual exclusion is shown in the following pseudocode, which uses the sign-up procedure from Example 14.1. The main idea is that each process executes some kind of *wait* action before calling sign-up and executes some kind of *signal* action afterward. The wait action should check to see if any other process is calling sign-up or executing any other procedure that might interfere with sign-up. If so, the action should make the process wait, only allowing the call to sign-up to occur when this is acceptable. The signal action allows a waiting process to proceed.

```
<initialze concurrency control>
cobegin
   begin
      <wait>      // wait for access
      sign-up(fred);            // critical section
      <signal>    // allow waiting process to proceed
   end;
   begin
      <wait>      // wait for access
      sign-up(bill);            // critical section
      <signal>    // allow waiting process to proceed
   end;
end;
```

Although the ideas behind *wait* and *signal* actions may seem straightforward, implementing these actions is tricky. In fact, in the early days of concurrent programming, several incorrect implementations were proposed before a good solution was finally discovered. We can understand some of the subtlety by examining a plausible but imperfect implementation.

Example 14.2

Let us try to implement wait and signal by using an ordinary integer variable as a "lock." Each process will test the variable before it enters its critical section. If another process is already in its critical section, then the lock will indicate that the process must wait. If the value of the lock indicates that the process may enter, then the process sets the lock to keep other processes out. This approach is shown in the following code:

```
lock := 0;
cobegin
   begin
```

```
        while lock=1 do end; // loop doing nothing until lock is 0

        lock:=1; // set lock to enter critical section
        sign-up(fred); // critical section
        lock:=0; // release lock
    end;
    begin
        while lock=1 do end; // loop doing nothing until lock is 0
        lock:=1; // set lock to enter critical section
        sign-up(bill); // critical section
        lock := 0; // release lock
    end;
end;
```

Each process (each of the two blocks) tests the lock before entering the critical section (i.e., before calling sign-up). If the lock is 1, indicating that another process is using sign-up, then the process loops until the lock is 0. When the lock is 0, the process enters the critical section, setting lock:=1 on entry and lock:=0 on exit. Using a loop to wait for some condition is called *busy waiting*.

The problem with using a shared variable for mutual exclusion is that the operation that reads the value of the variable is different from the operation that sets the variable. With the method shown in this code, it is possible for one process to test the variable and see that the lock is open, but before the process can lock other processes out, another process can also see that the lock is open and enter its critical section. This allows two processes to call sign-up at the same time.

A fundamental idea is concurrency control is atomic test-and-set. An atomic test-and-set, sometimes called TSL for Test and Set Lock, copies the value of the lock variable and sets it to a nonzero (locked) value, all in one atomic step. Although the value of the lock variable is being tested, no other process can enter its critical section. Most modern processors provide some form of test-and-set in their instruction set.

Deadlock

Deadlock occurs if a process can never continue because the state of another process. Deadlock can easily occur if there are two processes and two locks. Suppose that Process 1 first sets Lock 1 and then wait for Lock 2. If Process 2 first sets Lock 2 and then waits for Lock 1, then both processes are waiting for the lock held by the other process. In this situation, neither process can proceed and deadlock has occurred.

One technique that prevents deadlock is called *two-phase locking.* In two-phase locking, a process is viewed as a sequence of independent tasks. For each task, the process must first acquire all locks that are needed (or could be needed) to perform the task. Before proceeding from one task to the next, a process must release all locks. Thus there are two phases in the use of locks, a locking phase in which all locks are acquired and a release phase in which all locks are released. To avoid deadlock in the locking phase, all processes agree on an ordering of the locks and acquire locks in that order. You may find more information about locking policies and deadlock in books on operating systems and database transactions.

14.1.4 Semaphores

Semaphores, first proposed by Edsger W. Dijkstra in 1968, provide a way for an operating system to guarantee mutual exclusion without busy waiting. There are several different detailed implementations of semaphores. All use a counter or Boolean flag and require the operating system to execute the entire wait or signal procedure atomically. Atomicity prevents individual statements of one wait procedure from being interleaved with individual statements of another wait on the same semaphore.

A standard *semaphore* is represented by an integer variable, an integer maximum, and a queue of waiting processes. Initially, the integer variable is set to the maximum. The maximum indicates the number of processes that may enter a

critical section at the same time; in many situations the maximum is one. If a program contains several shared resources, the program may use a separate semaphore for each resource.

When a process waits on a semaphore, the wait operation checks the integer value of the semaphore. If the value is greater than 0, this indicates that a process may proceed. The value is decremented before proceeding to limit the number of processes that are allowed to proceed. (If the maximum is 1, then the semaphore is decremented to 0 and no other process can proceed.)

If a wait is executed on a semaphore whose integer value is 0, then the process is suspended and placed on a queue of waiting processes. Suspending a process is an operating system operation that keeps the process from continuing until the process is resumed. Here is a pseudocode showing how semaphore wait may be implemented:

```
procedure wait (s : Semaphore)
begin                       // This procedure must be executed atomically
   if s.value > 0 then
        s.value:= s.value - 1; // Enter section and decrement counter
   else
        suspend_on (s.queue); // Wait for other processes to finish
end;
```

When a process leaves a critical section, it must allow other processes waiting on the associated semaphore to proceed. The signal operation checks the semaphore queue to see if any process is suspended. If the queue is not empty, one of the suspended waiting processes is allowed to run. If no process is waiting, then the integer value of the semaphore is incremented. Here is a pseudocode showing how semaphore signal might be implemented:

```
procedure signal (s : Semaphore)
begin                         // This procedure must be executed atomically
   if length (s.queue) = 0 then
        s.value := s.value + 1; // Increase count allowing other processes to enter
   else
        allow_one_process (s.queue); // Wake up one suspended process
end
```

If allow_one_process wakes up the first process on the queue, then each waiting process will have to wait only for the processes ahead of it in the queue. This achieves bounded waiting, as defined in Subsection 14.1.3.

Originally, Dijkstra, used P() for Wait() because P is the first letter of the Dutch word *passeren*, which means to pass, and V() for Signal() because V is the first letter of the Dutch word *vrijg even*, which means to release.

14.1.5 Monitors

Semaphores provide mutual exclusion, but only if used properly. If a process calls wait before entering a critical section, but fails to call signal afterwards, this error may cause the program to deadlock. This kind of program error can be hard to find. Monitors, developed in the early 1970s, place the responsibility for synchronization on the operations

that access data. Monitors are similar to abstract data types, with all synchronization placed in the operations of the data type. This makes it easier to write correct client code.

A *monitor* consists of one or more procedures, an initialization sequence, and local data. The local data are accessible only by the monitor procedures, which are responsible for all synchronization associated with concurrent access to the data. In traditional terminology, a process *enters* the monitor by invoking one of its procedures. The synchronization code is generally designed so that only one process may be in the monitor at a time; any other process that calls a monitor procedure is suspended and waits for the monitor to become available.

In modern terminology, a monitor might be called a *synchronized object.* In an important early paper (Monitors: An Operating System Structuring Concept, Comm. ACM 17(10) 1974, pp. 549-557), C.A.R. Hoare showed that monitors can be implemented by use of semaphores and, conversely, semaphores can be implemented by use of monitors. In this sense, the two constructs are equally powerful synchronization constructs. However, monitors have proven to be a more useful program-structuring concept. We look at the main programming issues associated with monitors in Section 14.4 in our discussion of Java synchronized objects.

Team-Fly



Previous Next

Team-Fly

14.2 THE ACTOR MODEL

Actors are a general approach to concurrent computation. Each actor is a form of reactive object, executing some computation in response to a message and sending out a reply when the computation is done. Actors do not have any shared state, but use buffered asynchronous message passing for all communication. Although pure actor systems have not been widely successful, actors are an appealing metaphor and an interesting point on the spectrum of concurrent language design alternatives.

Actors originated through Carl Hewitt's work on the artificial intelligence system Planner in the early 1970s. The concept evolved as a result of approximately 20 years of subsequent effort by Hewitt and many others, influencing a range of concurrent object-oriented systems. There is no one actor system that embodies all of the concepts that have been developed in the literature. As a result, our discussion of actor systems is more a discussion of a point of view, with possible alternatives and embellishments, than an evaluation of a specific, concrete system.

An *actor* is an object that carries out its actions in response to communications it receives. There are three basic actions that an actor may perform:

- It may send communication to itself or other actors,
- It may create actors,
- It may specify a replacement behavior, which is essentially another actor that takes the place of the actor that creates it for the purpose of responding to later communication.

Actor computation is reactive, which means that computation is performed only in response to communication. An actor program does not explicitly create new processes and control their execution. Instead, an actor program creates some number of actors and sends them messages. All of these actors can react to messages concurrently, but there is no explicit concept of thread.

An actor is dormant until it receives communication. When an actor receives a message, the script of the actor may specify subsequent communication and a set of new actors to create. After executing its script, the actor returns to its dormant state. The replacement behavior specifies how the actor will behave when it receives another message. For example, if an actor representing the number 3 receives a communication *increment by* 1, its replacement behavior will be to represent the number 4. There is no assignment to local variables in the actor model; the only side effect of an actor is to specify a replacement behavior, which is an atomic state change that takes effect only after the other activities are complete.

In any computation, each actor receives a linearly ordered sequence of messages. However, messages are not guaranteed to arrive in the order in which they are sent. If one actor, A, sends two communications to another, B, then B may not receive them in the same order that A sent them. This is consistent with the idea that actors may be physically located in different places, connected by a communication network that might route different messages along different paths.

An important part of the actor model is the *mail system*, which routes and buffers messages between actors. Every message must be sent to a *mail address*; one actor may communicate with another if it knows its mail address. When an actor A specifies a replacement behavior, the replacement behavior will be the script for a new actor that receives all messages addressed to the mail address of A.

A message from one actor to another is called a *task.* A task has three parts:

- A unique tag, distinguishing it from other tasks in the system,
- A target, which is the mail address of the intended receiver,

- A communication, which is the data contained in the message.

Figure 14.1 shows a finite set represented by an actor. The actor in the upper left of the figure represents the set with elements 1, 4, and 7. This actor receives a task Insert 2. In response, the actor becomes the actor on the right that has elements 1, 2, 4, and 7. This actor receives the task Get min. As a result, the actor sends the message 1, which is the minimum element in the set, and becomes an actor that represents the set with elements 2, 4, and 7.

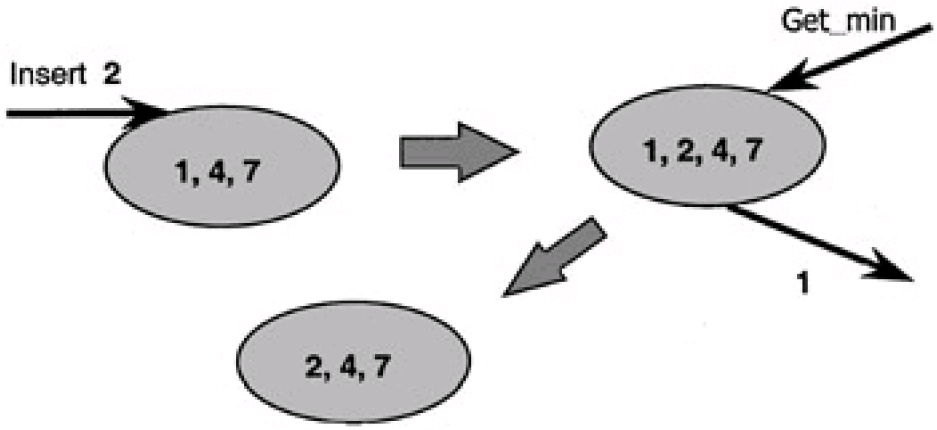


Figure 14.1: Messages and state change for a finite-set actor.

The configuration of an actor system consists of a finite set of actors and a finite set of pending or undelivered tasks. Because each task has a single, specified destination, we may visualize the system as a set of actors and task queues, with one queue per actor and each task on exactly one queue.

Although we will not look at the syntactic structure of actor programs in any detail, here is a short example to give some feel for the language. The code subsequent is something called a behavior definition, which defines a behavior and gives it a name but does not create an actor. A program that uses this behavior definition creates one or more stack_node actors and sends them tasks. As this example illustrates, a behavior may be defined as a function of parameters, which are called *acquaintances:*

```
a stack_node with acquaintances content and link
    if operation requested is push(new_content) then
        let P=new stack_node with current acquaintances
        become stack_node with acquaintances new_content and P
    if operation requested is a pop and content != nil then
        become forwarder to link
        send content to customer
```

A stack-node actor with the behavior defined by this code has two data fields (determined by acquaintances), content and link. This actor sits and waits for a task. The two tasks that the actor will process are push and pop. If the operation requested is a push, then the push task will include a content value called new_content. The actor creates a copy P of itself, as indicated by the expression new stack_node with current acquaintances and becomes a stack node with data new_content and link to the copy P of the actor's previous state. The reason why the actor "becomes" the new actor, instead of keeping its original state and creating a new actor linked to it, is that future tasks sent to this actor should reach the new stack with new_content added, not the old stack without these data.

The pop operation uses an actor specific concept called a forwarder. A forwarder actor does nothing itself, except forward all tasks to another actor. The reason for having forwarders is illustrated by the pop operation, in which the actor that receives the pop task would like to disappear and let another actor take its place. Instead of "disappearing,"

the actor becomes a forwarded to the actor it is linked to.

We can see how forwarders work by looking at some pictures. Suppose that three actors represent a stack, with data 3, 4, and 5, as shown here:



If the actor on the left, representing the top of the stack, receives a pop task, then the actor sends content 3 to the actor that sent the pop task and becomes a forwarder. This produces a stack of three actors, as shown here.

The forwarder has the same mail address as that of the actor with content 3, so any task sent to the top of the stack will now be forwarded to the actor with content 4. In effect, the forwarder node is invisible to any user of the stack. In fact, an actor implementation may garbage-collect forwarders and handle forwarding in the mail system.



Two interesting general aspects of actors are the concepts of replacement behavior and the use of buffered asynchronous communication. An actor changes state only by becoming another actor after it completes processing input. This is a very clean idea that eliminates state change within an actor while computing a response to a task.

The motivation for using asynchronous communication is that it is easily implemented on a wide area network. When actors are on different computers connected by a slow network, it is easier to implement asynchronous communication by means of standard IP-based protocols than it is to devise some form a synchronous communication. With asynchronous communication, it is important either to use some form of buffering or to use some kind of resend protocol to handle lost messages. Although buffers may overflow in practice, buffered communication is a convenient abstraction for many programming situations.

The actor design does not guarantee the order of message delivery. This is inconvenient in many situations. For example, consider an actor traversing a graph, using another stack actor to keep track of graph nodes to revisit. If there is no guarantee on the order of messages that push graph nodes onto the stack, the algorithm will not necessarily traverse the graph in depth-first order. The design rationale for basing actors on communication without guaranteed order is that order can be imposed by adding sequence numbers to messages.

There are also some pragmatic issues associated with programming when message delivery may take arbitrarily long. Returning to the example of searching a graph, using a stack to keep track of nodes that must be revisited, it seems difficult to tell whether the graph search is finished. The usual algorithm continues to visit nodes of the graph until the stack of remaining nodes is empty. However, if an actor representing a stack sends a message saying that the stack is empty, this may mean only that all push messages are still in the mail system somewhere and have not yet been delivered to the stack.

Although the simplicity of the actor model is appealing, these problems with message order, message delivery, and coordination between sequences of concurrent actions also help us appreciate the programming value of more complex concurrent languages.

Team-Fly

14.4 JAVA CONCURRENCY

Java concurrency is an integral part of the Java language and run-time system. We create a Java thread object by defining a class with a run() method, either by extending the Thread class or implementing the Runnable interface directly. When a thread object is activated, the run method runs as a separate thread inside the Java virtual machine (JVM). Java communication and synchronization are provided by separate mechanisms.

Java threads may communicate by calling methods of shared objects. For example, a producer thread may communicate with a consumer thread by using a queue object. When the producer thread wants to send its output to the consumer, the producer calls an enqueue method that places its output in the queue. When the consumer is ready for another item, the consumer thread may call a dequeue method, drawing another item off the queue.

Communication through a shared object does not synchronize the threads involved - each thread may attempt to execute a method independently. To avoid problems associated with shared resources (see Subsection 14.1.3), Java provides various synchronizations primitives. The Java language provides a semaphore primitive (maintaining a queue of waiting processes) and supports monitors directly in the form of synchronized objects, which are objects that allow only one thread to invokea method at a time.

A second form of Java concurrency, which we discuss briefly in Subsection 14.4.4, arises when Java programs running in one VM communicate with programs running in another. The Java remote method invocation (RMI) mechanism allows an object running in one JVM to invoke methods of an object running in another JVM. Java RMI is used for distributed Java programming, with distributed programming infrastructures such as JINI and JXTA relying on RMI to varying degrees.

The Java thread design is a successful language feature. Java makes concurrent programs portable across a variety of computing platforms, as concurrency is a standardized part of the language and run-time system. Java threads are used in the execution of many Java programs, as the Java garbage collector runs as a separate thread in the JVM.

Although Java has been adopted widely, there remain some difficulties in programming with Java threads. In part, these are just general problems associated with the difference between sequential and concurrent programming. Classes designed for use in sequential programs generally do not use synchronization and may not work well in multithreaded programs. Conversely, code with synchronization maybe unnecessarily inefficient in serial programs - synchronization imposes a locking overhead that is wasteful in a single-threaded program.

14.4.1 Threads, Communication, and Synchronization

We can create a Java thread by extending the class Thread, which implements the Runnable interface. When a class extends Thread, it inherits methods to start the thread and throw an exception to the thread:

start: A method that spawns a new thread by causing the virtual machine to execute the run method in parallel with the thread that creates the object.

interrupt: A method that suspends and throws an exception to the thread.

Here is an example subclass of Thread:

```
class PrintMany extends Thread {
    private String msg;
    public PrintMany (String m){msg=m; }
```

```
        public void run(){
            try{ for (;;) { System.out.print(msg +  " ");
                    sleep(10);

                }
            } catch (InterruptedException e){
                return;
            }
        }
    }
```

Suppose a program creates an object pm = new PrintMany("Hi"). When pm.start is called, a thread is created that prints the string "Hi" over and over. This continues until the thread is interrupted by a call to pm.interrupt.

Originally, Java threads had additional methods that affect their execution, including the following two methods:

Suspend: A method that stops execution of the thread until it is later resumed.

stop: A method that forcibly causes the thread to halt.

However, these have now been deprecated because programmers have found that these methods often cause more problems than they solve. The problem with Thread.suspend is that it can contribute to deadlock. If a thread holding a lock on a critical section is suspended, then no other thread can enter the critical section until the target thread is resumed. The problem with Thread.stop is related but almost the opposite. When a thread is stopped by Thread.stop, this causes all the locks it holds to be released. If any of the objects protected by these locks is in an inconsistent state, then other threads may now have access to damaged objects. This can cause the program to behave in unexpected and incorrect ways.

Threads running inside the same virtual machine are scheduled according to their priority. Threads with higher priority are generally executed in preference to threads with lower priority, but there is no guarantee that the highest priority thread will always be run. A Java thread inherits the priority of the thread that creates it and may be changed with the setPriority method.

Communication between threads

In a sense, Java does not make any special provisions for communication: Java threads communicate by assigning to shared variables or calling the methods of shared objects. By themselves, shared variables and shared objects can be problematic, as discussed in Subsection 14.1.3. Java does provide specific synchronization mechanisms that make access to shared objects safe.

Synchronization Primitives

Java synchronization is based on three basic mechanisms:

- Locks: Every object has a lock, used for mutual exclusion.
- Wait sets: Every object has a wait set, providing a form of semaphore.
- Thread termination: A process can pause until another thread terminates.

Locks are tested and set by synchronized blocks and methods, wait sets are used by methods wait, notify, and notifyAll that are defined on all objects, and thread termination is used by the join method on thread objects.

Locks are used by the synchronized statement, which has the form

---

```
synchronized( object ){
   statements
}
```

When executed, this statement computes a reference to the indicated object and locks its associated lock before executing the body of the statement. If the lock is already locked by another thread, the thread waits for the lock to become available. After executing the body of the statement, the thread releases the lock. Inside a method, it is common to use this as the lock object. However, it is possible to synchronize on the lock of any object.

Java locks are a *reentrant*, meaning that a thread may acquire a lock multiple times. This may occur if a thread executes one synchronized statement inside another or with synchronized methods that call each other. For example, the synchronized method f in the following class calls the synchronized method g:

```
public class LockTwice {
   public synchronized void f(){ g(); }
   public synchronized void g(){ System.out.println("Method g() called");}
}
```

When f is called, the calling thread acquires the lock for the LockTwice object. When f calls g, the thread attempts to acquire the same lock again. Because Java locks are reentrant, the calling thread may acquire the LockTwice object's lock again and both methods terminate and return. In systems that do not support reentrant locks, a second call to a synchronized method would deadlock. The semaphore implementation discussed in Subsection 14.1.4 is *not* reentrant.

The methods wait, notify, and notifyAll of class Object use the wait set of an object to provide concurrency control. A thread may suspend itself by using wait. A suspended thread does not execute any statements until another thread awakens it by using notify. The wait/notify combination can be used when threads have a producer-consumer relationship, for example. Because wait notifies the virtual machine scheduler that a process should be suspended, wait and notify are more efficient than busy waiting, which would involve a loop that repeatedly tests to see if a thread should proceed.

The notifyAll method is similar to notify, except that every thread in the wait set for the object is removed and reenabled for thread scheduling. Most Java books recommend calling the notify or notifyAll method of an object only after the calling thread has locked the object's lock.

The third form of synchronization, pausing until another thread terminates, is provided by the join method. Here is a short code example illustrating its use:

```
class Compute_thread extends Thread {
        private int result;
        public void run(){ result = f(...); }
        public int getResult(){ return result;}
}
```

```
...
Compute_thread t = new Compute_thread;
t.start()        // start compute thread
...
t.join(); x = t.getResult(); // wait and get result
...
```

In this code, a Compute thread object does some kind of computation to determine some result. The program creates a Compute thread object and starts the thread running so that it will compute a useful result. Later, when the result is needed, the program calls t.join() to wait for the Compute_thread to finish before requesting the result.

14.4.2 Synchronized Methods

Java threads typically communicate by calling methods on shared objects, with synchronization used to provide mutual exclusion. For example, if a transaction manager allows only one thread to commit at a time, the class defining a transaction manager might look like this:

```
class TransactionManager{ ...
    public synchronized void commitTransaction(. . . ){... }
...
}
```

When a method is declared synchronized, the effect is the same as placing the body of the method inside a synchronized statement, using the lock on the object that contains the method. Therefore, even if several threads have access to a Transaction-Manager object m, only one thread can execute m.commitTransaction(…) at a time. If one thread invokes m.commitTransaction(. . .) while another thread is in the process of doing so, the second thread will block and wait until the lock on object m is free.

A class may have some synchronized methods and some methods that are not synchronized. If a method is not declared synchronized, then more than one thread may call the unsynchronized method simultaneously. If two threads call an unsynchronized method at the same time, then each thread will have an activation record for a method call on its own run-time stack. However, the two activation records will point to the same (shared) object fields on the run-time heap, in effect allowing the threads to interact through shared variables. In general, a programmer may wish to leave as many methods unsynchronized as possible, allowing for maximal concurrency. However, care must be taken to synchronize enough methods to avoid placing an object in an inconsistent state.

Here is a simple example of a class with two synchronized methods and an unsynchronized method (similar LinkedCell classes appear in various books and articles on Java threads and on web pages with Java program examples):

```
class LinkedCell{ // Lisp-style cons cell containing
    protected double value; // value and link to next cell
    protected LinkedCell next;
    public LinkedCell (double v, LinkedCell t){
        value = v; next = t;
```

```
    }
    public synchronized double getValue(){
        return value;
    }
    public synchronized void setValue(double v){
        value = v; // assignment not atomic
    }
    public LinkedCell next(){ // no synch needed
        return next;
    }
}
```

This class implements a standard linked cell, as might be used to implement stacks, queues, or singly linked lists, with one data field and a next field that can be used to link one linked cell to the next. Note that the data has type double. This is important(for illustrating the need for synchronization) because assignment of doubles is not atomic. Because doubles are represented by 8 bytes and 32-bit processors normally move 4 bytes at a time, it takes two separate instructions to set or read a double.

If setValue were not synchronized and two threads called value setValue simultaneouosly, this could put the LinkedCell object in an *inconsistent state*. Specifically, consider two calls, c.setValue(x) and c. setValue(y), where x and y are doubles. One possible interleaving of steps in the two assignments to c.value is

> Set first half of c.value to first half of x
>
> Set first half of c.value to first half of y
>
> Set second half of c.value to second half of y
>
> Set first half of c.value to second half of x

After execution in this order, c.value does not contain either x or y, but rather contains some strange number it determined by combining some bits from the representation of x with some bits from the representation of y. Because the intended effect of setValue(z) is to place the value z in the cell, interleaved execution is inconsistent with the intended semantics of the method. It is necessary to synchronize calls to setValue.

The reason for synchronizing getValue is similar. If one thread calls getValue while another calls setValue, then the double returned by the call to getValue could contain half of the bits that were in the cell before setValue took effect and half of the bits after. To prevent getValue and setValue from running simultaneously, we synchronize both. More generally, although it is possible for two threads to read a data structure simultaneously, errors can occur if a read occurs during a write. The simplest way to prevent these errors is to lock out reads while a write is occurring. A more sophisticated approach, which may be worth the effort in some applications, is to use separate read and write locks, with one read allowing any number of other reads to proceed, but preventing any write.

The next method is not synchronized because, when two threads call next simultaneously, they both get the same correct answer. There is no method to change the next link of a cell, and pointer assignment is atomic in Java anyway.

Synchronized Methods and Inheritance

The synchronized designation is not part of the method signature, and therefore a subclass may override a synchronized method with an unsynchronized method. This programmer flexibility can be useful. However, it is also possible to destroy completely the locking policy of a class through inheritance and subtyping. Therefore, programmers must be very careful when extending classes in multithreaded programs.

14.4.3 Virtual Machine and Memory Model

General properties of the JVM are discussed in Section 13.4. In the Java architecture, Java source code is compiled to

an intermediate form called the Java bytecode, which is then interpreted by the JVM. When a Java program creates a new thread, this thread runs inside the JVM. As mentioned in Section 13.4, each Java thread has its own stack, but all threads share same heap. Therefore, when two threads call a method of the same object, the activation records associated with the method calls are on separate stacks, but the fields of the shared object reside in a single heap accessible to both threads. Although the basic Java virtual machine is designed to run on a single processor, multiprocessor implementations have also been developed and used.

In this subsection, we discuss two aspects of the JVM that are relevant to concurrency: concurrent garbage collection and the memory model used to manage reads and writes to shared locations.

Concurrent Garbage Collection

The JVM uses concurrent garbage collection. One advantage is performance. In sequential garbage collection, the program must stop long enough for the garbage collector to identify and reclaim unused memory. With concurrent garbage collection, the garbage collector is designed to run as other threads execute, keeping accessible memory in a consistent state so that other threads can do useful work. When properly tuned, this approach may keep an interactive program from exhibiting long pauses. Instead of pausing long enough to do a complete collection, the scheduler can switch back and forth between the garbage collector and a thread interacting with a user in a way that keeps the user from noticing any long garbage-collection pauses.

The actual design and implementation of concurrent garbage collectors is complicated and involves concepts that are not covered in this book. If you want to gain some appreciation for the issues involved, imagine running a mark-and-sweep algorithm in parallel with a program that modifies memory. As the garbage collector searches for reachable locations, starting from pointers stored on the stack, the program may modify the stack and modify pointers in reachable data. This makes it difficult to understand whether a cell that has not been marked as reachable is truly garbage. In fact, most garbage-collection algorithms, once started, must either finish the task or abandon all their work. Incremental and concurrent garbage collectors generally involve a concept called a *barrier*, which prevents threads from writing to areas of memory that are in use by the garbage collector.

Java Memory Model

The Java memory model is a specification of how shared memory can be cached and accessed by multiple threads. The specification is called a memory *model* because there are many possible implementations of the JVM. The memory model is a set of properties that every implementation must have, including both single-processor and multiprocessor implementations. The original memory model, presented in the *Java Language Specification*, has proven difficult to understand and unsatisfactory for implementers in various ways. Although it is likely that the memory model will eventually be revised to correct some of the known problems, we discuss the original design briefly in order to understand some of the subtle basic issues involved in concurrent memory access.

In the JVM, all threads share the same heap, which will be referred to as *shared memory* for the rest of this section. The virtual machine specification allows each thread to have a local memory, which will be referred to as the *thread cache*. The reason for each thread cache is to avoid coordinated reads and writes to shared memory when possible. If each thread can proceed independently, reading and writing to its own cache, this reduces the need for synchronization between threads.

Because there are several kinds of reads and writes in the memory model, it is useful to establish some precise terminology:

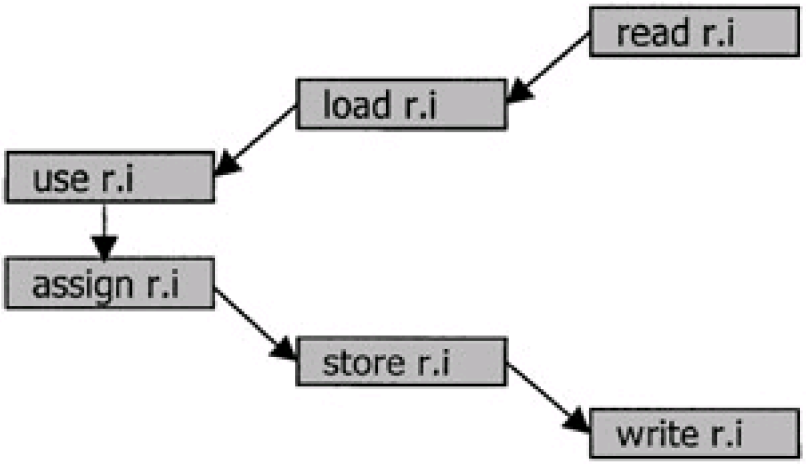
- *use* and *assign* are actions that read and write the thread cache,
- *load* and *store* are actions that transfer values between shared memory and cache,
- *read* and *write* are actions that read and write shared memory.

Bytecode programs execute instructions like getfield, which loads the value stored in a field of an object onto the operand stack (see Subsection 13.4.4) and putfield, which stores a value in the field of an object. In our discussion of the Java memory model, we do not need to distinguish among a getfield instruction, a getstatic instruction (which loads a value from a static field of a class), and array load instructions; werefer to all of these as *use* actions. Similarly, *assign* may be putfield, putstatic, or an array store instruction.

For example, consider a thread executing a compiled bytecode from this source code:

```
r.i = r.i+1;
```

In general, six memory actions will occur as a result of executing this statement. These actions may be interleaved with other actions by the same thread, as long as the order of actions conforms to the memory model. Here is an illustration showing only the six actions directly related to this assignment, in their expected order:



Before the thread computes the sum r.i+1, the value of field i of object r must be read from shared memory, loaded into the local cache of the thread, and used (placed on the operand stack of the thread). Similar steps occur in reverse order to place the value of r.i in shared memory. In the drawing, the left column shows actions that read and write the thread cache, the center column shows actions that transfer values between the thread cache and shared memory, and the right column shows actions that read and write shared memory. Although the effect of *store* r.i, for example, is to place a value in shared memory, the memory model includes separate *store* and *write* actions because the steps that transfer data may not take place atomically. The *store* action indicates the point when the value is read from the thread cache, and the *write* action indicates the point at which the value is actually placed shared memory. If another *assign* occurs after the *store* but before the *write*, it will change the value in the thread cache, but not the value that is placed in shared memory. An important goal in the design of the Java memory model is called *coherence*: For each variable, the uses and assigns to the variable should appear as if they acted directly on shared memory in an order that matches the order within each thread. In other words, threads may proceed independently. In the absence of synchronization, an assignment to a variable in one thread may occur before or after an assignment or use in another thread. However, the cache must be essentially invisible; the program should behave as if the actions operated on shared memory directly. Although coherence is a goal, it is not clear that the goal is actually achieved by the design.

The memory model specifies that each new thread starts with an empty local cache and, when a new variable is created, the variable is created in shared memory, not the thread cache. Here are the main constraints relating *use*, *assign*, *load*, and *store*:

- *Use* and *assign* actions by a thread must occur in the order specified by the program.
- A thread is not permitted to lose its most recent assign (more precisely, a store action by thread *T* on variable *V* must occur between an *assign* by *T* of *V* and a subsequent *load* by *T* of *V*).
- A thread is not permitted to write data from its cache to shared memory for no reason (more precisely, an *assign* action by thread *T* on variable *V* must be scheduled to occur between a *load* or *store* by *T* of

V and a subsequent *store* by *T* of *V*).

There are similar constraints relating *load*, *store*, *read*, and *write*:

- For every *load*, there must be a preceding *read* action.
- For every *store*, there must be a following *write* action,
- Actions on master copy of a variable are performed by the shared memory in the order requested by thread.

The *Java Language Specification* gives the implementer complete freedom beyond these constraints, stating that "Provided that all the constraints are obeyed, a *load* or *store* action may be issued at any time by any thread on any variable, at the whim of the implementation."

A complication in the Java memory model is a provision for so-called *prescientstores*, which are *store* actions that occur earlier than would otherwise be permitted. The purpose of allowing a prescient store is to allow Java compilers to perform certain optimizations that make properly synchronized programs run faster, but might allow some memory actions to execute out of order in programs that are not properly synchronized. In other words, the memory model lets some compilers choose to execute store actions early under conditions that are intended to preserve coherence for properly structured programs.

Here is the outline of a simple program (devised by William Pugh). The Java Memory Model is Futally Flawed, *Concurrency: Practice and Experience*, 12(6) 2000, pp. 445-455), whose behavior is affected by the relaxed prescient store rule (this is obviously not Java syntax; the idea is that we have a program that initializes x and y to 0, and then spawns two threads that each execute two assignments):

```
x=0;y=0;
Thread 1: a = x;y=1;
Thread 2: b = y;x=1;
```

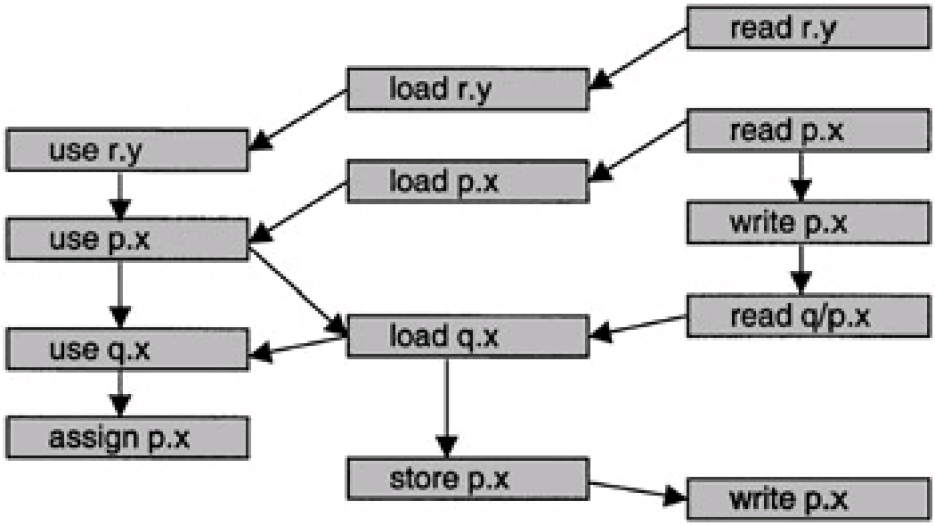
Without prescient stores, the two assignments in each thread must occur in the order shown. As a result, after both threads are executed, variables a and b can both be 0, or one can be 0 and the other 1. With prescient stores, the store actions can be done before the assign actions. This makes it legal for the write actions for both x and y to come before either of the read actions, and the threads can finish with a=b==1. The prescient store rule does not allow the load and store actions for a particular variable to be reordered, but a store involving variable y can be done before an assignment involving a and x.

William Pugh has studied the Java memory model (see paper mentioned above) Here is a simple program that, by his analysis, shows that the memory model is more restrictive than necessary:

```
//p & q are aliased
i = r.y;
j = p.x;
// concurrent write to p.x from another thread occurs here
k = q.x;
p.x = 42;
```

When executed in a manner consistent with the Java memory model, Pugh's analysis shows that the Java memory model *forces* a write to p.x from another thread to change the value of in q the thread cache, giving j and k different values in this program.

Here is a drawing showing the ordering of events required by the Java memory model. All of the actions in the drawing come from the preceding code, except for the write action halfway down the right-hand column, which comes from another thread:



Although it takes a little time, it is possible to check that this ordering between actions satisfies the constraints that make up the Java memory model. For example, all use and assign actions, which are in the left-hand column, are in the order corresponding to the assignments in the Java source code. Each use is preceded by a load that initializes the local cache, and similarly each load has a preceding read from shared memory.

The problem with this situation is that it forces a change in a thread cache that does not seem intuitively necessary. A programmer who writes this code and knows that p and q are aliases would be perfectly happy to have j and k get the same value. However, the implementation is forced to do extra work, setting the thread cache after the write from another thread. This requirement makes it more difficult to implement the Java memory model than it needs to be. The problem is particularly significant for multiprocessor implementations, as multiprocessors may reorder instructions for efficiency in situations like this. In fact, common single-processor implementations also appear not to enforce the full Java memory model correctly.

Although the details of the Java memory model are complicated, it should be clear from our brief discussion that it is difficult to design a good memory model. The two problems are that the memory model for any concurrent programming language with shared variables must be easy for programmers to understand so they can write correct and efficient programs that take advantage of concurrency. At the same time, the memory model must be implementable. This is particularly tricky for multiprocessors that may have their own form of processor caches and shared memory. Because Java is the first programming language that integrates concurrency and is widely used for general programming, it is likely that further experience with Java will lead to better understanding of memory models for concurrent languages.

14.4.4 Distributed Programming and Remote Method Invocation

RMI allows a thread running on one JVM to call a method of an object on another virtual machine across the network. This communication mechanism is one of the basic building blocks of distributed Java programming. Because RMI is an asymmetric operation (one thread executes a method call, the other does not), it is convenient to refer to the two threads by two different names. We will refer to the caller as the *client* and the thread providing the remote object as the *server*.

A typical server application creates some remote objects, makes references to them accessible, and waits for clients to invoke methods on these remote objects. There is a basic chicken-and-egg problem, however. Once we have some established connection between client and server, we can use this connection to pass references to objects. But how do we make the first connection between two systems? A solution used in distributed Java programming is the RMI registry, which is a remotely accessible directory of available objects.

Java provides three general facilities for distributed programming:

- *Locating remote objects*: A server can register its remote objects by using the RMI registry or the server can pass object references through other connectionsal ready established by the programs.
- *Communicating with remote objects*: Communication between remote objects is handled by RMI, which looks like standard Java method invocation to the programmer.
- *Transmitting objects across the network*: An object can be sent as an argument to a remote call or as the return value of such a call. When a Java object is transmitted, one virtual machine transmits the bytecode to execute the object and associated data to another. The receiving virtual machine loads the bytecode and creates the object.

Creating Remote Objects

There are several steps involved in creating an object that can be accessed remotely. The first step is to define a class that implements a *remote interface*, which is an interface that extends the interface java.rmi. Remote. Each method of the remote interface must declare java.rmi. RemoteException in its throws clause, in addition to any other exceptions. This allows a remote client to throw a remote exception to any object of this class.

When an object from a class that implements a remote interface is passed from the server virtual machine to a client virtual machine, RMI passes a remote *stub* for the object. The stub object has the same methods as the original object sitting on the server, but calls to methods on the stub object are sent across the network and executed on the server object. In the client program, all of this looks like regular Java programming: A call to an object that happens to be remote returns the stub of another object, and subsequent calls to methods on that object are transmitted across the network and executed on the server.

Compiling the server code requires two steps. The first step is to use the regular Java compiler to compile the source files, which contain the implementation of the remote interfaces and implementations and the server classes. The second step uses a second compiler to create stubs for the remote objects.

Dynamic Class Loading

An important feature of Java RMI is the way that the bytecode is transmitted across the network. This is possible because Java provides a uniform execution environment, the JVM, that runs standard bytecode regardless of the kind of hardware or operating system used. When RMI is used to pass an object from one virtual machine to another, the bytecode for the class of the object is transmitted and loaded, if the class has not already been installed on the receiving machine. This allows new types to be introduced into a remote virtual machine, extending the behavior of a program dynamically.

The RMI Registry

The RMI registry, running on a server, allows remote clients to get a reference to a remote object. The registry is typically used to locate only the first remote object that a client needs, as methods of the first object can return other remote objects.

The RMI system allows a programmer to bind a URL-formatted name of the form //host/objectname to a remote object. Once a remote object is registered on the server, callers can look up the object by name, obtain a remote object reference, and then remotely invoke methods on the object. For example, if helo is an object that can be called remotely to say "hello world," the following code binds the name HelloWorld to a reference to the helo object:

```
Naming.rebind( "//machine.stanford.edu/HelloWorld", helo);
```

This will allow a client program to get a stub for helo by a network access to the host (in this case machine.stanford.edu). There are several details involved in setting this all up properly that need not concern us. The main point is that a Java thread can get an object from a server across the network by using a symbolic name for the server and the object, and this is enough to let a client thread get more remote objects from this server.

Team-Fly



Previous Next

14.5 CHAPTER SUMMARY

In this chapter, we discussed some general aspects of concurrent and distributed programming and looked at three example languages: Actor systems, CML, and Java features providing threads, synchronization, and RMI. These three languages were chosen to illustrate different general approaches to communication and synchronization. Actors and CML are based on message passing, whereas Java threads communicate by shared variables and shared objects that may have one or more synchronized methods. Threads are implicit in actor systems, arising from concurrently sending more than one message to a set of concurrently reactive actors, whereas CML and Java programs must explicitly create threads.

General Issues in Concurrency

Concurrent programming languages define concurrent actions that may be executed simultaneously on several processors or in an interleaved fashion on a single processor. Concurrent programs generally have nondeterministic behavior, as the order of execution may depend on the speed of communication or the order in which process are scheduled. Independent threads may communicate in order to compute a useful result and coordinate to avoid problems associated with shared resources. When two threads attempt to update a shared data structure, for example, some coordination may be needed to keep the actions of each thread from interfering with the other. Without coordination, assignments by one thread may cancel assignments of another and leave the data structure in a state that is not meaningful to the program.

Locking, semaphores, and monitors are traditional operating systems mechanisms for coordinating the actions of independent processes. A lock is a kind of shared variable that may be set to indicate that a resource is locked or unlocked. When one process locks a resource, other processes should not access the resource. One problem with locking is that testing and setting a lock must be atomic. For this reason, locks with atomic test-and-set are often provided by hardware.

Semaphores structure a set of waiting processes into a list that may be given access to the shared resource when it becomes available. Processes using a semaphore must call *wait* before accessing the protected resource and *signal* when they are done. Both *wait* and *signal* are atomic actions that may be implemented by the operating system (or programming language run-time system) using locks. Monitors group synchronization with operations on data, eliminating the need for the calling process to call *wait* and *signal*.

Concurrent programs communicate by using shared resources or message passing. In principle, message-passing mechanisms can be synchronous or asynchronous, buffered or unbuffered, and preserve or not preserve transmission order. Actors and CML differ in all three ways, illustrating two very different sets of communication assumptions. As explored in the exercises, some of the remaining combinations are sensible language design choices and some are either not sensible or too weak for many programming problems.

	Synchronous		**Asynchronous**	
	Ordered	**Unordered**	**Ordered**	**Unordered**
Buffered				Actors
Unbuffered	CML			

Actor Systems

An actor is a form of reactive object that performs a computation in response to a message and may perform one or more of the following basic actions:

- Sending communication to itself or other actors,
- Creating actors,
- Specifying a replacement behavior.

Actors do not have any shared state, but use buffered asynchronous message passing for all communication. Actors change state atomically. There are no changes in the internal state of an actor while it is responding to a message. Instead, an actor changes state atomically as it completes its computation and specifies a replacement behavior. Some interesting aspects of the actor model are the mail system, which buffers and delivers messages, and the challenges of programming with only a synchronous communication without order guarantees.

Concurrent ML

CML is a concurrent extension of Standard ML that provides dynamic thread creation and synchronous message passing on typed channels. The most interesting aspects of CML, are the ways that communication and coordination are combined and the way that CML allows programmers to define their own communication and synchronization abstractions by using events. Events represent synchronous actions that are performed when a program synchronizes on them. CML event primitives can be used to implement selective computation, as illustrated by Dijkstra's guarded commands, and a variety of communication mechanisms such as multicast channels and various forms of protected shared variables.

Java Threads and Synchronization

Java makes concurrent programs portable across a variety of computing platforms. We create a Java thread object by defining a class with a run() method, either by extending the Thread class or by implementing the Runnable interface directly. When a thread object is activated, the run method runs as a separate thread inside the JVM. Java threads generally communicate by calling methods of shared objects. The Java language provides several synchronization primitives and supports monitors directly through objects with synchronized methods.

Java synchronization is achieved through three basic mechanisms:

- Locks: Every object has a lock, used for mutual exclusion.
- Wait sets: Every object has a wait set, providing a form of semaphore.
- Thread termination: A process can pause until another thread terminates.

Locks are tested and set by synchronized blocks and methods, wait sets are used by methods wait, notify, and notifyAll that are defined on all objects, and thread termination is used by the join method on thread objects.

The Java memory model is a specification of how shared memory can be cached and accessed by multiple threads. The original memory model, presented in *Java Language Specification*, has proven difficult to understand and unsatisfactory for implementers in various ways. Although it is likely that the memory model will eventually be revised to correct some of the known problems, we discussed the memory model briefly in order to understand some of the subtle basic issues involved in concurrent memory access.

Distributed Java Programming and Remote Method Invocation

RMI allows a thread running on one JVM to call a method of an object on another virtual machine across the network. A typical server application creates some remote objects, makes references to them accessible, and waits for clients to invoke methods on these remote objects. A server may make objects accessible by using the RMI registry, which is a remotely accessible directory of available objects. When RMI is used to pass an object from one virtual machine to another, the bytecode for the class of the object is transmitted and loaded if the class has not already been installed on the receiving machine. This allows new types to be introduced into a remote virtual machine, extending the behavior of a program dynamically.

Team-Fly

Chapter 15: The Logic Programming Paradigm and Prolog

Krzysztof R. Apt

15.1 HISTORY OF LOGIC PROGRAMMING

The logic programming paradigm has its roots in automated theorem proving from which it took the notion of a deduction. What is new is that in the process of deduction some values are computed. The creation of this programming paradigm is the outcome of a long history that for most of its course ran within logic and only later inside computer science. Logic programming is based on the syntax of first-order logic, which was originally proposed in the second half of nineteenth century by Gottlob Frege and later modified to the currently used form by Giuseppe Peano and Bertrand Russell.

In the 1930s, Kurt G¨odel and Jacques Herbrand studied the notion of computability based on derivations. These works can be viewed as the origin of the "computation as deduction" paradigm. Additionally, Herbrand discussed in his doctoral thesis a set of rules for manipulating algebraic equations on terms that can be viewed now as a sketch of a unification algorithm. Some 30 years later, in 1965, Alan Robinson published his fundamental paper (Robinson 1965) that lies at the foundation of the field of automated deduction. In this paper, he introduced the resolution principle, the notion of unification, and a unification algorithm. Using the resolution method, one can prove theorems of first-order logic, but another step was needed to see how one could compute within this framework.

This was eventually achieved in 1974 by Robert Kowalski (Predicate logic as a programming language, North-Holland, 1974.) Proc. IFIP'74,in which logic programs with a restricted form of resolution were introduced. The difference between this form of resolution and the one proposed by Robinson is that the syntax is more restricted, but proving now has a side effect in the form of a satisfying substitution. This substitution can be viewed as a result of a computation and consequently certain logical formulas can be interpreted as programs. In parallel, Alain Colmerauer and his colleagues worked on a programming language for natural language processing based on automated theorem proving. This ultimately led to creation of Prolog in1973. Kowalski and Colmerauer with his team interacted in the period from 1971 to1973. This influenced their views and helped them to crystallize the ideas.

Prolog can be seen as a practical realization of the idea of logic programs. It started as a programming language for applications in natural language processing, but soon after it was found that it can also be used as a general purpose programming language.A number of other attempts to realize the computation as deduction paradigm were proposed about the same time, notably by Cordell Green and Carl Hewitt, but the logic programming proposal, probably because it was the simplest and most versatile, became most successful.

Originally, Prolog was implemented by Philippe Roussel, a colleague of Colmerauer, in the form of an interpreter written in Algol-W. An important step forward was achieved by David H. Warren, who proposed in 1983 an abstract machine, now called Warren Abstract Machine (WAM), which consists of a machine architecture with an instruction set that serves as a target for machine independent Prolog compilers. WAM became a standard basis for implementing Prolog and other logic programming languages.

The logic programming paradigm influenced a number of developments in computer science. As early as the 1970s, it led to the creation of deductive databases that extend the relational databases by providing deduction capabilities. A further impetus to the subject came unexpectedly from the Japanese Fifth Generation Project for intelligent computing systems (1982-1991), in which logic programming was chosen as its basis. More recently, this paradigm led to constraint logic programming that realizes a general approach to computing in which the programming process is limited to a generation of constraints (requirements) and a solution of them, and to inductive logic programming, a logic-based approach to machine learning.

The above account of history of logic programming and Prolog shows its roots in logic and automated deduction. In

fact, Colmerauer and Roussel (1996) write "There is no question that Prolog is essentially a theorem prover à la Robinson. Our contribution was to transform that theorem prover into a programming language." This origin of the logic paradigm probably impeded its acceptance within computer science in times when imperative programming got impetus thanks to the creation of Pascal and C, the fields of verification and semantics of imperative programs gained ground, and when the artificial intelligence community already adopted Lisp as their language of choice.

Here, we offer an alternative presentation of the subject by focusing on the ordinary programming concepts (often implicitly) present in logic programming and by relating various of its ideas to those present in the imperative and functional programming paradigms.

Team-Fly



Previous Next

15.2 BRIEF OVERVIEW OF THE LOGIC PROGRAMMING PARADIGM

The logic programming paradigm differs substantially from other programming paradigms. When stripped to the bare essentials, it can be summarized by the following three features.

- Computing takes place over the domain of all terms defined over a "universal" alphabet.
- Values are assigned to variables by means of automatically generated substitutions, called *most general unifiers*. These values may contain variables, called *logical variables*.
- The control is provided by a single mechanism: automatic backtracking.

In our exposition of this programming paradigm, we shall stress the above three points. Even such a brief summary shows both the strength and weakness of the logic programming paradigm. Its strength lies in an enormous simplicity and conciseness; its weakness has to do with the restrictions to one control mechanism and the use of a single data type.

This framework has to be modified and enriched to accommodate it to the customary needs of programming, for example by providing various control constructs and by introducing the data type of integers with the customary arithmetic operations. This can be done and, in fact, Prolog and constraint logic programming languages are examples of such a customization of this framework.

15.2.1 Declarative Programming

Two additional features of logic programming are important to note. First, in its pure form it supports *declarative programming*. A declarative program admits two interpretations. The first one, called a *procedural interpretation*, explains *how* the computation takes place, where as the second one, called a *declarative interpretation*, is concerned with the question *what* is being computed.

Informally, the procedural interpretation is concerned with the *method*, whereas the declarative interpretation is concerned with the *meaning*. In the procedural interpretation, a declarative program is viewed as a description of an algorithm that can be executed. In the declarative interpretation, a declarative program is viewed as a formula, and one can reason about its correctness without any reference to the underlying computational mechanism. This makes declarative programs easier to understand and to develop.

As we shall see, in some situations the specification of a problem in the logic programming format already forms an algorithmic solution to the problem. So logic programming supports declarative programming and allows us to write *executable specifications*. It should be added, however, that in practice the Prolog programs obtained in this way are often inefficient, so this approach to programming has to be combined with various optimization techniques, and an appropriate understanding of the underlying computation mechanism is indispensable. To clarify this point, we shall present here a number of Prolog programs that are declarative and eliminate from them various sources of inefficiency.

This dual interpretation of declarative programs also accounts for the double use of logic programming as a formalism for programming and for knowledge representation, and explains the importance of logic programming in the field of artificial intelligence.

15.2.2 Interactive Programming

Another important feature of logic programming is that it supports *interactive programming*. In other words, the user can write a single program and interact with it by means of various queries of interest to which answers are produced. The Prolog systems greatly support such an interaction and provide simple means to compute one or more solutions to the submitted query, to submit another query, and to trace the execution by setting up, if desired, various check

points, all within the same "interaction loop." This leads to a flexible style of programming.

This is completely analogous to the way functional programs are used where the interaction is achieved by means of expressions that need to be evaluated using a given collection of function definitions.

In what follows, we shall introduce Prolog, the best known programming language based on the logic programming paradigm. Prolog is then based on a subset of first-order logic. We explain here how Prolog uses this syntax in a novel way (this characteristic is called *ambivalent syntax*) and extends it by a number of interesting features, notably by supporting infix notation and by providing so-called *anonymous* and *meta-variables*. These extensions amount to more than syntactic sugar. In fact, they make it possible to realize in Prolog higher-order programming and meta-programming in a simple way.

When discussing Prolog, it is useful to abstract from the programming language and first consider the underlying conceptual model provided by logic programming.

Team-Fly



Previous Next

Team-Fly

15.3 EQUATIONS SOLVED BY UNIFICATION AS ATOMIC ACTIONS

We begin by explaining how computing takes place at the "atomic level." In logic programming, the atomic actions are equations between terms (arbitrary expressions). They are executed by means of the unification process that attempts to solve such equations. In the process of solving, values are assigned to variables. These values can be arbitrary terms. In fact, the variables are all of one type that consists of the set of all terms.

This informal summary shows that the computation process in logic programming is governed by different principles than in the other programming paradigms.

15.3.1 Terms

In a more rigorous explanation, let us start by introducing an alphabet that consists of the following disjoint classes of symbols:

- *variables*, denoted by x, y, z ,… possibly with subscripts
- *function symbols*
- parentheses, "('and')"
- comma, ","

We also postulate that each function symbol has a fixed *arity*, that is the number of arguments associated with it. 0-ary function symbols are called *constants*, and are usually denoted by a, b, c, d ,…. Below we denote function symbols of positive arity by f, g, h ,….

Finally, *terms* are defined inductively as follows:

- a variable is a term,
- if f is an n-ary function symbol and $t_1,\ldots, t_n$ are terms, then $f(t_1,\ldots, t_n)$ is a term.

In particular every constant is a term. Variable-free terms are usually called *ground terms*. Below we denote terms by s, t, u, w ,….

For example, if a is a constant, x and y are variables, f is a binary function symbol and g a unary function symbol, then $f(f(x, g(b)), y)$ is a term.

Terms are fundamental concepts in mathematical logic, but at first sight they seem to be less common in computer science. However, they can be seen as a generalization of the concept of a string familiar from the theory of formal languages. In fact, strings can be viewed as terms built out of an alphabet the only function symbols of which are the concatenation operations in each arity (or alternatively, out of an alphabet the only function symbol of which is the binary concatenation operation assumed to be associative, say to the right). Another familiar example of terms are arithmetic expressions. These are terms built out of an alphabet in which, as the function symbols, we take the usual arithmetic operations of addition, subtraction, multiplication, and, say, integer division, and as constants 0, –1, 1 ,….

In logic programming, no specific alphabet is assumed. In fact, it is convenient to assume that in each arity an infinite supply of function symbols exists and that all terms are written in this "universal alphabet." These function symbols can be, in particular, the denotations of arithmetic operations, but no meaning is attached to these function symbols. This is in contrast to most of the imperative programming languages, in which for example the use of "+" in an expression implies that we refer to the addition operation. The other consequence of this choice is no types are assigned to terms. In fact, no types are assumed and consequently there is no distinction between, say, arithmetic expressions, Boolean expressions, and terms denoting lists. All these terms are considered as being of one type.

15.3.2 Substitutions

Unlike in imperative programming, in logic programming the variables can be uninitialized. Moreover, the possible values of variables are terms. So to properly explain the computation process we need to reexamine the notion of a state.

At any moment during the computation there will be only a finite number of variables that are *initialized* - these are variables to which, in the considered computation, some value was already assigned. Since these values are terms, we are naturally led to consider *substitutions*. These are finite mappings from variables to terms such that no variable is mapped to itself. So substitution provides information about which variables are initialized. (Note that no variable can be initialized to itself, which explains the restriction that no variable is mapped to itself.)

Substitutions then form a counterpart of the familiar notion of a *state* used in imperative programming. We denote a substitution by $\{x_1/t_1,\ldots, x_n/t_n\}$. This notation implies that $x_1,\ldots, x_n$ are different variables, $t_1,\ldots, t_n$ are terms and that no term t_i equals the variable x_i. We say then that the substitution $\{x_1/t_1,\ldots, x_n/t_n\}$*binds* thevariable x_i to the term t_i.

Using a substitution, we can evaluate a term in much the same way as using a state we can evaluate an expression in imperative programming languages. This process of evaluation is called an *application* of a substitution to a term. It is the outcome of a simultaneous replacement of each variable occurring in the domain of the substitution by the corresponding term. So, for example, the application of the substitution $\{x/f(z), y/g(z)\}$ to the term $h(x, y)$ yields the term $h(f(z), g(z))$. Here the variable x was replaced by the term $f(z)$ and the variable y by the term $g(z)$. In the same way, we define an application of a substitution to an atom, query, or a clause.

So an evaluation of a term using a substitution yields again a term. This is in contrast to imperative programming where an evaluation of an expression using a state yields a value that belongs to the type of this expression.

15.3.3 Most General Unifiers

As already mentioned, in logic programming the atomic actions are equations between terms and the unification process is used to determine their meaning. Before we discuss these matters in detail, let us consider some obvious examples of how solving equations can be used as an assignment.

We assume that all mentioned variables are uninitialized. By writing $x = a$, we assign the constant a to the variable x. Because in logic programming the equality "=" is symmetric, the same effect is achieved by writing $a = x$. More interestingly, by writing $x = f(y)$ (or, equivalently, $f(y) = x$) we assign the term $f(y)$ to the variable x. Since $f(y)$ is a term with a variable, we assigned to the variable x an expression with a variable in it. Recall that a variable that occurs in a value assigned to another variable is called a logical variable. Therefore, y is a logical variable here. The use of logical variables is an important distinguishing feature of logic programming and we devote the whole subsection 15.5.2 to an explanation of their use. Finally, by writing $f(y) = f(g(a))$, we assign the term $g(a)$ to the variable y, as this is the way to make these two terms equal.

These examples show that the equality "=" in logic programming and the assignment in C, also written using "=", are totally different concepts.

Intuitively, *unification* is the process of solving an equation between terms (i.e., of making two terms equal) in a least constraining way. The resulting substitution (if it exists) is called a *most general unifier* (*mgu*). For example, the equation $x = f(y)$ can be solved (i.e., the terms x and $f(y)$*unify*) in a number of ways, for instance, by means of each of the substitutions $\{x/f(y)\}$, $\{x/f(a), y/a\}$, $\{x/f(a), y/a, z/g(b)\}$, Clearly, only the first one is "least constraining." In fact, out of these three substitutions thefirst one is the only most general unifier of the equation $x = f(y)$. The notion of a least constraining substitution can be made precise by defining an order on substitutions. In this order, the substitution $\{x/f(y)\}$ is more general than $\{x/f(a), y/a\}$, etc.

Note that we made the terms x and $f(y)$ equal by instantiating only one of them. Such a special case of the unification is called *matching*, which is the way of assigning values in functional programming languages. Unification is more general than matching as the following, slightly less obvious, example shows. Consider the equation $f(x, a) = f(b, y)$. Here, the most general unifier is $\{x/b, y/a\}$. In contrast to the previous example, it is now not possible to make these two terms equal by instantiating only one of them.

The problem of deciding whether an equation between terms has a solution is called the *unification problem*.

Robinson (JACM, 12(1): 23-41, 1965) showed that the unification problem is decidable. More precisely, he introduced a unification algorithm with the following property. If an equation between terms has a solution, the algorithm produces an mgu, otherwise it reports a failure. An mgu of an equation is unique up to renaming of the variables.

15.3.4 A Unification Algorithm

In what follows, we discuss the unification process in more detail using an elegant unification algorithm introduced in Martelli and Montanari (An efficient unification algorithm, ACM Trans. Prog. Lang. and Systems; vol 4, 1982, pp. 258-282). This algorithm takes as input a finite set of term equations $\{s_1 = t_1, \ldots, s_n = t_n\}$ and tries to produce an mgu of them.

MARTELLI-MONTANARI ALGORITHM

Nondeterministically choose from the set of equations an equation of a form below and perform the associated action.

1.	$f(s_1, \ldots, s_n) = f(t_1, \ldots, t_n)$	*replace by the equations* $s_1 = t_1, \ldots, s_n = t_n$,
2.	$f(s_1, \ldots, s_n) = g(t_1, \ldots, t_m)$ *where* $f \neq g$	*halt with failure*,
3.	x = x	*delete the equation*,
4.	$t = x$ where t is not a variable	*replace by the equation* $x = t$,
5.	$x = t$ where x does not occur in t and x occurs elsewhere	*apply the substitution {x/t} to all other equations*
6.	$x = t$ where x occurs in t and x differs from t	*halt with failure*.

The algorithm terminates when no action can be performed or when failure arises. In case of success, by changing in the final set of equations all occurrences of "=" to "/" we obtain the desired mgu. Note that action (1) includes the case $c = c$ for every constant c which leads to deletion of such an equation. In addition, action (2) includes the case of two different constants.

To illustrate the operation of this algorithm, reconsider the equation $f(x, a) = f(b, y)$. Using action (1) it rewrites to the set of two equations, $\{x = b, a = y\}$. By action (4) we now get the set $\{x = b, y = a\}$. At this moment, the algorithm terminates and we obtain the mgu $\{x/b, y/a\}$.

So by interpreting the equality symbol as the request to find a most general unifier of the considered pair of terms, each equation is turned into an atomic action that either produces a substitution (a most general unifier) or *fails*. This possibility of a failure at the level of an atomic action is another distinguishing feature of logic programming.

By writing a sequence of equations, we can create very simple logic programs that either succeed and produce as output a substitution or fail. It is important to understand how the computation then proceeds. We illustrate it by means of three progressively more complex examples.

First, consider the sequence

$$f(x, a) = f(g(z), y),\ h(u) = h(d).$$

The first equation yields first the intermediate substitution $\{x/g(z), y/a\}$ and the second one the substitution $\{u/d\}$. By combining these two substitutions we obtain the substitution $\{x/g(z), y/a, u/d\}$ produced by this logic program.

As a slightly less obvious example, consider the sequence

$$f(x, a) = f(g(z), y),\ h(x, z) = h(u, d).$$

Here the intermediate substitution $\{x/g(z), y/a\}$ binds the variable x that also occurs in the second equation. This second equation needs to be evaluated first in the "current state," here represented by the substitution $\{x/g(z), y/a\}$, before being executed. This evaluation produces the equation $h(g(z), z) = h(u, d)$. This equation yields the most general unifier $\{u/g(d), z/d\}$ and the resulting final substitution is here $\{x/g(d), y/a, u/g(d), z/d\}$.

What happened here is that the substitution $\{u/g(d), z/d\}$ was *applied* to the intermediate substitution $\{x/g(z), y/a\}$. The

effect of an application of one substitution, say δ, to another, say γ, (or of *composition* of the substitutions) is obtained by

- applying δ to each of the terms that appear in the range of γ
- adding to the resulting substitution the bindings to the variables that are in the domain of δ but not in the domain of γ

In the above example, the first step yields the substitution {*x*/*g*(*d*), *y*/a}, and the second step adds the bindings *u*/*g*(*d*) and *z*/*d* to the final substitution. This process of substitution composition corresponds to an *update of a state* in imperative programming, and that is how we shall refer to it in the sequel.

As a final example consider the sequence

$$f(x, a) = f(g(z), y),\ h(x, z) = h(d, u).$$

It yields a failure. Indeed, after executing the first equation the variable *x* is bound to *g*(*z*), so the evaluation of the second equation yields *h*(*g*(*z*), *z*) = *h*(*d*, *u*) and no substitution makes equal (unifies) the terms *h*(*g*(*z*), *z*) and *h*(*d*, *u*).

It is useful to compare solving equations by unification with the assignment command. First, note that, in contrast to assignment, unification can assign an arbitrary term to a variable. Also, it can fail, something the assignment cannot do. On the other hand, using assignment one can modify the value of a variable, something unification can perform in a very limited way: by further instantiating the term used as a value. Thus, these atomic actions are incomparable.

Team-Fly



Previous Next

Team-Fly

15.4 CLAUSES AS PARTS OF PROCEDURE DECLARATIONS

Logic programming is a rule based formalism and Prolog is a rule based language. In this context, the rules are called *clauses*. To better understand the relationship between logic programming and imperative programming, we proceed in two steps and introduce a restricted form of clauses first.

15.4.1 Simple Clauses

Using unification, we can execute only extremely simplistic programs that consist of sequences of equations. We now enrich this framework by adding procedures. In logic programming they are modelled by means of *symbols*, sometimes called *predicates*. Below, we denote relation symbols by p, q, r,.... As in the case of the function symbols, we assume that each relation symbol has a fixed arity associated with it. When the arity is 0, the relation symbol is usually called a *propositional symbol*.

If p is an n-ary relation symbol and $t_1,\ldots, t_n$ are terms, then we call $p(t_1,\ldots, t_n)$ an *atom*. When $n = 0$ the propositional symbols coincide with atoms. Interestingly, as we shall see, such atoms are useful. Intuitively, a relation symbol corresponds to a *procedure identifier* and an atom to a *procedure call*. The equality symbol "=" is a binary relation symbol written in an infix form, so each equation is also an atom. However, the meaning of equality is determined, so it can be viewed as a built-in procedure, i.e., a procedure with a predefined meaning.

We still need to define the procedure declarations and to clarify the parameter mechanism used. Given an n-ary relation symbol p and atoms $A_1,\ldots, A_k$ we call an expression of the form

$$p(x_1,\ldots, x_n):\text{-}\ A_1,\ldots, A_k.$$

a *simple clause*. $p(x_1,\ldots, x_n)$ is called the *head* of the clause and $A_1,\ldots, A_k$ its *body*. The fullstop "." at the end of the clause is important: it signals to the compiler (or interpreter) that the end of the clause is encountered.

The procedural interpretation of a simple clause $p(x_1,\ldots, x_n):\text{-}\ A_1,\ldots, A_k$ is: "to establish $p(x_1,\ldots, x_n)$ establish $A_1,\ldots, A_k$", while the declarative interpretation is: "$p(x_1,\ldots, x_n)$ is true if $A_1,\ldots, A_k$ is true". The declarative interpretation explains why in the logic programming theory the reversed implication symbol "←" is used instead of ":-".

Finally, a *simple logic program* is a finite set of clauses. Such a program is activated by providing an initial *query*, which is a sequence of atoms. In the imperative programming jargon a query is then a program and a simple logic program is a set of procedure declarations. Intuitively, given a simple program, the set of its simple clauses with the same relation symbol in the head corresponds to the procedure declaration in the imperative languages. One of the syntactic confusions is that in logic programming the comma "," is used as a separator between the atoms constituting a query, where as in the imperative programming the semicolon ";" is used for this purpose.

15.4.2 Computation Process

A nondeterminism is introduced into this framework by allowing *multiple clauses* with the same relation symbol in the head. In the logic programming theory, this form of nondeterminism (called *don't know nondeterminism*) is retained by considering all computations that can be generated by means of multiple clauses and by retaining the ones that lead to a success. "Don't know" refers to the fact that in general we do not know which computation will lead to a success.

In Prolog, this computation process is made deterministic by ordering the clauses by the textual ordering and by employing automatic backtracking to recover from failures. Still, when designing Prolog programs, it is useful to have the don't know nondeterminism in mind. In fact, in explanations of Prolog programs phrases like "this program nondeterministically guesses an element such that …" are common. Let us explain now more precisely how the computing takes place in Prolog. To this end, we need to clarify the procedure mechanism used and the role played by the use of multiple clauses.

The procedure mechanism associated with the simple clauses introduced above is *call-by-name* according to which the formal parameters are simultaneously substituted by the actual ones. So this procedure mechanism can be simply explained by means of substitutions: given a simple clause $p(x_1,\ldots, x_n)$:- $A_1,\ldots, A_k$. a procedure call $p(t_1,\ldots, t_n)$ leads to an execution of the statement $(A_1,\ldots, A_k)\{x_1/t_1,\ldots, x_n/t_n\}$obtained by applying the substitution $\{x_1/t_1,\ldots, x_n/t_n\}$ to the statement $A_1,\ldots, A_k$.(We assume here that the variables of the clauses are appropriately renamed to avoid variable clashes.) Equivalently, we can say that the procedure call $p(t_1,\ldots, t_n)$leads to an execution of the statement $A_1,\ldots, A_k$ in the state (represented by a substitution) updated by the substitution $\{x_1/t_1,\ldots, x_n/t_n\}$.

The clauses are tried in the order they appear in the program text. The depth-first strategy is implied by the fact that a procedure call leads directly to an execution of the body of the selected simple clause. If at a certain stage a failure arises, the computation backtracks to the last choice point (a point in the computation at which one out of more applicable clauses was selected) and the subsequent simple clause is selected. If the selected clause was the last one, the computation backtracks to the previous choice point. If no choice point is left, a failure arises. Backtracking implies that the state is restored, so all the state updates performed since the creation of the last choice point are undone.

Let us illustrate now this definition of Prolog's computation process by considering the most known Prolog program the purpose of which is to append two lists. In Prolog, the empty list is denoted by [] and the list with head h and tail t by [h | t]. The term [a | [b | s]] abbreviates to a more readable form [a, b | s], the list [a |[b |[]]] abbreviates to [a, b] and similarly with longer lists. This notation can be used both for lists and for arbitrary terms that start with the list formation operator [.|..].

Then the following logic program defines by induction w.r.t. the first argument how to append two lists. Here and elsewhere we follow Prolog's syntactic conventions and denote variables by strings starting with an upper case letter. The names ending with "s" are used for the variables meant to be instantiated to lists.

```
% append(Xs, Ys, Zs) :- Zs is the result of concatenating the lists Xs and Ys.
append(Xs, Ys, Zs) :- Xs = [], Zs = Ys.
append(Xs, Ys, Zs) :- Xs = [H | Ts], Zs = [H | Us], append(Ts, Ys, Us).
```

In Prolog, the answers are generated as substitutions written in an equational form (as in the Martelli-Montanari algorithm presented above). In what follows, we display a query Q. as ?-Q. . Here "?-" is the system prompt and the fullstop "." signals the end of the query.

One can check then that the query

```
?- append([jan,feb,mar], [april,may], Zs).
```

yields Zs = [jan,feb,mar,april,may] as the answer and that the query

```
?- append([jan,feb,mar], [april,may], [jan,feb,mar,april,may]).
```

---

succeeds and yields the empty substitution as the answer.

In contrast, the query

```
?- append([jan,feb,mar], [april,may], [jan,feb,mar,april]).
```

---

fails. Indeed, the computation leads to the subsequent procedure calls

---

```
append([feb,mar], [april,may], [feb,mar,april]),
append([mar], [april,may], [mar,april]) and
append([], [april,may], [april]),
```

---

and the last one fails because the terms [april,may] and [april] don't unify.

15.4.3 Clauses

The last step in defining logic programs consists of allowing arbitrary atoms as heads of the clauses. Formally, given atoms H, $A_1,\ldots, A_k$, we call an expression of the form

$$H\text{:-}A_1,\ldots, A_k.$$

a *clause*. If $k=0$, that is if the clause's body is empty, such a clause is called a *fact* and the ":-" symbol is then omitted. If $k > 0$, that is, if the clause's body is nonempty, such a clause is called a *rule*.A *logic program* is then a finite set of clauses and a *pure Prolog program* is a finite sequence of clauses.

Given a pure Prolog program, we call the set of its clauses with the relation p in the head the *definition of p.* Definitions correspond to the procedure declarations in imperative programming and to the function definitions in functional programming. Variables that occur in the body of a clause but not in its head are called *local*. They correspond closely to the variables that are local to the procedure bodies in the imperative languages with the difference that in logic programs their declaration is implicit. Logic programming, like Pascal, does not have a block statement.

To explain how the computation process takes place for pure Prolog programs, we simply view a clause of the form

$$p(s_1,\ldots, s_n)\text{:-}A_1,\ldots, A_k.$$

as a shorthand for the simple clause

$$p(x_1,\ldots, x_n)\text{:-}(x_1,\ldots, x_n) = (s_1,\ldots, s_n), A_1,\ldots, A_k.$$

where $x_1,\ldots, x_n$ are fresh variables. We use here Prolog's syntactic facility according to which given a sequence $s_1,\ldots, s_n$ of terms $(s_1,\ldots, s_n)$ is also a term.

So given a procedure call $p(t_1,\ldots, t_n)$ if the above clause $p(s_1,\ldots, s_n)$:-$A_1,\ldots, A_k$ is selected, an attempt is made to unify $(t_1,\ldots, t_n)$ with $(s_1,\ldots, s_n)$. (As before, we assume here that no variable clashes arise; otherwise the variables of the clause should be appropriately renamed.) If the unification succeeds and produces a substitution θ, the state (represented by a substitution) is updated by applying to it θ and the computation continues with the statement $A_1,\ldots, A_k$ in this new state. Otherwise, a failure arises and the next clause is selected.

Therefore, by using clauses instead of simple clauses, unification is effectively lifted to a parameter mechanism. As a side effect, this makes the explicit use of unification, modelled by means of "=", superfluous. As an example, reconsider the above program appending two lists. Using the clauses it can be written in a much more succinct way, as the following program APPEND:

```
% append(Xs, Ys, Zs) :- Zs is the result of concatenating the lists Xs and Ys.
append([], Ys, Ys).
append([X | Xs], Ys, [X | Zs]) :- append(Xs, Ys, Zs).
```

Here, the implicit case analysis present in the previous program is in effect moved into the heads of the clauses. The use of terms in the heads of the clauses is completely analogous to the use of *patterns* in function definitions in functional programming.

To summarize, the characteristic elements of procedure declarations in logic programming, in contrast to imperative programming, are: the use of multiple rules and use of patterns to select among these rules.

Team-Fly

15.5 PROLOG'S APPROACH TO PROGRAMMING

The power and originality of the Prolog programming style lies in the combination of automatic backtracking with the use of relations and logical variables.

15.5.1 Multiple Uses of a Single Program

As a first illustration of the novelty of Prolog's approach to programming, we illustrate the possibility of using the same program for different purposes. The perhaps simplest example involves the following program MEMBER. We use in it a useful feature of Prolog, so-called anonymous variable, written as an "underscore" character "_." Each occurrence of "_" in a query or in a clause is interpreted as a *different* variable. Anonymous variables are analogous to the *wildcard pattern* feature of the ML or Haskell language.

```
% member(X, Xs):- X is a member of the list Xs.
  member(X, [X | ]).
  member(X, [_| Xs]):- member(X, Xs).
```

MEMBER can be used both for testing and for computing:

```
?- member(wed, [mon, wed, fri]).
yes
?- member(X, [mon, wed, fri]).
Xs = mon ;
Xs = wed ;
Xs = fri ;
no
```

Here ";" is the user's request to produce the next answer. If this request fails, the answer "no" is printed.

Consequently, given a variable X and two lists s and t, the query member(X, s), member(X, t). generates all elements that are present both in s and t. Operationally, the first call generates all members of s and the second call tests for each of them the membership in t.

Also the APPEND program can be used for a number of purposes, in particular to concatenate two lists and to split a list in all possible ways. For example, we have

```
?- append(Xs, Ys, [mon, wed, fri]).
 Xs = []
 Ys = [mon, wed, fri];
 Xs = [mon]
 Ys = [wed, fri];
 Xs = [mon, wed]
 Ys = [fri];
 Xs = [mon, wed, fri]
 Ys = [];
 no
```

This cannot be achieved with any functional programing version of the APPEND. The difference comes from the fact that in logic programming procedures are defined by means of the relations, whereas in functional programming functions are used. In fact, there is no distinction between input and output arguments in the procedures in logic programs.

To see two uses of append in a single program, consider a program that checks whether one list is a consecutive sublist of another one. The one line program SUBLIST that follows formalizes the following definition of a sublist:

- the list Xs is a sublist of the list Ys if Xs is a prefix of a suffix of Ys.

```
% sublist(Xs, Ys) :- Xs is a sublist of the list Ys.
sublist(Xs, Ys) :- append(_, Zs, Ys), append(Xs,_, Zs).
```

Here, both anonymous variables and Zs are local. In this rule Zs is a suffix of Ys and Xs is a prefix of Zs. This relation is illustrated in Figure 15.1.

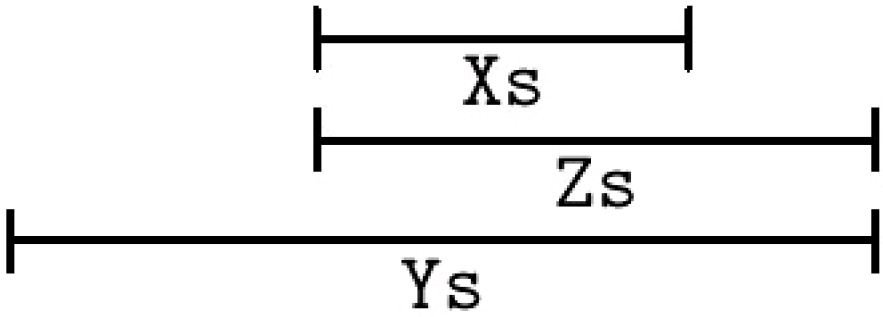


Figure 15.1: Xs is a sublist of the list Ys

Operationally, given two lists, as and bs, the query sublist(as, bs). leads to a generation of splits of the list bs through the call append (_, Zs, bs). Then for each generated suffix Zs of bs it is checked whether for some list, denoted by the anonymous variable, the call append (as,, Zs) succeeds. This happens when as is a prefix of Zs. So a typical use of this program involves backtracking.

15.5.2 Logical Variables

Let us return now to the logical variables. They are an important feature of logic programming, but it is easy to overlook their use. For example, they already appear in the computations involving the first version of the list concatenation program, and consequently, because of the way we defined the computation process, in the computations of the APPEND program. Indeed, given the query append ([jan, feb, mar], [april, may], Zs). the rule

```
append(Xs, Ys, Zs) :- Xs = [H | Ts], Zs = [H | Us], append(Ts, Ys, Us).
```

leads to the binding of the variable Zs to the term [jan | Us]. The value of the variable Us is computed later, by means of the call append([feb, mar], [april, may], Us). This call first binds Us to the term [feb | U1s], where U1s is a fresh variable, and hence Zs to the term [jan, feb | U1s]. This progressive building of the output using the logical variables is typical for Prolog. The real power of logical variables should become apparent after considering the following three original Prolog programs.

A type assignment

The typed lambda calculus and Curry's system of type assignment involves statements of the form s: τ, which should be read as "term s has type τ ." Finite sequences of such statements with s being a variable are called *environments* are denoted below by E. A statement of the form $E ? s : \tau$ should be read as "in the environment E the term s has type τ". The following three rules define by induction on the structure of lambda terms how to assign types to lambda terms:

$$\frac{}{x : t ? E}$$

$$\frac{}{E ? x : t}$$

$$\frac{E ? m : s \to t \quad E ? n : s}{E ? E (mn): t}$$

$$\frac{E , x : s ? m : t}{E ? (\lambda x.m): s \to t}$$

To encode the lambda terms as usual "first-order" terms, we use the unary function symbol var and two binary function symbols, lambda and apply. The lambda term x(a variable) is translated to the term var(x), the lambda term (mn) to the term apply(m, n), and the lambda term $\lambda x.m$ to the term lambda(x, m). For example, the lambda term $\lambda x.$ (xx) translates to lambda(x, apply(var(x), var(x))). The subtle point is thataccording to Prolog convention, lower case letters stand for constants, so for example var(x) is a ground term (i.e., a term without variables).

The above rules directly translate into the following Prolog program that refers to the previously defined member

relation.

```
:- op(1100, yfx, arrow).
% type(E, S, T):- lambda term S has type T in the environment E.
type(E, var(X), T):- member([X, T], E).
type(E, apply(M, N), T):- type(E, M, S arrow T), type(E, N, S).
type(E, lambda(X, M), (S arrow T)):- type([[X, S] | E], M, T).
```

For readability, we use here arrow as a binary function symbol written in infix notation. The first line declares this use of arrow together with a certain associativity and priority information (The details of this archaic, though useful, Prolog notation are not relevant here.)

As expected, the above program can be used to check whether a given (representation of a) lambda term has a given type. Less expected is that this program can also be used to compute a type assignment to a lambda term, if such an assignment exists, and to report a failure if no such assignment exists. To this end, given a lambda term *s*, it suffices to use the query type([], t, T)., where the empty list [] denotes the empty environment and where t is the translation of *s* to a first-order term. For instance, the query

```
?- type([], lambda(x, apply(var(x), var(x))), T).
```

fails. In fact, no type can be assigned to the lambda term $\lambda x.\ (xx)$. The computation first leads to the call

```
type([[x, S]], apply(var(x), var(x)), T)
```

and then to the call

```
type([[x, S]], var(x), S arrow T).
```

This in turn leads to the call

```
member([x, S arrow T], [[x, S]])
```

which fails, because the terms S arrow T and S do not unify. In the above computation, T is used as a logical variable.

The problem of computing a type assignment for lambda terms was posed and solved by Curry and Feys (1958). It is an important topic in the theory of lambda calculus that is of relevance for type inference in functional programming. The solution in Prolog given above is completely elementary. A typical use of this program does not involve backtracking. In fact, its power relies on unification.

A Sequence Program

Next, consider the following problem: arrange three 1s, three 2s *,…,* three 9s in sequence so that for all *i* ? [1,9] there are exactly *i* numbers between successive occurrences of *i*. An example of such a sequence is

```
1, 9, 1, 2, 1, 8, 2, 4, 6, 2, 7, 9, 4, 5, 8, 6, 3, 4, 7, 5, 3, 9, 6, 8, 3, 5, 7.
```

The desired program is an almost verbatim formalization of the problem in Prolog.

```
% sequence(Xs) :- Xs is a list of 27 variables.
sequence([_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_]).
% question(Ss) :- Ss is a solution to the problem.
question(Ss) :-
    sequence(Ss),
    sublist([9,_,_,_,_,_,_,_,_,9,_,_,_,_,_,_,_,_,9], Ss),
    sublist([8,_,_,_,_,_,_,_,8,_,_,_,_,_,_,_,8], Ss),
    sublist([7,_,_,_,_,_,_,7,_,_,_,_,_,_,7], Ss),
    sublist([6,_,_,_,_,_,6,_,_,_,_,_,6], Ss),
    sublist([5,_,_,_,_,5,_,_,_,_,5], Ss),
    sublist([4,_,_,_,4,_,_,_,4], Ss),
    sublist([3,_,_,3,_,_,3], Ss),
    sublist([2,_,2,_,2], Ss),
    sublist([1,_,1,_,1], Ss).
```

Note how the anonymous variables dramatically improve the readability of the program.

Operationally, the query ?- question (Ss). leads to the procedure call sequence (Ss) that instantiates the variable Ss to the list of 27 anonymous (so different) variables. Then each of the nine calls of the sublist procedure enforces an existence of a specific sublist pattern on Ss. Each pattern involves syntactically anonymous variables, each of them representing operationally a logical variable.

In spite of the fact that the program is simple and transparent, the resulting computation is involved because of a extensive use of backtracking. The query generates all six solutions to the problem.

Difference lists

One of the drawbacks of the concatenation of lists performed by the APPEND program is that for lists s and t the execution of the query append (s, t, Z) takes the number of steps that is proportional to the length of the first list, s. This is obviously inefficient. In an imperative setting, if one represents a list as a link list, to concatenate two lists it suffices to adjust one pointer.

Difference list is a generalization of the concept of a list that allows us to perform concatenation in constant time. The fact that many programs rely explicitly on list concatenation explains the importance of this concept.

In what follows, we use the subtraction operator "-" written in the infix form. Its use has nothing to do with arithmetic, though intuitively one should read it as the "difference." Formally, a *difference list* is a construct of the form [a_1,…, a_m|x] –x, where x is a variable and where we used the notation introduced inSub section 15.4.2. It *represents* the list [a_1,…, a_m] in a form amenable to a different definition of concatenation. Namely, consider two difference lists[a_1,…, a_m|x] –x and [b_1,…, b_n|y] –y. Then their concatenation is the difference list [a_1,…, a_m, b_1,…, b_n|y] –y 2.

This concatenation process is achieved by the following one line APPEND_DL program:

```
% append(Xs, Ys, Zs) :- the difference list Zs is the result of concatenating
%          the difference lists Xs and Ys.
append_dl(X-Y, Y-Z, X-Z).
```

For example, we have:

```
?- append_dl([a,b|X]-X, [c,d|Y]-Y, U).
U = [a,b,c,d|Y]-Y,
X = [c,d|Y]
```

which shows that U became instantiated to the difference list representing the list [a,b,c,d].

We shall illustrate the use of difference lists in Subsection 15.6.2.

15.6 ARITHMETIC IN PROLOG

The Prolog programs presented so far are *declarative* since they admit a dual reading as a formula. The treatment of arithmetic in Prolog compromises to some extent its declarative underpinnings. However, it is difficult to come up with a better solution than the one offered by the original designers of the language. The shortcomings of Prolog's treatment of arithmetic are overcome in the constraint logic programming languages.

15.6.1 Arithmetic Operators

Prolog provides integers and floating point numbers as built-in data structures, with the typical operations on them. These operations include the usual arithmetic operators such as +,-, * (multiplication), and // (integer division).

Now, according to the usual notational convention of logic programming and Prolog, the relation and function symbols are written in the *prefix form*, that is in front of the arguments. In contrast, in accordance with their usage in arithmetic, the binary arithmetic operators are written in *infix form*, that is between the arguments. Moreover, negation of a natural number can be written in the *bracketless prefix form*, that is, without brackets surrounding its argument.

This discrepancy in the syntax is resolved by considering the arithmetic operators as built-in function symbols written in the infix or bracketless prefix form with information about their associativity and binding power that allows us to disambiguate the arithmetic expressions.

Actually, Prolog provides a means to declare an *arbitrary* function symbol as an infix binary symbol or as a bracketless prefix unary symbol, with a fixed *priority* that determines its binding power and a certain *mnemonics* that implies some (or no)form of associativity. An example of such a declaration was the line:- op(1100, yfx, arrow). used in the above-type assignment program. Function symbols declared in this way are called *operators*. Arithmetic operators can be thus viewed as operators predeclared in the language "prelude."

In addition to the arithmetic operators we also have at our disposal infinitely many integer constants and infinitely many floating point numbers. In what follows, by a *number*, we mean either an integer constant or a floating point number. The arithmetic operators and the set of all numbers uniquely determine a set of terms. We call terms defined in this language *arithmetic expressions* and introduce the abbreviation *gae* for ground (i.e., variable free) arithmetic expressions.

15.6.2 Arithmetic Comparison Relations

With each gae, we can uniquely associate its *value*, computed in the expected way. Prolog allows us to compare the values of gaes by means of the customary six *arithmetic comparison relations*

<, =<, =:= ("equal"), =\=, ("different"), >=, and >.

The "equal" relation "=:=" should not be confused with the "is unifiable with" relation "=" discussed in Section 15.3.

The arithmetic comparison relations work on gaes and produce the expected outcome. For instance, > compares the values of two gaes and succeeds if the value of the first argument is larger than the value of the second and fails otherwise.

Thus, for example

```
?- 6*2 =:= 3*4.
yes
?- 7 > 3+4.
no
```

However, when one of the arguments of the arithmetic comparison relations is not a gae, the computation *ends in an error*.

For example, we have

```
?- [] < 5.
error in arithmetic expression: [] is not a number.
```

As a simple example of the use of the arithmetic comparison relations, consider the following program, which checks whether a list of numbers is ordered.

```
% ordered(Xs) :- Xs is an =<-ordered list of numbers
ordered([]).
ordered([_]).
ordered([X, Y | Xs]) :- X =< Y, ordered([Y | Xs]).
```

Recall that `[X, Y | Xs])` is the abbreviated Prolog notation for `[X | [Y | Xs]])`.

We now have

```
?- ordered([1,1,2,3]). yes
```

but also

```
?- ordered([1,X,1]).
instantiation fault in 1 =< X
```

Here, a run-time error took place because at a certain stage the comparison relation "=<" was applied to an argument that is not a number.

As another example, consider Prolog's version of the quicksort procedure of C.A.R. Hoare. According to this sorting procedure, a list is first partitioned into two sublists using an element X of it, one consisting of the elements smaller than X and the other consisting of the elements larger or equal than X. Then each sublist is quicksorted and the resulting sorted sublists are appended with the element X put in the middle. This can be expressed in Prolog by means of the following QUICKSORT program, where X is chosen to be the first element of the given list:

```
% qs(Xs, Ys):- Ys is an ordered permutation of the list Xs.
qs([], []).
qs([X | Xs], Ys):-
    part(X, Xs, Littles, Bigs),
    qs(Littles, Ls),
    qs(Bigs, Bs),
    append(Ls, [X | Bs], Ys).
% part(X, Xs, Ls, Bs):- Ls is a list of elements of Xs which are < X,
%                Bs is a list of elements of Xs which are >= X.
part(_, [], [], []).
part(X, [Y | Xs], [Y | Ls], Bs) :-X>Y, part(X, Xs, Ls, Bs).
part(X, [Y | Xs], Ls, [Y | Bs]) :- X =< Y, part(X, Xs, Ls, Bs).
```

We now have, for example

```
?- qs([7,9,8,1,5], Ys).
Ys = [1, 5, 7, 8, 9]
```

and also

```
?- qs([7,9,8,1,5], [1,5,7,9,8]).
no
```

The QUICKSORT program uses the append relation to concatenate the lists. Consequently, its efficiency can be improved using the difference lists introduced in Subsection 15.5.2. Conceptually, the calls of the append relation are first replaced by the corresponding calls of the append_dl relation. This yields a program defining the qs dl relation. Then unfolding the calls of append_dl leads to a program that does not use the APPEND_DL program anymore and performs the list concatenation "on the fly." This results in the program QUICKSORT_DL in which the definition of the qs relation is replaced by

---

```
% qs(Xs, Ys) :- Ys is an ordered permutation of the list Xs.
qs(Xs, Ys) :- qs_dl(Xs, Ys - []).
%qs_dl(Xs, Y) :- Y is a difference list representing the
%          ordered permutation of the list Xs.
qs_dl([], Xs - Xs).

qs_dl([X | Xs], Ys - Zs) :-
    part(X, Xs, Littles, Bigs),
    qs_dl(Littles, Ys - [X | Y1s]),
    qs_dl(Bigs, Y1s - Zs).
```

The first rule links the qs relation with the qs_dl relation.

15.6.3 Evaluation of Arithmetic Expressions

So far we have presented programs that use ground arithmetic expressions but have not yet introduced any means of evaluating them. For example, no facilities have been introduced so far to evaluate 3+4. All we can do at this stage is to check that the outcome is 7 by using the comparison relation =:= and the query 7 =:= 3+4. However, using the comparison relations it is not possible to *assign* the value of 3+4, that is 7, to a variable, say X. Note that the query X =:= 3+4. ends in an error, while the query X = 3+4. instantiates X to the term 3+4.

To overcome this problem, the binary *arithmetic evaluator* is used in Prolog. is an infix operator defined as follows.

Consider the call ist.

Then t has to be a ground arithmetic expression (gae).

The call of s is t results in the unification of the *value* of the gae t with s.

If t is not a gae then a run-time error arises.

Thus, for example, we have

```
?- 7 is 3+4.
```

```
yes
8 is 3+4.
no
?- X is 3+4.
X=7
?- X is Y+1.
! Error in arithmetic expression: not a number
```

As an example of the use of an arithmetic evaluator, consider the proverbial factorial function. It can be computed using the following program FACTORIAL:

```
% factorial(N, F) :- F is N!.
factorial(0, 1).
factorial(N, F) :- N > 0, N1 is N-1, factorial(N1, F1), F is N*F1.
```

Note the use of a local variable N1 in the atom N1 is N-1 to compute the decrement of N and the use of a local variable F1 to compute the value of N1 factorial. The atom N1 is N-1 corresponds to the assignment command N:= N-1 of imperative programming. The difference is that a new variable needs to be used to compute the value of N-1. Such uses of local variables are typical when computing with integers in Prolog.

As another example consider a Prolog program that computes the length of a list.

```
% length(Xs, N):- N is the length of the list Xs.
length([], 0).
length([_| Ts], N) :- length(Ts, M), N is M+1.
```

We then have

```
?- length([a,b,c], N).
N=3
```

An intuitive but incorrect version would use as the second clause

```
length([_| Ts], N+1) :- length(Ts, N).
```

With such definition we would get the following nonintuitive outcome:

```
?- length([a,b,c], N)
.N=0+1+1+1
```

The point is that the generated ground arithmetic expressions are not automatically evaluated in Prolog.

We conclude that arithmetic facilities in Prolog are quite subtle and require good insights to be properly used.

Team-Fly



15.7 CONTROL, AMBIVALENT SYNTAX, AND META-VARIABLES

In the framework discussed so far, no control constructs are present. Let us see now how they could be simulated by means of the features explained so far. Consider the customary **if** B **then** S **else** T **fi** construct. It can be modelled by means of the following two clauses:

```
p(x):-B,S.
p(x) :- not B, T.
```

where p is a new procedure identifier and all the variables of B, S and T are collected in **x**. To see how inefficiency creeps into this style of programming, consider two cases.

First, suppose that the first clause is selected and that B is true (i.e., succeeds). Then the computation continues with S. But in general B is an arbitrary query and because of the implicit nondeterminism present B can succeed in many ways. If the computation of S fails, these alternative ways of computing B will be automatically tried even though we know already that B is true.

Second, suppose that the first clause is selected and that B is false (that is fails). Then backtracking takes place and the second clause is tried. The computation proceeds by evaluating not B. This is completely unneeded, since we know at this stage that not B is true (that is, succeeds).

Note that omitting not B in the second rule would cause a problem in case a success of B were followed by a failure of S. Then upon backtracking T would be executed.

15.7.1 Cut

To deal with such problems, Prolog provides a low level built-in nullary relation symbol called *cut* and denoted by "!". To explain its meaning we rewrite first the above clauses using cut:

```
p(x):-B,!,S.
p(x):-T.
```

In the resulting analysis, two possibilities arise, akin to the above case distinction. First, if B is true (i.e., succeeds), then the cut is encountered. Its execution

- discards all alternative ways of computing B,

- discards the second clause, p(**x**):- T as a backtrackable alternative to the current selection of the first clause.

Both items have an effect that in the current computation some clauses are not available anymore.

Second, if B is false (i.e., fails), then backtracking takes place and the second clause is tried. The computation proceeds now by directly evaluating T.

So using the cut and the above rewriting we achieved the intended effect and modelled the **if** B **then** S **else** T **fi** construct in the desired way.

The above explanation of the effect of cut is a good starting point to provide its definition in full generality.

Consider the following definition of a relation p:

---

p(**s**1):- **A**1.
...
p(**s**i):- **B**,!,**C**..
...
(**s** k):- **A** k.

Here, the ith clause contains a cut atom. Now, suppose that during the execution of a query, a call p(**t**) is encountered and eventually the ith clause is used and the indicated occurrence of the cut is executed. Then the indicated occurrence of ! succeeds immediately, but additionally

1. all alternative ways of computing **B** are discarded, and
2. all computations of p(**t**) using the ith to k^{th} clause for p are discarded as backtrackable alternatives to the current selection of the *i*-clause.

The cut was introduced to improve the implicit control present through the combination of backtracking and the textual ordering of the clauses. Because of the use of patterns in the clause heads, the potential source of inefficiency can be sometimes hidden somewhat deeper in the program text. Reconsider for example the QUICKSORT program of Section 15.6 and the query ?- qs([7,9,8,1,5], Ys). To see that the resulting computation is inefficient, note that the second clause defining the part relation fails when 7 is compared with 9 and subsequently the last, third, clause is tried. At this moment 7 is again compared with 9. The same redundancy occurs when 1 is compared with 5. To avoid such inefficiencies the definition of part can be rewritten using cut as follows:

```
part( , [], [], []).
part(X, [Y | Xs], [Y | Ls], Bs) :-X>Y,!, part(X, Xs, Ls, Bs).
part(X, [Y | Xs], Ls, [Y | Bs]) :- part(X, Xs, Ls, Bs).
```

Of course, this improvement can be also applied to the QUICKSORT_DL program.

Cut clearly compromises the declarative reading of the Prolog programs. It has been one of the most criticized

features of Prolog. In fact, a proper use of cut requires a good understanding of Prolog's computation mechanism and a number of thumb rules were developed to help a Prolog programmer to use it correctly. A number of alternatives to cut were proposed. The most interesting of them, called *commit*, entered various constraint and parallel logic programming languages but is not present in standard Prolog.

15.7.2 Ambivalent Syntax and Meta-variables

Before we proceed, let us review first the basics of Prolog syntax mentioned so far.

1. Variables are denoted by strings starting with an upper case letter or "_"(underscore). In particular, Prolog allows so-called anonymous variables, written as "_" (underscore).
2. Relation symbols (procedure identifiers), function symbols, and nonnumeric constants are denoted by strings starting with a lower case letter.
3. Binary and unary function symbols can be declared as infix or bracketless prefix operators.

Now, in contrast to first-order logic, in Prolog the *same* name can be used both for function symbols and for relation symbols. Moreover, the same name can be used for function or relation symbols of different arity. This facility is called *ambivalent syntax*. A function or a relation symbol f of arity n is then referred to as f/n. Thus, in a Prolog program, we can use both a relation symbol p/2 and function symbols p/1 and p/2 and build syntactically legal terms or atoms like p(p(a, b), c, p(X)).

In presence of the ambivalent syntax, the distinction between function symbols and relation symbols and between terms and atoms disappears, but in the context of queries and clauses, it is clear which symbol refers to which syntactic category.

The ambivalent syntax together with Prolog's facility to declare binary function symbols (and thus also binary relation symbols) as infix operators allows us to pass queries, clauses and programs as arguments. In fact, ":-/2" is declared internally as an infix operator and so is the comma ",/2" between the atoms, so each clause is actually a term. This facilitates *meta-programming*, that is, writing programs that use other programs as data.

In what follows, we shall explain how meta-programming can be realized in Prolog. To this end, we need to introduce one more syntactic feature. Prolog permits the use of variables in the positions of atoms, both in the queries and in the clause bodies. Such a variable is called then a *meta-variable*. Computation in the presence of the meta-variables is defined as before since the mgus employed can also bind the meta-variables. Thus, for example, given the legal, albeit unusual, Prolog program (that uses the ambivalent syntax facility)

```
p(a).
a.
```

---

the execution of the Prolog query p(X), X. first leads to the query a. and then succeeds. Here, a is both a constant and a nullary relation symbol.

Prolog requires that the meta-variables are properly instantiated before they are executed. In other words, they need to evaluate to a nonnumeric term at the moment they are encountered in an execution. Otherwise, a run-time error arises. For example, for the above program and the query p(X), X, Y., the Prolog computation ends up in error once the query Y. is encountered.

15.7.3 Control Facilities

Let us now see how the ambivalent syntax in conjunction with meta-variables supports meta-programming. In this section we limit ourselves to (meta-)programs that show how to introduce new control facilities. We discuss here three

examples, each introducing a control facility actually available in Prolog as a built-in. More meta-programs will be presented in the next section once we introduce other features of Prolog.

Disjunction

To start with, we can define *disjunction* by means of the following simple program:

```
or(X,Y) :- X.
or(X,Y) :- Y.
```

A typical query is then or(Q, R), where Q and R are "conventional queries." Disjunction is a Prolog's built-in declared as an infix operator ";/2" and defined by means of the above two rules, with "or" replaced by ";". So instead of or(Q, R) one writes Q;R.

If-then-else

The other two examples involve the cut operator. The already discussed **if** B **then** S **else** T **fi** construct can be introduced by means of the now-familiar program

```
if_then_else(B, S, T):- B,!,S.
if_then_else(B, S, T):- T.
```

if_then_else(B, S, T):- B,!, S.if_then_else(B, S, T) :- T.

In Prolog, if_then_else is a built-in defined internally by the above two rules.if_then_else (B, S, T) is written as B->S;T, where "→ /2" is a built-in declared as an infix operator. As an example of its use, let us rewrite yet again the definition of the part relation used in the QUICKSORT program, this time using Prolog's B->S;T. To enforce the correct parsing, we need to enclose the B->S;T statement in brackets:

```
part( , [], [], []).
part(X, [Y | Xs], Ls, Bs) :-
 (X>Y->
 Ls = [Y | L1s], part(X, Xs, L1s, Bs)
;
 Bs=[Y | B1s], part(X, Xs, Ls, B1s)
).
```

Note that here we had to dispense with the use of patterns in the "output" positions of part and reintroduce the explicit use of unification in the procedure body. By introducing yet another B->S;T statement to deal with the case analysis in the second argument, we obtain a definition of the part relation that very much resembles a functional program:

```
part(X, X1s, Ls, Bs) :-
 (X1s=[]->
 Ls = [], Bs = []
;
 X1s=[Y | Xs],
 (X>Y->
 Ls = [Y | L1s], part(X, Xs, L1s, Bs)
;
 Bs=[Y | B1s], part(X, Xs, Ls, B1s)
 )
).
```

In fact, in this program all uses of unification boil down to matching and its use does not involve backtracking. This example explains how the use of patterns often hides an implicit case analysis. By making this case analysis explicit using the **if-then-else** construct we end up with longer programs. In the end the original solution with the cut seems to be closer to the spirit of the language.

Negation

Finally, consider the negation operation not that is supposed to reverse failure with success. That is, the intention is that the query not Q. succeeds iff the query Q. fails. This operation can be easily implemented by means of meta-variables and cut as follows:

```
not(X):- X, !, fail.
not(_).
```

fail/0 is Prolog's built-in with the empty definition. Thus, the call of the parameterless procedure fail always fails.

This cryptic two-line program employs several discussed features of Prolog. In the first line, X is used as a meta-variable. Now consider the call not(Q), where Q is aquery. If Q succeeds, then the cut is performed. This has the effect that all alternative ways of computing Q are discarded and also the second clause is discarded. Next, the built-in fail is executed and a failure arises. Because the only alternative clause was just discarded, the query not(Q) fails. If, on the other hand, the query Q fails, then backtracking takes place and the second clause, not(_), is selected. It immediately succeeds and so the initial query not(Q) succeeds. So this definition of not achieves the desired effect.

not/1 is defined internally by the above two line definition augmented with the appropriate declaration of it as a bracketless prefix unary symbol.

Call

Finally, let us mention that Prolog also provides an indirect way of using meta-variables by means of a built-in relation call/1. call/1 is defined internally by this rule:

```
call(X) :- X.
```

call/1 is often used to "mask" the explicit use of meta-variables, but the outcome is the same.

15.7.4 Negation as Failure

The distinction between successful and failing computations is one of the unique features of logic programming and Prolog. In fact, no counterpart of failing computations exists in other programming paradigms.

The most natural way of using failing computations is by employing the negation operator not that allows us to turn failure into success, by virtue of the fact that the query not Q. succeeds iff the query Q. fails. This way we can use not to represent negation of a Boolean expression. In fact, we already referred informally to this use of negation at the beginning of Section 15.7.

This suggests a declarative interpretation of the not operator as a classical negation. This interpretation is correct only if the negated query always terminates and is ground. Note, in particular, that given the procedure p defined by the single rule p:-p. the query not p. does not terminate. Also, for the query not(X = 1)., we get the following counterintuitive outcome:

```
?- not(X = 1).
no
```

Thus, to generate all elements of a list Ls that differ from 1, the correct query is member(X, Ls), not(X = 1). and not not (X = 1), member(X, Ls). One usually refers to the way negation is used in Prolog as "negation as failure." When properly used, it is a powerful feature as testified by the following jewel program. We consider the problem of determining a winner in a two-person finite game. Suppose that the moves in the game are represented by a relation move. The game is assumed to be finite, so we postulate that given a position pos the query move(pos, Y). generates finitely many answers, which are all possible moves from pos. A player loses if he is in a position pos from which no move exists, i.e., if the query move(pos, Y).fails.

A position is a winning one when a move exists that leads to a losing, i.e., non-winning position. Using the negation operator, this can be written as

```
% win(X) :- X is a winning position in the two-person finite game
%     represented by the relation move.
win(X) :- move(X, Y), not win(Y).
```

This remarkably concise program has a simple declarative interpretion. In contrast, the procedural interpretation is quite complex: the query win(pos). determines whether pos is a winning position by performing a minimax search on the 0-1 game tree represented by the relation move. In this recursive procedure, the base case appears when the call to move fails. Then the corresponding call of win also fails.

15.7.5 Higher-Order Programming and Meta-Programming in Prolog

Thanks to the ambivalent syntax and meta-variables, higher-order programming and another form of meta-programming can be easily realized in Prolog. To explain this, we need two more built-ins. Each of them belongs to a different category.

Term Inspection Facilities

Prolog offers a number of built-in relations that allow us to inspect, compare, and decompose terms. One of them is =../2 (pronounced *univ*) that allows us to switch between a term and its representation as a list. Instead of precisely describing its meaning, we just illustrate one of its uses by means the following query:

```
?- Atom =.. [square, [1,2,3,4], Ys].
Atom = square([1,2,3,4], Ys).
```

The left-hand side, here Atom, is unified with the term (or, equivalently, the atom),here square([1,2,3,4], Ys), represented by a list on the right-hand side, here [square,[1,2,3,4], Ys]. In this list representation of a term, the head of the list is the leading function symbol and the tail is the list of the arguments.

Using *univ*, one can construct terms and pass them as arguments. More interestingly, one can construct atoms and execute them using the meta-variable facility. This way it is possible to realize higher-order programming in Prolog in the sense that relations can be passed as arguments. To illustrate this point, consider the following program MAP:

```
% map(P, Xs, Ys) :- the list Ys is the result of applying P
%     elementwise to the list Xs.
map(P, [], []).
map(P, [X | Xs] , [Y | Ys]) :- apply(P, [X, Y]), map(P, Xs, Ys).
% apply(P, [X1, …, Xn]) :- execute the atom P(X1, …, Xn).
apply(P, Xs) :- Atom =.. [P|Xs], Atom.
```

In the last rule, *univ* is used to construct an atom. Note the use of the meta-variable Atom. MAP is Prolog's counterpart of the familiar higher-order functional program and it behaves in the expected way. For example, given the program % square(X, Y):- Y is the square of X. square(X, Y) :- Y is X*X. we get

```
?- map(square, [1,2,3,4], Ys).
Ys = [1, 4, 9, 16]
```

Program manipulation facilities

Another class of Prolog built-ins makes it possible to access and modify the program during its execution. We consider here a single built-in in this category, clause/2, that allows us to access the definitions of the relations present in the considered program. Again, consider first an example of its use in which we refer to the program MEMBER of Sub section 15.5.1.

```
?- clause(member(X,Y), Z).
Y=[X|_A],
Z = true ;
Y=[ A|_B],
Z = member(X,_B) ;
no
```

In general, the call clause(head, body) leads to a unification of the term head :- body with the successive clauses forming the definition of the relation in question. This relation, here member, is the leading symbol of the first argument of clause/2 that has to be a non-variable.

This built-in assumes that true is the body of a fact, here member(X, [X |_]). true/0 is Prolog's built-in that succeeds immediately. Thus, its definition consists just of the fact true. This explains the first answer. The second answer is the result of unifying the term member(X,Y) :- Z with (a renaming of) the second clause defining member, namely member(X, [_| Xs]):- member(X, Xs).

Using clause/2, we can construct Prolog interpreters written in Prolog, that is, *meta-interpreters*. Here is the simplest one:

```
% solve(Q) :- the query Q succeeds for the program accessible by clause/2.
solve(true) :- !.
solve((A,B)) :- !, solve(A), solve(B).
solve(A) :- clause(A, B), solve(B).
```

Recall that (A,B) is a legal Prolog term (with no leading function symbol). To understand this program, one needs to know that the comma between the atoms is declared internally as a right associative infix operator, so the query

A,B,C,D actually stands for the term (A,(B,(C,D))), etc.

The first clause states that the built-in true\ succeeds immediately. The second clause states that a query of the form *A*,**B** can be solved if *A* can be solved and **B** can be solved. Finally, the last clause states that an atomic query *A* can be solved if there exists a clause of the form *A* :-**B** such that the query **B** can be solved. The cuts are used here to enforce the a "definition by cases": either the argument of solve is true or a nonatomic query or else an atomic one.

To illustrate the behavior of the above meta-interpreter, assume that MEMBER is a part of the considered program. We then have

---

```
?- solve(member(X, [mon, wed, fri])).
X = mon ;
X = wed ;
X = fri ;
no
```

This meta-program forms a basis for building various types of interpreters for larger fragments of Prolog or for its extensions.

Team-Fly



Previous Next

Team-Fly

15.8 ASSESSMENT OF PROLOG

Prolog, because of its origin in automated theorem proving, is an unusual programming language. It leads to a different style of programming and to a different view of programming. A number of elegant Prolog programs presented here speak for themselves. We also noted that the same Prolog program can often be used for different purposes - for computing, testing or completing a solution, or for computing all solutions. Such programs cannot be easily written in other programming paradigms. Logical variables are a unique and, as we saw, very useful feature of logic programming. Additionally, pure Prolog programs have a dual interpretation as logical formulas. In this sense, Prolog supports declarative programming.

Both through the development of a programming methodology and ingenious implementations, great care was taken to overcome the possible sources of inefficiency. On the programming level, we already discussed cut and the difference lists. Programs such as FACTORIAL of Subsection 15.6.3 can be optimized by means of tail recursion. On the implementation level, efficiency is improved by such techniques as the last call optimization that can be used to optimize tail recursion, indexing that deals with the presence of multiple clauses, and a default omission of the occur-check (the test "*x* does not occur in *t*" in clause (5) of the Martelli-Montanari algorithm) that speeds up the unification process (although on rare occasions makes it unsound).

Prolog's only data type, the terms, is implicitly present in many areas of computer science. In fact, whenever the objects of interest are defined by means of grammars, for example first-order formulas, digital circuits, programs in any programming language, or sentences in some formal language, these objects can be naturally defined as terms. Prolog programs can then be developed starting with this representation of the objects as terms. Prolog's support for handling terms by means of unification and various term inspection facilities becomes handy. In short, symbolic data can be naturally handled in Prolog.

The automatic backtracking becomes very useful when dealing with search. Search is of paramount importance in many artificial intelligence applications and backtracking itself is most natural when dealing with the NP-complete problems. Moreover, the principle of "computation as deduction" underlying Prolog's computation process facilitates formalization of various forms of reasoning in Prolog. In particular, Prolog's negation operator not can be naturally used to support nonmonotonic reasoning. All this explains why Prolog is a natural language for programming artificial intelligence applications, such as automated theorem provers, expert systems, and machine learning programs where reasoning needs to be combined with computing, game playing programs, and various decision support systems.

Prolog is also an attractive language for computational linguistics applications and for compiler writing. In fact, Prolog provides support for so-called definite clause grammars (DCG). Thanks to this, a grammar written in the DCG form is already a Prolog program that forms a parser for this grammar. The fact that Prolog allows one to write executable specifications makes it also a useful language for rapid prototyping, in particular in the area of meta-programming.

For the sake of a balanced presentation let us discuss now Prolog's shortcomings.

Lack of Types

Types are used in programming languages to structure the data manipulated by the program and to ensure its correct use. In Prolog, one can define various types like binary trees and records. Moreover, the language provides a notation for lists and offers a limited support for the type of all numbers by means of the arithmetic operators and arithmetic comparison relations. However, Prolog does not support types in the sense that it does not check whether the queries use the program in the intended way.

Because of this absence of type checking, type errors are easy to make but difficult to find. For example, even though the APPEND program was meant to be used to concatenate two lists, it can also be used with nonlists as arguments:

```
?- append([a,b], f(c), Zs).
Zs = [a, b|f(c)]
```

and no error is reported. In fact, almost every Prolog program can be misused. Moreover, because of lack of type checking some improvements of the efficiency of the implementation cannot be carried out and various run-time errors cannot be prevented.

Subtle Arithmetic

We discussed already the subtleties arising in presence of arithmetic in Section 15.6. We noted that Prolog's facilities for arithmetic easily lead to run-time errors. It would be desirable to discover such errors at compile time but this is highly nontrivial.

Idiosyncratic Control

Prolog's control mechanisms are difficult to master by programmers accustomed to the imperative programming style. One of the reasons is that both bounded iteration (the for statement) and unbounded iteration (the while statement) need to be implemented by means of the recursion. For example, a nested for statement is implemented by means of nested tail recursion that is less easy to understand. Of course, one can introduce both constructs by means of meta-programming, but then their proper use is not enforced because of the lack of types. Additionally, as already mentioned, cut is a low-level mechanism that is not easy to understand.

Complex Semantics of Various Built-ins

Prolog offers a large number of built-ins. In fact, the ISO Prolog Standard describes 102 built-ins. Several of them are quite subtle. For example, the query not(not Q). tests whether the query Q. succeeds and this test is carried out without changing the state, i.e., without binding any of the variables. Moreover, it is not easy to describe precisely the meaning of some of the built-ins. For example, in the ISO Prolog Standard the operational interpretation of the **if-then-else** construct consists of 17 steps.

No Modules and No Objects

Finally, even though modules exist in many widely used Prolog versions, neither modules nor objects are present in ISO Prolog Standard.as This makes it difficult to properly structure Prolog programs and reuse them as components of other Prolog programs. It should be noted that thanks to Prolog's support for meta-programming, the object-programming style can be mimicked in Prolog in a simple way. But no compile-time checking of its proper use is then enforced then and errors in the program design will be discovered at best at the run-time. The same critique applies to Prolog's approach to higher-order programming and to meta-programming.

Of course, these limitations of Prolog were recognized by many researchers who came up with various good proposals on how to improve Prolog's control, how to add to it (or how to infer) types, and how to provide modules and objects. Research in the field of logic programming also has dealt with the precise relation between the procedural and declarative interpretation of logic programs and a declarative account of various aspects of Prolog, including negation and meta-programming. Also verification of Prolog programs and its semantics were extensively studied.

However, no single programming language proposal emerged yet that could be seen as a natural successor to Prolog in which the above shortcomings are properly solved. The language that comes closest to this ideal is Mercury (see http://www.cs.mu.oz.au/research/mercury/). Colmerauer designed a series of successors of Prolog, Prolog II, III, and IV that incorporated various forms of constraint processing into this programming style.

When assessing Prolog, it is useful to have in mind that it is a programming language designed in the early 1970s (and standardized in the 1990s). The fact that it is still widely used and that new applications for it keep being found testifies to its originality. No other programming language succeeded to embrace first-order logic in such an effective way.

Team-Fly

15.9 BIBLIOGRAPHIC REMARKS

For those interested in learning more about the origins of logic programming and of Prolog, the best place to start is Colmerauer and Roussel's account (The Birth of Prolog, in Bergin and Gibson, History of Programming Languages, ACM Press/Addison-Wesley, 1996, pp. 331-367). There a number of excellent books on programming in Prolog. The two deservedly most successful are Bratko (*PROLOG Programming for Artificial Intelligence*, Addison-Wesley, 2001) and Sterling and Shapiro (*The Art of Prolog*, MIT Press, 1994). The work by O'Keefe (*The Craft of Prolog*, MIT Press, 1990) discusses in depth the efficiency and pragmatics of programming in Prolog. The work by Aït-Kaci (*Warrens' Abstract Machine*, MIT Press, 1991. Out of print. Available at http://www.isg.sfu.ca/~hak/documents/wam.html) is an outstanding tutorial on the implementation of Prolog.

Team-Fly

Team-Fly

15.10 CHAPTER SUMMARY

We discussed the logic programming paradigm and its realization in Prolog. This paradigm has contributed a number of novel ideas in the area of programming languages. It introduced unification as a computation mechanism and it realized the concept of "computation as deduction". Additionally, it showed that a fragment of first-order logic can be used as a programming language and that declarative programming is an interesting alternative to structured programming in the imperative programming style.

Prolog is a rule-based language but thanks to a large number of built-ins it is a general purpose programming language. Programming in Prolog substantially differs from programming in the imperative programming style. Table 15.1 may help to relate the underlying concepts used in both programming styles.

Table 15.1

Logic Programming	Imperative Programming
equation solved by unification	assignment
relation symbol	procedure identifier
term	expression
atom	procedure call
query	program
definition of a relation	procedure declaration
local variable of a rule	local variable of a procedure
logic program	set of procedure declarations
"," between atoms	sequencing (";")
substitution	state
composition of substitutions	state update

Acknowledgements

Maarten van Emden and Jan Smaus provided K.R. Apt with useful comments on this chapter.

Team-Fly